

Principia Softwarica: The Version Control System `git`

version 0.1

Yoann Padioleau
`yoann.padioleau@gmail.com`

with code from
Ori Bernstein

May 18, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2019 Ori Bernstein.
MIT license.

Contents

1	Introduction	11
1.1	Motivations	11
1.2	The version control system Git (and <code>git9</code>)	11
1.3	Other version control systems	12
1.4	Getting started	13
1.5	Requirements	14
1.6	About this document	14
1.7	Copyright	14
1.8	Acknowledgments	15
2	Overview	16
2.1	Version control system principles	16
2.1.1	Storing past versions	16
2.1.2	Tracking changes	17
2.1.3	Concurrent development	17
2.1.4	Parallel development	17
2.1.5	Centralized versus distributed	17
2.2	<code>git</code> command-line interface	17
2.3	Git concepts and data structures	18
2.3.1	Object store	18
2.3.2	References	20
2.3.3	Staging area	20
2.4	Repository format: <code>.git/</code>	22
2.5	<code>hello/.git/</code>	23
2.5.1	Creating a repository	23
2.5.2	Staging a diff	24
2.5.3	Committing a diff	24
2.5.4	Managing branches	25
2.5.5	Inspecting objects	26
2.5.6	Other commands	27
2.6	Code organization	27
2.7	Software architecture	27
2.8	Book structure	27
I	Foundations	28
3	Core Data Structures	29
3.1	Secure Hash Algorithm (SHA1) hashes	29
3.1.1	Binary hashes	29

3.1.2	Hexadecimal hashes	29
3.1.3	Conversion functions	30
3.2	Object	30
3.2.1	Object types: <code>Gxxx</code>	31
3.2.2	Object flags: <code>Cxxx</code>	31
3.2.3	Reference counting	32
3.2.4	Blobs	33
3.2.5	Trees and <code>Tinfo</code>	33
3.2.6	Commits and <code>Cinfo</code>	34
3.3	Objects	34
3.3.1	<code>Objset</code>	34
3.3.2	<code>Objq</code>	36
3.4	References	38
3.4.1	Reference names	38
3.4.2	<code>HEAD</code>	38
3.4.3	Reference content	38
3.4.4	Reference store	38
3.4.5	Objectish references	38
3.5	<code>.git/INDEX9</code>	38
4	Reading from a Repository	39
4.1	Objects	39
4.1.1	<code>readobject()</code>	39
4.1.2	<code>readidxobject()</code>	39
4.1.3	<code>readloose()</code> and <code>Cloaded</code>	40
4.1.4	<code>parseobject()</code> and <code>Cparsed</code>	41
4.1.5	Decompression	42
4.1.6	Reading a blob	43
4.1.7	Reading a tree	43
4.1.8	Reading a commit	44
4.2	References	46
4.2.1	<code>.refs/</code> prefixes	46
4.2.2	<code>matchpfx</code>	47
4.3	Index	48
5	Writing to a Repository	49
5.1	Objects	49
5.1.1	Compression	49
5.1.2	Locking	49
5.1.3	Writing a blob	49
5.1.4	Writing a tree	49
5.1.5	Writing a commit	49
5.2	References	49
5.3	Index	49
6	git/conf	50
6.1	Repository configuration: <code>.git/config</code>	50
6.2	User configuration: <code>~/lib/git/config</code>	50
6.3	Usage	50
6.4	<code>main()</code>	50

6.5	gitinit() and findrepo()	51
6.6	git/conf -r	52
6.7	git/conf <key>	52
6.8	Advanced features	53
6.8.1	git/conf -a	53
6.8.2	git/conf -f <config>	54
7	git/fs	55
7.1	Usage	55
7.2	main()	55
7.3	Qxxx and qroot	56
7.4	Data structures	57
7.4.1	Gिताux and Crumb	57
7.4.2	Objlist	57
7.4.3	Uqid and Cache	58
7.5	gitxxx() skeletons	59
7.5.1	gitattach() and gitdir	59
7.5.2	gitclone() and gitdestroyfid()	60
7.5.3	gitopen()	61
7.5.4	gitstat()	61
7.5.5	gitread()	62
7.5.6	gitwalk1()	63
7.6	.git/fs/	64
7.6.1	.git/fs/ctl	65
7.6.2	.git/fs/object/	65
7.6.3	.git/fs/object/<blobid>	68
7.6.4	.git/fs/object/<treeid>/	68
7.6.5	.git/fs/object/<commitid>/	69
7.6.6	.git/fs/object/<commitid>/tree	71
7.6.7	.git/fs/object/<commitid>/parent	71
7.6.8	.git/fs/object/<commitid>/msg	71
7.6.9	.git/fs/object/<commitid>/hash	72
7.6.10	.git/fs/object/<commitid>/author	72
7.6.11	.git/fs/object/<commitid>/committer	72
7.6.12	.git/fs/HEAD/	72
7.6.13	.git/fs/branch/	73
7.7	git/fs -m	75
8	git/query	76
8.1	Usage	76
8.2	Commit expressions	76
8.3	main()	76
8.4	Data structures	78
8.4.1	Eval	78
8.5	resolverefs()	78
8.6	Commit expression evaluator	78
8.6.1	evalpostfix()	79
8.6.2	word()	80
8.6.3	push() and pop()	81
8.6.4	parent() and ^	81

8.6.5	<code>lca()</code> and <code>@</code>	82
8.6.6	<code>range()</code> and <code>..</code>	83
8.7	<code>paint()</code>	83
8.8	Advanced features	86
8.8.1	<code>git/query -c</code> , list changes between commits	86
8.8.2	<code>git/query -p</code> , full path listing	86
8.8.3	<code>git/query -r</code> , reversed listing	86
9	<code>git/walk</code>	88
9.1	Usage	88
9.2	<code>main()</code>	88
9.3	Advanced features	92
II	Basic Commands	93
10	Creating a Repository	94
10.1	Initializing a new repository: <code>git init</code>	94
10.1.1	<code>init.rc</code>	94
10.1.2	Usage	94
10.2	Copying a local repository: <code>cp -r</code>	95
10.3	Cloning a remote repository: <code>git clone</code>	95
11	Staging a Diff	96
11.1	Adding files: <code>git add</code>	96
11.1.1	Usage	96
11.1.2	<code>add.rc</code>	96
11.1.3	<code>gitup()</code> part 1	96
11.1.4	<code>drop()</code>	97
11.2	Removing files: <code>git rm</code>	97
12	Committing a Diff	98
12.1	Committing the index: <code>git commit</code>	98
12.1.1	Usage	98
12.1.2	<code>commit.rc</code>	98
12.1.3	<code>whoami()</code>	99
12.1.4	<code>gitup()</code> part 2	99
12.1.5	<code>findbranch()</code>	99
12.1.6	<code>parents()</code>	100
12.1.7	<code>editmsg()</code>	100
12.1.8	<code>commit()</code>	101
12.1.9	<code>update()</code>	101
12.2	<code>git/walk -fRMA</code>	102
12.3	<code>git/query \$branch</code>	102
12.4	<code>git/save</code>	102
12.4.1	Usage	102
12.4.2	<code>main()</code>	102
12.5	Advanced features	104
12.5.1	<code>git/commit -r</code> , revising a commit	104
12.5.2	<code>git/commit -p</code> , selecting hunks	104

13 Branching	106
13.1 <code>git branch</code>	106
13.1.1 Usage	106
13.1.2 <code>branch.rc</code>	106
13.2 Listing branches: <code>git branch</code>	109
13.3 Creating a branch: <code>git branch <name></code>	109
13.4 Deleting a branch: <code>git branch -d <name></code>	109
13.5 Checking out a branch: <code>git checkout</code>	109
13.5.1 Computing the index (and work tree) from trees	109
13.5.2 Creating a file from a blob	109
13.6 Resetting a branch: <code>git reset</code>	109
14 Merging	110
14.1 merging branches: <code>git merge</code>	110
14.1.1 Usage	110
14.1.2 <code>merge.rc</code>	110
14.1.3 <code>merge()</code>	111
14.1.4 <code>merge1()</code>	111
14.1.5 <code>mergeperm()</code>	112
14.2 Merging two trees	112
14.3 Merging two files	112
14.4 <code>diff3</code> and <code>merge3</code>	112
14.5 Merging conflicts	112
14.6 Committing merged branches: <code>git commit</code>	112
15 Inspecting	113
15.1 Showing the commit history: <code>git log</code>	113
15.1.1 Usage	113
15.1.2 <code>main()</code>	113
15.1.3 <code>showcommits()</code>	114
15.1.4 <code>show()</code>	115
15.1.5 <code>showquery()</code>	116
15.1.6 Advanced features	116
15.2 Representing changes	116
15.2.1 Tree changes	116
15.2.2 File changes	116
15.3 Showing file differences: <code>git diff</code>	116
15.3.1 Usage	116
15.3.2 <code>diff.rc</code>	116
15.3.3 <code>diff -u</code>	117
15.3.4 Computing tree changes	117
15.3.5 Computing file changes	117
15.3.6 Showing changes	117
16 Packing Objects	118
16.1 <code>Pack.t</code>	118
16.2 IO	118
16.2.1 Reading a pack	118
16.2.2 Writing a pack	119
16.3 Modifications to previous functions and data structures	119

16.3.1	Object Store	119
16.3.2	Refs	119
16.4	Delta	119
16.5	Pack commands	119
16.5.1	<code>git repack</code>	119
17	Exchanging	121
17.1	Upstream and remote origin	121
17.2	Pulling updates from another repository: <code>git pull</code>	121
17.2.1	Usage	121
17.2.2	<code>pull.rc</code>	121
17.2.3	Fetching objects locally	123
17.2.4	Fetching references	123
17.2.5	Fetching objects in a pack	123
17.3	<code>git/get</code>	123
17.3.1	Usage	123
17.3.2	<code>main()</code>	123
17.4	Object store walker	124
17.5	Pushing changes to another repository: <code>git push</code>	124
17.5.1	Usage	124
17.5.2	<code>push.rc</code>	124
17.5.3	Sending objects locally	125
17.5.4	Sending objects in a pack	125
17.6	<code>git/send</code>	125
17.6.1	Usage	125
17.6.2	<code>main()</code>	125
17.7	Cloning a remote repository: <code>git clone</code>	126
17.7.1	Usage	126
17.7.2	<code>clone.rc</code>	127
17.7.3	Fetching objects	129
17.7.4	Importing references	129
18	Networking	130
18.1	<code>git://</code> protocol	130
18.1.1	Reading packets	131
18.1.2	Writing packets	131
18.2	Capabilities	131
18.3	<code>git://</code> client	131
18.3.1	Fetching remote objects	131
18.3.2	Fetching pack files	131
18.3.3	Sending pack files	131
18.4	<code>git://</code> server	131
18.4.1	<code>git/serve</code>	131
III	Beyond the Basics	133
19	Core Algorithms	134
19.1	SHA1	135
19.1.1	Overview	135

19.1.2	32 bits operators	135
19.1.3	Entry point: <code>sha1()</code>	135
19.1.4	Endianness	135
19.1.5	Filling blocks	135
19.1.6	Looping over blocks	135
19.1.7	Padding	135
19.2	Zip and Unzip	135
19.3	Diff	135
19.3.1	Overview	135
19.3.2	Data structures	135
19.3.3	Entry point: <code>Diffs.diff()</code>	135
19.3.4	Edit distance basic algorithm ($O(n^2)$ in space/time)	135
19.3.5	Myers algorithm ($O(n)$ in space in best-case)	135
19.4	Diff3	135
19.4.1	Overview	135
19.4.2	Data structures	135
19.4.3	Entry point: <code>diff3()</code>	135
19.4.4	Displaying merging conflicts	135
20	Supporting Programs	136
20.1	<code>diff</code>	136
20.1.1	Usage	136
20.1.2	Data structures	136
20.1.3	<code>main()</code>	136
20.1.4	Advanced features	137
20.2	<code>patch</code>	137
20.2.1	Usage	137
20.2.2	Data structures	138
20.2.3	<code>main()</code>	138
20.2.4	Advanced features	139
20.3	<code>merge3</code>	139
20.3.1	Usage	139
20.3.2	Data structures	139
20.3.3	<code>main()</code>	139
21	Exchanging Patches	141
21.1	<code>git/export</code>	141
21.1.1	Usage	141
21.1.2	<code>export.rc</code>	141
21.2	<code>git/import</code>	143
21.2.1	Usage	143
21.2.2	<code>import.rc</code>	143
22	Advanced Features	146
22.1	Tracking symbolic links	146
22.2	Splitting a large repository: submodules	147

23 Advanced Commands	148
23.1 Tagging a commit with a name	148
23.1.1 Reading a tag	148
23.1.2 Writing a tag	148
23.2 <code>git/hist</code>	148
23.2.1 Usage	148
23.2.2 <code>hist.rc</code>	149
23.3 Reverting a change: <code>git revert</code>	149
23.3.1 Usage	149
23.3.2 <code>revert.rc</code>	149
23.4 Rebase: <code>git rebase</code>	150
23.4.1 Usage	150
23.4.2 <code>rebase.rc</code>	150
24 Advanced Networking	152
24.1 Other clients	152
24.1.1 <code>http://</code>	152
24.1.2 <code>ssh://</code>	152
24.2 Other servers	153
24.2.1 <code>http://</code>	153
25 Advanced Topics	154
25.1 Bytes versus ASCII versus UTF-8	154
25.2 Reliability and signals	154
25.3 Optimizations	154
25.3.1 Object cache	154
25.4 Fast import and export	155
25.5 Alternates	155
25.6 Other repository format	155
25.6.1 Bare repository	155
25.6.2 <code>.git</code> file	155
25.7 Security	155
26 Conclusion	156
A Debugging	157
A.1 Format dumpers	157
A.2 <code>git/fs -d</code>	158
A.3 <code>git/xxx -d</code>	158
B Error Management	160
C Utilities	161
C.1 Error malloc	161
C.2 String utilities	163
D Extra Code	165
D.1 <code>git9/</code>	165
D.1.1 <code>git9/common.rc</code>	165
D.1.2 <code>git9/add.rc</code>	165
D.1.3 <code>git9/branch.rc</code>	165

D.1.4	git9/clone.rc	165
D.1.5	git9/commit.rc	165
D.1.6	git9/compat.rc	165
D.1.7	git9/diff.rc	170
D.1.8	git9/export.rc	170
D.1.9	git9/import.rc	170
D.1.10	git9/hist.rc	170
D.1.11	git9/init.rc	170
D.1.12	git9/merge.rc	170
D.1.13	git9/pull.rc	170
D.1.14	git9/push.rc	170
D.1.15	git9/rebase.rc	170
D.1.16	git9/revert.rc	170
D.1.17	git9/rm.rc	170
D.1.18	git9/conf.c	170
D.1.19	git9/delta.c	170
D.1.20	git9/fs.c	174
D.1.21	git9/get.c	176
D.1.22	git9/git.h	185
D.1.23	git9/log.c	189
D.1.24	git9/objset.c	192
D.1.25	git9/ols.c	192
D.1.26	git9/pack.c	195
D.1.27	git9/proto.c	218
D.1.28	git9/query.c	227
D.1.29	git9/ref.c	230
D.1.30	git9/repack.c	233
D.1.31	git9/save.c	234
D.1.32	git9/send.c	240
D.1.33	git9/serve.c	245
D.1.34	git9/util.c	255
D.1.35	git9/walk.c	259
D.2	diff/	266
D.2.1	diff/diff.c	266
D.2.2	diff/diff.h	267
D.2.3	diff/diffdir.c	268
D.2.4	diff/diffio.c	271
D.2.5	diff/diffreg.c	278
D.2.6	diff/merge3.c	286
D.2.7	diff/util.c	290
D.3	misc/	291
D.3.1	misc/patch.c	291

Glossary 306

Index 307

References 307

Chapter 1

Introduction

The goal of this book is to explain with full details the source code of a version control system.

1.1 Motivations

Why a version control system (VCS)? Because I think you are a better programmer if you fully understand how things work under the hood, and a VCS is one of the tools a programmer uses the most. Indeed, programmers use VCSs—which manage efficiently changes to a file or set of files—to manage the vital product of their labour: source code.

VCSs allow to store and retrieve past versions of a file, and to record who made each change, when, where, and why. VCSs are mostly used by programmers but you can use a VCS to manage any text-based documents (e.g., the source of this book), configuration files, and even binaries.

For projects with a single developer, a VCS is useful to keep track of changes: it allows the programmer to easily go back to past versions if he messed things up. For projects with multiple developers, a VCS is almost mandatory: it allows programmers to collaborate with each other and even to work on and modify the same files concurrently.

Here are a few questions I hope this book will answer:

- What are the fundamental concepts of a VCS? What are the core algorithms behind a VCS?
- How does a VCS store efficiently multiple versions of a file?
- How does a VCS represent changes to a file, or changes to a set of files? How does a VCS represent the addition, deletion, or renaming of a file?
- How does a VCS allow multiple users to work on and modify the same files at the same time? How can a VCS help coordinate the actions of multiple users?
- How does a VCS support parallel developments in a project? How can some developers work on the main release, some developers on experimental features, and others on fixing bugs on previously shipped releases all at the same time? What is a branch? How does a VCS reconcile multiple branches?
- What is a merge algorithm? When is a merge safe? What is a merge conflict? How does a VCS resolve conflicts?

1.2 The version control system Git (and git9)

There are many open source VCSs (Subversion, Mercurial, etc.) with different user interfaces, storage strategies, concurrency models, or features. I will present a few of those VCSs in Section 1.3. For this book, I decided

to base the presentation on Git¹ because it is one of the most popular VCSs in the open source community. Moreover, when coupled with the hosting website GitHub², Git makes it really easy for people to collaborate with each other.

However, Git is a rather large project with more than 200 000 LOC. It is impossible to present all this code in a book of a reasonable size. There are a few clones of Git written in higher-level languages than C, for example, Dulwich³ written in Python with only 16 000 LOC. I could have based this book on Dulwich, but this would introduce another language in the Principia Softwarica book series in addition to C. This would also in turn require to present the code of the Python interpreter, which contains unfortunately more than 170 000 lines of C code (almost the size of all the Plan 9 programs together).

Fortunately, in 2021 Ori Bernstein decided to rewrite Git for Plan 9 and redesigned the program to use far less code. This is why I will explain in this book the code of `git9`⁴, which contains about 10 000 lines of code (LOC); most of the code is written in C but about 1000 LOC is written in `rc`, the Plan 9 shell (see SHELL book [Pad18]).

Like for most books in Principia Softwarica, I chose a Plan 9 program because those programs are simple, small, elegant, open source, and they form together a coherent set. Note that `git9` was not part of the original Plan 9 but `git9` follows the Plan 9 philosophy and coding style.

1.3 Other version control systems

Here are a few VCSs that I considered for this book, but which I ultimately discarded:

- The Revision Control System (RCS) [Tic85]⁵ was the first open source VCS. Like its predecessor, the Source Code Control System (SCCS) [Roc75]⁶, RCS uses *deltas* to store efficiently the past versions of a file (Section 19.3 will present the Diff algorithm to compute those deltas). However, RCS improves over SCCS by using *reverse-deltas* to enable the user to also retrieve quickly the last version of a file (a recurring operation).

RCS is still a reasonable choice to manage a small project with a single developer or a small set of developers who can share access to a common directory (for example, by using NFS and symbolic links). However, the use of *locks* in RCS to forbid multiple developers to modify the same file at the same time and its focus on individual files make RCS inadequate for large projects with many independent developers. Moreover, even if RCS is very limited compared to modern VCSs like Git, RCS still contains more than 17 500 LOC.

- The Concurrent Versions System CVS [Gru86]⁷ was the most popular VCS for over a decade before distributed VCSs (e.g., Git) took over. CVS introduced a few important innovations. First, CVS was designed to operate on a set of files at once⁸, with files possibly organized in a tree. With CVS, you can group together related changes to a set of files and you can easily retrieve past versions of a whole project. Secondly, CVS allows multiple users to work on and modify the same files *concurrently* (hence the 'C' in CVS). CVS relies on the `merge` program instead of locks (Section 19.4 will present the Diff3 algorithm which underlies the `merge` program). Finally, CVS introduced later in 1995 a *client/server* architecture where a *repository* could be stored on a remote machine. When coupled with the free hosting site Sourceforge⁹, CVS allowed developers to easily start and collaborate on new projects.

¹<https://git-scm.com/>

²<https://github.com>

³<https://www.dulwich.io/>

⁴<https://github.com/oridb/git9>

⁵<https://www.gnu.org/software/rcs/>

⁶<http://sccs.sourceforge.net/>

⁷<http://www.nongnu.org/cvs/>

⁸RCS can operate on multiple files by using shell wildcards, but it was not designed for it.

⁹<https://sourceforge.net>

CVS started as a few shell scripts using RCS as a backend. However, today CVS is a very large C program with more than 80 000 LOC. The backend file `src/rcs.s` contains already 9000 LOC.

- Git [CS14]¹⁰ was originally created by Linus Torvalds in 2005 to manage the source code of the Linux kernel. It followed a series of *distributed VCSs* (e.g., TeamWare, BitKeeper, Arch, Darcs, Monotone) designed to overcome the main limitations of CVS (a centralized VCS). Because a distributed VCS (DVCS) does not require access to a central repository, it allows developers to work independently offline. A DVCS makes it also easy and cheap to create *branches* representing parallel developments, and to reconcile later those branches (see Section 2.1.5 for more discussions on the advantages of distributed VCSs over centralized VCSs).

As I explained in Section 1.2, Git is a large program with more than 200 000 LOC spread over more than 400 files (not including the tests, GUIs, and extra contributions). The first version of Git was small and contained only 1000 LOC¹¹. However, it contained only a few low-level commands that were not easy to use. This forced programmers to develop and use the extra program *cogito*¹², which provided an extra layer of higher-level commands (called *porcelain*), but added many more lines of code.

- Mercurial [O’S09]¹³ is another popular DVCS started also in 2005 by another Linux kernel programmer: Matt Mackal. It is mostly written in Python and relies only on a few C files for critical operations. Its code is arguably easier to understand than Git but it still contains more than 100 000 LOC spread over more than 170 files (not including the tests, extensions, and extra contributions).

Figure 1.1 presents a timeline of major VCSs. `git9` is not as efficient and complete as Git or Mercurial, but it provides almost all of the essential features of those programs with more than an order of magnitude less code.

1.4 Getting started

To play with `git9`, you will first need to install the Plan 9 fork used in Principia Softwarica (see <https://www.principia-softwarica.org/getting-started.html>). Once installed, you can test `git9` under Plan 9 with the following commands:

```
1 $ cd /tests/
2 $ mkdir hello
3 $ cd hello/
4 $ git/init
5 ...
6 $ echo "Hello Git" > hello.txt
7 $ git/add hello.txt
8 $ git/commit -m "first commit" .
9 heads/master: 56adf1865
```

The command in Line 4 initializes a new *repository* by creating the appropriate metadata in the `/tests/hello/.git` subdirectory. Line 7 then adds the new file `hello.txt` to the *staging area* of Git (I will explain later in Section 2.3.3 what is a staging area). Finally, Line 8 *commits* what was staged to the repository and records this commit with the message “first commit”.

¹⁰<https://git-scm.com>

¹¹<https://github.com/git/git/commit/e83c5163316f89bfbde7d9ab23ca2e25604af290>

¹²<http://git.or.cz/cogito/>

¹³<https://www.mercurial-scm.org/>

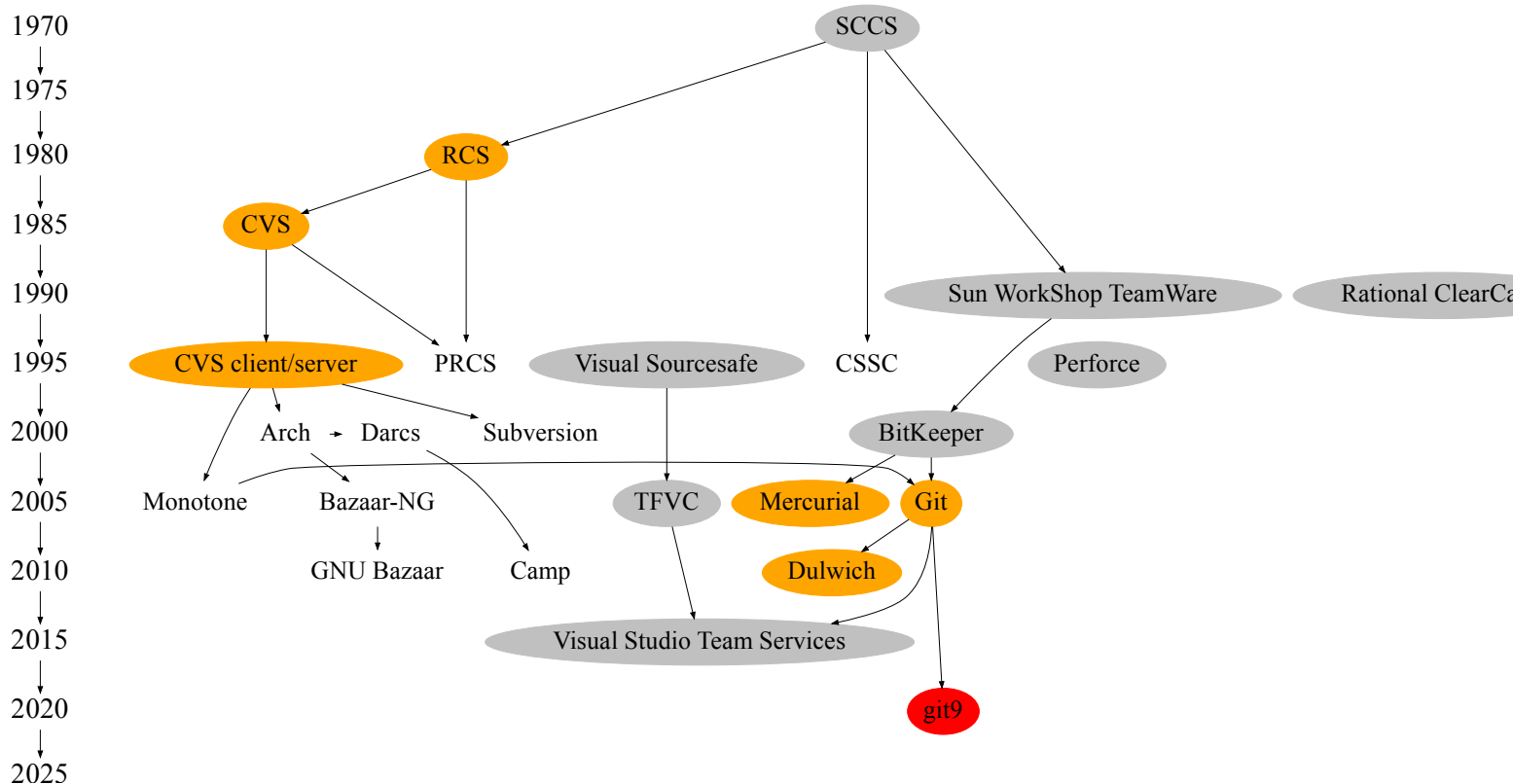


Figure 1.1: Version control systems timeline

1.5 Requirements

Because most of this book is made of C source code, you will need a good knowledge of the C programming language [KR88] to understand it. A significant part of `git9` is made of `rc` shell scripts so you will need to read the SHELL book [Pad18]. Moreover, those scripts are often using utilities such as `awk` or `sed` so reading the UTILITIES book [Pad25] would be useful.

You do not need to know Git, or more generally any VCSs, to understand this book. However, if while reading this book you have specific questions on the interface of Git, I suggest you to consult the man pages of Git¹⁴, or any of the books on Git (e.g., [CS14]).

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Ori Bernstein, so those parts are copyright by him. The prose is mine and is licensed under the All Rights Reserved License.

¹⁴<https://git-scm.com/docs>

1.8 Acknowledgments

I would like to acknowledge first the author of `git`⁹, Ori Bernstein, who wrote in some sense most of this book.

I would like also to thank Linus Torvalds, the original author of Git, for designing this simple but powerful VCS. Many of his design decisions seem obvious in retrospect but they are not: most VCSs chose different designs (for the storage, file permissions, for the way to handle renames, etc.) which in the end are more complicated and usually less efficient than in Git.

Chapter 2

Overview

Before showing the source code of `git9` in the following chapters, I first give an overview in this chapter of the general principles of a VCS. I also quickly describe the command-line interface of `git` (and `git9`), the format of a Git repository, and I use a simple terminal session containing `git` commands to illustrate its major features. Finally, I define terms, explain how the code is organized, and more generally give the background necessary to understand the code I will show later.

2.1 Version control system principles

The main requirements for a modern VCS are to be able to store past versions of a set of files, to track the changes to those files, and to help multiple developers to collaborate with each other on those files by supporting concurrent and parallel development (I will explain soon the differences between those two kinds of development). The next sections will each give more details about those requirements. I will also try to present the major techniques used by popular VCSs to satisfy those requirements. Finally, I will discuss two different architectures for VCSs: centralized and distributed.

2.1.1 Storing past versions

The first requirement of a VCS is to store past versions of a file, hence the 'V' in VCS. However, there is also an 'S' in VCS, which means it must be done in a systematic way. Indeed, one way to understand the usefulness of a VCS is to see how people are struggling when they are managing documents that evolve (e.g., source code, $\text{\LaTeX}$ files, notes) without a VCS. In that situation, people often use `cp` to *backup* files, but it is inefficient space-wise and error-prone.

The repository

Working copy

Delta

Compression

2.1.2 Tracking changes

The commit

Version identifier

Change granularity

diff

patch

2.1.3 Concurrent development

Locks

Merging

Merge conflicts

2.1.4 Parallel development

Branch

Merging

A direct acyclic graph of versions

2.1.5 Centralized versus distributed

2.2 git command-line interface

I just described the general principles of a VCS, and illustrated some of those principles with examples from RCS, CVS, or Git. I will now focus exclusively on Git and give more details about the interface of its command-line program: `git`.

The command-line interface of `git` is pretty simple:

```
1 $ git
2 usage: git <init|add|rm|commit|branch|checkout|reset|...> [options]
3 $ git init
4 Initialized empty Git repository in /tests/hello/.git/
5 $ git add hello.txt
6 $ git commit -m "first commit" --author "pad <todo@todo>"
```

`git` takes first a *command* as its first argument and then options and extra arguments specific to this command. For example, the command `init` at Line 3 does not need any extra arguments, but `add` at Line 5 requires at least the name of a file or directory, and `commit` at Line 6 usually requires command-line flags. I will gradually describe the different `git` commands, their options, and implementations in the following chapters.

Note that in most of this chapter I will use `git` and not `git9` commands to be more general. The reader can thus easily test those commands on Linux or macOS by using the regular `git` program. It is easy to adapt

those commands to `git9` on Plan 9. The main difference is that `git9` accepts commands separated by a `/` (as in `git/init` or `git/add foo.txt`) instead of space.

2.3 Git concepts and data structures

To understand how Git is implemented, and to some degree to learn how to use Git effectively, you need to know the few concepts underlying Git. Those concepts are the *object store*, the *reference*, and the *staging area*. They correspond also to `git9`'s main data structures, which I will describe fully in Chapter 3. The following sections will just give an overview of those data structures.

2.3.1 Object store

To really understand Git, you need to realize that at its core, Git is a simple *content-addressable storage system*. Given the *hash* of a content (Git uses SHA1 hashes, as explained below), Git can retrieve back quickly the full corresponding content. Git internally is simply a *persistent hash table*, also known as a *key/value store*.

In this store, Git manipulates mainly three kinds of values, known as *objects* in Git terminology (hence the term *object store*):

- A *blob*: to represent the content of a file at a specific time.
- A *tree*: to assign names to blobs or other subtrees
- A *commit*: to associate to a specific toplevel tree a message, an author, a date, and zero or more *parent* commits

You will see another kind of objects in Section 23.1 with the *tag*. In addition to those objects, Git maintains also *references* to specific commits and an *index* of specific blobs, as explained in the next sections.

Regarding the keys of the store, Git uses the SHA1 algorithm [sha93, EJ01] to compute the hash of an object. This algorithm associates an almost unique number of 160 bits (which amounts to 20 bytes, or 40 digits in hexadecimal format) to any content of arbitrary length. Section 19.1 presents the code of this algorithm and gives more information on SHA1. For example, given the content `Hello Git\n`, the SHA1 algorithm will return the number `9f4d96d5b00d98959ea9960f069585ce42b1349a` in hexadecimal format. Here the hashing is not very interesting because the hash is bigger than the original content, but in practice most files under a VCS are far bigger than 20 bytes.

SHA1 is a complex cryptographic hash function. It is outside the scope of this book to explain how and why it works, but what is important for Git is that SHA1 is an almost perfect hash function: given two different content, there is an almost zero probability that SHA1 will return the same hash number. Such an event is called a *collision*, and in practice it should never happen. Thanks to this almost perfect hash function, Git can identify any content of any length with just a 20 bytes number. In Git, the SHA1 of a file is similar to the *inode* of a file in a filesystem (see the KERNEL book [Pad14] for more information on filesystems and inodes)¹, except it references a specific version of a file. In the tree object, which associates names to blobs, Git uses the SHA1 of a blob to identify this blob. In the same way, in the commit object, Git uses the SHA1 of a tree to identify the toplevel tree a commit refers to.

Figure 2.1 presents the Git objects (and their relationships) of the Git repository created in Section 1.4 after the addition of another commit. The first commit, which added `hello.txt`, is at the top left of Figure 2.1. The second commit, below, added the two files `foo.txt` and `bar.txt` under a directory `dir1/`, and also modified `hello.txt`. I will describe soon in Section 2.4 how Git stores those objects on the disk.

Commit objects are linked together, as shown in Figure 2.1 with the second commit linked to the first commit through the `parents` field of the object. Those links represent the history of the repository, also known as its

¹The author of Git, Linus Torvalds, is also the author of Linux, which explains why he chose a design inspired by filesystems.

```

commit 19d977...-----+
|tree:                |   tree 2f092e...-----+       blob 9f4d96...+
| 2f092e... -----+-->| hello.txt 9f4d96.---+----->|Hello Git    | | |
|parents:             |   +-----+             |             |
| None                |   |             |             |
|Author: pad          |   +-----+             |             |
| <todo@todo>         |   |             |             |
|Date: Fri Sep 29     |   blob 557db0...+       |             |
| 14:04:46 2017 -0700|   +-->|Hello World  |   |             |
|Message:             |   |             |             |
| first commit        |   ++>tree 0c4b27...-----+ |             |
+-----+             |   ++ dir1 7e8bb2...    | | +-----+
                ^    | || hello.txt 557db0...++
                |    | |+-----+
commit 5d9dfe...---+---+| |
|tree:                | | |             blob 452a53...+
| 0c4b27...-----+---++ |             |baaaaaaaaaa | | | |
|parents:             | | |             +>|aaaaaaaaaa |
| 19d977-----+       | | |             | |aaaaaaaaar |
|Author: pad          | | | tree 7e8bb2...-----+ | +-----+
| <todo@todo>         | | | ++>| bar.txt 452a53.---+---+
|Date: Fri Sep 29     | | | | foo.txt 96ea4a.---+---+ blob 96ea4a...+
| 14:26:23 2017 -0700 | | | +-----+ | | foooooooooo |
|Message:             | | | +>| oooooooooo |
| second commit, subdir| | | | oooooooooo |
+-----+             | | | +-----+

```

Figure 2.1: Git objects relationships for hello/.git after two commits.

log. A commit can have multiple parents in the case of a *merge* (I will explain in Section 14.1 how to merge branches in Git). Thus, the log forms a *graph*, or more specifically a *direct acyclic graph* (DAG). Figure 2.2 shows such a graph for an additional series of commits following the first two commits of Figure 2.1. In this example, a developer *forked* first an experimental branch called **branch1** (I will explain briefly how to create a branch in Section 2.5.4 and fully in Section 13.3) following the second commit of Figure 2.1. At the same time, development on the main branch, called the *master branch* in Git terminology, continued and saw two commits: **commit4** and **commit6**. The experimental branch got *merged* later at **commit7** (I will explain in Section 14.1 how to merge branches in Git). This is why this commit has two parents. Later on, a developer created another branch called **experiment** that remained active and did not get merged yet; it follows in parallel the development of the master branch.

2.3.2 References

As I mentioned briefly in the previous section, Git maintains, in addition to the object store, *references* to specific commit objects. Those references have a *name* and a *content*. For example, in Figure 2.2, the reference named **refs/heads/experiment** points to the last commit of the **experiment** branch, and its content is the SHA1 of the **commit9** object. I will explain in Section 2.4 how Git stores those references on the disk.

Git maintains also a special reference called the *HEAD* which points to the last commit of the *current branch* (see the bottom of Figure 2.2). Its content is usually not a SHA1 but the name of another reference. By default, *HEAD* points to the master branch, as shown at the bottom of Figure 2.2, and so its content is the string **ref: refs/heads/master** (and **refs/heads/master** contains the SHA1 of the last commit of the master branch). When you switch branch (I will explain how to switch branch briefly in Section 2.5.4 and fully in Section 13.5), the content of the *HEAD* will change.

A reference pointing to the last commit of a branch is called a *head* in Git terminology and starts with **refs/heads/**. In Figure 2.2, those heads are **refs/heads/experiment** and **refs/heads/master**.

Git maintains a final set of special references called the *remotes*, for example **refs/remotes/origin/master** pointing to **commit8** in Figure 2.2. Those references are used when people are collaborating with each other and I will explain them in Chapter 17.

2.3.3 Staging area

The last important concept of Git is the *staging area*. Git operates mainly on three different areas, as shown at the top of Figure 2.3:

1. The *working copy* contains the current state of the files
2. The *repository* contains all past versions of all the files including the state of the files at the last commit
3. The *staging area* contains what the user wants to commit next

After you modify a set of files in your working copy, for example, **bar.txt**, **foo.txt**, and **misc.txt** at the right of Figure 2.3, you need to indicate which of those files and modifications should be part of the next commit. To do so, you need to *mark* those files by using the command **git add** (or **git rm** if you want to mark for deletion a file).

Once you marked all the relevant files, you can use the command **git commit** to commit the modifications to the repository. For example, in Figure 2.3, only the modifications to **foo.txt** and the deletion of **misc.txt** will be part of the next commit, and not the modifications to **bar.txt**.

Once you add a file in a repository, the file is said to be *tracked* in Git terminology. You can sometimes avoid manipulating explicitly the staging area by using instead the command **git commit -a** to automatically commit all the modifications to the tracked files. However, the staging area and **git add** give more flexibility

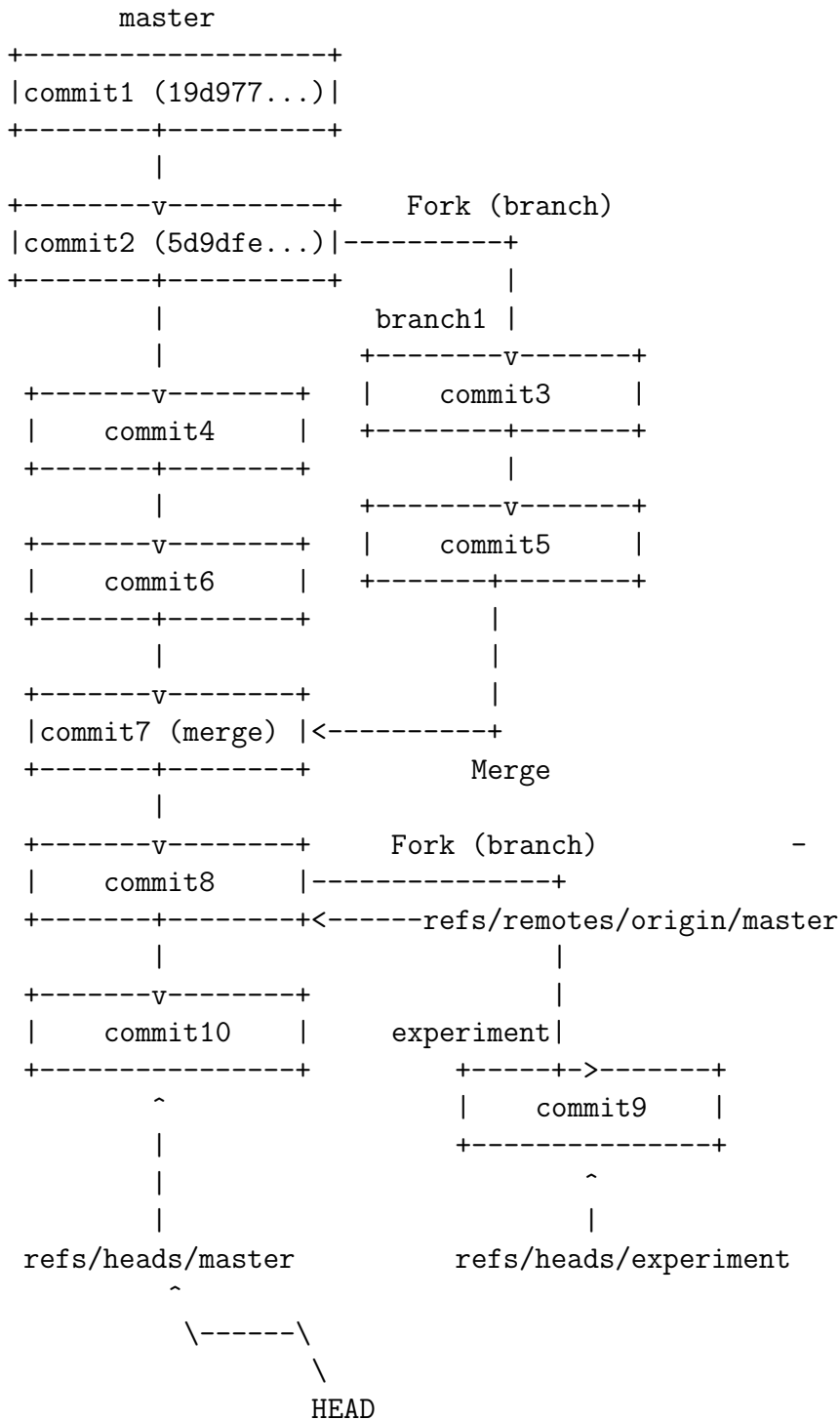


Figure 2.2: Commit graph and references.

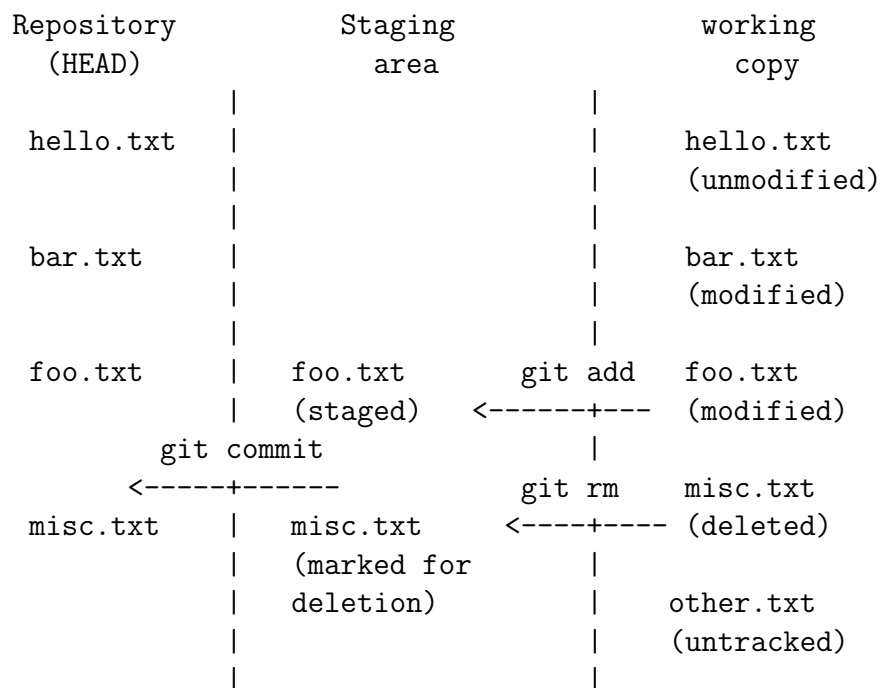


Figure 2.3: Git areas.

to the user by allowing to split a set of modifications in multiple commits. For example, in Figure 2.3, the modification to `bar.txt` can be put in another commit.

Internally, Git uses a data structure called the *index* (also known as the *directory cache*) to manage the staging area. I will explain later in Section 3.5 why this data structure is called an index.

2.4 Repository format: `.git/`

Now that you know the main concepts and data structures underlying Git, it will be easy to understand the format of the `.git/` directory. Here is the slightly edited output of the UNIX command `tree`, which recursively explores a directory and displays its content as a tree². The command is applied here to the `/tests/hello/.git/` directory after the last command in Section 1.4, when the repository contained just one commit:

```
1 $ pwd
/tests/hello
2 $ tree -F .git/
.git/
|-- HEAD
...
|-- index
|-- objects/
|   |-- 2f/
|   |   '-- 092e9cadfc1eb4a6d2febfddb941f4c1fe6fd6
|   |-- 19/
|   |   '-- d977a48d1d7b7ae8d520dd66190702dd4fb5bc
|   |-- 9f/
```

²See <http://mama.indstate.edu/users/ice/tree/> for more information about the `tree` program. The `-F` option used in the terminal session above is to add the special mark `'/'` at the end of directory names, to help differentiate regular files from subdirectories.

```
|   |   '-- 4d96d5b00d98959ea9960f069585ce42b1349a
.....
'-- refs/
    |-- heads/
    |   '-- master
.....
```

The objects of the key/value store are simply stored in separate files under `.git/objects/`. The hexadecimal digits of the SHA1 of the object are split in two parts: the first two digits are used for the directory name containing the object and the remaining digits for the filename of the object. For example, the file `.git/objects/9f/4d96d5b00d98959ea9960f069585ce42b1349a` contains the blob of `Hello Git\n` (see the blob and its SHA1 at the top right of Figure 2.1).

Git uses the first two digits of the SHA1 to classify objects in separate folders to avoid having too many files in the same directory. Indeed, this would slow down filesystem operations such as opening a file referenced by a specific SHA1 key, a recurring operation under Git (see the *KERNEL* book [Pad14] for more information on the performance of filesystem operations).

The references are also stored simply in separate files. Git uses the name of the reference as the path to a file under `.git/refs/` (e.g., `.git/refs/heads/master` in the example above).

Finally, the index and the HEAD are stored directly under `.git/` in separate files.

2.5 hello/.git/

In this section, I will give a short tutorial on how to use Git. I will explain the main commands of `git` by using a terminal session starting from an empty `hello/` directory. I will continue the series of commands I introduced in Section 1.4. I will also describe the semantics of those commands by showing their effects on the Git metadata stored under the `hello/.git/` subdirectory (by using mainly the output of the `tree` command, as in the previous section). This will hopefully help you to understand the code of those commands I will present later in the following chapters.

I will again use `git` instead of `git9` in the terminal sessions but the examples are easy to transpose to Plan 9.

2.5.1 Creating a repository

Here are the commands to create a fresh new repository:

```
1 $ cd /tests/
2 $ mkdir hello
3 $ cd hello/
4 $ git init
Initialized empty Git repository in /tests/hello/.git/
5 $ tree -F .git/
.git/
|-- HEAD
...
|-- objects/
'-- refs/
    |-- heads/
6 $ cat .git/HEAD
ref: refs/heads/master
```


The `init` command creates just the directory structure under `.git/`, without any objects or references in it (the `.git/objects/` and `.git/refs/heads` are empty directories above). There is also not yet an index file. There is a `HEAD` file, but its contents, shown with command 6 above references an head that does not exist yet under `.git/refs/heads/`.

2.5.2 Staging a diff

Here is the command to add a new file (or a new version of a file) to the repository:

```
1 $ echo "Hello Git" > hello.txt
2 $ git add hello.txt
3 $ tree -F .git/
.git/
|-- HEAD
...
|-- index
...
|-- objects/
|   |-- 9f/
|   |   '-- 4d96d5b00d98959ea9960f069585ce42b1349a
...
'-- refs/
    |-- heads/
4 $ cat .git/objects/9f/4d96d5b00d98959ea9960f069585ce42b1349a
x^K\312\3110R04'\360H\315\311\311Wp\317,\341^B^@4\201^Ec
```

Adding the first file in a repository has two effects:

1. The creation of a new blob object (here in `.git/objects/9f/4d96...`) containing essentially the compressed form of the content of the added file (hence the cryptic output of the `cat` command above). I will fully explain in Chapter 4 the format of a blob on the disk.
2. The creation of the index file. Again, I will describe precisely the format of the index file in Chapter 4, but the index essentially associates to every tracked filenames by the repository the SHA1 of the blob *currently* corresponding to the filename. In the example above, the index will associate to the filename `hello.txt` the SHA1 `9f4d96...`

You must use the command `add` to add a new file to the repository, when the file was not yet tracked by the repository. However, you must also use the command `add` to stage the modifications on a tracked file as in Figure 2.3. In that case, the `add` command also creates a new blob with the current content of the file. It also updates the index so the filename of the added file now points to the SHA1 of the new blob.

Note that using `git add` on an unmodified file will have no effect. Indeed, Git will compute the SHA1 of the current content, which will be identical to the SHA1 of an existing blob. Updating the index will have also no effect because the filename will reference the same SHA1.

2.5.3 Committing a diff

Here is the command to commit to the repository what was previously staged:

```

1 $ git commit -m "first commit" --author "pad <todo@todo>"
2 $ tree -F .git/
.git/
|-- HEAD
|-- index
|-- objects/
|   |-- 2f/
|   |   '-- 092e9cadfc1eb4a6d2febfddb941f4c1fe6fd6
|   |-- 19/
|   |   '-- d977a48d1d7b7ae8d520dd66190702dd4fb5bc
|   |-- 9f/
|   |   '-- 4d96d5b00d98959ea9960f069585ce42b1349a
'-- refs/
    |-- heads/
    |   '-- master
3 $ cat .git/refs/heads/master
19d977a48d1d7b7ae8d520dd66190702dd4fb5bc

```

The `commit` command has three effects:

1. The creation of a new tree object, in the example above `.git/objects/2f/092e...` (see Figure 2.1 for the content of this tree). This tree derives from the content of the index. Because the repository contains only one directory (the toplevel root containing just `hello.txt`), `git commit` will create just one tree object, but in general when a project has many directories `git commit` may create many new tree objects.
2. The creation of a new commit object, here `.git/objects/19/d977...`, referencing the newly created tree object (again, see Figure 2.1 for the content of this commit and its relationship to the tree object).
3. The update of the content of the head of the current branch (`refs/heads/master` according to the content of `.git/HEAD`) to contain the SHA1 of the newly created commit object (here `19d977...`), as shown by command 3 above.

Note that it can be tedious to specify each time on the command-line your name and email through the `--author` flag of `git commit`. Git can also leverage a configuration file containing such information in a `.gitconfig` file in your home directory as explained in Section 6.1.

2.5.4 Managing branches

Here is the command to create a new branch:

```

1 $ git branch experiment
2 $ tree -F .git/
.git/
|-- HEAD
...
'-- refs/
    |-- heads/
    |   |-- experiment
    |   '-- master
...
3 $ cat .git/HEAD

```

```

ref: refs/heads/master
4 $ cat .git/refs/heads/master
19d977a48d1d7b7ae8d520dd66190702dd4fb5bc
5 $ cat .git/refs/heads/experiment
19d977a48d1d7b7ae8d520dd66190702dd4fb5bc

```

As you can see, creating a new branch is an extremely cheap operation under Git. It consists just in adding a file under `.git/refs/heads/` with the name of the new branch (here `.git/refs/heads/experiment`), and the SHA1 commit of the current branch for its content (here `19d977...`). However, creating a branch does not switch to this branch as shown by command 3 above.

Here is the command to switch to the new branch:

```

6 $ git checkout experiment
Switched to branch 'experiment'
7 $ cat .git/HEAD
ref: refs/heads/experiment

```

Switching to a branch is usually a more costly operation. `git checkout` modifies `HEAD` (as shown by command 7 above), which is fast, but it also recomputes the index from the tree object that is referenced in the commit of the head of the new branch, as well as from all its subtrees (see Section 13.5.1 for the full description of this operation). It also updates all the files so that they contain the content of the blobs referenced in the index. The more the new branch differs from the current branch, the more costly switching to this new branch will be.

With `git9` you will need to use `git/branch -n experiment` to create a new branch and it will also automatically switch to this branch. There is no `git/checkout`; everything is done with `git/branch`.

2.5.5 Inspecting objects

Git provides a few commands to query the repository. Those commands are useful to understand the internal structures of Git. They are also useful for doing some archeology on the history of a project, as explained in Section ???. The first of those commands, `show`, shows the content of a Git object. `git show` takes the SHA1 of an object as a parameter (in hexadecimal format) and pretty prints its content in a readable format. Here are a few examples of this command:

```

1 $ tree -F .git/
...
|-- objects/
|   |-- 2f/
|   |   '-- 092e9cadfc1eb4a6d2febfddb941f4c1fe6fd6
|   |-- 19/
|   |   '-- d977a48d1d7b7ae8d520dd66190702dd4fb5bc
|   |-- 9f/
|   |   '-- 4d96d5b00d98959ea9960f069585ce42b1349a
...
2 $ git show 9f4d96d5b00d98959ea9960f069585ce42b1349a
Hello Git
3 $ git show 2f092e9cadfc1eb4a6d2febfddb941f4c1fe6fd6
tree 2f092e9cadfc1eb4a6d2febfddb941f4c1fe6fd6

hello.txt

```

```
4 $ git show 19d977a48d1d7b7ae8d520dd66190702dd4fb5bc
commit 19d977a48d1d7b7ae8d520dd66190702dd4fb5bc
Author: pad <todo@todo>
Date:   Fri Sep 29 14:04:46 2017 -0700
```

```
    first commit
```

```
diff --git a/hello.txt b/hello.txt
new file mode 100644
index 0000000..9f4d96d
--- /dev/null
+++ b/hello.txt
@@ -0,0 +1 @@
+Hello Git
```

The command `show` mainly opens the appropriate file under `.git/objects/`, decompresses the file, and finally displays its content.

Another important query command is `log` to see the full history of a repository. Here is the output of this command on the repository we used this far:

```
1 $ git log
commit 19d977a48d1d7b7ae8d520dd66190702dd4fb5bc
Author: pad <todo@todo>
Date:   Fri Sep 29 14:04:46 2017 -0700
```

```
    first commit
```

The command `log` mainly opens `.git/HEAD` to get a reference to the last commit. Then, it goes through the parent links of the commit object to recursively explore all the linked commit objects (stored again under `.git/objects/`).

2.5.6 Other commands

There are a few other important Git commands: `git merge`, `git status`, `git diff`, `git clone`, `git pull`, or `git push`, but I will introduce them later in this document.

2.6 Code organization

2.7 Software architecture

2.8 Book structure

Part I

Foundations

Chapter 3

Core Data Structures

3.1 Secure Hash Algorithm (SHA1) hashes

3.1.1 Binary hashes

```
⟨struct Hash 29a⟩≡ (185f)
    struct Hash {
        byte h[20];
    };
```

```
⟨global Zhash 29b⟩≡ (258)
    Hash    Zhash;
```

```
⟨function hasheq 29c⟩≡ (258)
    int
    hasheq(Hash *a, Hash *b)
    {
        return memcmp(a->h, b->h, sizeof(a->h)) == 0;
    }
```

```
⟨macro GETBE32 29d⟩≡ (185f)
    #define GETBE32(b)\
        (((b)[0] & 0xFFu1) << 24) | \
        (((b)[1] & 0xFFu1) << 16) | \
        (((b)[2] & 0xFFu1) << 8) | \
        (((b)[3] & 0xFFu1) << 0)
```

3.1.2 Hexadecimal hashes

```
⟨pragmas varargck 29e⟩≡ (185f) 157a▷
    #pragma varargck type "H" Hash
```

```
⟨gitinit() fmtinstall calls 29f⟩≡ (51f) 157b▷
    fmtinstall('H', Hfmt);
```

```
⟨function Hfmt 29g⟩≡ (258)
    int
    Hfmt(Fmt *fmt)
    {
        Hash h;
        int i, n, l;
        char c0, c1;

        l = 0;
```

```

h = va_arg(fmt->args, Hash);
for(i = 0; i < sizeof h.h; i++){
    n = (h.h[i] >> 4) & 0xf;
    c0 = (n >= 10) ? n-10 + 'a' : n + '0';
    n = h.h[i] & 0xf;
    c1 = (n >= 10) ? n-10 + 'a' : n + '0';
    l += fprintf(fmt, "%c%c", c0, c1);
}
return l;
}

```

3.1.3 Conversion functions

<function hparse 30a>≡ (258)

```

errorneg1
hparse(Hash *h, char *b)
{
    int i, c0, c1;

    for(i = 0; i < nelem(h->h); i++){
        if((c0 = charval(b[2*i+0])) == ERROR_NEG1)
            return ERROR_NEG1;
        if((c1 = charval(b[2*i+1])) == ERROR_NEG1)
            return ERROR_NEG1;
        h->h[i] = (c0 << 4) | c1;
    }
    return OK_0;
}

```

<function charval 30b>≡ (258)

```

int
charval(int c)
{
    if(c >= '0' && c <= '9')
        return c - '0';
    if(c >= 'a' && c <= 'f')
        return c - 'a' + 10;
    if(c >= 'A' && c <= 'F')
        return c - 'A' + 10;
    // else
    werrstr("invalid hex char");
    return ERROR_NEG1;
}

```

3.2 Object

<struct Object 30c>≡ (185f)

```

struct Object {
    /* Git data */
    Hash    hash;
    // enum<Gxxx>
    int type;

    <Object other fields 32b>
    <Object cache fields 31e>
    <Object indexing fields 31a>
    <Object extra fields 31b>
}

```

```

/* Everything below here gets cleared */

// raw object content after decompression
// option<ref_own<string>>, set after Cloaded
char    *all;
// ref_shared<string> point in Object.all after object header (used for GBlob too)
char    *data;
/* size excludes header */
vlong   size;

// those fields are set after Cparsed
/* Significant win on memory use */
union {
    <Object union fields 33b>
};
};

<Object indexing fields 31a>≡ (30c)
vlong   off;
vlong   len;
u32int  crc;

<Object extra fields 31b>≡ (30c)
Object  *next;
Object  *prev;

```

3.2.1 Object types: Gxxx

```

<enum Gxxx 31c>≡ (185f)
enum Gxxx {
    GNone    = 0,

    GCommit  = 1,
    GTree    = 2,
    GBlob     = 3,

    <Gxxx other cases 31d>
};

<Gxxx other cases 31d>≡ (31c) 148a▷
GDelta = 6,
GRdelta = 7,

```

3.2.2 Object flags: Cxxx

```

<Object cache fields 31e>≡ (30c) 154c▷
// enum<Cxxx>
int flag;

<enum Cxxx 31f>≡ (185f)
enum Cxxx {
    Cloaded = 1 << 0,
    Cparsed = 1 << 2,
    <Cxxx other cases 32a>
};

```



```

⟨Cxxx other cases 32a⟩≡ (31f)
    Cidx    = 1 << 3,
    Ccache  = 1 << 4,
    Cexist  = 1 << 5,
    Cthin   = 1 << 6,

```

3.2.3 Reference counting

```

⟨Object other fields 32b⟩≡ (30c)
    // shared objects
    int refs;

```

```

⟨function ref 32c⟩≡ (215b)
    Object*
    ref(Object *o)
    {
        o->refs++;
        return o;
    }

```

```

⟨function unref 32d⟩≡ (215b)
    void
    unref(Object *o)
    {
        if(!o)
            return;
        o->refs--;
        if(o->refs == 0)
            clear(o);
    }

```

```

⟨function clear 32e⟩≡ (215b)
    static void
    clear(Object *o)
    {
        if(!o)
            return;

        assert(o->refs == 0);
        ⟨clear() assert o->flag 32f⟩

        switch(o->type){
        ⟨clear() switch o->type cases 33e⟩
        default:
            break;
        }

        free(o->all);
        o->all = nil;
        o->data = nil;
        ⟨clear() reset o->flag 32g⟩
    }

```

```

⟨clear() assert o->flag 32f⟩≡ (32e) 33a▷
    assert(o->flag & Cloaded);

```

```

⟨clear() reset o->flag 32g⟩≡ (32e)
    o->flag &= ~(Cloaded|Cparsed);

```

```

⟨clear() assert o->flag 33a⟩+≡ (32e) <32f
    assert((o->flag & Ccache) == 0);

```

3.2.4 Blobs

3.2.5 Trees and Tinfo

```

⟨Object union fields 33b⟩≡ (30c) 34a▷
    // when GTree
    Tinfo *tree;

```

```

⟨struct Tinfo 33c⟩≡ (185f)
    struct Tinfo {
        /* Tree */
        // array<ref_own<Dirent>> len = nent
        Dirent *ent;
        int nent;
    };

```

```

⟨struct Dirent 33d⟩≡ (185f)
    struct Dirent {
        // ref_own<string>
        char *name;
        int mode;
        Hash h;
        ⟨Dirent other fields 146a⟩
    };

```

```

⟨clear() switch o->type cases 33e⟩≡ (32e) 34c▷
    case GTree:
        if(!o->tree)
            break;
        free(o->tree->ent);
        free(o->tree);
        o->tree = nil;
        break;

```

```

⟨function emptydir 33f⟩≡ (258)
    Object*
    emptydir(void)
    {
        static Object *e;

        if(e != nil)
            return ref(e);
        e = emalloc(sizeof(Object));
        e->type = GTree;
        e->tree = emalloc(sizeof(Tinfo));
        e->tree->ent = nil;
        e->tree->nent = 0;
        e->flag |= Cloaded|Cparsed;
        e->hash = Zhash;
        e->off = -1;
        ref(e);
        cache(e);
        return e;
    }

```

3.2.6 Commits and Cinfo

```
<Object union fields 34a>+≡ (30c) <33b
// when GCommit
Cinfo *commit;
```

```
<struct Cinfo 34b>≡ (185f)
struct Cinfo {
    /* Commit */
    // array<Hash> (0, 1, or 2 parents?)
    Hash *parent;
    int nparent;

    Hash tree;

    // ref_own<string>
    char *author;
    // option<ref_own<string>>
    char *committer;

    // ref<string> (len = nmsg), point in Object.all?
    char *msg;
    int nmsg;

    vlong ctime;
    vlong mtime;
};
```

```
<clear() switch o->type cases 34c>+≡ (32e) <33e
case GCommit:
    if(!o->commit)
        break;
    free(o->commit->parent);
    free(o->commit->author);
    free(o->commit->committer);
    free(o->commit);
    o->commit = nil;
    break;
```

```
<global zcommit(ref.c) 34d>≡ (232)
static Object zcommit = {
    .type=GCommit
};
```

3.3 Objects

3.3.1 Objset

```
<struct Objset 34e>≡ (185f)
struct Objset {
    // hash_array<Hash, ref<Object>>, len = nobj (alloc = sz)
    Object **obj;
    int nobj;
    int sz;
};
```

```

⟨function osinit 35a⟩≡ (192a)
void
osinit(Objset *s)
{
    s->sz = 16;
    s->nobj = 0;
    s->obj = eamalloc(s->sz, sizeof(Hash));
}

```

```

⟨function osclear 35b⟩≡ (192a)
void
osclear(Objset *s)
{
    free(s->obj);
}

```

Objset methods

```

⟨function osadd 35c⟩≡ (192a)
void
osadd(Objset *s, Object *o)
{
    u32int probe;
    ⟨osadd() other locals 35d⟩

    probe = GETBE32(o->hash.h) % s->sz;
    while(s->obj[probe]){
        if(hasheq(&s->obj[probe]->hash, &o->hash)){
            s->obj[probe] = o;
            return;
        }
        // else
        probe = (probe + 1) % s->sz;
    }
    assert(s->obj[probe] == nil);
    s->obj[probe] = o;
    s->nobj++;

    ⟨osadd() realloc if needed 35e⟩
}

```

```

⟨osadd() other locals 35d⟩≡ (35c)
Object **obj;
int i, sz;

```

```

⟨osadd() realloc if needed 35e⟩≡ (35c)
if(s->sz < 2*s->nobj){
    sz = s->sz;
    obj = s->obj;

    s->sz *= 2;
    s->nobj = 0;
    s->obj = eamalloc(s->sz, sizeof(Hash));
    for(i = 0; i < sz; i++)
        if(obj[i])
            // recurse!
            osadd(s, obj[i]);
    free(obj);
}

```

```

⟨function osfind 36a⟩≡ (192a)
Object*
osfind(Objset *s, Hash h)
{
    u32int probe;

    for(probe = GETBE32(h.h) % s->sz; s->obj[probe]; probe = (probe + 1) % s->sz)
        if(hasheq(&s->obj[probe]->hash, &h))
            return s->obj[probe];
    // else
    return nil;
}

```

```

⟨function oshas 36b⟩≡ (192a)
int
oshas(Objset *s, Hash h)
{
    return osfind(s, h) != nil;
}

```

3.3.2 Objq

```

⟨struct Objq 36c⟩≡ (185f)
struct Objq {
    // growing_array<Qelt> (len = nheap, allocated = heapsz)
    Qelt *heap;
    int nheap;
    int heapsz;
};

```

```

⟨struct Qelt 36d⟩≡ (185f)
struct Qelt {
    // ref<Object>
    Object *o;
    vlong ctime;
    //enum<Qcolor>
    int color;
};

```

Objq methods

```

⟨function qinit 36e⟩≡ (258)
void
qinit(Objq *q)
{
    memset(q, 0, sizeof(Objq));
    q->nheap = 0;
    q->heapsz = 8;
    q->heap = eamalloc(q->heapsz, sizeof(Qelt));
}

```

```

⟨function qclear 36f⟩≡ (258)
void
qclear(Objq *q)
{
    free(q->heap);
}

```

<function qput 37a>≡ (258)

```
void
qput(Objq *q, Object *o, int color)
{
    Qelt t;
    int i;

    assert(o->type == GCommit);
    <qput() realloc if needed 37b>
    q->heap[q->nheap].o = o;
    q->heap[q->nheap].color = color;
    q->heap[q->nheap].ctime = o->commit->ctime;
    <qput() resort by ctime 37c>
    q->nheap++;
}
```

<qput() realloc if needed 37b>≡ (37a)

```
if(q->nheap == q->heapsz){
    q->heapsz *= 2;
    q->heap = earealloc(q->heap, q->heapsz, sizeof(Qelt));
}
```

<qput() resort by ctime 37c>≡ (37a)

```
for(i = q->nheap; i > 0; i = (i-1)/2){
    if(q->heap[i].ctime < q->heap[(i-1)/2].ctime)
        break;
    // else exchange
    t = q->heap[i];
    q->heap[i] = q->heap[(i-1)/2];
    q->heap[(i-1)/2] = t;
}
```

<function qpop 37d>≡ (258)

```
int
qpop(Objq *q, Qelt *e)
{
    int i, l, r, m;
    Qelt t;

    if(q->nheap == 0)
        return 0;
    *e = q->heap[0];
    if(--q->nheap == 0)
        return 1;

    i = 0;
    q->heap[0] = q->heap[q->nheap];
    while(1){
        m = i;
        l = 2*i+1;
        r = 2*i+2;
        if(l < q->nheap && q->heap[m].ctime < q->heap[l].ctime)
            m = l;
        if(r < q->nheap && q->heap[m].ctime < q->heap[r].ctime)
            m = r;
        if(m == i)
            break;
        t = q->heap[m];
        q->heap[m] = q->heap[i];
        q->heap[i] = t;
    }
```

```
        i = m;  
    }  
    return 1;  
}
```

3.4 References

3.4.1 Reference names

3.4.2 HEAD

3.4.3 Reference content

3.4.4 Reference store

3.4.5 Objectish references

3.5 .git/INDEX9

Chapter 4

Reading from a Repository

4.1 Objects

4.1.1 readobject()

```
<function readobject 39a>≡ (215b)
  /// gitwalk1 | gtreegen | ... -> <>
  /*
    * Loads and returns a cached object.
    */
  Object*
  readobject(Hash h)
  {
    Object *o;

    if((o = readidxobject(nil, h, 0)) == nil)
      return nil;
    parseobject(o);
    ref(o);
    return o;
  }
```

4.1.2 readidxobject()

```
<function readidxobject 39b>≡ (215b)
  /// readobject | ?? -> <>
  static Object*
  readidxobject(Biobuf *idx, Hash h, int flag)
  {
    char path[Pathmax], hbuf[41];
    Object *obj, *new;
    int i, r;
    bool retried;
    Biobuf *f;
    vlong o;

    <readidxobject() if h in object cache 154d>
    <readidxobject() if Cthin flag 40a>
    <readidxobject() if Cidx flag 40b>
    new = nil;
    if(obj == nil){
      new = emalloc(sizeof(Object));
      new->id = objcache.nobj + 1;
      new->hash = h;
    }
```



```

        obj = new;
    }

    o = -1;
    retried = false;
retry:
    <readidxobject() look for object in packs 119a>
    // else
    snprintf(hbuf, sizeof(hbuf), "%H", h);
    snprintf(path, sizeof(path), ".git/objects/%c%c/%s", hbuf[0], hbuf[1], hbuf + 2);
    if((f = Bopen(path, OREAD)) != nil){
        if(readloose(f, obj, flag) == ERROR_NEG1)
            goto errorf;
        Bterm(f);
        parseobject(obj);
        cache(obj);
        return obj;
    }
    // else

    if(o == -1){
        if(retried)
            goto error;
        retried = true;
        refreshpacks();
        goto retry;
    }
errorf:
    Bterm(f);
error:
    free(new);
    return nil;
}

```

<readidxobject() if Cthin flag 40a>≡ (39b)

```

    if(flag & Cthin)
        flag &= ~Cidx;

```

<readidxobject() if Cidx flag 40b>≡ (39b)

```

    if(flag & Cidx)
        return nil;

```

4.1.3 readloose() and Cloaded

<function readloose 40c>≡ (215b)

```

    /// readobject -> readidxobject | ??? -> <>
    static int
    readloose(Biobuf *f, Object *o, int flag)
    {
        struct { char *tag; int type; } *p, types[] = {
            {"blob", GBlob},
            {"tree", GTree},
            {"commit", GCommit},
            <readloose() local types other elements 148d>
            {nil},
        };
        char *d, *s, *e, *ed;
        vlong sz, n;
        int l;

```

```

n = decompress(&d, f, nil);
if(n == ERROR_NEG1)
    return ERROR_NEG1;

s = d;
ed = d + n;
o->type = GNone;
for(p = types; p->tag; p++){
    l = strlen(p->tag);
    if(strncmp(s, p->tag, l) == 0){
        s += l;
        o->type = p->type;
        while(s != ed && !isspace(*s))
            s++;
        break;
    }
}
if(o->type == GNone){
    free(o->data);
    return ERROR_NEG1;
}
// else
sz = strtol(s, &e, 0);
if(e == s || *e++ != '\0'){
    werrstr("malformed object header");
    goto error;
}
if(sz != n - (e - d)){
    werrstr("mismatched sizes");
    goto error;
}
o->all = d;
o->data = e;
o->size = sz;
o->flag |= Cloaded|flag;
return OK_0;

error:
    free(d);
    return ERROR_NEG1;
}

```

4.1.4 parseobject() and Cparsed

```

⟨function parseobject 41⟩≡ (215b)
void
parseobject(Object *o)
{
    if(o->flag & Cparsed)
        return;
    switch(o->type){
        ⟨parseobject() switch o->type cases 43b⟩
        default:    break;
    }
    o->flag |= Cparsed;
}

```

4.1.5 Decompression

```
<gitinit() libflate initializations 42a>≡ (51f)
inflateinit();
deflateinit();
```

```
<function decompress 42b>≡ (215b)
/// readloose | ??? -> <>
errorneg1
decompress(void **p, Biobuf *b, vlong *csz)
{
    Buf d = {.len=0, .data=nil, .sz=0};

    if(bdecompress(&d, b, csz) == ERROR_NEG1){
        free(d.data);
        return ERROR_NEG1;
    }
    *p = d.data;
    return d.len;
}
```

```
<struct Buf 42c>≡ (215b)
struct Buf {
    int len;
    int sz;
    int off;
    char *data;
};
```

```
<function bdecompress 42d>≡ (215b)
int
bdecompress(Buf *d, Biobuf *b, vlong *csz)
{
    vlong o;

    o = Boffset(b);
    if(inflatezlib(d, bappend, b, breadc) == -1 || d->data == nil){
        free(d->data);
        return -1;
    }
    if (csz)
        *csz = Boffset(b) - o;
    return d->len;
}
```

```
<function bappend 42e>≡ (215b)
int
bappend(void *p, void *src, int len)
{
    Buf *b = p;
    char *n;

    while(b->len + len >= b->sz){
        b->sz = b->sz*2 + 64;
        n = realloc(b->data, b->sz);
        if(n == nil)
            return -1;
        b->data = n;
    }
    memmove(b->data + b->len, src, len);
    b->len += len;
}
```

```

    return len;
}

```

```

⟨function breadc 43a⟩≡ (215b)
int
breadc(void *p)
{
    return Bgetc(p);
}

```

4.1.6 Reading a blob

4.1.7 Reading a tree

```

⟨parseobject() switch o->type cases 43b⟩≡ (41) 44a▷
case GTree: parsetree(o); break;

```

```

⟨function parsetree 43c⟩≡ (215b)
static void
parsetree(Object *o)
{
    int m, a, entsz, nent;
    Dirent *t, *ent;
    char *p, *ep;

    p = o->data;
    ep = p + o->size;

    nent = 0;
    entsz = 16;
    ent = eamalloc(entsz, sizeof(Dirent));
    o->tree = emalloc(sizeof(Tinfo));
    while(p != ep){
        if(nent == entsz){
            entsz *= 2;
            ent = earealloc(ent, entsz, sizeof(Dirent));
        }
        t = &ent[nent++];
        m = strtol(p, &p, 8);
        if(*p != ' ')
            sysfatal("malformed tree %H: *p=(%d) %c", o->hash, *p, *p);
        p++;
        /*
         * only the stored permissions for the user
         * are relevant; git fills group and world
         * bits with whatever -- so to serve with
         * useful permissions, replicate the mode
         * of the git repo dir.
         */
        t->mode = gitdirmode;
        t->ismod = false;
        t->islink = false;
        if(m & 0777){
            a = (m & 0777)>>6;
            t->mode &= ((a<<6)|(a<<3)|a);
        }
        ⟨parsetree() if submodule 147b⟩
        ⟨parsetree() if symlink 146b⟩
        if(m & 0040000) /* dir */
    }
}

```

```

        t->mode |= DMDIR;
    t->name = p;
    p = memchr(p, 0, ep - p);
    if(p == nil || *p++ != 0 || ep - p < sizeof(t->h.h))
        sysfatal("malformed tree %H, remaining %d (%s)", o->hash, (int)(ep - p), p);
    memcpy(t->h.h, p, sizeof(t->h.h));
    p += sizeof(t->h.h);
}
o->tree->ent = ent;
o->tree->nent = nent;
}

```

4.1.8 Reading a commit

```

⟨parseobject() switch o->type cases 44a⟩≡ (41) <43b 148e>
    case GCommit: parsecommit(o); break;

⟨function parsecommit 44b⟩≡ (215b)
    static void
    parsecommit(Object *o)
    {
        char *p, *t, buf[512];
        int np;

        p = o->data;
        np = o->size;
        o->commit = emalloc(sizeof(Cinfo));
        while(1){
            if(scanword(&p, &np, buf, sizeof(buf)) == -1)
                break;
            if(strcmp(buf, "tree") == 0){
                if(scanword(&p, &np, buf, sizeof(buf)) == -1)
                    sysfatal("invalid commit: tree missing");
                if(hparse(&o->commit->tree, buf) == -1)
                    sysfatal("invalid commit: garbled tree");
            }else if(strcmp(buf, "parent") == 0){
                if(scanword(&p, &np, buf, sizeof(buf)) == -1)
                    sysfatal("invalid commit: missing parent");
                o->commit->parent = realloc(o->commit->parent, ++o->commit->nparent * sizeof(Hash));
                if(!o->commit->parent)
                    sysfatal("unable to malloc: %r");
                if(hparse(&o->commit->parent[o->commit->nparent - 1], buf) == -1)
                    sysfatal("invalid commit: garbled parent");
            }else if(strcmp(buf, "author") == 0){
                parseauthor(&p, &np, &o->commit->author, &o->commit->mtime);
            }else if(strcmp(buf, "committer") == 0){
                parseauthor(&p, &np, &o->commit->committer, &o->commit->ctime);
            }else if(strcmp(buf, "gpgsig") == 0){
                /* just drop it */
                if((t = strstr(p, "-----END PGP SIGNATURE-----")) == nil)
                    if((t = strstr(p, "-----END SSH SIGNATURE-----")) == nil)
                        if((t = strstr(p, "-----END SIGNED MESSAGE-----")) == nil)
                            sysfatal("malformed gpg signature");
                np -= t - p;
                p = t;
            }
            nextline(&p, &np);
        }
        while (np && isspace(*p)) {

```

```

        p++;
        np--;
    }
    o->commit->msg = p;
    o->commit->nmsg = np;
}

```

nextline()

```

⟨function nextline (git9/pack.c) 45a⟩≡ (215b)
static void
nextline(char **str, int *nstr)
{
    char *s;

    if((s = strchr(*str, '\n')) != nil){
        *nstr -= s - *str + 1;
        *str = s + 1;
    }
}

```

parseauthor()

```

⟨gitinit() set authorpat 45b⟩≡ (51f)
authorpat = regcomp("[\t ]*(.*)[\t ]+([0-9]+)[\t ]*([\\-+]?[0-9]+)?");

```

```

⟨function parseauthor 45c⟩≡ (215b)
// may raise sysfatal
static void
parseauthor(char **str, int *nstr, char **name, vlong *time)
{
    char buf[512];
    Resub m[4];
    vlong tz;
    char *p;
    int n, nm;

    if((p = strchr(*str, '\n')) == nil)
        sysfatal("malformed author line");
    n = p - *str;
    if(n >= sizeof(buf))
        sysfatal("overlong author line");
    memset(m, 0, sizeof(m));
    memcpy(buf, *str, n);
    buf[n] = '\0';
    *str = p;
    *nstr -= n;

    if(!regexec(authorpat, buf, m, nelem(m)))
        sysfatal("invalid author line %s", buf);
    nm = m[1].e.ep - m[1].s.sp;
    *name = emalloc(nm + 1);
    memcpy(*name, m[1].s.sp, nm);
    buf[nm] = '\0';

    nm = m[3].e.ep - m[3].s.sp;
    memcpy(buf, m[3].s.sp, nm);
    buf[nm] = '\0';
}

```

```

tz = atoll(buf);

nm = m[2].e.ep - m[2].s.sp;
memcpy(buf, m[2].s.sp, nm);
buf[nm] = '\0';
*time = atoll(buf) + 3600*(tz/100) + 60*(tz%100);

return;
}

```

4.2 References

<function readref (git9/ref.c) 46a> (232)
`errorneg1`
`readref(Hash *h, char *ref)`
`{`
`char buf[256], s[256];`
`errorneg1 r;`
`fdt f;`
`int n;`
<readref() (ref.c) other locals 46b>

`r = hparse(h, ref);`
`if(r != ERROR_NEG1)`
`return r;`
`// else`
`if(strcmp(ref, "HEAD") == 0){`
`snprint(buf, sizeof(buf), ".git/HEAD");`
`f = open(buf, OREAD);`
`if(f == ERROR_NEG1)`
`return ERROR_NEG1;`
`n = readn(f, s, sizeof(s) - 1);`
`if(n == ERROR_NEG1)`
`return ERROR_NEG1;`
`// else`
`s[n] = '\0';`
`strip(s);`
`r = hparse(h, s);`
`goto found;`
`}`
`// else`
<readref() (ref.c) try .git/refs prefixes 47a>
<readref() (ref.c) try other prefixes 47b>
`// else`
`return ERROR_NEG1;`

`found:`
`if(r == ERROR_NEG1 && strncmp(s, "ref: ", 5) == 0)`
`// recurse!`
`r = readref(h, s + 5);`
`return r;`
`}`

4.2.1 .refs/ prefixes

<readref() (ref.c) other locals 46b> (46a)
`static char *try[] =`

```

    {"", "refs/", "refs/heads/", "refs/remotes/", "refs/tags/", nil};
char **pfx;

⟨readref() (ref.c) try .git/refs prefixes 47a⟩≡ (46a)
for(pfx = try; *pfx; pfx++){
    snprintf(buf, sizeof(buf), ".git/%s%s", *pfx, ref);
    f = open(buf, OREAD);
    if(f == ERROR_NEG1)
        continue;
    n = readn(f, s, sizeof(s) - 1);
    if(n == ERROR_NEG1)
        // not closing??
        continue;
    // else
    s[n] = '\0';
    strip(s);
    r = hparse(h, s);
    close(f);
    goto found;
}

```

4.2.2 matchpfx

```

⟨readref() (ref.c) try other prefixes 47b⟩≡ (46a)
if((r = matchpfx(h, ref)) != ERROR_NEG1)
    return r;

```

```

⟨function matchpfx 47c⟩≡ (232)
static errorneg1
matchpfx(Hash *h, char *ref)
{
    int i, c;
    Hash pfx;
    char *p;

    memset(&pfx, 0, sizeof(Hash));
    for(i = 0, p = ref; *p; p++, i++){
        if((c = charval(*p)) == -1)
            return ERROR_NEG1;
        pfx.h[i/2] |= c;
        if((i & 1) == 0)
            pfx.h[i/2] <= 4;
    }
    return expandprefix(h, pfx, i*4);
}

```

```

⟨function expandprefix 47d⟩≡ (215b)
errorneg1
expandprefix(Hash *rh, Hash h, int npfx)
{
    int i, ndir;
    fdt fd;
    char buf[128];
    Dir *d;

    refreshpacks();
    if(npfx < 8 || npfx % 4 != 0)
        return ERROR_NEG1;
    for(i = 0; i < npackf; i++)

```



```

        if(searchindex(packf[i].idx, packf[i].nidx, h, npfx, rh) != -1)
            return OK_0;
    sprintf(buf, ".git/objects/%x", h.h[0]);
    if((fd = open(buf, OREAD)) == -1)
        return ERROR_NEG1;
    ndir = dirreadall(fd, &d);
    close(fd);
    if(ndir == -1)
        return ERROR_NEG1;
    for(i = 0; i < ndir; i++){
        snprintf(buf, sizeof(buf), "%x%s", h.h[0], d[i].name);
        if(hparse(rh, buf) == 0 && hashcmp(h.h, rh->h, npfx) == 0){
            free(d);
            return OK_0;
        }
    }
    free(d);
    return ERROR_NEG1;
}

```

4.3 Index

Chapter 5

Writing to a Repository

5.1 Objects

5.1.1 Compression

5.1.2 Locking

5.1.3 Writing a blob

5.1.4 Writing a tree

5.1.5 Writing a commit

5.2 References

5.3 Index

Chapter 6

git/conf

6.1 Repository configuration: .git/config

6.2 User configuration: ~/.lib/git/config

6.3 Usage

```
<function usage 50a>≡ (170a)
void
usage(void)
{
    fprintf(STDERR, "usage: %s [-f file] [-r] keys..\n", argv0);
    fprintf(STDERR, "\t-f:    use file 'file' (default: .git/config)\n");
    fprintf(STDERR, "\t-r:    print repository root\n");
    exits("usage");
}
```

6.4 main()

```
<function main 50b>≡ (170a)
void
main(int argc, char **argv)
{
    char repo[512];
    int nrel;
    <main() (conf.c) other locals 52e>

    ARGBEGIN{
        <main() (conf.c) command line processing 52c>
        default:    usage();          break;
    }ARGEND;

    gitinit(repo, sizeof(repo), &nrel);

    <main() (conf.c) if findroot 52d>
    // else
    <main() (conf.c) set default file 51d>

    for(i = 0; i < argc; i++){
        <main() (conf.c) show config entry for argv[i] 52f>
    }
```

```

    exits(nil);
}

⟨signature gitinit 51a⟩≡ (185f)
    void gitinit(char*, int, int*);

⟨global file 51b⟩≡ (170a)
    // array<ref_own<filename> (length=nfile)
    char    *file[32];

⟨global nfile 51c⟩≡ (170a)
    int nfile;

⟨main() (conf.c) set default file 51d⟩≡ (50b)
    if(nfile == 0){
        if((file[nfile++] = smprint("%s/.git/config", repo)) == nil)
            sysfatal("smprint: %r");
        if((p = getenv("home")) != nil)
            if((file[nfile++] = smprint("%s/lib/git/config", p)) == nil)
                sysfatal("smprint: %r");
        file[nfile++] = "/lib/git/config";
    }

```

6.5 gitinit() and findrepo()

```

⟨function gitinit 51e⟩≡ (258)
    /// conf.c:main | fs.c:main | ... -> <>
    void
    gitinit(char *root, int nroot, int *nrel)
    {
        char repo[512] = ".git";
        Dir *d;

        ⟨gitinit() initializations 51f⟩
        if(root != nil){
            findrepo(root, nroot, nrel);
            snprintf(repo, sizeof(repo), "%s/.git", root);
        }
        if((d = dirstat(repo)) == nil)
            sysfatal("stat %s: %r", repo);
        gitdirmode = d->mode & 0777;
        free(d);
    }

⟨gitinit() initializations 51f⟩≡ (51e)
    ⟨gitinit() fmtinstall calls 29f⟩
    ⟨gitinit() libflate initializations 42a⟩
    ⟨gitinit() set authorpat 45b⟩
    ⟨gitinit() set objcache 154b⟩

⟨global gitdirmode 51g⟩≡ (258)
    int gitdirmode = -1;

```

```

⟨function findrepo 52a⟩≡ (258)
/* Finds the directory containing the git repo. */
static void
findrepo(char *buf, int nbuf, int *nrel)
{
    char *p;
    char *suffix = "/.git/HEAD";

    if(getwd(buf, nbuf - strlen(suffix) - 1) == nil)
        sysfatal("getwd: %r");

    *nrel = 0;
    for(p = buf + strlen(buf); p != nil; p = strrchr(buf, '/')){
        strcpy(p, suffix);
        if(access(buf, AEXIST) == 0){
            p[p == buf] = '\0';
            return;
        }
        // else
        *nrel += 1;
        *p = '\0';
    }
    sysfatal("not a git repository");
}

```

6.6 git/conf -r

```

⟨global findroot 52b⟩≡ (170a)
bool findroot;

```

```

⟨main() (conf.c) command line processing 52c⟩≡ (50b) 53c▷
case 'r': findroot=true; break;

```

```

⟨main() (conf.c) if findroot 52d⟩≡ (50b)
if(findroot){
    print("%s\n", repo);
    exits(nil);
}

```

6.7 git/conf <key>

```

⟨main() (conf.c) other locals 52e⟩≡ (50b)
char *p, *s;
int i, j;

```

```

⟨main() (conf.c) show config entry for argv[i] 52f⟩≡ (50b)
// syntax is git/fs <sect>.<key> or just <key>
p = strchr(argv[i], '.');
if(p == nil){
    s = nil;
    p = argv[i];
}else{
    *p = '\0';
    p++;
    s = smprint("[%s]", argv[i]);
}

```

```

for(j = 0; j < nfile; j++)
    if(showconf(file[j], s, p))
        break;

⟨function showconf 53a⟩≡ (170a)
static bool
showconf(char *cfg, char *sect, char *key)
{
    Biobuf *f;
    bool found, foundsect;
    char *ln, *p;
    int nsect, nkey;

    f = Bopen(cfg, OREAD);
    if(f == nil)
        return false;

    found = false;
    nsect = sect ? strlen(sect) : 0;
    nkey = strlen(key);
    foundsect = (sect == nil);

    while((ln = Brdstr(f, '\n', 1)) != nil){
        p = strip(ln);
        if(*p == '[' && sect){
            foundsect = strncmp(sect, ln, nsect) == 0;
        }else if(foundsect && strncmp(p, key, nkey) == 0){
            p = strip(p + nkey);
            if(*p != '=')
                continue;
            p = strip(p + 1);
            print("%s\n", p);
            found = true;
            if(!showall){
                free(ln);
                goto done;
            }
        }
        free(ln);
    }
done:
    return found;
}

```

6.8 Advanced features

6.8.1 git/conf -a

```

⟨global showall 53b⟩≡ (170a)
bool showall;

⟨main() (conf.c) command line processing 53c⟩+≡ (50b) <52c 54>
case 'a':    showall=true;        break;

```

6.8.2 git/conf -f <config>

```
<main() (conf.c) command line processing 54>+≡
case 'f':
    if(nfile == nelem(file))
        sysfatal("too many configs");
    file[nfile++] = EARGF(usage());
    break;
```

(50b) <53c

Chapter 7

git/fs

7.1 Usage

```
<function usage (git9/fs.c) 55a>≡ (174)
void
usage(void)
{
    fprintf(STDERR, "usage: %s [-d]\n", argv0);
    fprintf(STDERR, "\t-d:    debug\n");
    exits("usage");
}
```

7.2 main()

```
<function main (git9/fs.c) 55b>≡ (174)
void
main(int argc, char **argv)
{
    char repo[512];
    int nelt;
    Dir *d;

    gitinit(repo, sizeof(repo), &nelt);
    // !!chdir!!
    if(chdir(repo) == ERROR_NEG1)
        sysfatal("chdir: %r");

    ARGBEGIN{
        <main() (fs.c) command line processing 75b>
        default: usage(); break;
    }ARGEND;

    if(argc != 0)
        usage();

    // can use just relative .git path because of chdir() above
    d = dirstat(".git");
    if(d == nil)
        sysfatal("dirstat .git: %r");
    username = strdup(d->uid);
    groupname = strdup(d->gid);
    free(d);
}
```



```

    (main() (fs.c) set branches 56e)
    postmountsrv(&gitsrv, nil, mntpt, MCREATE);
    exits(nil);
}

(global username 56a)≡ (174)
    char    *username;

(global groupname 56b)≡ (174)
    char    *groupname;

(global mntpt 56c)≡ (174)
    // can be changed by git/fs -m
    char    *mntpt = ".git/fs";

(global branches 56d)≡ (174)
    // growing_array<ref_own<string>>
    char    **branches = nil;

(main() (fs.c) set branches 56e)≡ (55b)
    branches = emalloc(sizeof(char*));
    branches[0] = nil;

(global gitsrv 56f)≡ (174)
    Srv gitsrv = {
        // will attach and set gitdir too
        .attach=gitattach,
        .walk1=gitwalk1,
        .clone=gitclone,
        .open=gitopen,
        .read=gitread,
        .stat=gitstat,
        .destroyfid=gitdestroyfid,
    };

```

7.3 Qxxx and qroot

```

(enum Qxxx 56g)≡ (174)
    enum Qxxx {
        Qroot,

        Qhead,
        Qbranch,
        Qobject,
        Qctl,

        // object/
        Qtree,
        Qcommit,
        Qmsg,
        Qparent,
        Qhash,
        Qauthor,
        Qcommitter,

        Qmax,
        Internal=1<<7,
    };

```

```

⟨constant qroot 57a⟩≡ (174)
char *qroot[] = {
    "HEAD",
    "branch",
    "object",
    "ctl",
};

```

7.4 Data structures

7.4.1 Gitaux and Crumb

```

⟨struct Gitaux 57b⟩≡ (174)
struct Gitaux {
    int ncrumb;
    // growing_array<ref_own<Crumb>> len = ncrumb
    Crumb *crumb;

    ⟨Gिताux object dir fields 57f⟩
    ⟨Gिताux other fields 57c⟩
};

```

```

⟨Gिताux other fields 57c⟩≡ (57b)
// option<ref_own<string>>, ex: ".git/refs/" for Qbranch
char *refpath;
int qdir;

```

```

⟨struct Crumb 57d⟩≡ (174)
struct Crumb {
    char *name;

    Qid qid;
    int mode;
    vlong mtime;

    // option<shared_ref<Object>>
    Object *obj;
};

```

```

⟨function crumb 57e⟩≡ (174)
static Crumb*
crumb(Gिताux *aux, int n)
{
    if(n < aux->ncrumb)
        return &aux->crumb[aux->ncrumb - n - 1];
    return nil;
}

```

7.4.2 Objlist

```

⟨Gिताux object dir fields 57f⟩≡ (57b)
/* For listing object dir */
// ref_own<Objlist> ??
Objlist *ols;
Object *olslast;

```

```

⟨struct Objlist 58a⟩≡ (185f)
    struct Objlist {
        int idx;

        // option<fdt> (None = -1)
        fdt fd;
        // ??
        int state;
        int stage;

        // .git/objects/
        // array<ref_own<Dir>> (len = ntop)
        Dir *top;
        int ntop;
        int topidx;

        // ??
        // array<ref_own<Dir>> (len = nloose)
        Dir *loose;
        int nloose;
        int looseidx;

        // .git/objects/pack/
        // option<array<ref_own<Dir>> (len = npack)
        Dir *pack;
        int npack;
        int packidx;

        int nent;
        int entidx;
    };

⟨function olsfree 58b⟩≡ (194b)
    void
    olsfree(Objlist *ols)
    {
        if(ols == nil)
            return;
        if(ols->fd != -1)
            close(ols->fd);
        free(ols->top);
        free(ols->loose);
        free(ols->pack);
        free(ols);
    }

```

7.4.3 Uqid and Cache

```

⟨struct Uqid 58c⟩≡ (174)
    struct Uqid {
        vlong    uqid;

        vlong    ppath;
        vlong    oid;

        int t;
        int idx;
    };

```

```
<struct Cache 59a>≡ (174)
```

```
struct Cache {
    // growing_array<ref_own<Uqid>> (len = n)
    Uqid *cache;
    int n;
    int max;
};
```

```
<global uqidcache 59b>≡ (174)
```

```
// hash<qid, Uqid>
Cache    uqidcache[512];
```

```
<function qpath 59c>≡ (174)
```

```
vlong
qpath(Crumb *p, int idx, vlong id, vlong t)
{
    int h, i;
    vlong pp;
    Cache *c;
    Uqid *u;

    pp = p ? p->qid.path : 0;
    h = (pp*333 + id*7 + t) & (nelem(uqidcache) - 1);
    c = &uqidcache[h];
    u = c->cache;
    for(i=0; i <c->n ; i++){
        if(u->ppath == pp && u->oid == id && u->t == t && u->idx == idx)
            return (u->uqid << 8) | t;
        u++;
    }
    if(c->n == c->max){
        c->max += c->max/2 + 1;
        c->cache = erealloc(c->cache, c->max*sizeof(Uqid));
    }
    nextqid++;
    c->cache[c->n] = (Uqid){nextqid, pp, id, t, idx};
    c->n++;
    return (nextqid << 8) | t;
}
```

```
<global nextqid 59d>≡ (174)
```

```
vlong    nextqid = Qmax;
```

```
<macro QDIR 59e>≡ (185f)
```

```
#define QDIR(qid)    ((int)(qid)->path & (0xff))
```

7.5 gitxxx() skeletons

7.5.1 gitattach() and gitdir

```
<global gitdir 59f>≡ (174)
```

```
char    gitdir[512];
```

```

⟨function gitattach 60a⟩≡ (174)
static void
gitattach(Req *r)
{
    Gitaux *aux;
    Dir *d;

    d = dirstat(".git");
    if(d == nil)
        sysfatal("git/fs: %r");
    if(getwd(gitdir, sizeof(gitdir)) == nil)
        sysfatal("getwd: %r");

    aux = emalloc(sizeof(Gitaux));
    aux->crumb = emalloc(sizeof(Crumb));
    aux->ncrumb = 1;
    ⟨gitattach() set aux->crumb 60b⟩

    r->ofcall.qid = (Qid){Qroot, 0, QTDIR};
    r->fid->qid = r->ofcall.qid;
    r->fid->aux = aux;

    respond(r, nil);
}

```

```

⟨gitattach() set aux->crumb 60b⟩≡ (60a)
aux->crumb[0].qid = (Qid){Qroot, 0, QTDIR};
aux->crumb[0].obj = nil;
aux->crumb[0].mode = DMDIR | 0555;
aux->crumb[0].mtime = d->mtime;
aux->crumb[0].name = estrdup("/");

```

7.5.2 gitclone() and gitdestroyfid()

```

⟨function gitclone 60c⟩≡ (174)
static char*
gitclone(Fid *o, Fid *n)
{
    Gitaux *aux, *oaux;
    int i;

    oaux = o->aux;
    aux = emalloc(sizeof(Gitaux));

    aux->ncrumb = oaux->ncrumb;
    aux->crumb = eamalloc(oaux->ncrumb, sizeof(Crumb));
    for(i = 0; i < aux->ncrumb; i++){
        aux->crumb[i] = oaux->crumb[i];
        aux->crumb[i].name = estrdup(oaux->crumb[i].name);
        if(aux->crumb[i].obj)
            aux->crumb[i].obj = ref(oaux->crumb[i].obj);
    }
    if(oaux->refpath)
        aux->refpath = strdup(oaux->refpath);
    aux->qdir = oaux->qdir;
    n->aux = aux;
    return nil;
}

```

```

⟨function gitdestroyfid 61a⟩≡ (174)
static void
gitdestroyfid(Fid *f)
{
    Gिताux *aux;
    int i;

    if((aux = f->aux) == nil)
        return;
    for(i = 0; i < aux->ncrumb; i++){
        if(aux->crumb[i].obj)
            unref(aux->crumb[i].obj);
        free(aux->crumb[i].name);
    }
    olsfree(aux->ols);
    free(aux->refpath);
    free(aux->crumb);
    free(aux);
}

```

7.5.3 gitopen()

```

⟨function gitopen 61b⟩≡ (174)
static void
gitopen(Req *r)
{
    Gिताux *aux;
    Crumb *c;

    aux = r->fid->aux;
    c = crumb(aux, 0);
    switch(r->ifcall.mode&3){
    case OREAD:
    case ORDWR:
        respond(r, nil);
        break;
    case OWRITE:
        respond(r, Eperm);
        break;
    case OEXEC:
        if((c->mode & 0111) == 0)
            respond(r, Eperm);
        else
            respond(r, nil);
        break;
    default:
        respond(r, "botched mode");
        break;
    }
}

```

7.5.4 gitstat()

```

⟨function gitstat 61c⟩≡ (174)
static void
gitstat(Req *r)
{
    Gिताux *aux;

```

```

Crumb *c;

aux = r->fid->aux;
c = crumb(aux, 0);

r->d.uid = estrdup9p(username);
r->d.gid = estrdup9p(groupname);
r->d.muid = estrdup9p(username);

r->d.qid = r->fid->qid;
r->d.mtime = c->mtime;
r->d.atime = c->mtime;
r->d.mode = c->mode;

if(c->obj)
    obj2dir(&r->d, c, c->obj, c->name);
else
    r->d.name = estrdup9p(c->name);
respond(r, nil);
}

```

```

⟨function obj2dir 62a⟩≡ (174)
static void
obj2dir(Dir *d, Crumb *c, Object *o, char *name)
{
    d->qid = c->qid;
    d->atime = c->mtime;
    d->mtime = c->mtime;
    d->mode = c->mode;

    d->name = estrdup9p(name);
    d->uid = estrdup9p(username);
    d->gid = estrdup9p(groupname);
    d->muid = estrdup9p(username);

    if(o->type == GBlob || o->type == GTag){
        d->qid.type = 0;
        d->mode &= 0777;
        d->length = o->size;
    }
}
}

```

7.5.5 gitread()

```

⟨function gitread 62b⟩≡ (174)
static void
gitread(Req *r)
{
    char buf[256], *e;
    Gitaux *aux;
    Object *o;
    Qid *q;

    aux = r->fid->aux;
    q = &r->fid->qid;
    o = crumb(aux, 0)->obj;
    e = nil;

```

```

switch(QDIR(q)){
⟨gitread() switch QDIR(q) cases 64b⟩
default:
    e = Egreg;
}
respond(r, e);
}

```

7.5.6 gitwalk1()

⟨function gitwalk1 63a⟩≡ (174)

```

static char*
gitwalk1(Fid *fid, char *name, Qid *q)
{
    char path[128];
    Gitaux *aux;
    Crumb *c, *o;
    char *e;
    Dir *d;
    Hash h;

    e = nil;
    aux = fid->aux;

    q->vers = 0;
    ⟨gitwalk1() if dotdot 63b⟩
    // else

    aux->crumb = realloc(aux->crumb, (aux->ncrumb + 1) * sizeof(Crumb));
    aux->ncrumb++;

    c = crumb(aux, 0);
    o = crumb(aux, 1);
    memset(c, 0, sizeof(Crumb));
    c->mode = o->mode;
    c->mtime = o->mtime;
    c->obj = o->obj ? ref(o->obj) : nil;

    switch(QDIR(&fid->qid)){
    ⟨gitwalk1() switch QDIR(&fid->qid) cases 64d⟩
    default:
        return Egreg;
    }

    c->name = estrdup(name);
    c->qid = *q;
    fid->qid = *q;
    return e;
}

```

⟨gitwalk1() if dotdot 63b⟩≡ (63a)

```

if(strcmp(name, "..") == 0){
    popcrumb(aux);
    c = crumb(aux, 0);
    *q = c->qid;
    fid->qid = *q;
    return nil;
}

```



```

⟨function popcrumb 64a⟩≡ (174)
static void
popcrumb(Gitaux *aux)
{
    Crumb *c;

    if(aux->ncrumb > 1){
        c = crumb(aux, 0);
        free(c->name);
        unref(c->obj);
        aux->ncrumb--;
    }
}

```

7.6 .git/fs/

```

⟨gitread() switch QDIR(q) cases 64b⟩≡ (62b) 65a▷
case Qroot:
    dirread9p(r, rootgen, aux);
    break;

```

```

⟨function rootgen 64c⟩≡ (174)
static errorneg1
rootgen(int i, Dir *d, void *p)
{
    Crumb *c;

    c = crumb(p, 0);
    if (i >= nelem(qroot))
        return ERROR_NEG1;

    d->mode = 0555 | DMDIR;
    d->name = estrdup9p(qroot[i]);
    d->qid.vers = 0;
    d->qid.type = strcmp(qroot[i], "ctl") == 0 ? 0 : QTDIR;
    d->qid.path = qpath(nil, i, i, Qroot);
    d->uid = estrdup9p(username);
    d->gid = estrdup9p(groupname);
    d->muid = estrdup9p(username);
    d->mtime = c->mtime;
    return OK_0;
}

```

```

⟨gitwalk1() switch QDIR(&fid->qid) cases 64d⟩≡ (63a) 65e▷
case Qroot:
    if(strcmp(name, "ctl") == 0){
        *q = (Qid){Qctl, 0, 0};
        c->mode = 0644;
        ⟨gitwalk1() when Qroot case, switch on name other cases 65f⟩
    }else{
        e = Eexist;
    }
    break;

```

7.6.1 .git/fs/ctl

```
<gitread() switch QDIR(q) cases 65a>+≡ (62b) <64b 66a>
    case Qctl:
        e = readctl(r);
        break;
```

```
<function readctl 65b>≡ (174)
    static char *
    readctl(Req *r)
    {
        fdt fd;
        int n;
        char data[1024], ref[512];
        char *s, *e;

        fd = open(".git/HEAD", OREAD);
        <readctl() sanity check fd 65c>
        /* empty HEAD is invalid */
        n = readn(fd, ref, sizeof(ref) - 1);
        <readctl() sanity check n 65d>
        close(fd);

        s = ref;
        ref[n] = '\0';
        if(strncmp(s, "ref:", 4) == 0)
            s += 4;
        while(*s == ' ' || *s == '\t')
            s++;
        if((e = strchr(s, '\n')) != nil)
            *e = '\0';
        if(strstr(s, "refs/") == s)
            s += strlen("refs/");

        snprintf(data, sizeof(data), "branch %s\nrepo %s\n", s, gitdir);
        readstr(r, data);
        return nil;
    }
```

```
<readctl() sanity check fd 65c>≡ (65b)
    if(fd == ERROR_NEG1)
        return Erepo;
```

```
<readctl() sanity check n 65d>≡ (65b)
    if(n <= 0)
        return Erepo;
```

```
<gitwalk1() switch QDIR(&fid->qid) cases 65e>+≡ (63a) <64d 67a>
    case Qctl:
        return Enodir;
```

7.6.2 .git/fs/object/

```
<gitwalk1() when Qroot case, switch on name other cases 65f>≡ (64d) 72g>
    }else if(strcmp(name, "object") == 0){
        *q = (Qid){Qobject, 0, QTDIR};
        c->mode = DMDIR | 0555;
```

```

⟨gitread() switch QDIR(q) cases 66a⟩+≡ (62b) <65a 68d>
case Qobject:
    if(o)
        objread(r, aux);
    else
        dirread9p(r, objgen, aux);
    break;

```

objgen()

```

⟨function objgen 66b⟩≡ (174)
static int
objgen(int i, Dir *d, void *p)
{
    Gitaux *aux;
    Object *o;
    Crumb *c;
    char name[64];
    Objlist *ols;
    Hash h;

    aux = p;
    c = crumb(aux, 0);

    if(!aux->ols)
        aux->ols = mkols();
    ols = aux->ols;
    o = nil;

    /* We tried to sent it, but it didn't fit */
    if(aux->olslast && ols->idx == i + 1){
        snprintf(name, sizeof(name), "%H", aux->olslast->hash);
        obj2dir(d, c, aux->olslast, name);
        return OK_0;
    }
    // else
    while(ols->idx <= i){
        if(olsnext(ols, &h) == -1)
            return ERROR_NEG1;
        if((o = readobject(h)) == nil){
            fprintf(STDERR, "corrupt object %H\n", h);
            return ERROR_NEG1;
        }
    }
    if(o != nil){
        snprintf(name, sizeof(name), "%H", o->hash);
        obj2dir(d, c, o, name);
        unref(aux->olslast);
        aux->olslast = ref(o);
        return OK_0;
    }
    // else
    return ERROR_NEG1;
}

```

mkols()

```

⟨function mkols 66c⟩≡ (194b)

```

```

Objlist*
mkols(void)
{
    Objlist *ols;

    ols = emalloc(sizeof(Objlist));
    if((ols->top = slurpdir(".git/objects", &ols->top)) == ERROR_NEG1)
        sysfatal("read top level: %r");
    if((ols->npack = slurpdir(".git/objects/pack", &ols->npack)) == ERROR_NEG1)
        ols->npack = nil;
    ols->fd = -1;
    return ols;
}

```

gitwalk1() for Qobject

```

⟨gitwalk1() switch QDIR(&fid->qid) cases 67a⟩+≡ (63a) ◁65e 69a▷
case Qobject:
    if(c->obj){
        e = objwalk1(q, o->obj, o, c, name, Qobject, aux);
    }else{
        if(hparse(&h, name) == ERROR_NEG1)
            return Ebadobj;
        c->obj = readobject(h);
        if(c->obj == nil)
            return Ebadobj;
        if(c->obj->type == GBlob || c->obj->type == GTag){
            c->mode = 0644;
            q->type = 0;
        }else{
            c->mode = DMDIR | 0755;
            q->type = QTDIR;
        }
        q->path = qpath(o, Qobject, c->obj->id, Qobject);
        q->vers = 0;
    }
    break;

```

objwalk1()

```

⟨function objwalk1 67b⟩≡ (174)
static char *
objwalk1(Qid *q, Object *o, Crumb *p, Crumb *c, char *name, vlong qdir, Gitaux *aux)
{
    Object *w, *l;
    char *e;
    int i, m;

    w = nil;
    e = nil;
    if(!o)
        return Eexist;
    switch(o->type){
        ⟨objwalk1() switch o->type cases 68c⟩
    }
    return e;
}

```

7.6.3 .git/fs/object/<blobid>

```
<function objread 68a>≡ (174)
static void
objread(Req *r, Gitaux *aux)
{
    Object *o;

    o = crumb(aux, 0)->obj;
    switch(o->type){
        <objread() switch o->type cases 68b>
    default:
        sysfatal("invalid object type %d", o->type);
    }
}

<objread() switch o->type cases 68b>≡ (68a) 68e>
case GBlob:
    readbuf(r, o->data, o->size);
    break;

<objwalk1() switch o->type cases 68c>≡ (67b) 69b>
case GBlob:
    e = Enotdir;
    break;
```

7.6.4 .git/fs/object/<treeid>/

```
<gitread() switch QDIR(q) cases 68d>+≡ (62b) <66a 69c>
case Qtree:
    objread(r, aux);
    break;

<objread() switch o->type cases 68e>+≡ (68a) <68b 69d>
case GTree:
    dirread9p(r, gtreegen, aux);
    break;

<function gtreegen 68f>≡ (174)
static int
gtreegen(int i, Dir *d, void *p)
{
    Object *o, *l, *e;
    Gitaux *aux;
    Crumb *c;
    int m;

    aux = p;
    c = crumb(aux, 0);
    e = c->obj;
    if(i >= e->tree->nent)
        return ERROR_NEG1;
    m = e->tree->ent[i].mode;
    <gtreegen() if submodule 147c>
    else if((o = readobject(e->tree->ent[i].h)) == nil)
        sysfatal("could not read object %H: %r", e->tree->ent[i].h);

    <gtreegen() if e->tree->ent[i].islink 146c>
```

```

d->qid.vers = 0;
d->qid.type = o->type == GTree ? QTDIR : 0;
d->qid.path = qpath(c, i, o->id, aux->qdir);
d->mode = m;
d->atime = c->mtime;
d->mtime = c->mtime;
d->uid = estrdup9p(username);
d->gid = estrdup9p(groupname);
d->muid = estrdup9p(username);
d->name = estrdup9p(e->tree->ent[i].name);
d->length = o->size;
return OK_0;
}

```

`<gitwalk1() switch QDIR(&fid->qid) cases 69a>+≡` (63a) <67a 70b>

```

case Qtree:
    e = objwalk1(q, o->obj, o, c, name, Qtree, aux);
    break;

```

`<objwalk1() switch o->type cases 69b>+≡` (67b) <68c 70c>

```

case GTree:
    q->type = 0;
    for(i = 0; i < o->tree->nent; i++){
        if(strcmp(o->tree->ent[i].name, name) != 0)
            continue;
        m = o->tree->ent[i].mode;
        w = readobject(o->tree->ent[i].h);

        <objwalk1() when GTree case, if o->tree->ent[i].ismod 147d>
        <objwalk1() when GTree case, if o->tree->ent[i].islink 146d>

        if(!w)
            return Ebadobj;
        q->type = (w->type == GTree) ? QTDIR : 0;
        q->path = qpath(p, i, w->id, qdir);
        c->mode = m;
        c->mode |= (w->type == GTree) ? (DMDIR|0755) : 0644;
        c->obj = w;
        break;
    }
    if(!w)
        e = Eexist;
    break;

```

7.6.5 .git/fs/object/<commitid>/

`<gitread() switch QDIR(q) cases 69c>+≡` (62b) <68d 71c>

```

case Qcommit:
    objread(r, aux);
    break;

```

`<objread() switch o->type cases 69d>+≡` (68a) <68e 148b>

```

case GCommit:
    dirread9p(r, gcommitgen, aux);
    break;

```

<function gcommitgen 70a>≡ (174)

```
static int
gcommitgen(int i, Dir *d, void *p)
{
    Object *o;
    Crumb *c;

    c = crumb(p, 0);
    o = c->obj;

    d->uid = estrdup9p(username);
    d->gid = estrdup9p(groupname);
    d->muid = estrdup9p(username);
    d->mode = 0444;
    d->atime = o->commit->ctime;
    d->mtime = o->commit->ctime;
    d->qid.type = 0;
    d->qid.vers = 0;

    switch(i){
<gcommitgen() switch i cases 71a>
    default:
        free(d->uid);
        free(d->gid);
        free(d->muid);
        return ERROR_NEG1;
    }
    // else
    return OK_0;
}
```

<gitwalk1() switch QDIR(&fid->qid) cases 70b>+≡ (63a) *<69a 72a>*

```
case Qcommit:
    e = objwalk1(q, o->obj, o, c, name, Qcommit, aux);
    break;
```

<objwalk1() switch o->type cases 70c>+≡ (67b) *<69b 148c>*

```
case GCommit:
    q->type = 0;
    c->mtime = o->commit->mtime;
    c->mode = 0644;
    assert(qdir == Qcommit || qdir == Qobject || qdir == Qtree || qdir == Qhead || qdir == Qcommitter);
    if(strcmp(name, "msg") == 0)
        q->path = qpath(p, 0, o->id, Qmsg);
    else if(strcmp(name, "parent") == 0)
        q->path = qpath(p, 1, o->id, Qparent);
    else if(strcmp(name, "hash") == 0)
        q->path = qpath(p, 2, o->id, Qhash);
    else if(strcmp(name, "author") == 0)
        q->path = qpath(p, 3, o->id, Qauthor);
    else if(strcmp(name, "committer") == 0)
        q->path = qpath(p, 3, o->id, Qcommitter);
    else if(strcmp(name, "tree") == 0){
        q->type = QTDIR;
        q->path = qpath(p, 4, o->id, Qtree);
        unref(c->obj);
        c->mode = DMDIR | gitdirmode;
        c->obj = readobject(o->commit->tree);
        if(c->obj == nil)
            sysfatal("could not read object %H: %r", o->commit->tree);
    }
```

```

}
else
    e = Eexist;
break;

```

7.6.6 .git/fs/object/<commitid>/tree

```

⟨gcommitgen() switch i cases 71a⟩≡ (70a) 71b▷
case 0:
    d->mode = DMDIR | gitdirmode;
    d->name = estrdup9p("tree");
    d->qid.type = QTDIR;
    d->qid.path = qpath(c, i, o->id, Qtree);
    break;

```

7.6.7 .git/fs/object/<commitid>/parent

```

⟨gcommitgen() switch i cases 71b⟩+≡ (70a) ◁71a 71e▷
case 1:
    d->name = estrdup9p("parent");
    d->qid.path = qpath(c, i, o->id, Qparent);
    break;

```

```

⟨gitread() switch QDIR(q) cases 71c⟩+≡ (62b) ◁69c 71f▷
case Qparent:
    readcommitparent(r, o);
    break;

```

```

⟨function readcommitparent 71d⟩≡ (174)
static void
readcommitparent(Req *r, Object *o)
{
    char *buf, *p, *e;
    int i, n;

    /* 40 bytes per hash, 1 per nl, 1 for terminator */
    n = o->commit->nparent * (40 + 1) + 1;
    buf = emalloc(n);
    p = buf;
    e = buf + n;
    for (i = 0; i < o->commit->nparent; i++)
        p = seprint(p, e, "%H\n", o->commit->parent[i]);
    readbuf(r, buf, p - buf);
    free(buf);
}

```

7.6.8 .git/fs/object/<commitid>/msg

```

⟨gcommitgen() switch i cases 71e⟩+≡ (70a) ◁71b 72b▷
case 2:
    d->name = estrdup9p("msg");
    d->qid.path = qpath(c, i, o->id, Qmsg);
    break;

```

```

⟨gitread() switch QDIR(q) cases 71f⟩+≡ (62b) ◁71c 72c▷
case Qmsg:
    readbuf(r, o->commit->msg, o->commit->nmsg);
    break;

```



```

<gitwalk1() switch QDIR(&fid->qid) cases 72a>+≡ (63a) <70b 73c>
    case Qmsg:
    case Qparent:
    case Qhash:
    case Qauthor:
    case Qcommitter:
        return Enodir;

```

7.6.9 .git/fs/object/<commitid>/hash

```

<gcommitgen() switch i cases 72b>+≡ (70a) <71e 72d>
    case 3:
        d->name = estrdup9p("hash");
        d->qid.path = qpath(c, i, o->id, Qhash);
        break;

```

```

<gitread() switch QDIR(q) cases 72c>+≡ (62b) <71f 72e>
    case Qhash:
        snprintf(buf, sizeof(buf), "%H\n", o->hash);
        readstr(r, buf);
        break;

```

7.6.10 .git/fs/object/<commitid>/author

```

<gcommitgen() switch i cases 72d>+≡ (70a) <72b
    case 4:
        d->name = estrdup9p("author");
        d->qid.path = qpath(c, i, o->id, Qauthor);
        break;

```

```

<gitread() switch QDIR(q) cases 72e>+≡ (62b) <72c 72f>
    case Qauthor:
        snprintf(buf, sizeof(buf), "%s\n", o->commit->author);
        readstr(r, buf);
        break;

```

7.6.11 .git/fs/object/<commitid>/committer

```

<gitread() switch QDIR(q) cases 72f>+≡ (62b) <72e 73b>
    case Qcommitter:
        snprintf(buf, sizeof(buf), "%s\n", o->commit->committer);
        readstr(r, buf);
        break;

```

7.6.12 .git/fs/HEAD/

```

<gitwalk1() when Qroot case, switch on name other cases 72g>+≡ (64d) <65f 73d>
    }else if(strcmp(name, "HEAD") == 0){
        *q = (Qid){Qhead, 0, QTDIR};
        c->mode = DMDIR | 0555;
        c->obj = readref(".git/HEAD");
    }

```

<function readref 73a>≡ (174)

```
static Object *
readref(char *pathstr)
{
    fdt f;
    char buf[128], path[128], *p, *e;
    Hash h;
    int n;

    snprintf(path, sizeof(path), "%s", pathstr);
    while(true){
        if((f = open(path, OREAD)) == ERROR_NEG1)
            return nil;
        if((n = readn(f, buf, sizeof(buf) - 1)) == ERROR_NEG1)
            return nil;
        close(f);
        buf[n] = '\0';
        if(strncmp(buf, "ref:", 4) != 0)
            break;

        // else
        p = buf + 4;
        while(isspace(*p))
            p++;
        if((e = strchr(p, '\n')) != nil)
            *e = 0;
        snprintf(path, sizeof(path), ".git/%s", p);
    }

    if(hparse(&h, buf) == ERROR_NEG1)
        return nil;

    return readobject(h);
}
```

<gitread() switch QDIR(q) cases 73b>+≡ (62b) *<72f 74a>*

```
case Qhead:
    /* Empty repositories have no HEAD */
    if(o == nil)
        r->ofcall.count = 0;
    else
        objread(r, aux);
    break;
```

<gitwalk1() switch QDIR(&fid->qid) cases 73c>+≡ (63a) *<72a 75a>*

```
case Qhead:
    e = objwalk1(q, o->obj, o, c, name, Qhead, aux);
    break;
```

7.6.13 .git/fs/branch/

<gitwalk1() when Qroot case, switch on name other cases 73d>+≡ (64d) *<72g*

```
}else if(strcmp(name, "branch") == 0){
    *q = (Qid){Qbranch, 0, QTDIR};
    aux->refpath = estrdup(".git/refs/");
    c->mode = DMDIR | 0555;
```

`<gitread() switch QDIR(q) cases 74a>+≡ (62b) <73b`

```
case Qbranch:
    if(o)
        objread(r, aux);
    else
        dirread9p(r, branchgen, aux);
    break;
```

`<function branchgen 74b>≡ (174)`

```
static int
branchgen(int i, Dir *d, void *p)
{
    Gitaux *aux;
    Dir *refs;
    Crumb *c;
    int n;

    aux = p;
    c = crumb(aux, 0);
    refs = nil;
    d->qid.vers = 0;
    d->qid.type = QTDIR;
    d->qid.path = qpath(c, i, branchid(aux, aux->refpath), Qbranch | Internal);
    d->mode = 0555 | DMDIR;
    d->uid = estrdup9p(username);
    d->gid = estrdup9p(groupname);
    d->muid = estrdup9p(username);
    d->mtime = c->mtime;
    d->atime = c->mtime;
    if((n = slurpdir(aux->refpath, &refs)) < 0)
        return -1;
    if(i < n){
        d->name = estrdup9p(refs[i].name);
        free(refs);
        return 0;
    }else{
        free(refs);
        return -1;
    }
}
```

`<function branchid 74c>≡ (174)`

```
static vlong
branchid(Gitaux *aux, char *path)
{
    int i;

    for(i = 0; branches[i]; i++)
        if(strcmp(path, branches[i]) == 0)
            goto found;
    branches = realloc(branches, sizeof(char *)*(i + 2));
    branches[i] = estrdup(path);
    branches[i + 1] = nil;

found:
    if(aux){
        if(aux->refpath)
            free(aux->refpath);
        aux->refpath = estrdup(branches[i]);
    }
}
```

```

    return i;
}

⟨gitwalk1() switch QDIR(&fid->qid) cases 75a⟩+≡ (63a) <73c
case Qbranch:
    if(strcmp(aux->refpath, ".git/refs/heads") == 0 && strcmp(name, "HEAD") == 0)
        snprintf(path, sizeof(path), ".git/HEAD");
    else
        snprintf(path, sizeof(path), "%s/%s", aux->refpath, name);
    q->type = QTDIR;
    d = dirstat(path);
    if(d && d->qid.type == QTDIR)
        q->path = qpath(o, Qbranch, branchid(aux, path), Qbranch);
    else if(d && (c->obj = readref(path)) != nil)
        q->path = qpath(o, Qbranch, c->obj->id, Qcommit);
    else
        e = Eexist;
    if(d != nil)
        c->mode = d->mode & ~0222;
    free(d);
    break;

```

7.7 git/fs -m

```

⟨main() (fs.c) command line processing 75b⟩≡ (55b) 158b▷
case 'm':
    mntpt = EARGF(usage());
    break;

```

Chapter 8

git/query

8.1 Usage

```
<function usage (git9/query.c) 76a>≡ (229b)  
void  
usage(void)  
{  
    fprintf(STDERR, "usage: %s [-pcr] query\n", argv0);  
    exits("usage");  
}
```

8.2 Commit expressions

8.3 main()

```
<function main (git9/query.c) 76b>≡ (229b)  
void  
main(int argc, char **argv)  
{  
    char repo[512];  
    int nrel;  
    // ref_own<string>, question  
    char *query;  
    // list<ref_own<Hash>>, answer  
    Hash *h;  
    <main() (query.c) other locals 77a>  
  
    ARGBEGIN{  
        <main() (query.c) command line processing 86b>  
        default:    usage();    break;  
    }ARGEND;  
    if(argc == 0)  
        usage();  
  
    <main() (query.c) fmtinstall 77d>  
  
    gitinit(repo, sizeof(repo), &nrel);  
    // !! chdir !!  
    if(chdir(repo) == ERROR_NEG1)  
        sysfatal("chdir: %r");  
  
    <main() (query.c) set objpfx 86g>
```

```

    <main() (query.c) set query derived from argv 77b>
    n = resolverefs(&h, query);
    free(query);
    <main() (query.c) sanity check n result 77c>

    <main() (query.c) if changes 86c>
    else{
        p = "";
        <main() (query.c) adjust p if fullpath 86h>
        for(j = 0; j < n; j++) {
            i = j;
            <main() (query.c) adjust i if reverse 87b>
            print("%s%H\n", p, h[i]);
        }
    }
    exits(nil);
}

<main() (query.c) other locals 77a>≡ (76b) 86f▷
    char *p, *e;
    int i, j, n;

<main() (query.c) set query derived from argv 77b>≡ (76b)
    // p = concat(argv, " ")
    for(i = 0, n = 0; i < argc; i++)
        n += strlen(argv[i]) + 1;
    query = emalloc(n+1);
    p = query;
    e = query + n;
    for(i = 0; i < argc; i++)
        p = seprint(p, e, "%s ", argv[i]);

<main() (query.c) sanity check n result 77c>≡ (76b)
    if(n == ERROR_NEG1)
        sysfatal("resolve: %r");

<main() (query.c) fmtinstall 77d>≡ (76b)
    fmtinstall('P', Pfmt);

<function Pfmt 77e>≡ (229b)
    int
    Pfmt(Fmt *f)
    {
        int i, n;

        n = 0;
        for(i = 0; i < npath; i++)
            n += fmtprint(f, "%s/", path[i]);
        return n;
    }

<global path(query.c) 77f>≡ (229b)
    char *path[128];

<global npath(query.c) 77g>≡ (229b)
    int npath;

```

8.4 Data structures

8.4.1 Eval

```
<struct Eval 78a>≡ (232)
struct Eval {
    // ref<string> (owner = main() query)
    char    *str;
    // pointer in str as we parse it
    char    *p;

    // stack<ref_own<Object>>, len = nstk
    Object  **stk;
    int nstk;
    int stksz;
};
```

8.5 resolverefs()

```
<function resolverefs 78b>≡ (232)
// main(get.c) -> <>
errorneg1
resolverefs(Hash **r, char *ref)
{
    Eval ev;
    Hash *h;
    int i;
    errorneg1 res;

    res = evalexpr(&ev, ref);

    if(res == ERROR_NEG1){
        free(ev.stk);
        return ERROR_NEG1;
    }
    h = eamalloc(ev.nstk, sizeof(Hash));
    for(i = 0; i < ev.nstk; i++)
        h[i] = ev.stk[i]->hash;
    *r = h;
    free(ev.stk);
    return ev.nstk;
}
```

8.6 Commit expression evaluator

```
<function evalexpr 78c>≡ (232)
errorneg1
evalexpr(Eval *ev, char *ref)
{
    memset(ev, 0, sizeof(*ev));
    ev->str = ref;
    ev->p = ref;

    while(true){
        if(evalpostfix(ev) == ERROR_NEG1)
            return ERROR_NEG1;
    }
}
```

```

    if(ev->p[0] == '\0')
        return OK_0;
    else if(take(ev, ":") || take(ev, "..")){
        if(evalpostfix(ev) == ERROR_NEG1)
            return ERROR_NEG1;
        if(ev->p[0] != '\0'){
            werrstr("junk at end of expression");
            return ERROR_NEG1;
        }
        return range(ev);
    }
}
}
}

```

8.6.1 evalpostfix()

```

⟨function evalpostfix 79⟩≡ (232)
    errorneg1
    evalpostfix(Eval *ev)
    {
        char name[256];
        Hash h;
        Object *o;
        errorneg1 ret;

        eatspace(ev);
        if(!word(ev, name, sizeof(name))){
            werrstr("expected name in expression");
            return ERROR_NEG1;
        }
        ret = readref(&h, name);
        ⟨evalpostfix() sanity check ret invalid ref 80b⟩
        ⟨evalpostfix() if Zhash 80e⟩
        else {
            o = readobject(h);
            ⟨evalpostfix() sanity check o invalid ref 80c⟩
        }
        push(ev, o);

        while(true){
            eatspace(ev);
            switch(ev->p[0]){
                case '^':
                case '~':
                    ev->p++;
                    ret = parent(ev);
                    ⟨evalpostfix() sanity check ret 80d⟩
                    break;
                case '@':
                    ev->p++;
                    ret = lca(ev);
                    ⟨evalpostfix() sanity check ret 80d⟩
                    break;
                default:
                    goto done;
                    break;
            }
        }
    }
done:

```



```

    return OK_0;
}

⟨function eatspace 80a⟩≡ (232)
void
eatspace(Eval *ev)
{
    while(isspace(ev->p[0]))
        ev->p++;
}

⟨evalpostfix() sanity check ret invalid ref 80b⟩≡ (79)
if(ret == ERROR_NEG1){
    werrstr("invalid ref %s", name);
    return ret;
}

⟨evalpostfix() sanity check o invalid ref 80c⟩≡ (79)
if(o == nil){
    werrstr("invalid ref %s (hash %H)", name, h);
    return ERROR_NEG1;
}

⟨evalpostfix() sanity check ret 80d⟩≡ (79)
if(ret == ERROR_NEG1)
    return ret;

⟨evalpostfix() if Zhash 80e⟩≡ (79)
if(hashcq(&h, &Zhash))
    o = &zcommit;

```

8.6.2 word()

```

⟨function word 80f⟩≡ (232)
bool
word(Eval *ev, char *b, int nb)
{
    char *p, *e;
    int n;

    p = ev->p;
    for(e = p; isword(*e) && strcmp(e, "..", 2) != 0; e++)
        /* nothing */;
    /* 1 for nul terminator */
    n = e - p + 1;
    if(n >= nb)
        n = nb;
    snprintf(b, n, "%s", p);
    ev->p = e;
    return n > 0;
}

⟨function isword 80g⟩≡ (232)
bool
isword(char e)
{
    return isalnum(e) || e == '/' || e == '-' || e == '_' || e == '.';
}

```

8.6.3 push() and pop()

```
<function push 81a>≡ (232)
void
push(Eval *ev, Object *o)
{
    <push() realloc if needed 81b>
    ev->stk[ev->nstk++] = o;
}
```

```
<push() realloc if needed 81b>≡ (81a)
if(ev->nstk == ev->stksz){
    ev->stksz = 2*ev->stksz + 1;
    ev->stk = erealloc(ev->stk, ev->stksz*sizeof(Object*));
}
```

```
<function pop 81c>≡ (232)
Object*
pop(Eval *ev)
{
    <pop() sanity check empty stack 81d>
    return ev->stk[--ev->nstk];
}
```

```
<pop() sanity check empty stack 81d>≡ (81c)
if(ev->nstk == 0)
    sysfatal("stack underflow");
```

8.6.4 parent() and ^

```
<function parent 81e>≡ (232)
static errorneg1
parent(Eval *ev)
{
    Object *o, *p;

    o = pop(ev);
    <parent() sanity check o is a commit 81f>
    /* Special case: first commit has no parent. */
    if(o->commit->nparent == 0)
        p = emptydir();
    else {
        p = readobject(o->commit->parent[0]);
        <parent() sanity check p 81g>
    }
    push(ev, p);
    return OK_0;
}
```

```
<parent() sanity check o is a commit 81f>≡ (81e)
if(o->type != GCommit){
    werrstr("not a commit: %H", o->hash);
    return ERROR_NEG1;
}
```

```
<parent() sanity check p 81g>≡ (81e)
if (p == nil){
    werrstr("no parent for %H", o->hash);
    return ERROR_NEG1;
}
```

8.6.5 lca() and @

```
<function lca 82a>≡ (232)
errorneg1
lca(Eval *ev)
{
    Object *a, *b;
    // array<ref<Object>>
    Object **o;
    int n;

    if(ev->nstk < 2){
        werrstr("ancestor needs 2 objects");
        return ERROR_NEG1;
    }
    n = 0;
    b = pop(ev);
    a = pop(ev);
    paint(&a->hash, 1, &b->hash, 1, &o, &n, Lca);
    if(n == 0)
        return ERROR_NEG1;
    push(ev, *o);
    free(o);
    return OK_0;
}
```

```
<function ancestor 82b>≡ (232)
/// sendpack -> <>
Object*
ancestor(Object *a, Object *b)
{
    // array<ref<Object>>
    Object **o;
    Object *r;
    int n;

    if(paint(&a->hash, 1, &b->hash, 1, &o, &n, Lca) == ERROR_NEG1 || n == 0)
        return nil;
    r = ref(o[0]);
    free(o);
    return r;
}
```

```
<paint() switch mode cases 82c>≡ (84b) 83d▷
case Lca:
    dprint(1, "found ancestor\n");
    o = nil;
    for(i = 0; i < keep.sz; i++){
        o = keep.obj[i];
        if(o != nil && oshas(&drop, o->hash) && !oshas(&skip, o->hash))
            break;
    }
    if(i == keep.sz){
        *nres = 0;
        *res = nil;
    }else{
        *nres = 1;
        *res = eamalloc(1, sizeof(Object*));
        (*res)[0] = o;
    }
    break;
```

8.6.6 range() and ..

```
⟨function range 83a⟩≡ (232)
static errorneg1
range(Eval *ev)
{
    Object *a, *b;
    // array<ref<Object>>
    Object **o;
    int i, n;

    b = pop(ev);
    a = pop(ev);
    ⟨range() if b has Zhash 83b⟩
    ⟨range() if a has Zhash 83c⟩
    if(a->type != GCommit || b->type != GCommit){
        werrstr("non-commit object in range");
        return ERROR_NEG1;
    }

    if(paint(&b->hash, 1, &a->hash, 1, &o, &n, Range) == ERROR_NEG1)
        return ERROR_NEG1;
    for(i = 0; i < n; i++)
        push(ev, o[i]);
    free(o);
    return OK_0;
}
```

```
⟨range() if b has Zhash 83b⟩≡ (83a)
if(hasheq(&b->hash, &Zhash))
    b = &zcommit;
```

```
⟨range() if a has Zhash 83c⟩≡ (83a)
if(hasheq(&a->hash, &Zhash))
    a = &zcommit;
```

```
⟨paint() switch mode cases 83d⟩+≡ (84b) ◁82c 230d▷
case Range:
    dprint(1, "found range\n");
    *res = eamalloc(nrange, sizeof(Object*));
    *nres = 0;
    for(i = nrange - 1; i >= 0; i--){
        o = range[i];
        if(!oshas(&drop, o->hash) && !oshas(&skip, o->hash)){
            (*res)[*nres] = o;
            (*nres)++;
        }
    }
    free(range);
    break;
```

8.7 paint()

```
⟨enum PaintMode 83e⟩≡ (232)
enum PaintMode {
    Lca,
    Range,
    ⟨PaintMode other cases 230c⟩
};
```

<enum PaintQcolor 84a>≡ (232)

```
enum PaintQcolor {
    Blank,

    Keep,
    Drop,
    Skip,
};
```

<function paint 84b>≡ (232)

```
/// lca | ancestor | range -> <>
static errorneg1
paint(Hash *head, int nhead, Hash *tail, int ntail, Object ***res, int *nres, int mode)
{
    int i;
    Objset keep, drop, skip;
    Objq objq;
    Qelt e;
    Object *o, *c;
    Object **range;
    int nrange;
    int nskip;

    osinit(&keep);
    osinit(&drop);
    osinit(&skip);
    qinit(&objq);
    range = nil;
    nrange = 0;
    nskip = 0;

    for(i = 0; i < nhead; i++){
        if(hasheq(&head[i], &Zhash))
            continue;
        o = readobject(head[i]);
        if(o == nil){
            fprintf(STDERR, "warning: %H does not point at commit\n", head[i]);
            werrstr("read head %H: %r", head[i]);
            return ERROR_NEG1;
        }
        if(o->type != GCommit){
            fprintf(STDERR, "warning: %H does not point at commit\n", o->hash);
            unref(o);
            continue;
        }
        dprint(1, "init: keep %H\n", o->hash);
        qput(&objq, o, Keep);
        unref(o);
    }
    for(i = 0; i < ntail; i++){
        if(hasheq(&tail[i], &Zhash))
            continue;
        o = readobject(tail[i]);
        if(o == nil){
            werrstr("read tail %H: %r", tail[i]);
            return ERROR_NEG1;
        }
        if(o->type != GCommit){
            fprintf(STDERR, "warning: %H does not point at commit\n", o->hash);
            unref(o);
        }
    }
}
```

```

        continue;
    }
    dprint(1, "init: drop %H\n", o->hash);
    qput(&objq, o, Drop);
    unref(o);
}

dprint(1, "finding twixt commits\n");
while(objq.nheap != 0 && qpop(&objq, &e)){
    if(e.color == Skip)
        nskip--;
    if(oshas(&skip, e.o->hash))
        continue;
    // else

    switch(e.color){
    case Keep:
        if(oshas(&keep, e.o->hash))
            continue;
        if(oshas(&drop, e.o->hash))
            e.color = Skip;
        else if(mode == Range){
            range = earealloc(range, nrange+1, sizeof(Object*));
            range[nrange++] = e.o;
        }
        osadd(&keep, e.o);
        break;
    case Drop:
        if(oshas(&drop, e.o->hash))
            continue;
        if(oshas(&keep, e.o->hash))
            e.color = Skip;
        osadd(&drop, e.o);
        break;
    case Skip:
        osadd(&skip, e.o);
        break;
    }
    o = readobject(e.o->hash);
    if(o->type != GCommit){
        werrstr("not a commit: %H", o->hash);
        goto error;
    }

    for(i = 0; i < o->commit->nparent; i++){
        c = readobject(e.o->commit->parent[i]);
        if(c == nil)
            goto error;
        if(c->type != GCommit){
            fprintf(STDERR, "warning: %H does not point at commit\n", c->hash);
            unref(c);
            continue;
        }
        dprint(2, "\tenqueue: %s %H\n", colors[e.color], c->hash);
        qput(&objq, c, e.color);
        unref(c);
        if(e.color == Skip)
            nskip++;
    }
    unref(o);
}

```

```

}

switch(mode){
  <paint() switch mode cases 82c>
}
osclear(&keep);
osclear(&drop);
osclear(&skip);
return OK_0;
error:
  dprint(1, "paint error: %r\n");
  free(objq.heap);
  free(range);
  return ERROR_NEG1;
}

```

8.8 Advanced features

8.8.1 git/query -c, list changes between commits

```

<global changes(query.c) 86a>≡ (229b)
  bool changes;

<main() (query.c) command line processing 86b>≡ (76b) 86e▷
  case 'c':  changes=true;  break;

<main() (query.c) if changes 86c>≡ (76b)
  if(changes){
    for(i = 1; i < n; i++)
      diffcommits(h[0], h[i]);
  }

```

8.8.2 git/query -p, full path listing

```

<global fullpath(query.c) 86d>≡ (229b)
  bool fullpath;

<main() (query.c) command line processing 86e>+≡ (76b) <86b 87a▷
  case 'p':  fullpath=true;  break;

<main() (query.c) other locals 86f>+≡ (76b) <77a
  char *objpfx;

<main() (query.c) set objpfx 86g>≡ (76b)
  if((objpfx = smprint("%s/.git/fs/object/", repo)) == nil)
    sysfatal("smprint: %r");

<main() (query.c) adjust p if fullpath 86h>≡ (76b)
  if (fullpath)
    p = objpfx;

```

8.8.3 git/query -r, reversed listing

```

<global reverse(query.c) 86i>≡ (229b)
  bool reverse;

```

$\langle \text{main()} (query.c) \text{ command line processing } 87a \rangle + \equiv$
`case 'r': reverse ^= 1; break;`

$(76b) \triangleleft 86e \ 159 \triangleright$

$\langle \text{main()} (query.c) \text{ adjust i if reverse } 87b \rangle \equiv$
`if (reverse)`
`i = n - 1 - j;`

$(76b)$

Chapter 9

git/walk

9.1 Usage

```
<function usage (git9/walk.c) 88a>≡ (265b)
void
usage(void)
{
    fprintf(STDERR, "usage: %s [-qbcI] [-f filt] [-b base] [paths...]\n", argv0);
    exits("usage");
}
```

9.2 main()

```
<global wdirpath(walk.c) 88b>≡ (265b)
char    wdirpath[1024];
```

```
<function main (git9/walk.c) 88c>≡ (265b)
void
main(int argc, char **argv)
{
    char *p, *e, *ln, *base, **argrel, *parts[4], xbuf[8];
    int i, j, c, line, *argn;
    fdt wfd;
    Biobuf *f, *o, *w;
    Hash h;
    Dir rn;

    gitinit(repopath, sizeof(repopath), &nrel);

    if(getwd(wdirpath, sizeof(wdirpath)) == nil)
        sysfatal("getwd: %r");

    // !! chdir !!
    if(chdir(repopath) == ERROR_NEG1)
        sysfatal("chdir: %r");

    if(access(".git/fs/ctl", AEXIST) != 0)
        sysfatal("no running git/fs");

    ARGBEGIN{
    case 'q':
        quiet++;
        break;
```

```

case 'c':
    rstr = "";
    tstr = "";
    mstr = "";
    astr = "";
    ustr = "";
    break;
case 'f':
    for(p = EARGF(usage()); *p; p++)
        switch(*p){
            case 'T':    printfg |= Tflg;    break;
            case 'A':    printfg |= Aflg;    break;
            case 'M':    printfg |= Mflg;    break;
            case 'R':    printfg |= Rflg;    break;
            case 'U':    printfg |= Uflg;    break;
            default:     usage();             break;
        }
    break;
case 'b':
    isindexed = 0;
    base = EARGF(usage());
    if(resolveref(&h, base) == -1)
        sysfatal("no such ref '%s'", base);
    bdir = smprint(".git/fs/object/%H/tree", h);
    break;
case 'I':
    /* invalidate index */
    staleidx = 1;
    break;
case 'r':
    findslashes(EARGF(usage()));
    break;
default:
    usage();
}ARGEND;

if(printfg == 0)
    printfg = Tflg | Aflg | Mflg | Rflg;

argrel = emalloc(argc*sizeof(char*));
argn = emalloc(argc*sizeof(int));
for(i = 0; i < argc; i++){
    argrel[i] = reporel(argv[i]);
    argn[i] = strlen(argrel[i]);
}

if(isindexed && !staleidx){
    if((f = Bopen(".git/INDEX9", OREAD)) == nil){
        if(access(".git/index9", AEXIST) == 0){
            fprintf(2, "index format conversion needed:\n");
            fprintf(2, "\tcd %s && git/fs\n", repopath);
            fprintf(2, "\t@{cd .git/index9/removed >[2]/dev/null && walk -f | sed 's/^/R NOQID 0 /'} >> .git\n");
            fprintf(2, "\t@{cd .git/fs/HEAD/tree && walk -f | sed 's/^/T NOQID 0 /'} >> .git/INDEX9\n");
            exits("noindex");
        }
        staleidx = 1;
        goto Stale;
    }
}

nidx = 0;

```

```

idxsz = 32;
idx = emalloc(idxsz*sizeof(Idxxent));

line = 0;
while((ln = Brdstr(f, '\n', 1)) != nil){
    line++;
    /* allow blank lines */
    if(ln[0] == 0 || ln[0] == '\n')
        continue;
    if(getfields(ln, parts, nelem(parts), 0, " \t") != nelem(parts))
        sysfatal(".git/INDEX9:%d: corrupt index", line);
    cleannname(parts[3]);
    if(nidx == idxsz){
        idxsz += idxsz/2;
        idx = erealloc(idx, idxsz*sizeof(Idxxent));
    }
    idx[nidx].state = *parts[0];
    idx[nidx].qid = parseqid(parts[1]);
    idx[nidx].mode = strtol(parts[2], nil, 8);
    idx[nidx].path = estrdup(parts[3]);
    idx[nidx].order = nidx;
    nidx++;
    free(ln);
}
Bterm(f);
} else {
Stale:
    nwdir = 0;
    wdirs = 32;
    wdir = emalloc(wdirs*sizeof(Idxxent));

    if(chdir(bdir) == -1)
        sysfatal("chdir: %r");

    /* load whole tree into index when stale */
    intree = 1;
    if(staleidx || argc == 0)
        loadwdir(".");
    else for(i = 0; i < argc; i++)
        loadwdir(argrel[i]);

    if(chdir(repopath) == -1)
        sysfatal("chdir: %r");

    /* use as index */
    idx = wdir;
    nidx = nwdir;
    idxsz = wdirs;

    cleanidx = 1;
}
qsort(idx, nidx, sizeof(Idxxent), idxcmp);

nwdir = 0;
wdirs = 32;
wdir = emalloc(wdirs*sizeof(Idxxent));

intree = 0;
if(argc == 0)
    loadwdir(".");

```

```

else for(i = 0; i < argc; i++)
    loadwdir(argrel[i]);
qsort(wdir, nwdir, sizeof(Idxent), idxcmp);

if((o = Bfdopen(1, OWRITE)) == nil)
    sysfatal("open out: %r");

i = 0;
j = 0;
while(i < nidx || j < nwdir){
    /* find the last entry we tracked for a path */
    while(i+1 < nidx && strcmp(idx[i].path, idx[i+1].path) == 0){
        staleidx = 1;
        i++;
    }
    while(j+1 < nwdir && strcmp(wdir[j].path, wdir[j+1].path) == 0)
        j++;
    if(i < nidx && !pfxmatch(idx[i].path, argrel, argn, argc)){
        i++;
        continue;
    }
    if(i >= nidx)
        c = 1;
    else if(j >= nwdir)
        c = -1;
    else
        c = strcmp(idx[i].path, wdir[j].path);
    /* exists in both index and on disk */
    if(c == 0){
        if(idx[i].state == 'R'){
            if(checkedin(&idx[i], 0))
                show(o, Rflg, rstr, idx[i].path);
            else{
                idx[i].state = 'U';
                staleidx = 1;
            }
        }else if(idx[i].state == 'A' && !checkedin(&idx[i], 1))
            show(o, Aflg, astr, idx[i].path);
        else if(!samedata(&idx[i], &wdir[j]))
            show(o, Mflg, mstr, idx[i].path);
        else
            show(o, Tflg, tstr, idx[i].path);
        i++;
        j++;
    }
    /* only exists in index */
    }else if(c < 0){
        if(checkedin(&idx[i], 0))
            show(o, Rflg, rstr, idx[i].path);
        else{
            idx[i].state = 'U';
            staleidx = 1;
        }
        i++;
    }
    /* only exists on disk */
    }else{
        if(checkedin(&wdir[j], 0)){
            if(samedata(nil, &wdir[j]))
                show(o, Tflg, tstr, wdir[j].path);
            else
                show(o, Mflg, mstr, wdir[j].path);
        }
    }
}

```

```

        }else if(printflg & Uflg && pfxmatch(wdir[j].path, argrel, argn, argc))
            show(o, Uflg, ustr, wdir[j].path);
        j++;
    }
}
Bterm(o);

if(isindexed && staleidx)
if((wfd = create(".git/INDEX9.new", OWRITE, 0644)) != -1){
    if((w = Bfdopen(wfd, OWRITE)) == nil){
        close(wfd);
        goto Nope;
    }
    for(i = 0; i < nidx; i++){
        while(i+1 < nidx && strcmp(idx[i].path, idx[i+1].path) == 0)
            i++;
        if(idx[i].state == 'U')
            continue;
        Bprint(w, "%c %Q %o %s\n",
            idx[i].state,
            idx[i].qid,
            idx[i].mode,
            idx[i].path);
    }
    Bterm(w);
    nulldir(&rn);
    rn.name = "INDEX9";
    if(remove(".git/INDEX9") == -1)
        if(access(".git/INDEX9", AEXIST) == 0)
            goto Nope;
    if(dirwstat(".git/INDEX9.new", &rn) == -1)
        sysfatal("rename: %r");
}

Nope:
    if(!dirty)
        exits(nil);

    p = xbuf;
    e = p + sizeof(xbuf);
    for(i = 0; (1 << i) != Tflg; i++)
        if(dirty & (1 << i))
            p = seprint(p, e, "%c", "RMAUT"[i]);
    *p = '\0';
    exits(xbuf);
}

```

9.3 Advanced features

Part II

Basic Commands

Chapter 10

Creating a Repository

10.1 Initializing a new repository: `git init`

10.1.1 `init.rc`

```
<git9/init.rc 94a>≡
#!/bin/rc -e
rfork ne
. /lib/git/common.rc

<init.rc command line processing 94c>

dir=$1
if(~ $#dir 0)
    dir=.
if(~ $#branch 0)
    branch=master
if(test -e $dir/.git)
    die $dir/.git already exists
<init.rc set $name derived from $dir 94b>
<init.rc check $upstream flag 121a>

mkdir -p $dir/.git/refs/^(heads remotes)
mkdir -p $dir/.git/^(fs objects)
>$dir/.git/config {
    echo '[core]'
    echo ' repositoryformatversion = p9.0'
    <init.rc set origin in .git/config 121b>
    echo '[branch "'$branch'"]'
    echo ' remote = origin'
}
>$dir/.git/INDEX9
>$dir/.git/HEAD {
    echo ref: refs/heads/$branch
}
exit ''

<init.rc set $name derived from $dir 94b>≡ (94a)
name='{basename '{cleannname -d '{pwd} $dir}}
```

10.1.2 Usage

```
<init.rc command line processing 94c>≡ (94a)
flagfmt='u:upstream upstream,b:branch branch'; args='name'
```

```
eval '{auxrc/getflags $*} || exec auxrc/usage
```

10.2 Copying a local repository: `cp -r`

10.3 Cloning a remote repository: `git clone`

Chapter 11

Staging a Diff

11.1 Adding files: git add

11.1.1 Usage

```
<add.rc command line processing 96a>≡ (96b)
    flagfmt='r:remove'; args='file ...'
    eval '{auxrc/getflags $*} || exec auxrc/usage
```

11.1.2 add.rc

```
<git9/add.rc 96b>≡
    #!/bin/rc -e
    rfork ne
    . /lib/git/common.rc

    # for setting $gitroot, $gitrel
    gitup

    <add.rc command line processing 96a>

    s=A

    <add.rc if $remove flag 97e>
    <add.rc sanity check $* 96c>

    <add.rc set $paths derived from $* 97b>
    walk -f ./ $paths | grep -v '^(/)?\.git/' | \
        sed 's/^/'$s' NOQID 0 /' >> .git/INDEX9
    exit ''

    <add.rc sanity check $* 96c>≡ (96b)
    if(~ $#* 0)
        exec auxrc/usage
```

11.1.3 gitup() part 1

```
<fn gitup(common.rc) 96d>≡ (165a)
    fn gitup{
        gitroot='${nl{git/conf -r >[2]/dev/null}'
        if(~ $#gitroot 0)
            die 'not a git repository'
        gitrel='${nl{pwd | drop $gitroot | sed 's@~/@@'}}
```

```

        if(~ $#gitrel 0)
            gitrel='.'
        if(! cd $gitroot)
            die cd $gitroot: no repo there

        <gitup() (common.rc) start git/fs 99c>
    }

    <global nl (common.rc) 97a>≡ (165a)
        nl='
        ,

    <add.rc set $paths derived from $* 97b>≡ (96b)
        paths='$nl{cleannname -d $gitrel $* | drop $gitroot}

```

11.1.4 drop()

```

    <fn drop (common.rc) 97c>≡ (165a)
        fn drop {
            awk '
            BEGIN{ARGC=0}
            {
                if(index($0, ARGV[1]) == 1)
                    $0=substr($0, length(ARGV[1])+1)
                print
            }
            ' $*
        }

```

11.2 Removing files: git rm

```

    <git9/rm.rc 97d>≡
        #!/bin/rc -e

        exec git/add -r $*

    <add.rc if $remove flag 97e>≡ (96b)
        if(~ $remove 1)
            s=R

```

Chapter 12

Committing a Diff

12.1 Committing the index: git commit

12.1.1 Usage

```
<commit.rc command line processing 98a>≡ (98b)
    flagfmt='m:msg message, r:revise, e:edit, p:partial'; args='[file ...]'
    eval '{auxrc/getflags $*} || exec auxrc/usage
```

12.1.2 commit.rc

```
<git9/commit.rc 98b>≡
    #!/bin/rc -e
    rfork ne
    . /lib/git/common.rc

    <fn findbranch(commit.rc) 99d>
    <fn cleanmsg(commit.rc) 101a>
    <fn editmsg(commit.rc) 100e>
    <fn parents(commit.rc) 100b>
    <fn commit(commit.rc) 101c>
    <fn update(commit.rc) 101d>
    <fn filterdiff(commit.rc) 105a>
    <fn pickhunks(commit.rc) 104d>
    <fn apply(commit.rc) 105b>

    gitup

    <commit.rc command line processing 98a>

    <commit.rc create $msgfile.tmp 100d>

    files=()
    <commit.rc adjust $files if merge parents 112b>
    if(! ~ $#* 0)
        files=($files '$nl{git/walk -fRMA -c '$nl{cleannname -d $gitrel $*}}')
    if(~ $status '' || ~ $#files 0 && ! test -f .git/merge-parents && ~ $#revise 0)
        die 'nothing to commit'
    <commit.rc if $partial 104c>

    @{
        flag e +

        # prepare the commit
```

```

    whoami
    findbranch
    parents
    editmsg

    commit
    update
} || die 'could not commit:' $status
exit ''

```

12.1.3 whoami()

<fn whoami(common.rc) 99a> ≡ (165a)

```

fn whoami{
    name='${nl}{git/conf user.name}
    email='${nl}{git/conf user.email}
    <whomai() (common.rc) if adm keys 99b>
    if(~ $name '')
        name=$user
    if(~ $email '')
        email=$user@$sysname
    status=''
}

```

<whomai() (common.rc) if adm keys 99b> ≡ (99a)

```

if(test -f /adm/keys.who){
    if(~ $name '')
        name='${nl}{awk -F'|' ' $1=="$user" {x=$3} END{print x}' </adm/keys.who}
    if(~ $email '')
        email='${nl}{awk -F'|' ' $1=="$user" {x=$5} END{print x}' </adm/keys.who}
}

```

12.1.4 gitup() part 2

<gitup() (common.rc) start git/fs 99c> ≡ (96d)

```

gitfs=$gitroot/.git/fs
startfs=()
mkdir -p $gitfs
if(! test -e $gitfs/ctl)
    startfs=true
if(! grep -s '^repo '$gitroot'$' $gitfs/ctl >[2]/dev/null)
    startfs=true
if(~ $#startfs 1)
    git/fs >[1=2]
if not
    status=''

```

12.1.5 findbranch()

<fn findbranch(commit.rc) 99d> ≡ (98b)

```

fn findbranch{
    branch='{git/branch}
    <findbranch() (commit.rc) set $refpath and $initial 100a>
}

```

```

<findbranch() (commit.rc) set $refpath and $initial 100a>≡ (99d)
  if(test -e $gitfs/branch/$branch/tree){
    refpath=.git/refs/$branch
    initial=false
  }
  if not if(test -e $gitfs/object/$branch/tree){
    refpath=.git/HEAD
    initial=false
  }
  if not if(! test -e $gitfs/HEAD/tree){
    refpath=.git/refs/$branch
    initial=true
  }
  if not
    die 'invalid branch:' $branch

```

12.1.6 parents()

```

<fn parents(commit.rc) 100b>≡ (98b)
  fn parents{
    <parents() (commit.rc) if revise 104a>
    <parents() (commit.rc) if merge parents 112c>
    <parents() (commit.rc) if initial 100c>
    if not
      parents='{git/query $branch}'
  }

```

```

<parents() (commit.rc) if initial 100c>≡ (100b)
  if not if(~ $initial true)
    parents=()

```

12.1.7 editmsg()

```

<commit.rc create $msgfile.tmp 100d>≡ (98b)
  wdir=/mnt/git
  ramfs -m $wdir
  msgfile=$wdir/git-msg.$pid
  if(~ $#msg 1)
    echo $msg >$msgfile.tmp
  <commit.rc when create $msgfile.tmp, else if revise 104b>

```

```

<fn editmsg(commit.rc) 100e>≡ (98b)
  fn editmsg{
    if(! test -s $msgfile.tmp){
      >$msgfile.tmp {
        echo '# Author:' $name '<'$email'>'
        echo '#'
        for(p in $parents)
          echo '# parent:' $p
        git/walk -fAMR $files | subst '~' '#'
        echo '#'
        echo '# Commit message:'
      }
      edit=1
    }
    if(! ~ $#edit 0){
      giteditor='{git/conf core.editor}'
      if(~ $#editor 0)

```

```

        editor=$giteditor
        if(~ $#editor 0)
            editor=hold
        $editor $msgfile.tmp
    }
    cleanmsg < $msgfile.tmp > $msgfile
    if(! test -s $msgfile)
        die 'empty commit message'
}

⟨fn cleanmsg(commit.rc) 101a⟩≡ (98b)
# Remove commentary lines.
# Remove leading and trailing empty lines.
# Combine consecutive empty lines between paragraphs.
# Remove trailing spaces from lines.
# Ensure there's trailing newline.
fn cleanmsg{
    awk '
    /^[ \t]*$/ {next}
    /^[ \t]*$/ {empty = 1; next}

    wet && empty {printf "\n"}
    {wet = 1; empty = 0}
    {sub(/[\t]+$/, ""); print $0}
    ,
}

⟨fn subst(common.rc) 101b⟩≡ (165a)
fn subst {
    awk '
    BEGIN{ARGC=0}
    {sub(ARGV[1], ARGV[2]); print}
    ' $*
}

```

12.1.8 commit()

```

⟨fn commit(commit.rc) 101c⟩≡ (98b)
fn commit{
    msg='' {cat $msgfile}
    if(! ~ $#parents 0)
        pflags='-p'~$parents
    hash='{git/save -n "$name -e "$email -m "$msg $pflags $files || die $status}
}

```

12.1.9 update()

```

⟨fn update(commit.rc) 101d⟩≡ (98b)
fn update{
    mkdir -p '{basename -d $refpath}
    # Paranoia: let's not mangle the repo.
    if(~ $#hash 0)
        die 'botched commit'
    rm -f .git/merge-parents
    echo $branch: $hash
    echo $hash > $refpath
    for(f in $files){
        if(! test -e $f && ! test -e .git/object/$hash/tree/$f)

```

```

        echo R NOQID 0 $f >> .git/INDEX9
    if not
        echo T NOQID 0 $f >> .git/INDEX9
}
}

```

12.2 git/walk -fRMA

12.3 git/query \$branch

12.4 git/save

12.4.1 Usage

```

⟨function usage (git9/save.c) 102a⟩≡ (239c)
void
usage(void)
{
    fprintf(STDERR, "usage: %s -n name -e email -m message -d date [files...]\n", argv0);
    exits("usage");
}

```

12.4.2 main()

```

⟨function main (git9/save.c) 102b⟩≡ (239c)
void
main(int argc, char **argv)
{
    char *ln, *dstr, *parts[4], cwd[1024];
    int i, r, line, ncwd;
    Hash th, ch;
    vlong date;
    Biobuf *f;
    Object *t;

    gitinit(nil, 0, nil);

    if(access(".git", AEXIST) != 0)
        sysfatal("could not find git repo: %r");
    if(getwd(cwd, sizeof(cwd)) == nil)
        sysfatal("getcwd: %r");
    dstr = nil;
    date = time(nil);
    ncwd = strlen(cwd);

    ARGBEGIN{
    case 'm':
        commitmsg = EARGF(usage());
        break;
    case 'n':
        authorname = EARGF(usage());
        break;
    case 'e':
        authoremail = EARGF(usage());
        break;
}

```

```

case 'N':
    committername = EARGF(usage());
    break;
case 'E':
    committeremail = EARGF(usage());
    break;
case 'd':
    dstr = EARGF(usage());
    break;
case 'p':
    if(nparents >= Maxparents)
        sysfatal("too many parents");
    if(resolveref(&parents[nparents++], EARGF(usage())) == -1)
        sysfatal("invalid parent: %r");
    break;
default:
    usage();
    break;
}ARGEND;

if(commitmsg == nil)
    sysfatal("missing message");
if(authorname == nil)
    sysfatal("missing name");
if(authoremail == nil)
    sysfatal("missing email");
if((committername == nil) != (committeremail == nil))
    sysfatal("partially specified committer");
if(committername == nil && committeremail == nil){
    committername = authorname;
    committeremail = authoremail;
}
if(dstr){
    date=strtoll(dstr, &dstr, 10);
    if(strlen(dstr) != 0)
        sysfatal("could not parse date %s", dstr);
}
for(i = 0; i < argc; i++){
    cleannname(argv[i]);
    if(*argv[i] == '/' && strncmp(argv[i], cwd, ncwd) == 0)
        argv[i] += ncwd;
    while(*argv[i] == '/')
        argv[i]++;
}
qsort(argv, argc, sizeof(*argv), namecmp);

t = findroot();
nidx = 0;
idxsz = 32;
idx = emalloc(idxsz*sizeof(Idxxent));
if((f = Bopen(".git/INDEX9", OREAD)) == nil)
    sysfatal("open index: %r");
line = 0;
while((ln = Brdstr(f, '\n', 1)) != nil){
    line++;
    if(ln[0] == 0 || ln[0] == '\n')
        continue;
    if(getfields(ln, parts, nelem(parts), 0, "\t") != nelem(parts))
        sysfatal(".git/INDEX9:%d: corrupt index", line);
    if(nidx == idxsz){

```



```

        idxsz += idxsz/2;
        idx = realloc(idx, idxsz*sizeof(Idxsent));
    }
    cleannname(parts[3]);
    idx[nidx].state = *parts[0];
    idx[nidx].qid = parseqid(parts[1]);
    idx[nidx].mode = strtol(parts[2], nil, 8);
    idx[nidx].path = strdup(parts[3]);
    idx[nidx].order = nidx;
    nidx++;
    free(ln);
}
Bterm(f);
qsort(idx, nidx, sizeof(Idxsent), idxcmp);
r = treeify(t, argv, argv + argc, 0, &th);
if(r == -1)
    sysfatal("could not commit: %r");
mkcommit(&ch, date, th);
print("%H\n", ch);
exits(nil);
}

```

12.5 Advanced features

12.5.1 git/commit -r, revising a commit

```

<parents() (commit.rc) if revise 104a>≡ (100b)
    if(! ~ $#revise 0)
        parents='{cat $gitfs/HEAD/parent}

<commit.rc when create $msgfile.tmp, else if revise 104b>≡ (100d)
    if not if(~ $#revise 1){
        msg=1
        echo revising commit '{cat $gitfs/HEAD/hash}
        cat $gitfs/HEAD/msg >$msgfile.tmp
    }

```

12.5.2 git/commit -p, selecting hunks

```

<commit.rc if $partial 104c>≡ (98b)
    if(~ $#partial 1){
        for(i in $files)
            pickhunks $i
        apply
    }

<fn pickhunks (commit.rc) 104d>≡ (98b)
    fn pickhunks{
        p=$wdir/patch
        if(test -f .git/fs/HEAD/tree/$i)
            #<[12]/fd/0 >[13]/fd/1 { diff -u .git/fs/HEAD/tree/$1 $1 | filterdiff >>$p }
            { diff -u .git/fs/HEAD/tree/$1 $1 | filterdiff >>$p } || status=''
        if not
            diff -u /dev/null $1 >>$p || status=''
    }

```

`<fn filterdiff(commit.rc) 105a>≡` (98b)

```
fn filterdiff{
    awk '
        function dump(){
            if(n == 0)
                return

            print "--- a/"f >>"/dev/cons"
            print "+++ b/"f >>"/dev/cons"
            for(i = 0; i < n; i++)
                print lines[i] >>"/dev/cons"
            printf("Commit this hunk?[n] ") >>"/dev/cons"
            getline ans < "/dev/cons"
            if(ans ~ /[yY]/)
                for(i = 0; i < n; i++)
                    print lines[i]

            n = 0
        }
        /^---/{ next }
        /^\\+\\+\\+/{
            f = substr($0, 5)
            print "--- "f
            print "+++ "f
            n = 0
            next
        }
        /^@@/{ dump() }
        { lines[n++] = $0 }
        END{ dump() }
    ,
}
}
```

`<fn apply(commit.rc) 105b>≡` (98b)

```
fn apply{
    t=$wdir/tree/
    p=$wdir/patch
    ln='{wc -l $p}'
    if(test $ln(1) -lt 3)
        die 'no hunks seleted'
    f='{patch -n $p}'
    mkdir $t
    @{{cd .git/fs/HEAD/tree; tar c $f >[2]/dev/null} | @{{cd $t; tar xT}}
    for(i in $f){
        d='{basename -d $i}'
        if(! ~ $d .){
            mkdir -p $t^$d
            bind -bc $t^$d $d
        }
    }
    bind -bc $t .
    patch <$p >/dev/null || die 'failed partial patch: '$status
}
}
```

Chapter 13

Branching

13.1 git branch

13.1.1 Usage

```
<branch.rc command line processing 106a>≡ (106b)
  flagfmt='a:listall, b:baseref ref, r:remove, n:newbr, s:stay, m:merge, M:nomod'
  args='[branch]'
  eval '{auxrc/getflags $*} || exec auxrc/usage
```

13.1.2 branch.rc

```
<git9/branch.rc 106b>≡
  #!/bin/rc -e
  rfork en
  . /lib/git/common.rc

  gitup

  <branch.rc command line processing 106a>

  modified=()
  deleted=()
  failed=()
  if(~ $#* 0){
    if(~ $#listall 0)
      awk ' $1=="branch"{print $2}' < $gitfs/ctl
    if not
      cd .git/refs/ && walk -f heads remotes | sort
    exit
  }
  if(! ~ $#* 1)
    exec auxrc/usage

  branch=$1
  if(~ $branch refs/heads/*)
    new=$name
  if not if(~ $branch heads/*)
    new=refs/$branch
  if not
    new=refs/heads/$branch

  # where are we now?
  orig='{git/query HEAD}
```

```

origbranch=refs/{awk ' $1=="branch"{print $2}' < $gitfs/ctl}

# if we're switching branches to a branch that doesn't exist,
# but we have a remote branch that exists upstream, create a
# new head to mirror it.
if(~ $#newbr 0){
    if(! ~ $#baseref 0)
        die update would clobber $branch with $baseref
    if(! test -e .git/$new){
        baseref='${nl{echo -n $new | sed s@refs/heads/@refs/remotes/origin/@}
        if(! base='{git/query $baseref})
            exit 'bad ref'
    }
}

# figure out where we want the new branch to point to
if (~ $#baseref 1)
    base='{git/query $baseref} || exit 'bad ref'
if not if(~ $#newbr 0)
    base='{git/query $new} || exit 'bad ref'
if not
    base='{git/query HEAD} || exit 'bad ref'

modified='${nl{git/query -c HEAD $base | grep '^[-]' | subst '^..' }
deleted='${nl{git/query -c HEAD $base | grep '^-' | subst '^..' }

# if we remove the current branch without switching, bad things happen
if(~ $remove 1 && ~ $origbranch $new)
    die 'cannot remove current branch'
# if we're not merging, don't clobber existing changes.
if(~ $#merge 0 && ~ $#remove 0){
    if(! ~ $#modified 0 || ! ~ $#deleted 0){
        git/walk -fRMA $modified $deleted ||
            die 'uncommitted changes would be clobbered'
    }
}

if(~ $remove 1){
    rm -f .git/$new
    echo 'removed branch' $new
    exit
}

commit='{git/query $base} || die 'branch does not exist:' $base
basedir='{git/query -p $base}
dirtypaths=()
if(! ~ $#modified 0 || ! ~ $#deleted 0)
    dirtypaths='${nl{git/walk -cfRMA $modified $deleted}

>>[3].git/INDEX9{
    for(d in $deleted){
        if(! test -d $d){
            rm -f $d
            echo R NOQID 0 $d >>[1=3]
        }
    }
    for(m in $modified $deleted) if(! ~ $m $dirtypaths){
        d='${nl{basename -d $m}
        mkdir -p $d
        # Modifications can turn a file into
        # a directory, or vice versa, so we
        # need to delete and copy the files

```

```

# over.
a=dir
b=dir
if(test -f $m)
    a=file
if(test -f $basedir/tree/$m)
    b=file
if(! ~ $a $b){
    rm -rf $m
    echo R NOQID 0 $m >>[1=3]
}
if(~ $b file){
    echo T NOQID 0 $m >>[1=3]
    cp -x -- $basedir/tree/$m $m || {echo cp failed: $m; failed=1}
    touch -c $m
}
}
}
modified=()
deleted=()

for(ours in $dirtypaths){
    common=$gitfs/object/$orig/tree/$ours
    theirs=$gitfs/object/$base/tree/$ours
    merge1 $ours $ours $common $theirs
    st=$status
    if(! ~ $st '')
        >[1=2] echo merge failed $ours: $st
}
dirtypaths=()

if(~ $new */)
    mkdir -p .git/{basename -d $new}
echo $commit > .git/$new
if(! ~ $#stay 0)
    exit
if(~ $failed 1){
    echo 'pull failed: fix errors and try again' >[1=2]
    exit nopull
}
echo ref: $new > .git/HEAD
echo $new: '{git/query $new}'
exit ''

```

- 13.2 Listing branches: `git branch`
- 13.3 Creating a branch: `git branch <name>`
- 13.4 Deleting a branch: `git branch -d <name>`
- 13.5 Checking out a branch: `git checkout`
 - 13.5.1 Computing the index (and work tree) from trees
 - 13.5.2 Creating a file from a blob
- 13.6 Resetting a branch: `git reset`

Chapter 14

Merging

14.1 merging branches: git merge

14.1.1 Usage

```
<merge.rc command line processing 110a>≡ (110b)
    flagfmt=''; args='theirs'
    eval '{auxrc/getflags $*} || exec auxrc/usage
```

14.1.2 merge.rc

```
<git9/merge.rc 110b>≡
    #!/bin/rc -e
    rfork ne
    . /lib/git/common.rc

    <fn merge(merge.rc) 111a>

gitup

<merge.rc command line processing 110a>

if(! ~ $#* 1)
    exec auxrc/usage

theirs='{git/query $1}
ours='{git/query HEAD}
base='{git/query $theirs ^ ' ' ^ $ours ^ '@}'

if(~ $base $theirs)
    die 'nothing to merge, doofus'
if(! git/walk -q)
    die 'dirty work tree, refusing to merge'
if(~ $base $ours){
    >[1=2] echo 'fast forwarding...'
    echo $theirs > .git/refs/{git/branch}
    git/revert .
    exit ''
}
echo $ours >> .git/merge-parents
echo $theirs >> .git/merge-parents

merge $ours $base $theirs
echo 'merge complete: remember to commit'
```

```
exit ''
```

14.1.3 merge()

```
<fn merge(merge.rc) 111a>≡ (110b)
fn merge{
    ourbr=$gitfs/object/$1/tree
    basebr=$gitfs/object/$2/tree
    theirbr=$gitfs/object/$3/tree

    all='${nl}{git/query -c $1 $2; git/query -c $2 $3} | sed 's/^.../' | sort | uniq}
    for(f in $all){
        ours=$ourbr/$f
        base=$basebr/$f
        theirs=$theirbr/$f
        merge1 ./ $f $ours $base $theirs
    }
}
```

14.1.4 merge1()

```
<fn merge1(common.rc) 111b>≡ (165a)
# merge1 out ours base theirs
fn merge1 {@{
    rfork e
    n=$pid
    out=$1
    ours=$2
    base=$3
    theirs=$4
    tmp=$out.tmp
    while(test -f $tmp){
        tmp=$tmp.$n
        n='{echo $n + 1 | hoc}
    }

    if(! test -f $ours)
        ours=/dev/null
    if(! test -f $base)
        base=/dev/null
    if(! test -f $theirs)
        theirs=/dev/null

    if(mergeperm $ours $base $theirs){
        mkdir -p '${nl}{basename -d $tmp}
        if(! merge3 $ours $base $theirs > $tmp)
            echo merge needed: $out >[1=2]
        mv $tmp $out
        git/add $out
        chmod $mergedperms $out
    }
    if not {
        rm -f $tmp $out
        echo R NOQID 0 $out >> $gitroot/.git/INDEX9
    }
}}
```


14.1.5 mergeperm()

```
<fn mergeperm(common.rc) 112a>≡ (165a)
fn mergeperm {
    if(~ $1 /dev/null && cmp $2 $3>/dev/null)
        status=gone
    if not if (~ $3 /dev/null && cmp $1 $2>/dev/null)
        status=gone
    if not {
        mergedperms='-x'
        if(test -x $2){
            if(test -x $1 -a -x $3)
                mergedperms='+x'
        }
        if not{
            if(test -x $1 -o -x $3)
                mergedperms='+x'
        }
        status=()
    }
}
```

14.2 Merging two trees

14.3 Merging two files

14.4 diff3 and merge3

14.5 Merging conflicts

14.6 Committing merged branches: git commit

```
<commit.rc adjust $files if merge parents 112b>≡ (98b)
if(test -f .git/merge-parents)
    files='${nl}{git/query -c '{cat .git/merge-parents} | sed 's/^...//'}'
```

```
<parents() (commit.rc) if merge parents 112c>≡ (100b)
if not if(test -f .git/merge-parents)
    parents='{cat .git/merge-parents | sort | uniq}'
```

Chapter 15

Inspecting

15.1 Showing the commit history: `git log`

15.1.1 Usage

```
<function usage (git9/log.c) 113a>≡ (191b)
static void
usage(void)
{
    fprintf(STDERR, "usage: %s [-s] [-e expr | -c commit] files..\n", argv0);
    exits("usage");
}
```

15.1.2 `main()`

```
<global commitid(log.c) 113b>≡ (191b)
char    *commitid;
```

```
<function main (git9/log.c) 113c>≡ (191b)
void
main(int argc, char **argv)
{
    char path[1024], repo[1024], *p, *r;
    int i, nrel, nrepo;

    ARGBEGIN{
    case 'e':
        queryexpr = EARGF(usage());
        break;
    case 'c':
        commitid = EARGF(usage());
        break;
    case 's':
        shortlog++;
        break;
    case 'n':
        msgcount = atoi(EARGF(usage()));
        break;
    default:
        usage();
        break;
    }ARGEND;

    gitinit(repo, sizeof(repo), &nrel);
```

```

nrepo = strlen(repo);

if(argc != 0){
    if(getwd(path, sizeof(path)) == nil)
        sysfatal("getwd: %r");
    if(strncmp(path, repo, nrepo) != 0)
        sysfatal("path shifted??");
    p = path + nrepo;
    pathfilt = emalloc(sizeof(Pfilt));
    for(i = 0; i < argc; i++){
        if(*argv[i] == '/'){
            if(strncmp(argv[i], repo, nrepo) != 0)
                continue;
            r = smprint("./%s", argv[i]+nrepo);
        }else
            r = smprint("./%s/%s", p, argv[i]);
        cleannname(r);
        filteradd(pathfilt, r);
        free(r);
    }
}
// !!chdir!!
if(chdir(repo) == ERROR_NEG1)
    sysfatal("chdir: %r");

//9front-only
//tmfmtinstall();
out = Bfdopen(STDOUT, OWRITE);

if(queryexpr != nil)
    showquery(queryexpr);
else
    showcommits(commitid);

Bterm(out);
exits(nil);
}

```

15.1.3 showcommits()

```

⟨function showcommits 114⟩≡ (191b)
static void
showcommits(char *c)
{
    Object *o, *p;
    Qelt e;
    int i;
    Hash h;

    if(c == nil)
        c = "HEAD";
    if(resolveref(&h, c) == -1)
        sysfatal("resolve %s: %r", c);
    if((o = readobject(h)) == nil)
        sysfatal("load %H: %r", h);
    if(o->type != GCommit)
        sysfatal("%s: not a commit", c);

    qinit(&objq);

```

```

osinit(&done);
qput(&objq, o, 0);

while(qpop(&objq, &e) && (msgcount == -1 || msgcount > 0)){
    if(matchesfilter(e.o)){
        show(e.o);
        if(msgcount != -1)
            msgcount--;
    }
    for(i = 0; i < e.o->commit->nparent; i++){
        if(oshas(&done, e.o->commit->parent[i]))
            continue;
        if((p = readobject(e.o->commit->parent[i])) == nil)
            sysfatal("load %H: %r", o->commit->parent[i]);
        osadd(&done, p);
        qput(&objq, p, 0);
    }
    unref(e.o);
}
}

```

15.1.4 show()

```

⟨function show 115⟩≡ (191b)
static int
show(Object *o)
{
    //Tm tm;
    char *p, *q, *e;

    assert(o->type == GCommit);
    if(shortlog){
        p = o->commit->msg;
        e = p + o->commit->nmsg;
        q = nextline(p, e);
        Bprint(out, "%H ", o->hash);
        Bwrite(out, p, q - p);
        Bputc(out, '\n');
    }else{
        //9front only
        //tmtime(&tm, o->commit->mtime, tzload("local"));
        Bprint(out, "Hash:\t%H\n", o->hash);
        Bprint(out, "Author:\t%s\n", o->commit->author);
        if(o->commit->committer != nil
            && strcmp(o->commit->author, o->commit->committer) != 0)
            Bprint(out, "Committer:\t%s\n", o->commit->committer);
        //Bprint(out, "Date:\t%r\n", tmfmt(&tm, "WW MMM D hh:mm:ss z YYYY"));
        Bprint(out, "Date:\t%s", ctime(o->commit->mtime));
        Bprint(out, "\n");
        p = o->commit->msg;
        e = p + o->commit->nmsg;
        for(; p != e; p = q){
            q = nextline(p, e);
            Bputc(out, '\t');
            Bwrite(out, p, q - p);
            Bputc(out, '\n');
            if(q != e)
                q++;
        }
    }
}

```

```

        Bprint(out, "\n");
    }
    Bflush(out);
    return 1;
}

```

15.1.5 showquery()

```

⟨function showquery 116a⟩≡ (191b)
static void
showquery(char *q)
{
    Object *o;
    Hash *h;
    int n, i;

    if((n = resolverefs(&h, q)) == -1)
        sysfatal("resolve: %r");
    for(i = 0; i < n && (msgcount == -1 || msgcount > 0); i++){
        if((o = readobject(h[i])) == nil)
            sysfatal("read %H: %r", h[i]);
        if(matchesfilter(o)){
            show(o);
            if(msgcount != -1)
                msgcount--;
        }
        unref(o);
    }
    exits(nil);
}

```

15.1.6 Advanced features

```
git log -s short log
```

15.2 Representing changes

15.2.1 Tree changes

15.2.2 File changes

15.3 Showing file differences: git diff

15.3.1 Usage

```

⟨diff.rc command line processing 116b⟩≡ (116c)
    flagfmt='c:commit branch, s:summarize, u:uncommitted'; args='[file ...]'
    eval '{auxrc/getflags $*} || exec auxrc/usage

```

15.3.2 diff.rc

```

⟨git9/diff.rc 116c⟩≡
#!/bin/rc
rfork ne
. /lib/git/common.rc

```

gitup

<diff.rc command line processing 116b>

```
bparam=(-b $commit)
if(~ $#commit 0){
    commit=HEAD
    cparam=()
    bparam=()
}

files=()
filt=MAR
if(~ $#uncommitted 1)
    filt=MARU
if(! ~ $#* 0)
    files='{cleanname -d $gitrel $*}'

branch='{git/query -p $commit}'
if(~ $summarize 1 || ~ $uncommitted 1){
    git/walk $bparam -r$gitrel -f$filt $cparam $files
    exit
}

showed=()
mntgen /mnt/scratch
bind $branch/tree/ /mnt/scratch/a
bind . /mnt/scratch/b
for(f in '$nl{git/walk $bparam -c -f$filt $cparam $files}'){
    if(~ $#showed 0){
        echo diff '{git/query $commit} uncommitted
        showed=1
    }
    cd /mnt/scratch
    a=a/$f
    b=b/$f
    if(! test -f a/$f)
        a=/dev/null
    if(! test -f b/$f)
        b=/dev/null
    diff -u $a $b
}
exit ''
```

15.3.3 diff -u

15.3.4 Computing tree changes

15.3.5 Computing file changes

15.3.6 Showing changes

Chapter 16

Packing Objects

16.1 Pack.t

16.2 IO

16.2.1 Reading a pack

```
<function loadpack 118>≡ (215b)
static int
loadpack(Packf *pf, char *name)
{
    char buf[128];
    int i, ifd;
    Dir *d;

    memset(pf, 0, sizeof(Packf));
    snprintf(buf, sizeof(buf), ".git/objects/pack/%s.idx", name);
    snprintf(pf->path, sizeof(pf->path), ".git/objects/pack/%s.pack", name);
    /*
     * if we already have the pack open, just
     * steal the loaded info
     */
    for(i = 0; i < npackf; i++){
        if(strcmp(pf->path, packf[i].path) == 0){
            pf->pack = packf[i].pack;
            pf->idx = packf[i].idx;
            pf->nidx = packf[i].nidx;
            packf[i].idx = nil;
            packf[i].pack = nil;
            return 0;
        }
    }
    if((ifd = open(buf, OREAD)) == -1)
        return -1;
    if((d = dirfstat(ifd)) == nil){
        close(ifd);
        return -1;
    }
    pf->nidx = d->length;
    pf->idx = emalloc(pf->nidx);
    if(readn(ifd, pf->idx, pf->nidx) != pf->nidx){
        close(ifd);
        free(pf->idx);
        free(d);
    }
```

```

        return -1;
    }
    close(ifd);
    free(d);
    return 0;
}

```

16.2.2 Writing a pack

16.3 Modifications to previous functions and data structures

16.3.1 Object Store

```

⟨readidxobject() look for object in packs 119a⟩≡ (39b)
    for(i = 0; i < npackf; i++){
        o = searchindex(packf[i].idx, packf[i].nidx, h, SHA1dlen*8, nil);
        if(o != -1){
            if((f = openpack(&packf[i])) == nil)
                goto error;
            if((r = Bseek(f, o, 0)) != -1)
                r = readpacked(f, obj, flag);
            closepack(&packf[i]);
            if(r == -1)
                goto error;
            parseobject(obj);
            cache(obj);
            return obj;
        }
    }
}

```

16.3.2 Refs

16.4 Delta

16.5 Pack commands

16.5.1 git repack

Usage

```

⟨function usage (git9/repack.c) 119b⟩≡ (233c)
    void
    usage(void)
    {
        fprintf(STDERR, "usage: %s [-d]\n", argv0);
        exits("usage");
    }
}

```

main.c

```

⟨function main (git9/repack.c) 119c⟩≡ (233c)
    void
    main(int argc, char **argv)
    {

```



```

char path[128], **names;
int fd, nrefs;
Hash *refs, h;
Dir rn;

ARGBEGIN{
case 'd':
    chattygit++;
    break;
default:
    usage();
}ARGEND;

gitinit(nil, 0, nil);
refs = nil;
if((nrefs = listrefs(&refs, &names)) == -1)
    sysfatal("load refs: %r");
if((fd = create(TMPPATH("pack.tmp"), OWRITE, 0644)) == -1)
    sysfatal("open %s: %r", TMPPATH("pack.tmp"));
if(writepack(fd, refs, nrefs, nil, 0, &h) == -1)
    sysfatal("writepack: %r");
if(indexpack(TMPPATH("pack.tmp"), TMPPATH("idx.tmp"), h) == -1)
    sysfatal("indexpack: %r");
close(fd);

nulldir(&rn);
rn.name = path;
snprint(path, sizeof(path), "%H.pack", h);
if(dirwstat(TMPPATH("pack.tmp"), &rn) == -1)
    sysfatal("rename pack: %r");
snprint(path, sizeof(path), "%H.idx", h);
if(dirwstat(TMPPATH("idx.tmp"), &rn) == -1)
    sysfatal("rename pack: %r");
if(cleanup(h) == -1)
    sysfatal("cleanup: %r");
exits(nil);
}

```

Chapter 17

Exchanging

17.1 Upstream and remote origin

```
<init.rc check $upstream flag 121a>≡ (94a)
    if(~ $#upstream 0){
        upstream='{git/conf 'defaults "origin".baseurl'}'
        if(! ~ $#upstream 0)
            upstream=$upstream/$name
    }
```

```
<init.rc set origin in .git/config 121b>≡ (94a)
    if(! ~ $#upstream 0){
        echo '[remote "origin"]'
        echo ' url = '$upstream
    }
```

17.2 Pulling updates from another repository: git pull

17.2.1 Usage

```
<pull.rc command line processing 121c>≡ (121d)
    flagfmt='d:debug, q:quiet, f:fetchonly, u:upstream upstream'
    args=''
    eval '{auxrc/getflags $*} || exec auxrc/usage
```

17.2.2 pull.rc

```
<git9/pull.rc 121d>≡
    #!/bin/rc -e
    rfork en
    . /lib/git/common.rc

    fn update{
        upstream=$1
        url=$2
        dir=$3
        dflag=()
        if(! ~ $#debug 0)
            dflag='-d'
        heads='${nl}{
            # safe to split in awk, branch names may not
            # have spaces or newlines.
```

```

        git/query '$nl{
            cd .git/refs && \
            walk -f -emp heads remotes | \
            sort -rn | \
            awk '{print $2}'
        }
    }
    if(! ~ $#heads 0)
        heads='-h'^$heads
    {git/get $dflag -u $upstream $heads $url >[2=3] || die $status} | awk '
    /~remote refs\/(heads|tags)\/\/{
        ref=$2
        hash=$3
        gsub("^refs/heads", "refs/remotes/'$upstream'", ref)
        outfile = ".git/"ref
        print outfile > "/env/outfile"
        system("mkdir -p '$nl{basename -d $outfile}");\
        close("/env/outfile")
        print hash > outfile;
        close(outfile);
    }
    ' |[3] tr '\x0d' '\x0a'
}

gitup

<pull.rc command line processing 121c>

if(~ $#upstream 0)
    upstream=origin
remote='$nl{git/conf 'remote "'$upstream'.url'}'
if(~ $#remote 0){
    remote=$upstream
    upstream=THEM
}

update $upstream $remote
if (~ $fetchonly 1)
    exit

local='{git/branch}
remote='{git/branch | subst '~(refs/)heads' 'remotes/'$upstream}

# we have local commits, but the remote hasn't changed.
# in this case, we want to keep the local commits untouched.
if(~ '{git/query HEAD $remote @} '{git/query $remote}){
    echo 'up to date' >[1=2]
    exit
}

# The remote repository and our HEAD have diverged: we
# need to merge.
if(! ~ '{git/query HEAD $remote @} '{git/query HEAD}){
    >[1=2]{
        echo ours:      '{git/query HEAD}
        echo theirs:    '{git/query $remote}
        echo common:    '{git/query HEAD $remote @}
        echo git/merge $remote
    }
    exit diverged
}

```

```

oldcommit='{git/query $local}
newcommit='{git/query $remote}

if(~ $#quiet 0){
    git/log -s -e $oldcommit'..' $newcommit
    echo $remote': ' $oldcommit '=>' $newcommit
}

git/branch -mnb $remote $local
exit ''

```

17.2.3 Fetching objects locally

Find top common commits

Find missing objects

Iterating over objects

Adding Objects

17.2.4 Fetching references

Refspecs

Selecting the refs

Parsing ref tuples

17.2.5 Fetching objects in a pack

17.3 git/get

17.3.1 Usage

```

⟨function usage (git9/get.c) 123a⟩≡ (184)
void
usage(void)
{
    fprintf(2, "usage: %s [-dl] [-b br] [-u upstream] remote\n", argv0);
    fprintf(2, "\t-t-b br: only fetch matching branch 'br'\n");
    fprintf(2, "remote:  fetch from this repository\n");
    exits("usage");
}

```

17.3.2 main()

```

⟨function main (git9/get.c) 123b⟩≡ (184)
void
main(int argc, char **argv)
{
    char *s;
    Conn c;

    ARGBEGIN{

```

```

case 'b':  fetchbranch=EARGF(usage()); break;
case 'u':  upstream=EARGF(usage());   break;
case 'd':  chattygit++;               break;
case 'l':  listonly++;                break;
case 'h':
    s = EARGF(usage());
    if(nheads < nelem(heads))
        if(hparse(&heads[nheads], s) == 0)
            nheads++;
    break;
default:
    usage();
    break;
}ARGEND;

gitinit(nil, 0, nil);

if(argc != 1)
    usage();

if(gitconnect(&c, argv[0], "upload") == -1)
    sysfatal("could not dial %s: %r", argv[0]);
if(fetchpack(&c) == -1)
    sysfatal("fetch failed: %r");
closeconn(&c);
exits(nil);
}

```

17.4 Object store walker

17.5 Pushing changes to another repository: git push

17.5.1 Usage

<push.rc command line processing 124a>≡ (124b)

```

flagfmt='a:pushall, b:branch branch, f:force, d:debug,
        r:remove remove, u:upstream upstream' args=''
eval '{auxrc/getflags $*} || exec auxrc/usage

```

17.5.2 push.rc

<git9/push.rc 124b>≡

```

#!/bin/rc -e
rfork en
. /lib/git/common.rc

gitup

<push.rc command line processing 124a>
if(! ~ $#* 0)
    exec auxrc/usage

if(~ $pushall 1)
    branch='${nl{cd .git/refs/heads && walk -f}}
if(~ $#branch 0)
    branch='{git/branch}

```

```

if(~ $#branch 0)
    die 'no branches'
if(~ $force 1)
    force=-f
if(~ $debug 1)
    debug='-d'

if(~ $#upstream 0)
    upstream=origin

remotes='${nl}{git/conf -a 'remote "'$upstream'".url'}'
if(~ $#remotes 0)
    remotes=$upstream
branch=-b~$branch
if(! ~ $#remove 0)
    remove=-r~$remove
for(remote in $remotes){
    updates='${nl}{git/send $debug $force $branch $remove $remote} || die $status
    for(ln in $updates){
        u='{echo $ln}
        refpath='{echo $u(2) | subst '~refs/heads/' '~.git/refs/remotes/'$upstream'/'}'
        switch($u(1)){
        case update;
            mkdir -p '{basename -d $refpath}
            echo $u(4) > $refpath
            echo $u(2)~':~' $u(3) '=>' $u(4)
        case delete;
            echo $u(2)~': removed'
            rm -f $refpath
        case uptodate;
            echo $u(2)~': up to date'
        }
    }
}
exit ''

```

17.5.3 Sending objects locally

17.5.4 Sending objects in a pack

17.6 git/send

17.6.1 Usage

```

⟨function usage (git9/send.c) 125a⟩≡ (244)
void
usage(void)
{
    fprintf(STDERR, "usage: %s remote [reponame]\n", argv0);
    exits("usage");
}

```

17.6.2 main()

```

⟨function main (git9/send.c) 125b⟩≡ (244)
void
main(int argc, char **argv)

```

```

{
    char *br;
    Conn c;

    ARGBEGIN{
    default:
        usage();
        break;
    case 'd':
        chattygit++;
        break;
    case 'f':
        force++;
        break;
    case 'r':
        if(nremoved == nelem(removed))
            sysfatal("too many deleted branches");
        removed[nremoved++] = EARGF(usage());
        break;
    case 'a':
        sendall++;
        break;
    case 'b':
        br = EARGF(usage());
        if(strncmp(br, "refs/heads/", strlen("refs/heads/")) == 0)
            br = smprint("%s", br);
        else if(strncmp(br, "heads/", strlen("heads/")) == 0)
            br = smprint("refs/%s", br);
        else
            br = smprint("refs/heads/%s", br);
        branch = erealloc(branch, (nbranch + 1)*sizeof(char*));
        branch[nbranch] = br;
        nbranch++;
        break;
    }ARGEND;

    gitinit(nil, 0, nil);
    if(argc != 1)
        usage();
    if(gitconnect(&c, argv[0], "receive") == -1)
        sysfatal("git connect: %s: %r", argv[0]);
    if(sendpack(&c) == -1)
        sysfatal("send failed: %r");
    closeconn(&c);
    exits(nil);
}

```

17.7 Cloning a remote repository: git clone

17.7.1 Usage

```

⟨clone.rc command line processing 126⟩≡ (127)
    flagfmt='d:debug, b:branch branch'; args='remote [local]'
    eval '{auxrc/getflags $*} || exec auxrc/usage

```

17.7.2 clone.rc

```
<git9/clone.rc 127>≡
#!/bin/rc
rfork en
. /lib/git/common.rc

<clone.rc command line processing 126>

if(~ $debug 1)
    debug=(-d)

remote=$1
local=$2

if(~ $#remote 0)
    exec auxrc/usage
if(~ $#local 0)
    local='${nl{basename '`${nl{echo $remote | sed 's@.*/(~/+)*$@1@'} } .git}}
if(~ $#branch 1)
    branchflag=(-b $branch)

if(test -e $local && ~ '{ls $local | sed 1q | wc -l} 1)
    die 'destination already exists:' $local

fn clone{
    flag +e
    mkdir -p $local/.git
    mkdir -p $local/.git/fs
    mkdir -p $local/.git/objects/pack/
    mkdir -p $local/.git/refs/heads/

    cd $local

    >>.git/config {
        echo '[remote "origin"]'
        echo '  url='$remote
    }
    {git/get $debug $branchflag $remote >[2=3] | awk '
        BEGIN{
            headref=""
            if(ENVIRON["branch"] != "")
                headref="refs/remotes/origin/"ENVIRON["branch"]
            headhash=""
        }
        /^symref / && headref == "" {
            if($2 == "HEAD"){
                gsub("^refs/heads", "refs/remotes/origin", $3)
                gsub("^refs/tags", "refs/remotes/origin/tags", $3)
            }
        }
        /^remote /{
            if($2=="HEAD"){
                headhash=$3
            }else if(match($2, "^refs/(heads|tags)/")){
                gsub("^refs/heads", "refs/remotes/origin", $2)
                if($2 == headref || (headref == "" && $3 == headhash))
                    headref=$2
                outfile = ".git/" $2
                outdir = outfile
            }
        }
    }
```



```

        gsub("/?[^/]*/?$", "", outdir)
        system(sprintf("mkdir -p %q", outdir))
        print $3 > outfile
        close(outfile)
    }
}
END{
    if(headref != ""){
        remote = headref;
        reldir = headref;
        gsub("/?[^/]*/?$", "", reldir)
        gsub("^refs/remotes/origin", "refs/heads", headref)
        system("mkdir -p '{basename -d .git/"headref"}");
        system("cp .git/" remote " .git/" headref)
        print "ref: " headref > ".git/HEAD"
    }else if(headhash != ""){
        print "warning: detached head "headhash > "/fd/2"
        print headhash > ".git/HEAD"
    }
}
}' | [3] tr '\x0d' '\x0a' || die 'could not clone repository'

tree=.git/fs/HEAD/tree
lbranch='{git/branch}
rbranch='{echo $lbranch | subst 'heads' 'remotes/origin'}
echo checking out repository...
if(test -f .git/refs/$rbranch){
    mkdir -p '{basename -d .git/refs/$lbranch}
    cp .git/refs/$rbranch .git/refs/$lbranch
    git/fs
    @ {builtin cd $tree && tar cif /fd/1 .} | @ {tar xf /fd/0} \
        || die 'checkout failed:' $status
    {for(f in '$nl{cd $tree && walk -f})
        echo 'T NOQID 0 '$f' > .git/INDEX9
}
if not{
    echo no default branch >[1=2]
    echo check out your code with git/branch >[1=2]
}
}

fn sigint {
    echo cancelled clone $remote: cleaning $local >[1=2]
    umount $local/.git/fs >[2]/dev/null
    rm -rf $local
    exit interrupted
}

@{clone}
st=$status
if(! ~ $st ''){
    echo failed to clone $remote: cleaning $local >[1=2]
    umount $local/.git/fs >[2]/dev/null
    rm -rf $local
    exit $st
}
exit ''

```

17.7.3 Fetching objects

17.7.4 Importing references

Chapter 18

Networking

```
⟨function gitconnect 130a⟩≡ (226c)
/// ??? -> <>
int
gitconnect(Conn *c, char *uri, char *direction)
{
    char proto[Nproto], host[Nhost], port[Nport], path[Npath];

    memset(c, 0, sizeof(Conn));
    c->rfd = c->wfd = c->cfd = -1;

    if(localrepo(uri, path, sizeof(path)) == 0)
        return servelocal(c, path, direction);

    if(parseuri(uri, proto, host, port, path) == -1){
        werrstr("bad uri %s", uri);
        return -1;
    }
    if(strcmp(proto, "ssh") == 0)
        return dialssh(c, host, port, path, direction);
    else if(strcmp(proto, "git") == 0)
        return dialgit(c, host, port, path, direction);
    else if(strcmp(proto, "hgit") == 0)
        return dialhgit(c, host, port, path, direction, 1);
    else if(strcmp(proto, "gits") == 0)
        return dialhgit(c, host, port, path, direction, 0);
    else if(strcmp(proto, "http") == 0 || strcmp(proto, "https") == 0)
        return dialhttp(c, host, port, path, direction);
    werrstr("unknown protocol %s", proto);
    return -1;
}
```

18.1 git:// protocol

```
⟨function dialgit 130b⟩≡ (226c)
static int
dialgit(Conn *c, char *host, char *port, char *path, char *direction)
{
    char *ds;
    int fd;

    if((ds = netmkaddr(host, "tcp", port)) == nil)
        return -1;
    dprint(1, "dial %s git-%s-pack %s\n", ds, direction, path);
```

```

    fd = dial(ds, nil, nil, nil);
    if(fd == -1)
        return -1;
    c->type = ConnGit;
    c->rfd = fd;
    c->wfd = dup(fd, -1);
    return githandshake(c, host, path, direction);
}

```

18.1.1 Reading packets

18.1.2 Writing packets

18.2 Capabilities

18.3 git:// client

18.3.1 Fetching remote objects

18.3.2 Fetching pack files

```

read_pkt_refs()

_handle_upload_pack_head()

_handle_upload_pack_tail()

```

18.3.3 Sending pack files

18.4 git:// server

18.4.1 git/serve

Usage

```

⟨function usage (git9/serve.c) 131a⟩≡ (254)
void
usage(void)
{
    fprintf(STDERR, "usage: %s [-dw] [-r rel]\n", argv0);
    exits("usage");
}

```

main.c

```

⟨function main (git9/serve.c) 131b⟩≡ (254)
void
main(int argc, char **argv)
{
    char *repo, cmd[32], buf[512];
    Conn c;

    ARGBEGIN{
    case 'd':
        chattygit++;

```

```

        break;
case 'r':
    pathpfx = EARGF(usage());
    if(*pathpfx != '/')
        sysfatal("path prefix must begin with '/'");
    break;
case 'w':
    allowwrite++;
    break;
default:
    usage();
    break;
}ARGEND;

interactive = 0;
if(rfork(RFNAMEG) == -1)
    sysfatal("rfork: %r");
if(pathpfx != nil){
    if(bind(pathpfx, "/", MREPL) == -1)
        sysfatal("bind: %r");
}
if(rfork(RFNOMNT) == -1)
    sysfatal("rfork: %r");

initconn(&c, 0, 1);
if(readpkt(&c, buf, sizeof(buf)) == -1)
    sysfatal("readpkt: %r");
repo = parsecmd(buf, cmd, sizeof(cmd));
cleannname(repo);
if(strncmp(repo, "../", 3) == 0)
    fail(&c, "invalid path %s\n", repo);
if(bind(repo, "/", MREPL) == -1)
    fail(&c, "no such repo: %s", repo);
if(chdir("/") == -1)
    fail(&c, "no such repo");

gitinit(nil, 0, nil);

if(strcmp(cmd, "git-receive-pack") == 0)
    recvpack(&c);
else if(strcmp(cmd, "git-upload-pack") == 0)
    servpack(&c);
else
    fail(&c, "unsupported command '%s'", cmd);
exits(nil);
}

```

Part III

Beyond the Basics

Chapter 19

Core Algorithms

19.1 SHA1

19.1.1 Overview

19.1.2 32 bits operators

19.1.3 Entry point: `sha1()`

19.1.4 Endianess

19.1.5 Filling blocks

19.1.6 Looping over blocks

19.1.7 Padding

19.2 Zip and Unzip

19.3 Diff

19.3.1 Overview

19.3.2 Data structures

19.3.3 Entry point: `Diffs.diff()`

19.3.4 Edit distance basic algorithm ($O(n^2)$ in space/time)

19.3.5 Myers algorithm ($O(n)$ in space in best-case)

19.4 Diff3

19.4.1 Overview

19.4.2 Data structures

19.4.3 Entry point: `diff3()`

19.4.4 Displaying merging conflicts

Chapter 20

Supporting Programs

20.1 diff

20.1.1 Usage

```
<function usage(diff) 136a>≡ (266)
void
usage(void)
{
    fprintf(STDERR, "usage: %s [-abcefmmrw] file1 ... file2\n", argv0);
    exits("usage");
}
```

20.1.2 Data structures

20.1.3 main()

```
<function main(diff) 136b>≡ (266)
void
main(int argc, char *argv[])
{
    int i;
    Dir *fsb, *tsb;

    Binit(&stdout, 1, OWRITE);
    ARGBEGIN{
    case 'e':
    case 'f':
    case 'n':
    case 'c':
    case 'a':
    case 'u':
        mode = ARGV();
        break;
    case 'w':
        bflag = 2;
        break;

    case 'b':
        bflag = 1;
        break;

    case 'r':
        rflag = 1;
```

```

        break;

    case 'm':
        mflag = 1;
        break;

    case 'h':
    default:
        usage();
}ARGEND;

if (argc < 2)
    usage();

if ((tsb = dirstat(argv[argc-1])) == nil)
    sysfatal("can't stat %s", argv[argc-1]);
if (argc > 2) {
    if (!DIRECTORY(tsb))
        sysfatal("not directory: %s", argv[argc-1]);
    mflag = 1;
} else {
    if ((fsb = dirstat(argv[0])) == nil)
        sysfatal("can't stat %s", argv[0]);
    if (DIRECTORY(fsb) && DIRECTORY(tsb))
        mflag = 1;
    free(fsb);
}
free(tsb);
for (i = 0; i < argc-1; i++)
    diff(argv[i], argv[argc-1], 0);

done(anychange);
/*NOTREACHED*/
}

```

```

⟨function done(diff) 137a⟩≡ (266)
void
done(int status)
{
    switch(status)
    {
    case 0:
        exits("");
    case 1:
        exits("some");
    default:
        exits("error");
    }
}

```

20.1.4 Advanced features

20.2 patch

20.2.1 Usage

```

⟨function usage(patch.c) 137b⟩≡ (303)
void

```

```
usage(void)
{
    fprintf(STDERR, "usage: %s [-nR] [-p nstrip] [-r rejfile] [patch...]\n", argv0);
    exits("usage");
}
```

20.2.2 Data structures

20.2.3 main()

```
<function main(patch.c) 138>≡ (303)
void
main(int argc, char **argv)
{
    Biobuf *f;
    Patch *p;
    int i, ok;

    ARGBEGIN{
    case 'd':
        workdir = EARGF(usage());
        break;
    case 'r':
        rejfile = EARGF(usage());
        break;
    case 'p':
        strip = atoi(EARGF(usage()));
        break;
    case 'n':
        dryrun++;
        break;
    case 'R':
        reverse++;
        break;
    default:
        usage();
        break;
    }ARGEND;

    ok = 1;
    if(rejfile != nil){
        rejfd = create(rejfile, OWRITE, 0644);
        if(rejfd == -1)
            fail("open %s: %r", rejfile);
    }
    if(argc == 0){
        if((f = Bfdopen(0, OREAD)) == nil)
            fail("open stdin: %r");
        if((p = parse(f, "stdin")) == nil)
            fail("parse patch: %r");
        if(workdir != nil && chdir(workdir) == -1)
            fail("chdir %s: %r", workdir);
        if(apply(p, "stdin") == -1){
            fprintf(2, "apply stdin: %r\n");
            ok = 0;
        }
        freepatch(p);
        Bterm(f);
        ok = finish(ok);
    }
```

```

}else{
    for(i = 0; ok && i < argc; i++){
        if((f = Bopen(argv[i], OREAD)) == nil)
            fail("open %s: %r", argv[i]);
        if((p = parse(f, argv[i])) == nil)
            fail("parse patch: %r");
        if(workdir != nil && chdir(workdir) == -1)
            fail("chdir %s: %r", workdir);
        if(apply(p, argv[i]) == -1){
            fprintf(2, "apply %s: %r\n", argv[i]);
            ok = 0;
        }
        freepatch(p);
        Bterm(f);
        ok = finish(ok);
    }
}
exits(ok ? nil : "failed");
}

```

20.2.4 Advanced features

20.3 merge3

20.3.1 Usage

```

⟨function usage (diff/merge3.c) (diff) 139a⟩≡ (289)
void
usage(void)
{
    fprintf(STDERR, "usage: %s theirs base ours\n", argv0);
    exits("usage");
}

```

20.3.2 Data structures

20.3.3 main()

```

⟨function main (diff/merge3.c) (diff) 139b⟩≡ (289)
void
main(int argc, char **argv)
{
    Diff l, r;
    char *x;

    ARGBEGIN{
        default:
            usage();
    }ARGEND;

    if(argc != 3)
        usage();

    Binit(&stdout, 1, OWRITE);
    memset(&l, 0, sizeof(l));
    memset(&r, 0, sizeof(r));

```

```
calcdiff(&l, argv[1], argv[1], argv[0], argv[0]);
calcdiff(&r, argv[1], argv[1], argv[2], argv[2]);

if(l.binary || r.binary)
    sysfatal("cannot merge binaries");

x = merge(&l, &r);

freediff(&l);
freediff(&r);
exits(x);
}
```

Chapter 21

Exchanging Patches

21.1 git/export

21.1.1 Usage

```
<export.rc command line processing 141a>≡ (141b)
    flagfmt='o:patchdir patchdir'; args='[query]'
    eval '{auxrc/getflags $*} || exec auxrc/usage
```

21.1.2 export.rc

```
<git9/export.rc 141b>≡
    #!/bin/rc
    rfork ne
    . /lib/git/common.rc

    patchname=/tmp/git.patchname.$pid
    patchfile=/tmp/git.patchfile.$pid
    fn sigexit{
        rm -f $patchname $patchfile
    }

    gitup

    <export.rc command line processing 141a>

    if(~ $#patchdir 1 && ! test -d $patchdir)
        mkdir -p $patchdir

    q=$*
    if(~ $#q 0)
        q=HEAD
    commits='{git/query $q || die $status}'
    n=1
    m=$#commits

    # sleazy hack: we want to run
    # under rfork m for the web ui,
    # so don't error if we can't mount
    mntgen /mnt/scratch >[2]/dev/null || status=''
    for(c in $commits){
        cp='{git/query -p $c}'
        pp='{git/query -p $c''}'
```


21.2 git/import

21.2.1 Usage

```
<import.rc command line processing 143a>≡ (143b)
flagfmt='n:nocommit'; args='file ...'
eval '{auxrc/getflags $*} || exec auxrc/usage
```

21.2.2 import.rc

```
<git9/import.rc 143b>≡
#!/bin/rc
rfork ne
. /lib/git/common.rc

diffpath=/tmp/gitimport.$pid.diff
fn sigexit {
    rm -f $diffpath
}

fn apply1 {
    adate='{seconds $adate}
    files='${nl}{patch -np1 < $diffpath}
    if(! git/walk -q $files){
        >[1=2] {
            echo patch would clobber files:
            git/walk $files
            exit clobber
        }
    }
    echo applying $msg | sed 1q
    if(! files='${nl}{patch -p1 < $diffpath})
        die 'patch failed'
    for(f in $files){
        if(test -e $f)
            git/add $f
        if not
            git/add -r $f
    }
    git/walk -fRMA $files
    if(~ $#nocommit 0){
        if(hash='{git/save -n $aname -e $amail -N $name -E $email -m $msg -d $adate $parents $files)){
            echo $hash > $refpath
            for(f in $files)
                echo T NOQID 0 $f >> .git/INDEX9
        }
    }
    exit ''
}

fn apply @{
    git/fs
    amail='',
    aname='',
    msg='',
    whoami
    parents='-p'~'{git/query HEAD}
    branch='{git/branch}
```



```

if(test -e $gitfs/branch/$branch/tree)
    refpath=.git/refs/$branch
if not if(test -e $gitfs/object/$branch/tree)
    refpath=.git/HEAD
if not
    die 'invalid branch:' $branch
awk '
function doapply(){
    if(aname == "" || aemail == "" || date == "" || gotmsg == "")
        exit("missing headers");
    printf "%s", aname > "/env/aname"
    printf "%s", aemail > "/env/aemail"
    printf "%s", date > "/env/adate"
    if(system("rc -c apply1") != 0)
        exit("patch failed");
    close("/env/aname")
    close("/env/aemail")
    close("/env/adate")
    close(ENVIRON["diffpath"])
    applied = 1
}
BEGIN{
    state="headers"
}
state=="diff" && !/^[-+ @]|^diff|^$|^\\ No newline at end of file$/ {
    state="footers";
    doapply();
}
state=="footers" && /\n$/ {
    state="headers"
    next
}
state=="headers" && /^From: / {
    sub(/^From: [ \t]*/ , "", $0);
    aname=$0;
    aemail=$0;
    sub(/[ \t]*<.*$/ , "", aname);
    sub(/^<[ \t]*<*/ , "", aemail);
    sub(>[ \t]*>*/ , "", aemail);
}
state=="headers" && /^Date: / {
    sub(/^Date: [ \t]*/ , "", $0)
    date=$0
}
state=="headers" && /^Subject: / {
    sub(/^Subject: [ \t]*([Rr][Ee]:)?[ \t]*(\[^\]]*\)[ \t]*)*/ , "", $0);
    gotmsg = 1
    print > "/env/msg"
}
state=="headers" && /\n$/ {
    state="body"
}
(state=="headers" || state=="body") && (/^diff / || /^---( |$)/){
    state="diff"
    next
}
state=="body" && /\n$/ {
    empty=1
    next
}
}

```

```

state=="body" {
    if(empty)
        printf "\n" > "/env/msg"
    empty=0
    sub(/[ ]+$/, "")
    print > "/env/msg"
}
state=="diff" {
    print > ENVIRON["diffpath"]
}
END{
    if(!failed && state == "diff" && doapply() != 0){
        print "unable to apply patch" > "/fd/2"
        exit("mismatch")
    }
}
,
}

gitup

<import.rc command line processing 143a>

patches=(/fd/0)
if(! ~ $#* 0)
    patches='{cleanname -d $gitrel $*}'
for(p in $patches){
    # upas serves the decoded header and body separately,
    # so we cat them together when applying a upas message.
    #
    # this allows mime-encoded or line-wrapped patches.
    if(test -d $p && test -f $p/header && test -f $p/body)
        {{cat $p/header; echo; cat $p/body} | apply} || die $status
    if not
        apply < $p
}
exit ''

```

Chapter 22

Advanced Features

22.1 Tracking symbolic links

`<Dirent other fields 146a>≡ (33d) 147a▷`
bool_byte islink;

`<parsetree() if symlink 146b>≡ (43c)`
if(m == 0120000){ /* symlink */
t->mode = 0;
t->islink = true;
}

`<gtreegen() if e->tree->ent[i].islink 146c>≡ (68f)`
if(e->tree->ent[i].islink)
if((l = walklink(aux, o->data, o->size, 0, &m)) != nil)
o = l;

`<objwalk1() when GTree case, if o->tree->ent[i].islink 146d>≡ (69b)`
if(w && o->tree->ent[i].islink)
if((l = walklink(aux, w->data, w->size, 1, &m)) != nil)
w = l;

`<function walklink 146e>≡ (174)`
static Object*
walklink(Gitau *aux, char *link, int nlink, int ndotdot, int *mode)
{
char *p, *e, *path;
Object *o, *n;
int i;

path = emalloc(nlink + 1);
memcpy(path, link, nlink);
cleanname(path);

o = crumb(aux, ndotdot)->obj;
assert(o->type == GTree);
for(p = path; *p; p = e){
n = nil;
e = p + strcspn(p, "/");
if(*e == '/')
*e++ = '\\0';
/*
* cleanname guarantees these show up at the start of the name,
* which allows trimming them from the end of the trail of crumbs
* instead of needing to keep track of full parentage.
*/

```

    */
    if(strcmp(p, "..") == 0)
        n = crumb(aux, ++ndotdot)->obj;
    else if(o->type == GTree)
        for(i = 0; i < o->tree->nent; i++)
            if(strcmp(o->tree->ent[i].name, p) == 0){
                *mode = o->tree->ent[i].mode;
                n = readobject(o->tree->ent[i].h);
                break;
            }
        o = n;
        if(o == nil)
            break;
    }
    free(path);
    for(i = 0; o != nil && i < aux->ncrumb; i++)
        if(crumb(aux, i)->obj == o)
            return nil;
    return o;
}

```

22.2 Splitting a large repository: submodules

$\langle \text{Dirent } \textit{other fields } 147a \rangle + \equiv$ (33d) $\triangleleft 146a$
 bool_byte ismod;

$\langle \text{parsetree()} \textit{ if submodule } 147b \rangle \equiv$ (43c)
 if(m == 0160000){ /* module */
 t->mode |= DMDIR;
 t->ismod = true;
 }

$\langle \text{gtreegen()} \textit{ if submodule } 147c \rangle \equiv$ (68f)
 if(e->tree->ent[i].ismod)
 o = emptydir();

$\langle \text{objwalk1()} \textit{ when GTree case, if o->tree->ent[i].ismod } 147d \rangle \equiv$ (69b)
 if(!w && o->tree->ent[i].ismod)
 w = emptydir();

Chapter 23

Advanced Commands

23.1 Tagging a commit with a name

```
<Gxxx other cases 148a>+≡ (31c) <31d
    GTag      = 4,
```

```
<objread() switch o->type cases 148b>+≡ (68a) <69d
    case GTag:
        readbuf(r, o->data, o->size);
        break;
```

```
<objwalk1() switch o->type cases 148c>+≡ (67b) <70c
    case GTag:
        e = Eimpl;
        break;
```

23.1.1 Reading a tag

```
<readloose() local types other elements 148d>≡ (40c)
    {"tag", GTag},
```

```
<parseobject() switch o->type cases 148e>+≡ (41) <44a
    case GTag: parsetag(o);    break;
```

```
<function parsetag 148f>≡ (215b)
    static void
    parsetag(Object *)
    {
    }
```

23.1.2 Writing a tag

23.2 git/hist

23.2.1 Usage

```
<hist.rc command line processing 148g>≡ (149a)
    flagfmt='n:num count'; args='files...'
    eval '{auxrc/getflags $*} || exec auxrc/usage
    if(~ $#* 0) exec auxrc/usage
```

23.2.2 hist.rc

```
<git9/hist.rc 149a>≡
#!/bin/rc
rfork ne
. /lib/git/common.rc
gitup

<hist.rc command line processing 148g>

fn dodiff {
    while(t='{read}'){
        h=$t(1)
        o=$gitfs/object/'{git/query $h~}
        c=$gitfs/object/$h
        echo 'Hash:' $h
        echo -n 'Date: '; date '{walk -em $c/msg}
        echo -n 'Author: '; cat $c/author
        echo -n 'Message: '; cat $c/msg
        echo
        for(f in $files){
            curr=$c/tree/$f
            prev=$o/tree/$f
            if(! test -e $curr)
                curr=/dev/null
            if(! test -e $prev)
                prev=/dev/null
            diff -u $prev $curr
        }
        echo --' '
        echo ⑨
        echo
    }
}

files='{cleanname -d $gitrel $*}
args=(-s $files)
if(! ~ $#num 0)
    args=(-n $num $args)
git/log $args | dodiff
```

23.3 Reverting a change: git revert

23.3.1 Usage

23.3.2 revert.rc

```
<git9/revert.rc 149b>≡
#!/bin/rc
rfork en
. /lib/git/common.rc

gitup

flagfmt='c:query query' args='file ...'
if (! eval '{auxrc/getflags $*} || ~ $#* 0)
    exec auxrc/usage
```

```

if(~ $#query 0)
    query=HEAD
commit='{git/query -p $query}'

files='${nl}{cleanname -d $gitrel $* | drop $gitroot}'
for(f in '${nl}{git/walk -c -fRM -b $query $files}){
    mkdir -p '{basename -d $f}'
    cp -x -- $commit/tree/$f $f
    touch -c $f
    git/add $f
}
exit ''

```

23.4 Rebase: git rebase

23.4.1 Usage

23.4.2 rebase.rc

```

⟨git9/rebase.rc 150⟩≡
#!/bin/rc -e

rfork en

. /lib/git/common.rc
gitup

flagfmt='s:src src, n:nocommit'; args='onto'
eval '{auxrc/getflags $*} || exec auxrc/usage'
if(~ $#* 0) exec auxrc/usage

if(~ $#src 0)
    src='{git/branch}'
nflag=''
if(~ $nocommit 1)
    nflag='-n'

dst='{git/query $1}'
com='{git/query $dst $src @}'
if(~ $dst $com)
    die 'nothing to rebase, doofus'
git/log -se $dst'..'${src} | awk '
BEGIN{
    src=ENVIRON["src"];
    dst=ENVIRON["dst"];
}
{
    if(!done)
        print "git/branch -nb "dst" rebase.wip";
    c=$1; $1="";
    print "git/export "c" | git/import '$nflag'#"0;
    done=1
}
END{
    if(!done)
        print "git/branch -nb "dst" ";
    else{

```

```
        print "git/branch -nb rebase.wip "src  
        print "git/branch -r rebase.wip";  
    }  
},
```


Chapter 24

Advanced Networking

24.1 Other clients

24.1.1 http://

```
<function dialhttp 152a>≡ (226c)
static int
dialhttp(Conn *c, char *host, char *port, char *path, char *direction)
{
    char *geturl, *suff, *hsep, *psep, *isep;

    suff = "";
    hsep = "";
    psep = "";
    isep = "";
    if(port && strlen(port) != 0)
        hsep = ":";
    if(path && path[0] != '/')
        psep = "/";
    if(path && path[0] && path[strlen(path)-1] != '/')
        isep = "/";
    memset(c, 0, sizeof(*c));
    geturl = smprintf("https://%s%s%s%s%s%sinfo/refs?service=git-%s-pack",
        host, hsep, port, psep, path, suff, isep, direction);
    c->type = ConnHttp;
    c->url = smprintf("https://%s%s%s%s%s%s%git-%s-pack",
        host, hsep, port, psep, path, suff, isep, direction);
    c->cfd = webclone(c, geturl);
    free(geturl);
    if(c->cfd == -1)
        return -1;
    c->rfd = webopen(c, "body", OREAD);
    c->wfd = -1;
    if(c->rfd == -1)
        return -1;
    if(issmarthttp(c, direction) == -1)
        return -1;
    c->direction = estrdup(direction);
    return 0;
}
```

24.1.2 ssh://

```
<function dialssh 152b>≡ (226c)
```

```

static int
dialssh(Conn *c, char *host, char *, char *path, char *direction)
{
    int pid, pfd[2];
    char cmd[64];

    if(pipe(pfd) == -1)
        sysfatal("unable to open pipe: %r");
    pid = fork();
    if(pid == -1)
        sysfatal("unable to fork");
    if(pid == 0){
        close(pfd[1]);
        dup(pfd[0], 0);
        dup(pfd[0], 1);
        snprintf(cmd, sizeof(cmd), "git-%s-pack", direction);
        dprint(1, "exec ssh '%s' '%s' %s\n", host, cmd, path);
        execl("/bin/ssh", "ssh", host, cmd, path, nil);
        sysfatal("exec: %r");
    }
    close(pfd[0]);
    c->type = ConnSsh;
    c->rfd = pfd[1];
    c->wfd = dup(pfd[1], -1);
    return 0;
}

```

24.2 Other servers

24.2.1 http://

Chapter 25

Advanced Topics

25.1 Bytes versus ASCII versus UTF-8

25.2 Reliability and signals

25.3 Optimizations

25.3.1 Object cache

<global objcache 154a>≡ (215b)
Objset objcache;

<gitinit() set objcache 154b>≡ (51f)
osinit(&objcache);

<Object cache fields 154c>+≡ (30c) <31e
int id;

<readidxobject() if h in object cache 154d>≡ (39b)
if((obj = osfind(&objcache, h)) != nil){
 if(flag & Cidx){
 /*
 * If we're indexing, we need to be careful
 * to only return objects within this pack,
 * so if the objects exist outside the pack,
 * we don't index the wrong copy.
 */
 if(!(obj->flag & Cidx))
 return nil;
 if(obj->flag & Cloaded)
 return obj;
 o = Boffset(idx);
 if(Bseek(idx, obj->off, 0) == -1)
 return nil;
 if(readpacked(idx, obj, flag) == -1)
 return nil;
 if(Bseek(idx, o, 0) == -1)
 sysfatal("could not restore offset");
 cache(obj);
 return obj;
 }
 if(obj->flag & Cloaded)
 return obj;
}

25.4 Fast import and export

25.5 Alternates

25.6 Other repository format

25.6.1 Bare repository

25.6.2 .git file

25.7 Security

Chapter 26

Conclusion

Appendix A

Debugging

A.1 Format dumpers

```
<pragmas varargck 157a>+≡ (185f) <29e
    #pragma varargck type "T" int
    #pragma varargck type "O" Object*
    #pragma varargck type "Q" Qid
```

```
<gitinit() fmtinstall calls 157b>+≡ (51f) <29f
    fmtinstall('T', Tfmt);
    fmtinstall('O', Ofmt);
    fmtinstall('Q', Qfmt);
```

```
<function Tfmt 157c>≡ (258)
    int
    Tfmt(Fmt *fmt)
    {
        int t;
        int l;

        t = va_arg(fmt->args, int);
        switch(t){
        case GNone: l = fmtprint(fmt, "none"); break;
        case GCommit: l = fmtprint(fmt, "commit"); break;
        case GTree: l = fmtprint(fmt, "tree"); break;
        case GBlob: l = fmtprint(fmt, "blob"); break;
        case GTag: l = fmtprint(fmt, "tag"); break;
        case GDelta: l = fmtprint(fmt, "delta"); break;
        case GRdelta: l = fmtprint(fmt, "gdelta"); break;
        default: l = fmtprint(fmt, "?%d?", t); break;
        }
        return l;
    }
```

```
<function Ofmt 157d>≡ (258)
    int
    Ofmt(Fmt *fmt)
    {
        Object *o;
        int l;

        o = va_arg(fmt->args, Object *);
        print("== %H (%T) ==\n", o->hash, o->type);
        switch(o->type){
        case GTree:
```

```

        l = fprintf(fmt, "tree\n");
        break;
case GBlob:
    l = fprintf(fmt, "blob %s\n", o->data);
    break;
case GCommit:
    l = fprintf(fmt, "commit\n");
    break;
case GTag:
    l = fprintf(fmt, "tag\n");
    break;
default:
    l = fprintf(fmt, "invalid: %d\n", o->type);
    break;
}
return l;
}

```

<function Qfmt 158a>≡ (258)

```

int
Qfmt(Fmt *fmt)
{
    Qid q;

    q = va_arg(fmt->args, Qid);
    if(q.path == ~OULL && q.vers == ~OUL && q.type == 0xff)
        return fprintf(fmt, "NOQID");
    else
        return fprintf(fmt, "%llx.%lud.%hhx", q.path, q.vers, q.type);
}

```

A.2 git/fs -d

<main() (fs.c) command line processing 158b>+≡ (55b) <75b

```

case 'd':
    chatty9p++;
    break;

```

A.3 git/xxx -d

<global chattygit 158c>≡ (258)

```

int chattygit;

```

<macro dprint 158d>≡ (185f)

```

#define dprint(lvl, ...) \
    if(chattygit >= lvl) _dprint(__VA_ARGS__)

```

<function _dprint 158e>≡ (258)

```

void
_dprint(char *fmt, ...)
{
    va_list ap;

    va_start(ap, fmt);
    vfprint(STDERR, fmt, ap);
    va_end(ap);
}

```

$\langle \text{main()} (query.c) \text{ command line processing } 159 \rangle + \equiv$
case 'd': chattygit++; break;

(76b) $\triangleleft 87a$

Appendix B

Error Management

```
<fn die(common.rc) 160a>≡ (165a)
  fn die{
    >[1=2] echo $0: $*
    exit "$*"
  }
```

```
<constant Egreg 160b>≡ (174)
  #define Egreg "wat"
```

```
<constant Enodir 160c>≡ (174)
  #define Enodir "not a directory"
```

```
<constant Eperm 160d>≡ (174)
  #define Eperm "permission denied"
```

```
<constant Eexist 160e>≡ (174)
  #define Eexist "file does not exist"
```

```
<constant Enotdir 160f>≡ (174)
  #define Enotdir "is not a directory"
```

```
<constant E2long 160g>≡ (174)
  #define E2long "path too long"
```

```
<constant Erepo 160h>≡ (174)
  #define Erepo "unable to read repo"
```

```
<constant Eimpl 160i>≡ (174)
  #define Eimpl "not implemented"
```

```
<constant Ebadobj 160j>≡ (174)
  #define Ebadobj "invalid object"
```

Appendix C

Utilities

C.1 Error malloc

⟨function emalloc 161a⟩≡ (258)

```
void *
emalloc(ulong n)
{
    void *v;

    v = mallocz(n, 1);
    if(v == nil)
        sysfatal("malloc: %r");
    setmalloctag(v, getcallerpc(&n));
    return v;
}
```

⟨function eamalloc 161b⟩≡ (258)

```
void *
eamalloc(ulong n, ulong sz)
{
    uvlong na;
    void *v;

    na = (uvlong)n*(uvlong)sz;
    if(na >= (1ULL<<30))
        sysfatal("alloc: overflow");
    v = mallocz(na, 1);
    if(v == nil)
        sysfatal("malloc: %r");
    setmalloctag(v, getcallerpc(&n));
    return v;
}
```

⟨function erealloc 161c⟩≡ (258)

```
void *
erealloc(void *p, ulong n)
{
    void *v;

    v = realloc(p, n);
    if(v == nil)
        sysfatal("realloc: %r");
    setmalloctag(v, getcallerpc(&p));
    return v;
}
```

<function earealloc 162a>≡ (258)

```
void *
earealloc(void *p, ulong n, ulong sz)
{
    uulong na;
    void *v;

    na = (uulong)n*(uulong)sz;
    if(na >= (1ULL<<30))
        sysfatal("alloc: overflow");
    v = realloc(p, na);
    if(v == nil)
        sysfatal("realloc: %r");
    setmalloctag(v, getcallerpc(&p));
    return v;
}
```

<function estrdup 162b>≡ (258)

```
char*
estrdup(char *s)
{
    s = strdup(s);
    if(s == nil)
        sysfatal("strdup: %r");
    setmalloctag(s, getcallerpc(&s));
    return s;
}
```

<function emalloc(diff) 162c>≡ (291a)

```
void *
emalloc(unsigned n)
{
    register void *p;

    if ((p = malloc(n)) == 0)
        sysfatal("malloc: %r");
    return p;
}
```

<function erealloc(diff) 162d>≡ (291a)

```
void *
erealloc(void *p, unsigned n)
{
    void *rp;

    if ((rp = realloc(p, n)) == 0)
        sysfatal("realloc: %r");
    return rp;
}
```

<function emalloc(patch.c) 162e>≡ (303)

```
void *
emalloc(ulong n)
{
    void *v;

    v = mallocz(n, 1);
    if(v == nil)
        fail("malloc: %r");
    setmalloctag(v, getcallerpc(&n));
    return v;
}
```

```

⟨function erealloc(patch.c) 163a⟩≡ (303)
void *
erealloc(void *v, ulong n)
{
    if(n == 0)
        n++;
    v = realloc(v, n);
    if(v == nil)
        fail("malloc: %r");
    setmalloctag(v, getcallerpc(&n));
    return v;
}

```

C.2 String utilities

```

⟨macro isblank 163b⟩≡ (185f)
#define isblank(c) \
    (((c) != '\n') && isspace(c))

```

```

⟨function strip 163c⟩≡ (258)
char *
strip(char *s)
{
    char *e;

    while(isspace(*s))
        s++;
    e = s + strlen(s);
    while(e > s && isspace(*--e))
        *e = 0;
    return s;
}

```

```

⟨function scanword 163d⟩≡ (215b)
/*
 * Scans for non-empty word, copying it into buf.
 * Strips off word, leading, and trailing space
 * from input.
 *
 * Returns -1 on empty string or error, leaving
 * input unmodified.
 */
static errorneg1
scanword(char **str, int *nstr, char *buf, int nbuf)
{
    char *p;
    int n;
    errorneg1 r;

    r = ERROR_NEG1;
    p = *str;
    n = *nstr;
    while(n && isblank(*p)){
        n--;
        p++;
    }

    for(; n && *p && !isspace(*p); p++, n--){

```

```

    r = OK_0;
    *buf++ = *p;
    nbuf--;
    if(nbuf == 0)
        return ERROR_NEG1;
}
while(n && isblank(*p)){
    n--;
    p++;
}
*buf = '\0';
*str = p;
*nstr = n;
return r;
}

```

Appendix D

Extra Code

D.1 git9/

D.1.1 git9/common.rc

```
<git9/common.rc 165a>≡  
  <global nl(common.rc) 97a>  
  
  <fn die(common.rc) 160a>  
  
  <fn subst(common.rc) 101b>  
  <fn drop(common.rc) 97c>  
  
  <fn whoami(common.rc) 99a>  
  
  <fn mergeperm(common.rc) 112a>  
  <fn merge1(common.rc) 111b>  
  
  <fn gitup(common.rc) 96d>
```

D.1.2 git9/add.rc

D.1.3 git9/branch.rc

D.1.4 git9/clone.rc

D.1.5 git9/commit.rc

D.1.6 git9/compat.rc

```
<git9/compat.rc 165b>≡  
  #!/bin/rc  
  
  rfork e  
  
  opts=()  
  args=()  
  nl=''  
  ,  
  
  fn cmd_init{  
    while(! ~ $#* 0){  
      switch($1){
```

```

        case --bare
            # ignore
        case --
            # go likes to use these
        case -*
            die unknown command init $*
        case *
            args=($args $1)
    }
    shift
}
git/init $opts $args
}

fn cmd_clone{
    branch=()
    while( ! ~ $#* 0){
        switch($1){
            case -b
                branch=$2
                shift
            case --
                # go likes to use these
            case -*
                die unknown command clone $*
            case *
                args=($args $1)
        }
        shift
    }
    git/clone $opts $args
    if(~ $#branch 1)
        git/branch -n -b $1 origin/$1
}

fn cmd_pull{
    if(~ $1 -*)
        die unknown options for pull $*
    git/pull
}

fn cmd_fetch{
    while(~ $#* 0){
        switch($1){
            case --all
                opts=($opts -a)
            case -f
                opts=($opts -u $2)
                shift
            case --
                # go likes to use these
            case -*
                die unknown command clone $*
            case *
                args=($args $1)
        }
        shift
    }
    git/pull -f $opts
}

```

```

fn cmd_checkout{
    if(~ $1 -*)
        die unknown command pull $*
    if(~ $#* 0)
        die git checkout branch
    git/branch $b
}

fn cmd_submodule {
    if(test -f .gitmodules)
        die 'submodules unsupported'
}

fn cmd_rev-parse{
    while(~ $1 -*){
        switch($1){
            case --git-dir
                echo $gitroot/.git
                shift
            case --abbrev-ref
                echo '{dcmd git9/branch | sed s@^heads/@@g}'
                shift
            case *
                die unknown option $opt
        }
        shift
    }
}

fn cmd_show-ref{
    if(~ $1 -*)
        die unknown command pull $*
    filter=cat
    if(~ $#* 0)
        filter=cat
    if not
        filter='-e(^|/)'~$*~$'
    for(b in `nl{cd $gitroot/.git/refs/ && walk -f}`)
        echo '{cat $gitroot/.git/refs/$b} refs/$b'
}

fn cmd_rev-parse{
    switch($1){
        case --git-dir
            echo '{git/conf -r}^/.git
        case *
            die 'unknown rev-parse '$*
    }
}

fn cmd_remote{
    if({! ~ $#* 3 && ! ~ $#* 4} || ! ~ $1 add)
        die unimplemented remote cmd $*
    name=$2
    url=$3
    if(~ $3 '--')
        url=$4
    >>$gitroot/.git/config{

```



```

        echo '[remote "'$name']',
        echo ' url='$url
    }
}

fn cmd_log{
    count=()
    format=''
    while(~ $1 -*){
        switch($1){
            case --format
                format=$2
                shift
            case '--format= '*
                format='{echo $1 | sed 's/--format=//g'}'
            case -n
                count=-n$2
                shift
            case -n*
                count=$1
            case *
                dprint option $opt
        }
        shift
    }
    @{cd $gitroot && git/fs}
    switch($format){
        case ''
            git/log $count
        case '%H:%ct'
            for(c in '{git/log -s $count| awk '{print $1}'}')
                echo $c:'{mtime $gitroot/.git/fs/object/$c/msg}'
        case '%h %cd'
            for(c in '{git/log -s $count| awk '{print $1}'}')
                echo $c '{date '{mtime $gitroot/.git/fs/object/$c/msg}}'
    }
}

}

fn cmd_show{
    cmd_log -n1 $*
}

fn cmd_ls-remote{
    if(~ $1 -q)
        shift
    remote='${nl}{git/conf 'remote "'$1'".url}'
    if(~ $#remote 0)
        remote=$1
    git/get -l $remote | awk '/^remote/{print $3"\t"$2}'
}

fn cmd_version{
    echo git version 2.2.0
}

fn cmd_status{
    echo
}

```

```

fn usage{
    echo 'git <command> <args>' >[1=2]
    exit usage
}

fn die {
    >[1=2] echo git $_cmdname: $*
    exit $_cmdname: $*
}

_cmdname=$1
if(~ $0 *compat){
    ramfs -m /n/gitcompat
    touch /n/gitcompat/git
    bind $0 /n/gitcompat/git
    path=( /n/gitcompat $path )
    exec rc
}

if(~ $#gitcompatdebug 1)
    echo running $* >>/tmp/gitlog

if(~ $1 -c)
    shift 2
if(~ $1 -c*)
    shift 1
if(! test -f '/env/fn#cmd_'$1)
    die git $1: command not implemented
if(! ~ $1 init && ! ~ $1 clone)
    gitroot='{git/conf -r} || die repo

if(~ $#gitcompatdebug 1)
    cmd_$1 $*(2-) | tee >>/tmp/gitlog
if not
    cmd_$1 $*(2-)
exit ''

```

D.1.7 git9/diff.rc

D.1.8 git9/export.rc

D.1.9 git9/import.rc

D.1.10 git9/hist.rc

D.1.11 git9/init.rc

D.1.12 git9/merge.rc

D.1.13 git9/pull.rc

D.1.14 git9/push.rc

D.1.15 git9/rebase.rc

D.1.16 git9/revert.rc

D.1.17 git9/rm.rc

D.1.18 git9/conf.c

```
<git9/conf.c 170a>≡
#include <u.h>
#include <libc.h>
#include <ctype.h>

#include "git.h"

<global findroot 52b>
<global showall 53b>
<global nfile 51c>
<global file 51b>

<function showconf 53a>

<function usage 50a>
<function main 50b>
```

D.1.19 git9/delta.c

```
<function hash 170b>≡ (173b)
static u64int
hash(void *p, int n)
{
    return murmurhash2(p, n);
}

<function addblk 170c>≡ (173b)
static void
addblk(Dtab *dt, void *buf, int len, int off, u64int h)
{
    int i, sz, probe;
    Dblock *db;

    probe = h % dt->sz;
```

```

while(dt->b[probe].buf != nil){
    if(len == dt->b[probe].len && memcmp(buf, dt->b[probe].buf, len) == 0)
        return;
    probe = (probe + 1) % dt->sz;
}
assert(dt->b[probe].buf == nil);
dt->b[probe].buf = buf;
dt->b[probe].len = len;
dt->b[probe].off = off;
dt->b[probe].hash = h;
dt->nb++;
if(dt->sz < 2*dt->nb){
    sz = dt->sz;
    db = dt->b;
    dt->sz *= 2;
    dt->nb = 0;
    dt->b = eamalloc(dt->sz, sizeof(Dblock));
    for(i = 0; i < sz; i++)
        if(db[i].buf != nil)
            addblk(dt, db[i].buf, db[i].len, db[i].off, db[i].hash);
    free(db);
}
}

```

⟨function lookup 171a⟩≡ (173b)

```

static Dblock*
lookup(Dtab *dt, uchar *p, int n)
{
    int probe;
    u64int h;

    h = hash(p, n);
    for(probe = h % dt->sz; dt->b[probe].buf != nil; probe = (probe + 1) % dt->sz){
        if(dt->b[probe].hash != h)
            continue;
        if(n != dt->b[probe].len)
            continue;
        if(memcmp(p, dt->b[probe].buf, n) != 0)
            continue;
        return &dt->b[probe];
    }
    return nil;
}

```

⟨function nextblk 171b⟩≡ (173b)

```

static int
nextblk(uchar *s, uchar *e)
{
    u32int gh;
    uchar *p;

    if((e - s) < Minchunk)
        return e - s;
    p = s + Minchunk;
    if((e - s) > Maxchunk)
        e = s + Maxchunk;
    gh = 0;
    while(p != e){
        gh = (gh<<1) + geartab[*p++];
        if((gh & Splitmask) == 0)

```

```

        break;
    }
    return p - s;
}

```

⟨function dtinit 172a⟩≡ (173b)

```

void
dtinit(Dtab *dt, Object *obj)
{
    uchar *s, *e;
    u64int h;
    vlong n, o;

    o = 0;
    s = (uchar*)obj->data;
    e = s + obj->size;
    dt->o = ref(obj);
    dt->nb = 0;
    dt->sz = 128;
    dt->b = eamalloc(dt->sz, sizeof(Dblock));
    dt->base = (uchar*)obj->data;
    dt->nbase = obj->size;
    while(s != e){
        n = nextblk(s, e);
        h = hash(s, n);
        addblk(dt, s, n, o, h);
        s += n;
        o += n;
    }
}

```

⟨function dtclear 172b⟩≡ (173b)

```

void
dtclear(Dtab *dt)
{
    unref(dt->o);
    free(dt->b);
}

```

⟨function emitdelta 172c⟩≡ (173b)

```

static int
emitdelta(Delta **pd, int *nd, int cpy, int off, int len)
{
    Delta *d;

    *nd += 1;
    *pd = earealloc(*pd, *nd, sizeof(Delta));
    d = &(*pd)[*nd - 1];
    d->cpy = cpy;
    d->off = off;
    d->len = len;
    return len;
}

```

⟨function stretch 172d⟩≡ (173b)

```

static int
stretch(Dtab *dt, Dblock *b, uchar *s, uchar *e, int n)
{
    uchar *p0, *p, *q, *eb;

```

```

    if(b == nil)
        return n;
    p = s + n;
    q = dt->base + b->off + n;
    p0 = p;
    if(dt->nbase < (1<<24)-1)
        eb = dt->base + dt->nbase;
    else
        eb = dt->base + (1<<24)-1;
    while(1){
        if(p == e || q == eb)
            break;
        if(*p != *q)
            break;
        p++;
        q++;
    }
    return n + (p - p0);
}

```

⟨function deltify 173a⟩≡ (173b)

```

Delta*
deltify(Object *obj, Dtab *dt, int *pnd)
{
    Delta *d;
    Dblock *b;
    uchar *s, *e;
    vlong n, o;

    o = 0;
    d = nil;
    s = (uchar*)obj->data;
    e = s + obj->size;
    *pnd = 0;
    while(s != e){
        n = nextblk(s, e);
        b = lookup(dt, s, n);
        n = stretch(dt, b, s, e, n);
        if(b != nil)
            emitdelta(&d, pnd, 1, b->off, n);
        else
            emitdelta(&d, pnd, 0, o, n);
        s += n;
        o += n;
    }
    return d;
}

```

⟨git9/delta.c 173b⟩≡

```

#include <u.h>
#include <libc.h>

#include "git.h"

enum {
    Minchunk    = 128,
    Maxchunk    = 8192,
    Splitmask   = (1<<8)-1,
};

```

```
static u32int geartab[] = {
    0x67ed26b7, 0x32da500c, 0x53d0fee0, 0xce387dc7, 0xcd406d90, 0x2e83a4d4, 0x9fc9a38d, 0xb67259dc,
    0xca6b1722, 0x6d2ea08c, 0x235cea2e, 0x3149bb5f, 0x1beda787, 0x2a6b77d5, 0x2f22d9ac, 0x91fc0544,
    0xe413acfa, 0x5a30ff7a, 0xad6fdd0, 0x444fd0f5, 0x7ad87864, 0x58c5ff05, 0x8d2ec336, 0x2371f853,
    0x550f8572, 0x6aa448dd, 0x7c9ddbcbf, 0x95221e14, 0x2a82ec33, 0xcbec5a78, 0xc6795a0d, 0x243995b7,
    0x1c909a2f, 0x4fde51c, 0x635d334b, 0x0e2b9999, 0x2702968d, 0x856de1d5, 0x3325d60e, 0xeb6a7502,
    0xec2a9844, 0x0905835a, 0xa1820375, 0xa4be5cab, 0x96a6c058, 0x2c2ccd70, 0xba40fce3, 0xd794c46b,
    0x8fbae83e, 0xc3aa7899, 0x3d3ff8ed, 0xa0d42b5b, 0x571c0c97, 0xd2811516, 0xf7e7b96c, 0x4fd2fcdb,
    0xe2fdec94, 0x282cc436, 0x78e8e95c, 0x80a3b613, 0xcfbee20c, 0xd4a32d1c, 0x2a12ff13, 0x6af82936,
    0xe5630258, 0x8efa6a98, 0x294fb2d1, 0xdeb57086, 0x5f0fddb3, 0xeceda7ce, 0x4c87305f, 0x3a6d3307,
    0xe22d2942, 0x9d060217, 0x1e42ed02, 0xb6f63b52, 0x4367f39f, 0x055cf262, 0x03a461b2, 0x5ef9e382,
    0x386bc03a, 0x2a1e79c7, 0xf1a0058b, 0xd4d2dea9, 0x56baf37d, 0x5daff6cc, 0xf03a951d, 0xae7de45,
    0xa8f4581e, 0x3960b555, 0xffbfff6d, 0xbe702a23, 0x8f5b6d6f, 0x061739fb, 0x98696f47, 0x3fd596d4,
    0x151eac6b, 0xa9fcc4f5, 0x69181a12, 0x3ac5a107, 0xb5198fe7, 0x96bcb1da, 0x1b5ddf8e, 0xc757d650,
    0x65865c3a, 0x8fc0a41a, 0x87435536, 0x99eda6f2, 0x41874794, 0x29cff4e8, 0xb70efd9a, 0x3103f6e7,
    0x84d2453b, 0x15a450bd, 0x74f49af1, 0x60f664b1, 0xa1c86935, 0xfdafbce1, 0xe36353e3, 0x5d9ba739,
    0xbc0559ba, 0x708b0054, 0xd41d808c, 0xb2f31723, 0x9027c41f, 0xf136d165, 0xb5374b12, 0x9420a6ac,
    0x273958b6, 0xe6c2fad0, 0xebdc1f21, 0xfb33af8b, 0xc71c25cd, 0xe9a2d8e5, 0xbeb38a50, 0xbceb7cc2,
    0x4e4e73f0, 0xcd6c251d, 0xde4c032c, 0x4b04ac30, 0x725b8b21, 0x4eb8c33b, 0x20d07b75, 0x0567aa63,
    0xb56b2bb7, 0xc1f5fd3a, 0xcafd35ca, 0x470dd4da, 0xfe4f94cd, 0xfb8de424, 0xe8dbcf40, 0xfe50a37a,
    0x62db5b5d, 0xf32f4ab6, 0x2c4a8a51, 0x18473dc0, 0xfe0cbb6e, 0xfe399efd, 0xdf34ecc9, 0x6ccd5055,
    0x46097073, 0x139135c2, 0x721c76f6, 0x1c6a94b4, 0x6eee014d, 0x8a508e02, 0x3da538f5, 0x280d394f,
    0x5248a0c4, 0x3ce94c6c, 0x9a71ad3a, 0x8493dd05, 0xe43f0ab6, 0x18e4ed42, 0x6c5c0e09, 0x42b06ec9,
    0x8d330343, 0xa45b6f59, 0x2a573c0c, 0xd7fd3de6, 0xeedeab68, 0x5c84dafc, 0xbbd1b1a8, 0xa3ce1ad1,
    0x85b70bed, 0xb6add07f, 0xa531309c, 0x8f8ab852, 0x564de332, 0xeac9ed0c, 0x73da402c, 0x3ec52761,
    0x43af2f4d, 0xd6ff45c8, 0x4c367462, 0xd553bd6a, 0x44724855, 0x3b2aa728, 0x56e5eb65, 0xeaf16173,
    0x33fa42ff, 0xd714bb5d, 0xfbd0a3b9, 0xaf517134, 0x9416c8cd, 0x534cf94f, 0x548947c2, 0x34193569,
    0x32f4389a, 0xfe7028bc, 0xed73b1ed, 0x9db95770, 0x468e3922, 0x0440c3cd, 0x60059a62, 0x33504562,
    0x2b229fbd, 0x5174dca5, 0xf7028752, 0xd63c6aa8, 0x31276f38, 0x0646721c, 0xb0191da8, 0xe00e6de0,
    0x9eac1a6e, 0x9f7628a5, 0xed6c06ea, 0x0bb8af15, 0xf119fb12, 0x38693c1c, 0x732bc0fe, 0x84953275,
    0xb82ec888, 0x33a4f1b3, 0x3099835e, 0x028a8782, 0x5fdd51d7, 0xc6c717b3, 0xb06caf71, 0x17c8c111,
    0x61bad754, 0x9fd03061, 0xe09df1af, 0x3bc9eb73, 0x85878413, 0x9889aaf2, 0x3f5a9e46, 0x42c9f01f,
    0x9984a4f4, 0xd5de43cc, 0xd294daed, 0xbecba2d2, 0xf1f6e72c, 0x5551128a, 0x83af87e2, 0xf0342ba,
};
```

<function hash 170b>

<function addblk 170c>

<function lookup 171a>

<function nextblk 171b>

<function dtinit 172a>

<function dtclear 172b>

<function emitdelta 172c>

<function stretch 172d>

<function deltify 173a>

D.1.20 git9/fs.c

```
<git9/fs.c 174>≡
#include <u.h>
#include <libc.h>
#include <ctype.h>
```

```

#include <fcntl.h>
#include <thread.h>
#include <9p.h>

#include "git.h"

<enum Qxxx 56g>

typedef struct Gitaux Gitaux;
typedef struct Crumb Crumb;
typedef struct Cache Cache;
typedef struct Uqid Uqid;

<struct Crumb 57d>
<struct Gitaux 57b>

<struct Uqid 58c>
<struct Cache 59a>

<constant qroot 57a>

<constant Eperm 160d>
<constant Eexist 160e>
<constant Enotdir 160f>
<constant E2long 160g>
<constant Enodir 160c>
<constant Erepo 160h>
<constant Eimpl 160i>
<constant Egreg 160b>
<constant Ebadobj 160j>

<global gitdir 59f>
<global username 56a>
<global groupname 56b>
<global mntpt 56c>
<global branches 56d>
<global uqidcache 59b>
<global nextqid 59d>

static Object* walklink(Gitaux *, char *, int, int, int*);

<function qpath 59c>

<function crumb 57e>

<function popcrumb 64a>

<function branchid 74c>

<function obj2dir 62a>

<function rootgen 64c>

<function branchgen 74b>
<function gtreegen 68f>
<function gcommitgen 70a>

<function objgen 66b>
<function objread 68a>

```



```

⟨function readcommitparent 71d⟩
⟨function gitattach 60a⟩
⟨function walklink 146e⟩
⟨function objwalk1 67b⟩
⟨function readref 73a⟩
⟨function gitwalk1 63a⟩
⟨function gitclone 60c⟩
⟨function gitdestroyfid 61a⟩
⟨function readctl 65b⟩
⟨function gitread 62b⟩
⟨function gitopen 61b⟩
⟨function gitstat 61c⟩
⟨global gitsrv 56f⟩
⟨function usage (git9/fs.c) 55a⟩
⟨function main (git9/fs.c) 55b⟩

```

D.1.21 git9/get.c

```

⟨function okrefname 176a⟩≡ (184)
/*
 * Checks the rules for a refname at
 * git/Documentation/protocol-common.txt
 */
int
okrefname(char *s)
{
    if(strcmp(s, "HEAD") == 0)
        return 1;
    if(strncmp(s, "refs/", 5) == 0)
        return okref(s);
    return 0;
}

⟨function resolveremote 176b⟩≡ (184)
int
resolveremote(Hash *h, char *ref)
{
    char buf[128], *s;
    int r, f;

    ref = strip(ref);
    if((r = hparse(h, ref)) != ERROR_NEG1)
        return r;
    /* Slightly special handling: translate remote refs to local ones. */
    if(strcmp(ref, "HEAD") == 0){
        snprintf(buf, sizeof(buf), ".git/HEAD");
    }else if(strstr(ref, "refs/heads") == ref){

```

```

    ref += strlen("refs/heads");
    snprintf(buf, sizeof(buf), ".git/refs/remotes/%s/%s", upstream, ref);
} else if (strstr(ref, "refs/tags") == ref) {
    ref += strlen("refs/tags");
    snprintf(buf, sizeof(buf), ".git/refs/tags/%s/%s", upstream, ref);
} else {
    return -1;
}

r = ERROR_NEG1;
s = strip(buf);
if ((f = open(s, OREAD)) == -1)
    return -1;
if (readn(f, buf, sizeof(buf)) >= 40)
    r = hparse(h, buf);
close(f);

if (r == -1 && strstr(buf, "ref:") == buf)
    return resolveremote(h, buf + strlen("ref:"));
return r;
}

```

<function rename 177a> ≡ (184)

```

int
rename(char *pack, char *idx, Hash h)
{
    char name[128];
    Dir st;

    nulldir(&st);
    st.name = name;
    snprintf(name, sizeof(name), "%H.pack", h);
    if (access(name, AEXIST) == 0)
        fprintf(2, "warning, pack %s already fetched\n", name);
    else if (dirwstat(pack, &st) == -1)
        return -1;
    snprintf(name, sizeof(name), "%H.idx", h);
    if (access(name, AEXIST) == 0)
        fprintf(2, "warning, pack %s already indexed\n", name);
    else if (dirwstat(idx, &st) == -1)
        return -1;
    return 0;
}

```

<function checkhash 177b> ≡ (184)

```

int
checkhash(int fd, vlong sz, Hash *hcomp)
{
    DigestState *st;
    Hash hexpect;
    char buf[Pktmax];
    vlong n, r;
    int nr;

    if (sz < 28) {
        werrstr("undersize packfile");
        return -1;
    }

    st = nil;

```

```

n = 0;
while(n != sz - 20){
    nr = sizeof(buf);
    if(sz - n - 20 < sizeof(buf))
        nr = sz - n - 20;
    r = readn(fd, buf, nr);
    if(r != nr)
        return -1;
    st = sha1((uchar*)buf, nr, nil, st);
    n += r;
}
sha1(nil, 0, hcomp->h, st);
if(readn(fd, hexpect.h, sizeof(hexpect.h)) != sizeof(hexpect.h))
    sysfatal("truncated packfile");
if(!hasheq(hcomp, &hexpect)){
    werrstr("bad hash: %H != %H", *hcomp, hexpect);
    return -1;
}
return 0;
}

```

⟨function mkoutpath 178a⟩≡ (184)

```

int
mkoutpath(char *path)
{
    char s[128];
    char *p;
    int fd;

    snprintf(s, sizeof(s), "%s", path);
    for(p=strchr(s+1, '/'); p; p=strchr(p+1, '/')){
        *p = 0;
        if(access(s, AEXIST) != 0){
            fd = create(s, OREAD, DMDIR | 0775);
            if(fd == -1)
                return -1;
            close(fd);
        }
        *p = '/';
    }
    return 0;
}

```

⟨function prefixed 178b⟩≡ (184)

```

int
prefixed(char *s, char *pfx)
{
    return strncmp(s, pfx, strlen(pfx)) == 0;
}

```

⟨function branchmatch 178c⟩≡ (184)

```

int
branchmatch(char *br, char *pat)
{
    char name[128];

    if(prefixed(pat, "refs/heads"))
        snprintf(name, sizeof(name), "%s", pat);
    else if(prefixed(pat, "heads/"))
        snprintf(name, sizeof(name), "refs/%s", pat);
}

```

```

    else
        snprintf(name, sizeof(name), "refs/heads/%s", pat);
    return strcmp(br, name) == 0;
}

⟨function fail 179a⟩≡ (184)
void
fail(char *pack, char *idx, char *msg, ...)
{
    char buf[ERRMAX];
    va_list ap;

    va_start(ap, msg);
    snprintf(buf, sizeof(buf), msg, ap);
    va_end(ap);

    remove(pack);
    remove(idx);
    fprintf(2, "%s", buf);
    exits(buf);
}

⟨function enqueueparent 179b⟩≡ (184)
void
enqueueparent(Object *q, Object *o)
{
    Object *p;
    int i;

    if(o->type != GCommit)
        return;
    for(i = 0; i < o->commit->nparent; i++){
        if((p = readobject(o->commit->parent[i])) == nil)
            continue;
        qput(q, p, 0);
        unref(p);
    }
}

⟨function fmtcaps 179c⟩≡ (184)
void
fmtcaps(Conn *c, char *caps, int ncaps)
{
    char *p, *e;

    p = caps;
    e = caps + ncaps;
    *p = 0;
    if(c->multiack)
        p = seprint(p, e, " multi_ack");
    if(c->sideband64k)
        p = seprint(p, e, " side-band-64k");
    else if(c->sideband)
        p = seprint(p, e, " side-band");
    assert(p != e);
}

⟨function sbread 179d⟩≡ (184)
int
sbread(Conn *c, char *buf, int nbuf, char **pbuf)

```

```

{
    int n;

    assert(nbuf >= Pktmax);
    if(!c->sideband && !c->sideband64k){
        *pbuf = buf;
        return readn(c->rfd, buf, nbuf);
    }else{
        *pbuf = buf+1;
        while(1){
            n = readpkt(c, buf, nbuf);
            if(n <= 0)
                return n;
            else if(buf[0] == 1 && n > 1)
                return n - 1;
            else if(buf[0] == 3)
                fprintf(2, "error: %s\n", buf+1);
            else if(buf[0] < 1 || buf[0] > 3)
                fprintf(2, "unknown sideband(%c:%d) data: %s\n", buf[0], buf[0], buf+1);

        }
    }
}

```

⟨function fetchpack 180⟩≡ (184)

```

int
fetchpack(Conn *c)
{
    char spinner[] = {'|', '/', '-', '\\'};
    char buf[Pktmax], caps[512], *sp[3], *ep;
    char *packtmp, *idxtmp, **ref, *rp;
    Hash h, *have, *want;
    int nref, refs, first, nsent;
    int i, j, l, n, spin, req, pfd;
    vlong packs;
    Objset hadobj;
    Object *o;
    Objq haveq;
    Qelt e;

    nref = 0;
    refs = 16;
    first = 1;
    have = eamalloc(refs, sizeof(have[0]));
    want = eamalloc(refs, sizeof(want[0]));
    ref = eamalloc(refs, sizeof(ref[0]));
    while(1){
        n = readpkt(c, buf, sizeof(buf));
        if(n == -1)
            return -1;
        if(n == 0)
            break;

        if(first && n > strlen(buf)){
            parsecaps(buf + strlen(buf) + 1, c);
            if(c->symfrom[0] != 0)
                print("symref %s %s\n", c->symfrom, c->symto);
        }
        first = 0;
    }
}

```

```

if(getfields(buf, sp, nelem(sp), 1, " \t\n\r") < 2)
    sysfatal("invalid ref line");
if(strstr(sp[1], "^{}"))
    continue;
if(!okrefname(sp[1]))
    sysfatal("remote side sent invalid ref: %s", sp[1]);
if(fetchbranch && !branchmatch(sp[1], fetchbranch))
    continue;
else if(strcmp(sp[1], "HEAD") != 0
&& !prefixed(sp[1], "refs/heads/")
&& !prefixed(sp[1], "refs/tags/"))
    continue;
if(refsz == nref + 1){
    refsz *= 2;
    have = earealloc(have, refsz, sizeof(have[0]));
    want = earealloc(want, refsz, sizeof(want[0]));
    ref = earealloc(ref, refsz, sizeof(ref[0]));
}
if(hparse(&want[nref], sp[0]) == ERROR_NEG1)
    sysfatal("invalid hash %s", sp[0]);
if (resolveremote(&have[nref], sp[1]) == -1)
    memset(&have[nref], 0, sizeof(have[nref]));
ref[nref] = estrdup(sp[1]);
nref++;
}
if(listonly){
    flushpkt(c);
    goto showrefs;
}

if(writephase(c) == -1)
    sysfatal("write: %r");
req = 0;
fmtcaps(c, caps, sizeof(caps));
for(i = 0; i < nref; i++){
    if(hashcq(&have[i], &want[i]))
        goto skip;
    for(j = 0; j < i; j++)
        if(hashcq(&want[i], &want[j]))
            goto skip;
    if((o = readobject(want[i])) != nil){
        unref(o);
        continue;
    }
    if(fmtpkt(c, "want %H%s\n", want[i], caps) == -1)
        sysfatal("could not send want for %H", want[i]);
    caps[0] = 0;
    req = 1;
skip:;
}
flushpkt(c);

nsent = 0;
qinit(&haveq);
osinit(&hadobj);
/*
 * We know we have these objects, and we want to make sure that
 * they end up at the front of the queue. Send the 'have lines'
 * first, and then enqueue their parents for a second round of
 * sends.

```

```

*/
for(i = 0; i < nref; i++){
    if(hasheq(&have[i], &Zhash) || oshas(&hadobj, have[i]))
        continue;
    if((o = readobject(have[i])) == nil)
        sysfatal("missing expected object: %H", have[i]);
    if(fmtpkt(c, "have %H", o->hash) == -1)
        sysfatal("write: %r");
    enqueueparent(&haveq, o);
    osadd(&hadobj, o);
    unref(o);
    nsent++;
}
/*
* The other branches we have probably make sense to send,
* since often we'll be pulling a new branch with objects
* that we already have; it's not entirely clear what we
* want to do here.
*/
for(i = 0; i < nheads; i++){
    if((o = readobject(heads[i])) == nil)
        sysfatal("missing expected object: %H", have[i]);
    if(fmtpkt(c, "have %H", o->hash) == -1)
        sysfatal("write: %r");
    enqueueparent(&haveq, o);
    osadd(&hadobj, o);
    unref(o);
    nsent++;
}
/*
* While we could short circuit this and check if upstream has
* acked our objects, for the first 256 haves, this is simple
* enough.
*
* Also, doing multiple rounds of reference discovery breaks
* when using smart http.
*/
while(req && qpop(&haveq, &e) && nsent < 256){
    if(oshas(&hadobj, e.o->hash))
        continue;
    if((o = readobject(e.o->hash)) == nil)
        sysfatal("missing object we should have: %H", e.o->hash);
    if(fmtpkt(c, "have %H", o->hash) == -1)
        sysfatal("write: %r");
    enqueueparent(&haveq, o);
    osadd(&hadobj, o);
    unref(o);
    nsent++;
}
osclear(&hadobj);
qclear(&haveq);
if(!req)
    flushpkt(c);
if(fmtpkt(c, "done\n") == -1)
    sysfatal("write: %r");
if(!req)
    goto showrefs;
if(readphase(c) == -1)
    sysfatal("read: %r");

```

```

if((packtmp = smprint(".git/objects/pack/fetch.%d.pack", getpid())) == nil)
    sysfatal("smprint: %r");
if((idxtmp = smprint(".git/objects/pack/fetch.%d.idx", getpid())) == nil)
    sysfatal("smprint: %r");
if(mkoutpath(packtmp) == -1)
    sysfatal("could not create %s: %r", packtmp);
if((pfd = create(packtmp, ORDWR, 0664)) == -1)
    sysfatal("could not create %s: %r", packtmp);

fprintf(2, "fetching... ");
packsz = 0;
if(c->multiack){
    for(i = 0; i < nsent; i++){
        if(readpkt(c, buf, sizeof(buf)) == -1)
            sysfatal("read: %r");
        if(strncmp(buf, "NAK\n", 4) == 0)
            break;
        if(strncmp(buf, "ACK ", 4) == 0){
            if(getfields(buf, sp, nelem(sp), 1, " \t") == 2)
                break;
            continue;
        }
        sysfatal("bad response: '%s'", buf);
    }
}
if(readpkt(c, buf, sizeof(buf)) == -1)
    sysfatal("read: %r");
if(!c->sideband && !c->sideband64k && !c->multiack){
    /*
     * Work around torvalds git bug: we get duplicate have lines
     * sometimes, even though the protocol is supposed to start the
     * pack file immediately.
     *
     * Skip ahead until we read 'PACK' off the wire
     */
    while(1){
        if(readn(c->rfd, buf, 4) != 4)
            sysfatal("fetch packfile: short read");
        if(strncmp(buf, "PACK", 4) == 0)
            break;
        buf[4] = 0;
        l = strtol(buf, &ep, 16);
        if(ep != buf + 4)
            sysfatal("fetch packfile: junk pktline");
        if(readn(c->rfd, buf, l-4) != l-4)
            sysfatal("fetch packfile: short read");
    }
    if(write(pfd, "PACK", 4) != 4)
        sysfatal("write pack header: %r");
    packsiz = 4;
}
spin = 0;
while(1){
    n = sbread(c, buf, sizeof buf, &rp);
    if(n == 0)
        break;
    if(n == -1 || write(pfd, rp, n) != n)
        sysfatal("fetch packfile: %r");
    if(interactive && spin++ % 100 == 0)
        fprintf(2, "\b%c", spinner[spin/100 % nelem(spinner)]);
}

```



```

        packsz += n;
    }
    fprintf(2, "\n");
    closeconn(c);
    if(seek(pfd, 0, 0) == -1)
        fail(packtmp, idxtmp, "packfile seek: %r");
    if(checkhash(pfd, packsz, &h) == -1)
        fail(packtmp, idxtmp, "corrupt packfile: %r");
    close(pfd);
    if(indexpack(packtmp, idxtmp, h) == -1)
        fail(packtmp, idxtmp, "could not index fetched pack: %r");
    if(rename(packtmp, idxtmp, h) == -1)
        fail(packtmp, idxtmp, "could not rename indexed pack: %r");

showrefs:
    for(i = 0; i < nref; i++){
        print("remote %s %H local %H\n", ref[i], want[i], have[i]);
        free(ref[i]);
    }
    free(ref);
    free(want);
    free(have);
    return 0;
}

```

<git9/get.c 184>≡

```

#include <u.h>
#include <libc.h>

```

```

#include "git.h"

```

```

char *fetchbranch;
char *upstream = "origin";
Hash heads[64];
int nheads;
int listonly;

```

<function okrefname 176a>

<function resolveremote 176b>

<function rename 177a>

<function checkhash 177b>

<function mkoutpath 178a>

<function prefixed 178b>

<function branchmatch 178c>

<function fail 179a>

<function enqueueparent 179b>

<function fmtcaps 179c>

<function sbread 179d>

<function fetchpack 180>

<function usage (git9/get.c) 123a>

<function main (git9/get.c) 123b>

D.1.22 git9/git.h

<macro GETBE16 185a>≡ (185f)

```
#define GETBE16(b)\
    (((b)[0] & 0xFFu1) << 8) | \
    (((b)[1] & 0xFFu1) << 0))
```

<macro GETBE64 185b>≡ (185f)

```
#define GETBE64(b)\
    (((b)[0] & 0xFFu11) << 56) | \
    (((b)[1] & 0xFFu11) << 48) | \
    (((b)[2] & 0xFFu11) << 40) | \
    (((b)[3] & 0xFFu11) << 32) | \
    (((b)[4] & 0xFFu11) << 24) | \
    (((b)[5] & 0xFFu11) << 16) | \
    (((b)[6] & 0xFFu11) << 8) | \
    (((b)[7] & 0xFFu11) << 0))
```

<macro PUTBE16 185c>≡ (185f)

```
#define PUTBE16(b, n)\
    do{ \
        (b)[0] = (n) >> 8; \
        (b)[1] = (n) >> 0; \
    } while(0)
```

<macro PUTBE32 185d>≡ (185f)

```
#define PUTBE32(b, n)\
    do{ \
        (b)[0] = (n) >> 24; \
        (b)[1] = (n) >> 16; \
        (b)[2] = (n) >> 8; \
        (b)[3] = (n) >> 0; \
    } while(0)
```

<macro PUTBE64 185e>≡ (185f)

```
#define PUTBE64(b, n)\
    do{ \
        (b)[0] = (n) >> 56; \
        (b)[1] = (n) >> 48; \
        (b)[2] = (n) >> 40; \
        (b)[3] = (n) >> 32; \
        (b)[4] = (n) >> 24; \
        (b)[5] = (n) >> 16; \
        (b)[6] = (n) >> 8; \
        (b)[7] = (n) >> 0; \
    } while(0)
```

<git9/git.h 185f>≡

```
#include <bio.h>
#include <mp.h>
#include <libsec.h>
#include <flate.h>
#include <regexp.h>
```

```

typedef struct Conn Conn;
typedef struct Hash Hash;
typedef struct Delta Delta;
typedef struct Cinfo Cinfo;
typedef struct Tinfo Tinfo;
typedef struct Object Object;
typedef struct Objset Objset;
typedef struct Pack Pack;
typedef struct Buf Buf;
typedef struct Dirent Dirent;
typedef struct Idxent Idxent;
typedef struct Objlist Objlist;
typedef struct Dtab Dtab;
typedef struct Dblock Dblock;
typedef struct Objq Objq;
typedef struct Qelt Qelt;
typedef struct Idxent Idxent;

enum {
    Pathmax      = 512,
    Npackcache    = 32,
    Hashsz        = 20,
    Pktmax        = 65536,
    KiB           = 1024,
    MiB           = 1024*KiB,
};

<enum Gxxx 31c>
<enum Cxxx 31f>

enum {
    ConnGit,
    ConnGit9,
    ConnSsh,
    ConnHttp,
};

<struct Objlist 58a>

<struct Hash 29a>

struct Conn {
    int type;
    fdt rfd;
    fdt wfd;

    /* capabilities */
    char symfrom[256];
    char symto[256];
    char multiack;
    char sideband;
    char sideband64k;
    char report;

    /* only used by http */
    fdt cfd;
    char *url; /* note, first GET uses a different url */
    char *dir;
    char *direction;
};

```

⟨*struct* Dirent 33d⟩

⟨*struct* Object 30c⟩

⟨*struct* Tinfo 33c⟩

⟨*struct* Cinfo 34b⟩

⟨*struct* Objset 34e⟩

⟨*struct* Qelt 36d⟩

⟨*struct* Objq 36c⟩

```
struct Dtab {
    Object  *o;
    uchar   *base;
    int nbase;
    Dblock  *b;
    int nb;
    int sz;
};
```

```
struct Dblock {
    uchar   *buf;
    int len;
    int off;
    u64int  hash;
};
```

```
struct Delta {
    int cpy;
    int off;
    int len;
};
```

```
struct Idxent {
    char   *path;
    Qid qid;
    int mode;
    int order;
    char   state;
};
```

⟨*macro* GETBE16 185a⟩

⟨*macro* GETBE32 29d⟩

⟨*macro* GETBE64 185b⟩

⟨*macro* PUTBE16 185c⟩

⟨*macro* PUTBE32 185d⟩

⟨*macro* PUTBE64 185e⟩

⟨*macro* QDIR 59e⟩

⟨*macro* isblank 163b⟩

```
extern Reprog  *authorpat;
extern Objset  objcache;
extern vlong   cachemax;
extern Hash    Zhash;
extern int     chattygit;
extern int     interactive;
```

```

extern int gitdirmode;

<pragmas varargck 29e>
int Hfmt(Fmt*);
int Tfmt(Fmt*);
int Ofmt(Fmt*);
int Qfmt(Fmt*);

<signature gitinit 51a>

/* object io */
int resolverefs(Hash **, char *);
int resolveref(Hash *, char *);
int listrefs(Hash **, char ***);
Object *ancestor(Object *, Object *);
int findtwixt(Hash *, int, Hash *, int, Object ***, int *);
Object *readobject(Hash);
Object *clearedobject(Hash, int);
int expandprefix(Hash*, Hash, int);
void parseobject(Object *);
int indexpack(char *, char *, Hash);
int writepack(int, Hash*, int, Hash*, int, Hash*);
int hasheq(Hash *, Hash *);
Object *ref(Object *);
void unref(Object *);
void cache(Object *);
Object *emptydir(void);
int entcmp(void*, void*);

/* object sets */
void osinit(Objset *);
void osclear(Objset *);
void osadd(Objset *, Object *);
int oshas(Objset *, Hash);
Object *osfind(Objset *, Hash);

/* object listing */
Objlist *mkols(void);
int olsnext(Objlist *, Hash *);
void olsfree(Objlist *);

/* util functions */
<macro dprint 158d>
void _dprint(char *, ...);
void *eamalloc(ulong, ulong);
void *emalloc(ulong);
void *earealloc(void *, ulong, ulong);
void *erealloc(void *, ulong);
char *estrdup(char *);
int slurpdir(char *, Dir **);
errorneg1 hparse(Hash *, char *);
int hassuffix(char *, char *);
int swapsuffix(char *, int, char *, char *, char *);
char *strip(char *);
int showprogress(int, int);
u64int murmurhash2(void*, usize);
Qid parseqid(char*);
int charval(int);

/* packing */

```

```

void    dtinit(Dtab *, Object*);
void    dtclear(Dtab*);
Delta*  deltify(Object*, Dtab*, int*);

/* proto handling */
int readpkt(Conn*, char*, int);
int writepkt(Conn*, char*, int);
int fmtpkt(Conn*, char*, ...);
int flushpkt(Conn*);
void    initconn(Conn*, int, int);
int gitconnect(Conn *, char *, char *);
int readphase(Conn *);
int writephase(Conn *);
void    closeconn(Conn *);
void    parsecaps(char *, Conn *);
int okref(char*);

/* queues */
void    qinit(Objq*);
void    qclear(Objq*);
void    qput(Objq*, Object*, int);
int qpop(Objq*, Qelt*);

```

D.1.23 git9/log.c

```

⟨function filteradd 189a⟩≡ (191b)
void
filteradd(Pfilt *pf, char *path)
{
    char *p, *e;
    int i;

    if((e = strchr(path, '/')) != nil)
        p = smprint("%.*s", (int)(e - path), path);
    else
        p = strdup(path);

    while(e != nil && *e == '/')
        e++;
    for(i = 0; i < pf->nsub; i++){
        if(strncmp(pf->sub[i].elt, p) == 0){
            pf->sub[i].show = pf->sub[i].show || (e == nil);
            if(e != nil)
                filteradd(&pf->sub[i], e);
            free(p);
            return;
        }
    }
    pf->sub = earealloc(pf->sub, pf->nsub+1, sizeof(Pfilt));
    pf->sub[pf->nsub].elt = p;
    pf->sub[pf->nsub].show = (e == nil);
    pf->sub[pf->nsub].nsub = 0;
    pf->sub[pf->nsub].sub = nil;
    if(e != nil)
        filteradd(&pf->sub[pf->nsub], e);
    pf->nsub++;
}

```

```

⟨function lookup (git9/log.c) 189b⟩≡ (191b)

```

```

Hash
lookup(Pfilt *pf, Object *o)
{
    int i;

    if(o == nil)
        return Zhash;
    for(i = 0; i < o->tree->nent; i++)
        if(strcmp(o->tree->ent[i].name, pf->elt) == 0)
            return o->tree->ent[i].h;
    return Zhash;
}

```

<function matchesfilter1 190a>≡ (191b)

```

int
matchesfilter1(Pfilt *pf, Object *t, Object *pt)
{
    Object *a, *b;
    Hash ha, hb;
    int i, r;

    if(pf->show)
        return 1;
    if(t != nil){
        if(pt != nil && t->type != pt->type)
            return 1;
        if(t->type != GTree)
            return 0;
    }

    for(i = 0; i < pf->nsub; i++){
        ha = lookup(&pf->sub[i], t);
        hb = lookup(&pf->sub[i], pt);
        if(hasheq(&ha, &hb))
            continue;
        a = readobject(ha);
        b = readobject(hb);
        r = matchesfilter1(&pf->sub[i], a, b);
        unref(a);
        unref(b);
        if(r)
            return 1;
    }
    return 0;
}

```

<function matchesfilter 190b>≡ (191b)

```

int
matchesfilter(Object *o)
{
    Object *t, *p, *pt;
    int i, r;

    assert(o->type == GCommit);
    if(pathfilt == nil)
        return 1;
    if((t = readobject(o->commit->tree)) == nil)
        sysfatal("read %H: %r", o->commit->tree);
    for(i = 0; i < o->commit->nparent; i++){
        if((p = readobject(o->commit->parent[i])) == nil)

```

```

        sysfatal("read %H: %r", o->commit->parent[i]);
    if((pt = readobject(p->commit->tree)) == nil)
        sysfatal("read %H: %r", o->commit->tree);
    r = matchesfilter1(pathfilt, t, pt);
    unref(p);
    unref(pt);
    if(r)
        return 1;
}
if(o->commit->nparent == 0)
    return matchesfilter1(pathfilt, t, nil);
return 0;
}

```

<function nextline 191a>≡ (191b)

```

static char*
nextline(char *p, char *e)
{
    for(; p != e; p++)
        if(*p == '\n')
            break;
    return p;
}

```

<git9/log.c 191b>≡

```

#include <u.h>
#include <libc.h>
#include "git.h"

typedef struct Pfilt Pfilt;
struct Pfilt {
    char    *elt;
    int show;
    Pfilt    *sub;
    int nsub;
};

```

```

Biobuf    *out;
char      *queryexpr;
<global commitid(log.c) 113b>
int shortlog;
int msgcount = -1;

```

```

Objset    done;
Objq      objq;
Pfilt     *pathfilt;

```

<function filteradd 189a>

<function lookup (git9/log.c) 189b>

<function matchesfilter1 190a>

<function matchesfilter 190b>

<function nextline 191a>

<function show 115>

⟨function showquery 116a⟩

⟨function showcommits 114⟩

⟨function usage (git9/log.c) 113a⟩

⟨function main (git9/log.c) 113c⟩

D.1.24 git9/objset.c

⟨git9/objset.c 192a⟩≡

```
#include <u.h>
#include <libc.h>

#include "git.h"

⟨function osinit 35a⟩
⟨function osclear 35b⟩

⟨function osadd 35c⟩

⟨function osfind 36a⟩

⟨function oshas 36b⟩
```

D.1.25 git9/ols.c

⟨function crackidx 192b⟩≡ (194b)

```
static int
crackidx(char *path, int *np)
{
    int fd;
    char buf[4];

    if((fd = open(path, OREAD)) == -1)
        return -1;
    if(seek(fd, 8 + 255*4, 0) == -1)
        return -1;
    if(readn(fd, buf, sizeof(buf)) != sizeof(buf))
        return -1;
    *np = GETBE32(buf);
    return fd;
}
```

⟨function isloosedir 192c⟩≡ (194b)

```
int
isloosedir(char *s)
{
    return strlen(s) == 2 && isxdigit(s[0]) && isxdigit(s[1]);
}
```

⟨function endswith 192d⟩≡ (194b)

```
int
endswith(char *n, char *s)
{
    int nn, ns;

    nn = strlen(n);
```

```

    ns = strlen(s);
    return nn > ns && strcmp(n + nn - ns, s) == 0;
}

```

<function olsreadpacked 193a>≡ (194b)

```

int
olsreadpacked(Objlist *ols, Hash *h)
{
    char *p;
    int i, j;

    i = ols->packidx;
    j = ols->entidx;

    if(ols->state == Siter)
        goto step;
    for(i = 0; i < ols->npack; i++){
        if(!endswith(ols->pack[i].name, ".idx"))
            continue;
        if((p = smprint(".git/objects/pack/%s", ols->pack[i].name)) == nil)
            sysfatal("smprint: %r");
        ols->fd = crackidx(p, &ols->nent);
        free(p);
        if(ols->fd == -1)
            continue;
        j = 0;
        while(j < ols->nent){
            if(readn(ols->fd, h->h, sizeof(h->h)) != sizeof(h->h))
                continue;
            ols->state = Siter;
            ols->packidx = i;
            ols->entidx = j;
            return 0;
        }
    step:
        j++;
    }
    close(ols->fd);
}
ols->state = Sinit;
return -1;
}

```

<function olsreadloose 193b>≡ (194b)

```

int
olsreadloose(Objlist *ols, Hash *h)
{
    char buf[64], *p;
    int i, j, n;

    i = ols->topidx;
    j = ols->looseidx;
    if(ols->state == Siter)
        goto step;
    for(i = 0; i < ols->ntop; i++){
        if(!isloosedir(ols->top[i].name))
            continue;
        if((p = smprint(".git/objects/%s", ols->top[i].name)) == nil)
            sysfatal("smprint: %r");
        ols->fd = open(p, OREAD);
        free(p);
    }
    step:
        j++;
    }
    close(ols->fd);
}
ols->state = Sinit;
return -1;
}

```

```

    if(ols->fd == -1)
        continue;
    while((ols->nloose = dirread(ols->fd, &ols->loose)) > 0){
        j = 0;
        while(j < ols->nloose){
            n = snprintf(buf, sizeof(buf), "%s%s", ols->top[i].name, ols->loose[j].name);
            if(n >= sizeof(buf))
                goto step;
            if(hparse(h, buf) == ERROR_NEG1)
                goto step;
            ols->state = Siter;
            ols->topidx = i;
            ols->looseidx = j;
            return 0;
step:
            j++;
        }
        free(ols->loose);
        ols->loose = nil;
    }
    close(ols->fd);
    ols->fd = -1;
}
ols->state = Sinit;
return -1;
}

```

⟨function olsnext 194a⟩≡ (194b)

```

int
olsnext(Objlist *ols, Hash *h)
{
    if(ols->stage == 0){
        if(olsreadloose(ols, h) != -1){
            ols->idx++;
            return 0;
        }
        ols->stage++;
    }
    if(ols->stage == 1){
        if(olsreadpacked(ols, h) != -1){
            ols->idx++;
            return 0;
        }
        ols->stage++;
    }
    return -1;
}

```

⟨git9/ols.c 194b⟩≡

```

#include <u.h>
#include <libc.h>
#include <ctype.h>
#include "git.h"

```

```

enum {
    Sinit,
    Siter,
};

```

⟨function crackidx 192b⟩

<function isloosedir 192c>

<function endswith 192d>

<function olsreadpacked 193a>

<function olsreadloose 193b>

<function mkols 66c>

<function olsfree 58b>

<function olsnext 194a>

D.1.26 git9/pack.c

<macro max 195a>≡ (215b)
#define max(x, y) ((x) > (y) ? (x) : (y))

<function cache 195b>≡ (215b)
void
cache(Object *o)
{
 Object *p;

 if(o == lruhead)
 return;
 if(o == lrutail)
 lrutail = lrutail->prev;
 if(!(o->flag & Cexist)){
 osadd(&objcache, o);
 o->id = objcache.nobj;
 o->flag |= Cexist;
 }
 if(o->prev != nil)
 o->prev->next = o->next;
 if(o->next != nil)
 o->next->prev = o->prev;
 if(lrutail == o){
 lrutail = o->prev;
 if(lrutail != nil)
 lrutail->next = nil;
 }else if(lrutail == nil)
 lrutail = o;
 if(lruhead)
 lruhead->prev = o;
 o->next = lruhead;
 o->prev = nil;
 lruhead = o;

 if(!(o->flag & Ccache)){
 o->flag |= Ccache;
 ref(o);
 ncache += o->size;
 }
 while(ncache > cachemax && lrutail != lruhead){
 p = lrutail;

```

    lrutail = p->prev;
    if(lrutail != nil)
        lrutail->next = nil;
    p->flag &= ~Ccache;
    p->prev = nil;
    p->next = nil;
    ncache -= p->size;
    unref(p);
}
}

```

<function refreshpacks 196a>≡ (215b)

```

static void
refreshpacks(void)
{
    Packf *pf, *new;
    int i, n, l, nnew;
    Dir *d;

    if((n = slurpdir(".git/objects/pack", &d)) == -1)
        return;
    nnew = 0;
    new = eamalloc(n, sizeof(Packf));
    for(i = 0; i < n; i++){
        l = strlen(d[i].name);
        if(l > 4 && strcmp(d[i].name + l - 4, ".idx") != 0)
            continue;
        d[i].name[l - 4] = 0;
        if(loadpack(&new[nnew], d[i].name) != -1)
            nnew++;
    }
    for(i = 0; i < npackf; i++){
        pf = &packf[i];
        free(pf->idx);
        if(pf->pack != nil)
            Bterm(pf->pack);
    }
    free(packf);
    packf = new;
    npackf = nnew;
    free(d);
}

```

<function openpack 196b>≡ (215b)

```

static Biobuf*
openpack(Packf *pf)
{
    vlong t;
    int i, best;

    if(pf->pack != nil){
        pf->refs++;
        return pf->pack;
    }
    /*
     * If we've got more packs open
     * than we want cached, try to
     * free up the oldest ones.
     *
     * If we can't find a slot, this

```

```

* isn't fatal; we can just use
* another fd.
*/
while(openpacks >= Npackcache){
    t = (1ull<<62)-1;
    best = -1;
    for(i = 0; i < npackf; i++){
        if(&packf[i] != pf
            && packf[i].pack != nil
            && packf[i].opentm < t
            && packf[i].refs == 0){
            t = packf[i].opentm;
            best = i;
        }
    }
    if(best == -1){
        fprintf(2, "no available pack slots\n");
        break;
    }
    Bterm(packf[best].pack);
    packf[best].pack = nil;
    openpacks--;
}
openpacks++;
pf->opentm = nsec();
pf->refs++;
if((pf->pack = Bopen(pf->path, OREAD)) == nil)
    return nil;
return pf->pack;
}

```

```

⟨function closepack 197a⟩≡ (215b)
static void
closepack(Packf *pf)
{
    pf->refs--;
}

```

```

⟨function crc32 197b⟩≡ (215b)
static u32int
crc32(u32int crc, char *b, int nb)
{
    static u32int crctab[256] = {
        0x00000000, 0x77073096, 0xee0e612c, 0x990951ba, 0x076dc419, 0x706af48f, 0xe963a535,
        0x9e6495a3, 0x0edb8832, 0x79dcb8a4, 0xe0d5e91e, 0x97d2d988, 0x09b64c2b, 0x7eb17cbd,
        0xe7b82d07, 0x90bf1d91, 0x1db71064, 0x6ab020f2, 0xf3b97148, 0x84be41de, 0x1dad47d,
        0x6ddde4eb, 0xf4d4b551, 0x83d385c7, 0x136c9856, 0x646ba8c0, 0xfd62f97a, 0x8a65c9ec,
        0x14015c4f, 0x63066cd9, 0xfa0f3d63, 0x8d080df5, 0x3b6e20c8, 0x4c69105e, 0xd56041e4,
        0xa2677172, 0x3c03e4d1, 0x4b04d447, 0xd20d85fd, 0xa50ab56b, 0x35b5a8fa, 0x42b2986c,
        0xdbbbc9d6, 0xacbcf940, 0x32d86ce3, 0x45df5c75, 0xdcd60dcf, 0xabd13d59, 0x26d930ac,
        0x51de003a, 0xc8d77518, 0xbfd06116, 0x21b4f4b5, 0x56b3c423, 0xcfba9599, 0xb8bda50f,
        0x2802b89e, 0x5f058808, 0xc60cd9b2, 0xb10be924, 0x2f6f7c87, 0x58684c11, 0xc1611dab,
        0xb6662d3d, 0x76dc4190, 0x01db7106, 0x98d220bc, 0xefd5102a, 0x71b18589, 0x06b6b51f,
        0x9fbfe4a5, 0xe8b8d433, 0x7807c9a2, 0x0f00f934, 0x9609a88e, 0xe10e9818, 0x7f6a0dbb,
        0x086d3d2d, 0x91646c97, 0xe6635c01, 0xb66b51f4, 0x1c6c6162, 0x856530d8, 0xf262004e,
        0x6c0695ed, 0x1b01a57b, 0x8208f4c1, 0xf50fc457, 0x65b0d9c6, 0x12b7e950, 0x8bbeb8ea,
        0xfcb9887c, 0x62dd1ddf, 0x15da2d49, 0x8cd37cf3, 0xfbd44c65, 0x4db26158, 0x3ab551ce,
        0xa3bc0074, 0xd4bb30e2, 0x4adfa541, 0x3dd895d7, 0xa4d1c46d, 0xd3d6f4fb, 0x4369e96a,
        0x346ed9fc, 0xad678846, 0xda60b8d0, 0x44042d73, 0x33031de5, 0xaa0a4c5f, 0xdd0d7cc9,
        0x5005713c, 0x270241aa, 0xbe0b1010, 0xc90c2086, 0x5768b525, 0x206f85b3, 0xb966d409,

```

```

0xce61e49f, 0x5edef90e, 0x29d9c998, 0xb0d09822, 0xc7d7a8b4, 0x59b33d17, 0x2eb40d81,
0xb7bd5c3b, 0xc0ba6cad, 0xedb88320, 0x9abfb3b6, 0x03b6e20c, 0x74b1d29a, 0xead54739,
0x9dd277af, 0x04db2615, 0x73dc1683, 0xe3630b12, 0x94643b84, 0x0d6d6a3e, 0x7a6a5aa8,
0xe40ecf0b, 0x9309fff9, 0x0a00ae27, 0x7d079eb1, 0xf00f9344, 0x8708a3d2, 0x1e01f268,
0x6906c2fe, 0xf762575d, 0x806567cb, 0x196c3671, 0x6e6b06e7, 0xfed41b76, 0x89d32be0,
0x10da7a5a, 0x67dd4acc, 0xf9b9df6f, 0x8ebeeff9, 0x17b7be43, 0x60b08ed5, 0xd6d6a3e8,
0xa1d1937e, 0x38d8c2c4, 0x4fdff252, 0xd1bb67f1, 0xa6bc5767, 0x3fb506dd, 0x48b2364b,
0xd80d2bda, 0xaf0a1b4c, 0x36034af6, 0x41047a60, 0xdf60efc3, 0xa867df55, 0x316e8eef,
0x4669be79, 0xcb61b38c, 0xbc66831a, 0x256fd2a0, 0x5268e236, 0xcc0c7795, 0xbb0b4703,
0x220216b9, 0x5505262f, 0xc5ba3bbe, 0xb2bd0b28, 0x2bb45a92, 0x5cb36a04, 0xc2d7ffa7,
0xb5d0cf31, 0x2cd99e8b, 0x5bdeae1d, 0x9b64c2b0, 0xec63f226, 0x756aa39c, 0x026d930a,
0x9c0906a9, 0xeb0e363f, 0x72076785, 0x05005713, 0x95bf4a82, 0xe2b87a14, 0x7bb12bae,
0x0cb61b38, 0x92d28e9b, 0xe5d5be0d, 0x7cdcefb7, 0x0bdbdf21, 0x86d3d2d4, 0xf1d4e242,
0x68ddb3f8, 0x1fda836e, 0x81be16cd, 0xf6b9265b, 0x6fb077e1, 0x18b74777, 0x88085ae6,
0xff0f6a70, 0x66063bca, 0x11010b5c, 0x8f659eff, 0xf862ae69, 0x616bffd3, 0x166ccf45,
0xa00ae278, 0xd70dd2ee, 0x4e048354, 0x3903b3c2, 0xa7672661, 0xd06016f7, 0x4969474d,
0x3e6e77db, 0xaed16a4a, 0xd9d65adc, 0x40df0b66, 0x37d83bf0, 0xa9bcae53, 0xdeb9ec5,
0x47b2cf7f, 0x30b5ffe9, 0xbdbdf21c, 0xcabac28a, 0x53b39330, 0x24b4a3a6, 0xbad03605,
0xcdd70693, 0x54de5729, 0x23d967bf, 0xb3667a2e, 0xc4614ab8, 0x5d681b02, 0x2a6f2b94,
0xb40bbe37, 0xc30c8ea1, 0x5a05df1b, 0x2d02ef8d

```

```

};
int i;

crc ^= 0xFFFFFFFF;
for(i = 0; i < nb; i++)
    crc = (crc >> 8) ^ crctab[(crc ^ b[i]) & 0xFF];
return crc ^ 0xFFFFFFFF;
}

```

⟨function readvint 198a⟩≡ (215b)

```

static vlong
readvint(char *p, char *ep, char **pp)
{
    int s, c;
    vlong n;

    s = 0;
    n = 0;
    do {
        if(p == ep)
            return -1;
        c = *p++;
        n |= (vlong)(c & 0x7f) << s;
        s += 7;
    } while (c & 0x80 && s < 63);
    *pp = p;
    return n;
}

```

⟨function applydelta 198b⟩≡ (215b)

```

static int
applydelta(Object *dst, Object *base, char *d, int nd)
{
    char *r, *b, *ed, *er;
    vlong o, l, n, nr, c;

    ed = d + nd;
    b = base->data;
    n = readvint(d, ed, &d);
    if(n == -1 || n >= 1LL << 31 || n != base->size){

```

```

    werrstr("mismatched source size");
    return -1;
}

nr = readvint(d, ed, &d);
if(nr == -1 || nr >= 1LL << 31){
    werrstr("invalid pack: %r");
    return -1;
}
r = emalloc(nr + 64);
n = snprintf(r, 64, "%T %lld", base->type, nr) + 1;
dst->all = r;
dst->type = base->type;
dst->data = r + n;
dst->size = nr;
er = dst->data + nr;
r = dst->data;

while(d != ed){
    c = *d++;
    /* copy from base */
    if(c & 0x80){
        o = 0;
        l = 0;
        /* Offset in base */
        if(d != ed && (c & 0x01)) o |= (*d++ & 0xff) << 0;
        if(d != ed && (c & 0x02)) o |= (*d++ & 0xff) << 8;
        if(d != ed && (c & 0x04)) o |= (*d++ & 0xff) << 16;
        if(d != ed && (c & 0x08)) o |= (*d++ & 0xff) << 24;

        /* Length to copy */
        if(d != ed && (c & 0x10)) l |= (*d++ & 0xff) << 0;
        if(d != ed && (c & 0x20)) l |= (*d++ & 0xff) << 8;
        if(d != ed && (c & 0x40)) l |= (*d++ & 0xff) << 16;
        if(l == 0) l = 0x10000;

        if(o < 0 || l < 0 || l > er - r || o + l > base->size){
            werrstr("garbled delta: out of bounds copy");
            return -1;
        }
        memmove(r, b + o, l);
        r += l;
    } /* inline data */
    }else{
        if(c > ed - d || c > er - r){
            werrstr("garbled delta: write past object");
            return -1;
        }
        memmove(r, d, c);
        d += c;
        r += c;
    }
}
if(r != er){
    werrstr("truncated delta");
    return -1;
}

return nr;
}

```


⟨function readrdelta 200a⟩≡ (215b)

```
static int
readrdelta(Biobuf *f, Object *o, int nd, int flag)
{
    Object *b;
    Hash h;
    char *d;
    int n;

    d = nil;
    if(Bread(f, h.h, sizeof(h.h)) != sizeof(h.h))
        goto error;
    if(hashcq(&o->hash, &h))
        goto error;
    if((n = decompress(&d, f, nil)) == -1)
        goto error;
    o->len = Boffset(f) - o->off;
    if(d == nil || n != nd)
        goto error;
    if((b = readidxobject(f, h, flag|Cthin)) == nil)
        goto error;
    if(applydelta(o, b, d, n) == -1)
        goto error;
    free(d);
    return 0;
error:
    free(d);
    return -1;
}
```

⟨function readodelta 200b⟩≡ (215b)

```
static int
readodelta(Biobuf *f, Object *o, vlong nd, vlong p, int flag)
{
    Object b;
    char *d;
    vlong r;
    int c, n;

    d = nil;
    if((c = Bgetc(f)) == -1)
        return -1;
    r = c & 0x7f;
    while(c & 0x80 && r < (1ULL<<56)){
        if((c = Bgetc(f)) == -1)
            return -1;
        r = ((r + 1)<<7) | (c & 0x7f);
    }

    if(r > p || r >= (1ULL<<56)){
        werrstr("junk offset -%lld (from %lld)", r, p);
        goto error;
    }
    if((n = decompress(&d, f, nil)) == -1)
        goto error;
    o->len = Boffset(f) - o->off;
    if(d == nil || n != nd)
        goto error;
    if(Bseek(f, p - r, 0) == -1)
        goto error;
}
```

```

memset(&b, 0, sizeof(Object));
if(readpacked(f, &b, flag) == -1)
    goto error;
if(applydelta(o, &b, d, nd) == -1)
    goto error;
clear(&b);
free(d);
return 0;
error:
    free(d);
    return -1;
}

```

⟨function readpacked 201⟩≡ (215b)

```

static int
readpacked(Biobuf *f, Object *o, int flag)
{
    int c, s, n, t;
    vlong l, p;
    Buf b;

    p = Boffset(f);
    c = Bgetc(f);
    if(c == -1)
        return -1;
    l = c & 0xf;
    s = 4;
    t = (c >> 4) & 0x7;
    if(!t){
        werrstr("unknown type for byte %x at %lld", c, p);
        return -1;
    }
    while(c & 0x80){
        if((c = Bgetc(f)) == -1)
            return -1;
        l |= (vlong)(c & 0x7f) << s;
        s += 7;
    }
    if(l >= (1ULL << 32)){
        werrstr("object too big");
        return -1;
    }
    switch(t){
    default:
        werrstr("invalid object at %lld", Boffset(f));
        return -1;
    case GCommit:
    case GTree:
    case GTag:
    case GBlob:
        b.sz = 64 + l;
        b.data = emalloc(b.sz);
        n = snprintf(b.data, 64, "%T %lld", t, l) + 1;
        b.len = n;
        if(bdecompress(&b, f, nil) == -1){
            free(b.data);
            return -1;
        }
        o->len = Boffset(f) - o->off;
        o->type = t;
    }
}

```

```

        o->all = b.data;
        o->data = b.data + n;
        o->size = b.len - n;
        break;
case G0delta:
    if(readodelta(f, o, l, p, flag) == -1)
        return -1;
    break;
case GRdelta:
    if(readrdelta(f, o, l, flag) == -1)
        return -1;
    break;
}
o->flag |= Cloaded|flag;
return 0;
}

```

<function hashcmp 202a>≡ (215b)

```

int
hashcmp(uchar *a, uchar *b, uint nbit)
{
    int x, y, i, r;

    i = nbit/8;
    r = memcmp(a, b, i);
    if(r != 0 || nbit % 8 == 0)
        return r;
    x = a[i] >> (8 - nbit % 8);
    y = b[i] >> (8 - nbit % 8);
    return x - y;
}

```

<function searchindex 202b>≡ (215b)

```

vlong
searchindex(char *idx, int nidx, Hash h, int npfx, Hash *hret)
{
    uint lo, hi, hidx;
    vlong o, oo, nent;
    int i, r;
    void *s;

    o = 8;
    if(nidx < 8 + 256*4)
        return -1;
    if(npfx < 8)
        return -1;
    /*
     * Read the fanout table. The fanout table
     * contains 256 entries, correspondding to
     * the first byte of the hash. Each entry
     * is a 4 byte big endian integer, containing
     * the total number of entries with a leading
     * byte <= the table index, allowing us to
     * rapidly do a binary search on them.
     */
    if (h.h[0] == 0){
        lo = 0;
        hi = GETBE32(idx + o);
    } else {
        o += h.h[0]*4 - 4;
    }
}

```

```

        lo = GETBE32(idx + o);
        hi = GETBE32(idx + o + 4);
    }
    if(hi == lo)
        goto notfound;
    nent = GETBE32(idx + 8 + 255*4);

    /*
     * Now that we know the range of hashes that the
     * entry may exist in, search them
     */
    r = -1;
    s = nil;
    hidx = -1;
    o = 8 + 256*4;
    while(lo < hi){
        hidx = (hi + lo)/2;
        s = idx + o + hidx*sizeof(h.h);
        r = hashcmp(h.h, s, npfx);
        if(r < 0)
            hi = hidx;
        else if(r > 0)
            lo = hidx + 1;
        else
            break;
    }
    if(r != 0)
        goto notfound;
    if(hret != nil)
        memcpy(hret, s, sizeof(Hash));

    /*
     * We found the entry. If it's 32 bits, then we
     * can just return the oset, otherwise the 32
     * bit entry contains the oset to the 64 bit
     * entry.
     */
    oo = 8;          /* Header */
    oo += 256*4;      /* Fanout table */
    oo += Hashsz*nent; /* Hashes */
    oo += 4*nent;     /* Checksums */
    oo += 4*hidx;     /* Offset offset */
    if(oo < 0 || oo + 4 > nidx)
        goto err;
    i = GETBE32(idx + oo);
    o = i & 0xffffffffULL;
    /*
     * Large offsets (i.e. 64-bit) are encoded as an index
     * into the next table with the MSB bit set.
     */
    if(o & (1ull << 31)){
        o &= 0x7fffffffULL;
        oo = 8;          /* Header */
        oo += 256*4;      /* Fanout table */
        oo += Hashsz*nent; /* Hashes */
        oo += 4*nent;     /* Checksums */
        oo += 4*nent;     /* 32-bit Offsets */
        oo += 8*o;        /* 64-bit Offset offset */
        if(oo < 0 || oo + 8 >= nidx)
            goto err;
    }

```

```

        o = GETBE64(idx + oo);
    }
    return o;

err:
    werrstr("out of bounds read");
    return -1;
notfound:
    werrstr("not present");
    return -1;
}

⟨function clearedobject 204a⟩≡ (215b)
/*
 * Creates and returns a cached, cleared object
 * that will get loaded some other time. Useful
 * for performance if need to mark that a blob
 * exists, but we don't care about its contents.
 *
 * The refcount of the returned object is 0, so
 * it doesn't need to be unrefed.
 */
Object*
clearedobject(Hash h, int type)
{
    Object *o;

    if((o = osfind(&objcache, h)) != nil)
        return o;

    o = emalloc(sizeof(Object));
    o->hash = h;
    o->type = type;
    osadd(&objcache, o);
    o->id = objcache.nobj;
    o->flag |= Cexist;
    return o;
}

⟨function objcmp 204b⟩≡ (215b)
int
objcmp(void *pa, void *pb)
{
    Object *a, *b;

    a = *(Object**)pa;
    b = *(Object**)pb;
    return memcmp(a->hash.h, b->hash.h, sizeof(a->hash.h));
}

⟨function hwrite 204c⟩≡ (215b)
static int
hwrite(Biobuf *b, void *buf, int len, DigestState **st)
{
    *st = sha1(buf, len, nil, *st);
    return Bwrite(b, buf, len);
}

```

<function objectcrc 205a>≡ (215b)

```
static u32int
objectcrc(Biobuf *f, Object *o)
{
    char buf[8096];
    int n, r;

    o->crc = 0;
    Bseek(f, o->off, 0);
    for(n = o->len; n > 0; n -= r){
        r = Bread(f, buf, n > sizeof(buf) ? sizeof(buf) : n);
        if(r == -1)
            return -1;
        if(r == 0)
            return 0;
        o->crc = crc32(o->crc, buf, r);
    }
    return 0;
}
```

<function indexpack 205b>≡ (215b)

```
int
indexpack(char *pack, char *idx, Hash ph)
{
    char hdr[4*3], buf[8];
    int nobj, npct, nvalid, nbig;
    int i, n, pct;
    Object *o, **obj;
    DigestState *st;
    char *valid;
    Biobuf *f;
    Hash h;
    int c;

    if((f = Bopen(pack, OREAD)) == nil)
        return -1;
    if(Bread(f, hdr, sizeof(hdr)) != sizeof(hdr)){
        werrstr("short read on header");
        return -1;
    }
    if(memcmp(hdr, "PACK\0\0\0\2", 8) != 0){
        werrstr("invalid header");
        return -1;
    }

    pct = 0;
    npct = 0;
    nvalid = 0;
    nobj = GETBE32(hdr + 8);
    obj = eamalloc(nobj, sizeof(Object*));
    valid = eamalloc(nobj, sizeof(char));
    if(interactive)
        fprintf(2, "indexing %d objects: 0%%", nobj);
    while(nvalid != nobj){
        n = 0;
        for(i = 0; i < nobj; i++){
            if(valid[i]){
                n++;
                continue;
            }
        }
    }
}
```

```

    pct = showprogress((npct*100)/nobj, pct);
    if(obj[i] == nil){
        o = emalloc(sizeof(Object));
        o->off = Boffset(f);
        obj[i] = o;
    }
    o = obj[i];
    /*
     * We can seek around when packing delta chains.
     * Be extra careful while we don't know where all
     * the objects start.
     */
    Bseek(f, o->off, 0);
    if(readpacked(f, o, Cidx) == -1)
        continue;
    sha1((uchar*)o->all, o->size + strlen(o->all) + 1, o->hash.h, nil);
    valid[i] = 1;
    cache(o);
    npct++;
    n++;
    if(objectcrc(f, o) == -1)
        return -1;
}
if(n == nvalid)
    sysfatal("fix point reached too early: %d/%d: %r", nvalid, nobj);
nvalid = n;
}
if(interactive)
    fprintf(2, "\b\b\b\b100%%\n");
Bterm(f);

st = nil;
qsort(obj, nobj, sizeof(Object*), objcmp);
if((f = Bopen(idx, OWRITE)) == nil)
    return -1;
if(hwrite(f, "\xff0c\x00\x00\x00\x02", 8, &st) != 8)
    goto error;
/* fanout table */
c = 0;
for(i = 0; i < 256; i++){
    while(c < nobj && (obj[c]->hash.h[0] & 0xff) <= i)
        c++;
    PUTBE32(buf, c);
    if(hwrite(f, buf, 4, &st) == -1)
        goto error;
}
for(i = 0; i < nobj; i++){
    o = obj[i];
    if(hwrite(f, o->hash.h, sizeof(o->hash.h), &st) == -1)
        goto error;
}

for(i = 0; i < nobj; i++){
    PUTBE32(buf, obj[i]->crc);
    if(hwrite(f, buf, 4, &st) == -1)
        goto error;
}

nbig = 0;
for(i = 0; i < nobj; i++){

```

```

    if(obj[i]->off < (1ull<<31))
        PUTBE32(buf, obj[i]->off);
    else{
        PUTBE32(buf, (1ull << 31) | nbig);
        nbig++;
    }
    if(hwrite(f, buf, 4, &st) == -1)
        goto error;
}
for(i = 0; i < nobj; i++){
    if(obj[i]->off >= (1ull<<31)){
        PUTBE64(buf, obj[i]->off);
        if(hwrite(f, buf, 8, &st) == -1)
            goto error;
    }
}
if(hwrite(f, ph.h, sizeof(ph.h), &st) == -1)
    goto error;
sha1(nil, 0, h.h, st);
Bwrite(f, h.h, sizeof(h.h));

free(obj);
free(valid);
Bterm(f);
return 0;

error:
    free(obj);
    free(valid);
    Bterm(f);
    return -1;
}

```

⟨function deltaordercmp 207a⟩≡ (215b)

```

static int
deltaordercmp(void *pa, void *pb)
{
    Meta *a, *b;
    vlong cmp;

    a = *(Meta**)pa;
    b = *(Meta**)pb;
    if(a->obj->type != b->obj->type)
        return a->obj->type - b->obj->type;
    cmp = (b->path - a->path);
    if(cmp != 0)
        return (cmp < 0) ? -1 : 1;
    cmp = a->mtime - b->mtime;
    if(cmp != 0)
        return (cmp < 0) ? -1 : 1;
    return memcmp(a->obj->hash.h, b->obj->hash.h, sizeof(a->obj->hash.h));
}

```

⟨function writeordercmp 207b⟩≡ (215b)

```

static int
writeordercmp(void *pa, void *pb)
{
    Meta *a, *b, *ahd, *bhd;

    a = *(Meta**)pa;

```



```

    b = *(Meta**)pb;
    ahd = (a->head == nil) ? a : a->head;
    bhd = (b->head == nil) ? b : b->head;
    if(ahd->mtime != bhd->mtime)
        return bhd->mtime - ahd->mtime;
    if(ahd != bhd)
        return (uintptr)bhd - (uintptr)ahd;
    if(a->nchain != b->nchain)
        return a->nchain - b->nchain;
    return a->mtime - b->mtime;
}

```

<function addmeta 208a>≡ (215b)

```

static void
addmeta(Metavec *v, Objset *has, Object *o, vlong pathid, vlong mtime)
{
    Meta *m;

    if(oshas(has, o->hash))
        return;
    osadd(has, o);
    if(v == nil)
        return;
    m = emalloc(sizeof(Meta));
    m->obj = o;
    m->path = pathid;
    m->mtime = mtime;

    if(v->nmeta == v->metasz){
        v->metasz = 2*v->metasz;
        v->meta = earealloc(v->meta, v->metasz, sizeof(Meta*));
    }
    v->meta[v->nmeta++] = m;
}

```

<function freemeta 208b>≡ (215b)

```

static void
freemeta(Meta *m)
{
    free(m->delta);
    free(m);
}

```

<function loadtree 208c>≡ (215b)

```

static int
loadtree(Metavec *v, Objset *has, Hash tree, char *dpath, vlong mtime)
{
    Object *t, *o;
    Dirent *e;
    vlong dh, eh;
    int i, k, r;
    char *p;

    if(oshas(has, tree))
        return 0;
    if((t = readobject(tree)) == nil)
        return -1;
    if(t->type != GTree){
        fprintf(2, "load: %H: not tree\n", t->hash);
        unref(t);
    }
}

```

```

    return 0;
}
dh = murmurhash2(dpath, strlen(dpath));
addmeta(v, has, t, dh, mtime);
for(i = 0; i < t->tree->nent; i++){
    e = &t->tree->ent[i];
    if(oshas(has, e->h))
        continue;
    if(e->ismod)
        continue;
    k = (e->mode & DMDIR) ? GTree : GBlob;
    o = clearedobject(e->h, k);
    if(k == GTree){
        p = smprint("%s/%s", dpath, e->name);
        r = loadtree(v, has, e->h, p, mtime);
        free(p);
        if(r == -1)
            return -1;
    }else{
        eh = murmurhash2(e->name, strlen(e->name));
        addmeta(v, has, o, dh^eh, mtime);
    }
}
unref(t);
return 0;
}

```

<function loadcommit 209a>≡ (215b)

```

static int
loadcommit(Metavec *v, Objset *has, Hash h)
{
    Object *c;
    int r;

    if(osfind(has, h))
        return 0;
    if((c = readobject(h)) == nil)
        return -1;
    if(c->type != GCommit){
        fprintf(2, "load: %H: not commit\n", c->hash);
        unref(c);
        return 0;
    }
    addmeta(v, has, c, 0, c->commit->ctime);
    r = loadtree(v, has, c->commit->tree, "", c->commit->ctime);
    unref(c);
    return r;
}

```

<function readmeta 209b>≡ (215b)

```

static int
readmeta(Hash *theirs, int ntheirs, Hash *ours, int nours, Meta ***m)
{
    Object **obj;
    Objset has;
    int i, nobj;
    Metavec v;

    *m = nil;
    osinit(&has);

```

```

v.nmeta = 0;
v.metasz = 64;
v.meta = eamalloc(v.metasz, sizeof(Meta*));
if(findtwixt(theirs, theirs, ours, ours, &obj, &nobj) == -1)
    sysfatal("load twixt: %r");

if(nobj == 0)
    return 0;
for(i = 0; i < ours; i++)
    if(!hasheq(&ours[i], &Zhash))
        if(loadcommit(nil, &has, ours[i]) == -1)
            goto out;
for(i = 0; i < nobj; i++)
    if(loadcommit(&v, &has, obj[i]->hash) == -1)
        goto out;
osclear(&has);
*m = v.meta;
return v.nmeta;
out:
osclear(&has);
free(v.meta);
return -1;
}

```

⟨function deltasz 210a⟩≡ (215b)

```

static int
deltasz(Delta *d, int nd)
{
    int i, sz;
    sz = 32;
    for(i = 0; i < nd; i++)
        sz += d[i].cpy ? 7 : d[i].len + 1;
    return sz;
}

```

⟨function pickdeltas 210b⟩≡ (215b)

```

static void
pickdeltas(Meta **meta, int nmeta)
{
    Meta *m, *p;
    Object *o;
    Delta *d;
    int i, j, nd, sz, pct, best;

    pct = 0;
    dprint(1, "picking deltas\n");
    if(interactive)
        fprintf(2, "deltifying %d objects: 0%%", nmeta);
    qsort(meta, nmeta, sizeof(Meta*), deltaordercmp);
    for(i = 0; i < nmeta; i++){
        m = meta[i];
        pct = showprogress((i*100) / nmeta, pct);
        m->delta = nil;
        m->ndelta = 0;
        if(m->obj->type == GCommit || m->obj->type == GTag)
            continue;
        if((o = readobject(m->obj->hash)) == nil)
            sysfatal("readobject %H: %r", m->obj->hash);
        dtinit(&m->dtab, o);
        if(i >= 11)

```

```

        dtclear(&meta[i-11]->dtab);
best = o->size;
for(j = max(0, i - 10); j < i; j++){
    p = meta[j];
    /* long chains make unpacking slow */
    if(p->nchain >= 128 || p->obj->type != o->type)
        continue;
    d = deltify(o, &p->dtab, &nd);
    sz = deltasz(d, nd);
    if(sz + 32 < best){
        /*
         * if we already picked a best delta,
         * replace it.
         */
        free(m->delta);
        best = sz;
        m->delta = d;
        m->ndelta = nd;
        m->nchain = p->nchain + 1;
        m->prev = p;
        m->head = p->head;
        if(m->head == nil)
            m->head = p;
    }else
        free(d);
}
unref(o);
}
for(i = max(0, nmeta - 10); i < nmeta; i++)
    dtclear(&meta[i]->dtab);
if(interactive)
    fprintf(2, "\b\b\b\b100%%\n");
}

```

<function compread 211a>≡ (215b)

```

static int
compread(void *p, void *dst, int n)
{
    Buf *b;

    b = p;
    if(n > b->sz - b->off)
        n = b->sz - b->off;
    memcpy(dst, b->data + b->off, n);
    b->off += n;
    return n;
}

```

<function compwrite 211b>≡ (215b)

```

static int
compwrite(void *p, void *buf, int n)
{
    return hwrite(((Compout *)p)->bfd, buf, n, &(((Compout*)p)->st);
}

```

<function hcompress 211c>≡ (215b)

```

static int
hcompress(Biobuf *bfd, void *buf, int sz, DigestState **st)
{
    int r;

```

```

Buf b ={
    .off=0,
    .data=buf,
    .sz=sz,
};
Compout o = {
    .bfd = bfd,
    .st = *st,
};

r = deflatezlib(&o, compwrite, &b, compread, 6, 0);
*st = o.st;
return r;
}

```

```

⟨function append 212a⟩≡ (215b)
static void
append(char **p, int *len, int *sz, void *seg, int nseg)
{
    if(*len + nseg >= *sz){
        while(*len + nseg >= *sz)
            *sz += *sz/2;
        *p = erealloc(*p, *sz);
    }
    memcpy(*p + *len, seg, nseg);
    *len += nseg;
}

```

```

⟨function encodedelta 212b⟩≡ (215b)
static int
encodedelta(Meta *m, Object *o, Object *b, void **pp)
{
    char *p, *bp, buf[16];
    int len, sz, n, i, j;
    Delta *d;

    sz = 128;
    len = 0;
    p = emalloc(sz);

    /* base object size */
    buf[0] = b->size & 0x7f;
    n = b->size >> 7;
    for(i = 1; n > 0; i++){
        buf[i - 1] |= 0x80;
        buf[i] = n & 0x7f;
        n >>= 7;
    }
    append(&p, &len, &sz, buf, i);

    /* target object size */
    buf[0] = o->size & 0x7f;
    n = o->size >> 7;
    for(i = 1; n > 0; i++){
        buf[i - 1] |= 0x80;
        buf[i] = n & 0x7f;
        n >>= 7;
    }
    append(&p, &len, &sz, buf, i);
    for(j = 0; j < m->ndelta; j++){

```

```

d = &m->delta[j];
if(d->cpy){
    n = d->off;
    bp = buf + 1;
    buf[0] = 0x81;
    buf[1] = 0x00;
    for(i = 0; i < sizeof(buf); i++) {
        buf[0] |= 1<<i;
        *bp++ = n & 0xff;
        n >>= 8;
        if(n == 0)
            break;
    }

    n = d->len;
    if(n != 0x10000) {
        buf[0] |= 0x1<<4;
        for(i = 0; i < sizeof(buf)-4 && n > 0; i++){
            buf[0] |= 1<<(i + 4);
            *bp++ = n & 0xff;
            n >>= 8;
        }
    }
    append(&p, &len, &sz, buf, bp - buf);
}else{
    n = 0;
    while(n != d->len){
        buf[0] = (d->len - n < 127) ? d->len - n : 127;
        append(&p, &len, &sz, buf, 1);
        append(&p, &len, &sz, o->data + d->off + n, buf[0]);
        n += buf[0];
    }
}
}
*pp = p;
return len;
}

```

⟨function packhdr 213a⟩≡ (215b)

```

static int
packhdr(char *hdr, int ty, int len)
{
    int i;

    hdr[0] = ty << 4;
    hdr[0] |= len & 0xf;
    len >>= 4;
    for(i = 1; len != 0; i++){
        hdr[i-1] |= 0x80;
        hdr[i] = len & 0x7f;
        len >>= 7;
    }
    return i;
}

```

⟨function packoff 213b⟩≡ (215b)

```

static int
packoff(char *hdr, vlong off)
{
    int i, j;

```

```

char rbuf[8];

rbuf[0] = off & 0x7f;
for(i = 1; (off >>= 7) != 0; i++)
    rbuf[i] = (--off & 0x7f)|0x80;

j = 0;
while(i > 0)
    hdr[j++] = rbuf[--i];
return j;
}

```

```

⟨function genpack 214⟩≡ (215b)
static int
genpack(int fd, Meta **meta, int nmeta, Hash *h, int odelta)
{
    int i, nh, nd, res, pct, ret;
    DigestState *st;
    Biobuf *bfd;
    Meta *m;
    Object *o, *po, *b;
    char *p, buf[32];

    st = nil;
    ret = -1;
    pct = 0;
    dprint(1, "generating pack\n");
    if((fd = dup(fd, -1)) == -1)
        return -1;
    if((bfd = Bfdopen(fd, OWRITE)) == nil)
        return -1;
    if(hwrite(bfd, "PACK", 4, &st) == -1)
        return -1;
    PUTBE32(buf, 2);
    if(hwrite(bfd, buf, 4, &st) == -1)
        return -1;
    PUTBE32(buf, nmeta);
    if(hwrite(bfd, buf, 4, &st) == -1)
        return -1;
    qsort(meta, nmeta, sizeof(Meta*), writeordercmp);
    if(interactive)
        fprintf(2, "writing %d objects: 0%%", nmeta);
    for(i = 0; i < nmeta; i++){
        pct = showprogress((i*100)/nmeta, pct);
        m = meta[i];
        m->off = Boffset(bfd);
        if(m->off == -1)
            goto error;
        if((o = readobject(m->obj->hash)) == nil)
            return -1;
        if(m->delta == nil){
            nh = packhdr(buf, o->type, o->size);
            if(hwrite(bfd, buf, nh, &st) == -1)
                goto error;
            if(hcompress(bfd, o->data, o->size, &st) == -1)
                goto error;
        }else{
            if((b = readobject(m->prev->obj->hash)) == nil)
                goto error;
            nd = encodedelta(m, o, b, &p);

```

```

    unref(b);
    if(odelta && m->prev->off != 0){
        nh = 0;
        nh += packhdr(buf, G0delta, nd);
        nh += packoff(buf+nh, m->off - m->prev->off);
        if(hwrite(bfd, buf, nh, &st) == -1)
            goto error;
    }else{
        nh = packhdr(buf, GRdelta, nd);
        po = m->prev->obj;
        if(hwrite(bfd, buf, nh, &st) == -1)
            goto error;
        if(hwrite(bfd, po->hash.h, sizeof(po->hash.h), &st) == -1)
            goto error;
    }
    res = hcompress(bfd, p, nd, &st);
    free(p);
    if(res == -1)
        goto error;
}
unref(o);
}
if(interactive)
    fprintf(2, "\\b\\b\\b\\b100%%\n");
sha1(nil, 0, h->h, st);
if(Bwrite(bfd, h->h, sizeof(h->h)) == -1)
    goto error;
ret = 0;
error:
    if(Bterm(bfd) == -1)
        return -1;
    return ret;
}

```

```

⟨function writepack 215a⟩≡ (215b)
int
writepack(int fd, Hash *theirs, int ntheirs, Hash *ours, int nours, Hash *h)
{
    Meta **meta;
    int i, r, nmeta;

    if((nmeta = readmeta(theirs, ntheirs, ours, nours, &meta)) == -1)
        return -1;
    pickdeltas(meta, nmeta);
    r = genpack(fd, meta, nmeta, h, 0);
    for(i = 0; i < nmeta; i++)
        freemeta(meta[i]);
    free(meta);
    return r;
}

```

```

⟨git9/pack.c 215b⟩≡
#include <u.h>
#include <libc.h>
#include <ctype.h>

#include "git.h"

typedef struct Buf Buf;
typedef struct Metavec Metavec;

```



```

typedef struct Meta Meta;
typedef struct Compout Compout;
typedef struct Packf Packf;

⟨macro max 195a⟩

struct Metavec {
    Meta    **meta;
    int nmeta;
    int metasz;
};

struct Meta {
    Object *obj;
    vlong path;
    vlong mtime;

    /* The best delta we picked */
    Meta *head;
    Meta *prev;
    Delta *delta;
    int ndelta;
    int nchain;

    /* Only used for delta window */
    Dtab dtab;

    /* Only used for writing offset deltas */
    vlong off;
};

struct Compout {
    Biobuf *bfd;
    DigestState *st;
};

⟨struct Buf 42c⟩

struct Packf {
    char path[128];
    char *idx;
    vlong nidx;

    int refs;
    Biobuf *pack;
    vlong opentm;
};

static int readpacked(Biobuf *, Object *, int);
static Object *readidxobject(Biobuf *, Hash, int);

⟨global objcache 154a⟩
Object *lruhead;
Object *lrutail;
vlong ncache;
vlong cachemax = 128*MiB;
Packf *packf;
int npackf;
int openpacks;

```

<function clear 32e>
<function unref 32d>
<function ref 32c>
<function cache 195b>
<function loadpack 118>
<function refreshpacks 196a>
<function openpack 196b>
<function closepack 197a>
<function crc32 197b>
<function bappend 42e>
<function breadc 43a>
<function bdecompress 42d>
<function decompress 42b>
<function readvint 198a>
<function applydelta 198b>
<function readrdelta 200a>
<function readodelta 200b>
<function readpacked 201>
<function readloose 40c>
<function hashcmp 202a>
<function searchindex 202b>
<function scanword 163d>
<function nextline (git9/pack.c) 45a>
<function parseauthor 45c>
<function parsecommit 44b>
<function parsetree 43c>
<function parsetag 148f>
<function parseobject 41>
<function readidxobject 39b>
<function expandprefix 47d>
<function readobject 39a>
<function clearedobject 204a>
<function objcmp 204b>

<function hwrite 204c>
 <function objectcrc 205a>
 <function indexpack 205b>
 <function deltaordercmp 207a>
 <function writeordercmp 207b>
 <function addmeta 208a>
 <function freemeta 208b>
 <function loadtree 208c>
 <function loadcommit 209a>
 <function readmeta 209b>
 <function deltasz 210a>
 <function pickdeltas 210b>
 <function compread 211a>
 <function compwrite 211b>
 <function hcompress 211c>
 <function append 212a>
 <function encodedelta 212b>
 <function packhdr 213a>
 <function packoff 213b>
 <function genpack 214>
 <function writepack 215a>

D.1.27 git9/proto.c

<constant Useragent 218a>≡ (226c)
 #define Useragent "useragent git/2.24.1"

 <constant Contenthdr 218b>≡ (226c)
 #define Contenthdr "headers Content-Type: application/x-git-%s-pack-request"

 <constant Accepthdr 218c>≡ (226c)
 #define Accepthdr "headers Accept: application/x-git-%s-pack-result"

 <function matchcap 218d>≡ (226c)
 char *
 matchcap(char *s, char *cap, int full)
 {
 if(strncmp(s, cap, strlen(cap)) == 0)
 if(!full || strlen(s) == strlen(cap))

```

        return s + strlen(cap);
    return nil;
}

```

⟨function parsecaps 219a⟩≡ (226c)

```

void
parsecaps(char *caps, Conn *c)
{
    char *p, *n, *s, *t;

    for(p = caps; p != nil; p = n){
        n = strchr(p, ' ');
        if(n != nil)
            *n++ = 0;
        if(matchcap(p, "report-status", 1) != nil)
            c->report = 1;
        if(matchcap(p, "multi_ack", 1) != nil)
            c->multiack = 1;
        else if(matchcap(p, "side-band", 1) != nil)
            c->sideband = 1;
        else if(matchcap(p, "side-band-64k", 1) != nil)
            c->sideband64k = 1;
        else if((s = matchcap(p, "symref=", 0)) != nil){
            if((t = strchr(s, ':')) == nil)
                continue;
            *t++ = '\0';
            snprintf(c->symfrom, sizeof(c->symfrom), s);
            snprintf(c->symto, sizeof(c->symto), t);
        }
    }
}

```

⟨function tracepkt 219b⟩≡ (226c)

```

void
tracepkt(int v, char *pfx, char *b, int n)
{
    char *f;
    int o, i;

    if(chattygit < v)
        return;
    o = 0;
    f = emalloc(n*4 + 1);
    for(i = 0; i < n; i++){
        if(isprint(b[i])){
            f[o++] = b[i];
            continue;
        }
        f[o++] = '\\';
        switch(b[i]){
            case '\\': f[o++] = '\\'; break;
            case '\n': f[o++] = 'n'; break;
            case '\r': f[o++] = 'r'; break;
            case '\v': f[o++] = 'v'; break;
            case '\0': f[o++] = '0'; break;
            default:
                f[o++] = 'x';
                f[o++] = "0123456789abcdef"[(b[i]>>4)&0xf];
                f[o++] = "0123456789abcdef"[(b[i]>>0)&0xf];
                break;
        }
    }
}

```

```

    }
}
f[o] = '\0';
fprintf(2, "%s %04x:\t%s\n", pfx, n, f);
free(f);
}

```

⟨function readpkt 220a⟩≡ (226c)

```

int
readpkt(Conn *c, char *buf, int nbuf)
{
    char len[5];
    char *e;
    int n;

    if(readn(c->rfd, len, 4) != 4){
        werrstr("pktline: short read from transport");
        return -1;
    }
    len[4] = 0;
    n = strtol(len, &e, 16);
    if(n == 0){
        dprint(1, "=r=> 0000\n");
        return 0;
    }
    if(e != len + 4 || n <= 4)
        sysfatal("pktline: bad length '%s'", len);
    n -= 4;
    if(n >= nbuf)
        abort();//sysfatal("pktline: undersize buffer");
    if(readn(c->rfd, buf, n) != n)
        return -1;
    if(n > 4 && strncmp(buf, "ERR ", 4) == 0){
        if((e = strrchr(buf, '\n')) != nil)
            *e = '\0';
        werrstr("%s", buf + 4);
        return -1;
    }
    buf[n] = 0;
    tracepkt(1, "=r=>", buf, n);
    return n;
}

```

⟨function writepkt 220b⟩≡ (226c)

```

int
writepkt(Conn *c, char *buf, int nbuf)
{
    char len[5];

    snprintf(len, sizeof(len), "%04x", nbuf + 4);
    if(write(c->wfd, len, 4) != 4)
        return -1;
    if(write(c->wfd, buf, nbuf) != nbuf)
        return -1;
    tracepkt(1, "<=w=", buf, nbuf);
    return 0;
}

```

⟨function fmtpkt 221a⟩≡ (226c)

```
int
fmtpkt(Conn *c, char *fmt, ...)
{
    char pkt[Pktmax];
    va_list ap;
    int n;

    va_start(ap, fmt);
    n = vsnprint(pkt, sizeof(pkt), fmt, ap);
    n = writepkt(c, pkt, n);
    va_end(ap);
    return n;
}
```

⟨function flushpkt 221b⟩≡ (226c)

```
int
flushpkt(Conn *c)
{
    dprint(1, "<=w= 0000\n");
    return write(c->wfd, "0000", 4);
}
```

⟨function grab 221c⟩≡ (226c)

```
static void
grab(char *dst, int n, char *p, char *e)
{
    int l;

    l = e - p;
    if(l >= n)
        sysfatal("overlong component");
    memcpy(dst, p, l);
    dst[l] = 0;
}
```

⟨function parseuri 221d⟩≡ (226c)

```
static int
parseuri(char *uri, char *proto, char *host, char *port, char *path)
{
    char *s, *p, *q;
    int hasport;

    print("uri: \"%s\"\n", uri);

    p = strstr(uri, "://");
    if(p == nil)
        snprint(proto, Nproto, "ssh");
    else if(strncmp(uri, "git+", 4) == 0)
        grab(proto, Nproto, uri + 4, p);
    else
        grab(proto, Nproto, uri, p);
    *port = 0;
    hasport = 1;
    if(strcmp(proto, "git") == 0)
        snprint(port, Nport, "9418");
    else if(strncmp(proto, "https", 5) == 0)
        snprint(port, Nport, "443");
    else if(strncmp(proto, "http", 4) == 0)
        snprint(port, Nport, "80");
}
```

```

else if(strncmp(proto, "hjit", 5) == 0)
    snprintf(port, Nport, "17021");
else if(strncmp(proto, "gits", 5) == 0)
    snprintf(port, Nport, "9419");
else
    hasport = 0;
s = (p != nil) ? p + 3 : uri;
p = nil;
if(!hasport){
    p = strstr(s, ":");
    if(p != nil)
        p++;
}
if(p == nil)
    p = strstr(s, "/");
if(p == nil || strlen(p) == 1){
    werrstr("missing path");
    return -1;
}

q = memchr(s, ':', p - s);
if(q){
    grab(host, Nhost, s, q);
    grab(port, Nport, q + 1, p);
}else{
    grab(host, Nhost, s, p);
}

snprintf(path, Npath, "%s", p);
return 0;
}

```

```

⟨function webclone 222⟩≡ (226c)
static int
webclone(Conn *c, char *url)
{
    char buf[16];
    int n, conn;

    if((c->cfd = open("/mnt/web/clone", ORDWR)) < 0)
        goto err;
    if((n = read(c->cfd, buf, sizeof(buf)-1)) == -1)
        goto err;
    buf[n] = 0;
    conn = atoi(buf);

    /* github will behave differently based on useragent */
    if(write(c->cfd, Useragent, sizeof(Useragent)) == -1)
        return -1;
    dprint(1, "open url %s\n", url);
    if(fprint(c->cfd, "url %s", url) == -1)
        goto err;
    free(c->dir);
    c->dir = smprint("/mnt/web/%d", conn);
    return 0;
err:
    if(c->cfd != -1)
        close(c->cfd);
    return -1;
}

```

<function webopen 223a>≡ (226c)

```
static int
webopen(Conn *c, char *file, int mode)
{
    char path[128];
    int fd;

    snprintf(path, sizeof(path), "%s/%s", c->dir, file);
    if((fd = open(path, mode)) == -1)
        return -1;
    return fd;
}
```

<function issmarthttp 223b>≡ (226c)

```
static int
issmarthttp(Conn *c, char *direction)
{
    char buf[Pktmax+1], svc[128];
    int fd, n;

    if((fd = webopen(c, "contenttype", OREAD)) == -1)
        return -1;
    n = readn(fd, buf, sizeof(buf) - 1);
    close(fd);
    if(n == -1)
        return -1;
    buf[n] = '\0';
    snprintf(svc, sizeof(svc), "application/x-git-%s-pack-advertisement", direction);
    if(strcmp(svc, buf) != 0){
        werrstr("dumb http protocol not supported");
        return -1;
    }

    if((n = readpkt(c, buf, sizeof(buf))) == -1)
        sysfatal("http read: %r");
    buf[n] = 0;
    snprintf(svc, sizeof(svc), "# service=git-%s-pack\n", direction);
    if(strncmp(svc, buf, n) != 0){
        werrstr("invalid initial packet line");
        return -1;
    }
    if(readpkt(c, buf, sizeof(buf)) != 0){
        werrstr("protocol garble: expected flushpkt");
        return -1;
    }
    return 0;
}
```

<function githandshake 223c>≡ (226c)

```
static int
githandshake(Conn *c, char *host, char *path, char *direction)
{
    char *p, *e, cmd[512];

    p = cmd;
    e = cmd + sizeof(cmd);
    p = seprint(p, e - 1, "git-%s-pack %s", direction, path);
    if(host != nil)
        p = seprint(p + 1, e, "host=%s", host);
    if(writepkt(c, cmd, p - cmd + 1) == -1){
```



```

        fprintf(2, "failed to write message\n");
        closeconn(c);
        return -1;
    }
    return 0;
}

```

```

⟨function dialhgit 224a⟩≡ (226c)
static int
dialhgit(Conn *c, char *host, char *port, char *path, char *direction, int auth)
{
    char *ds;
    int pid, pfd[2];

    if((ds = netmkaddr(host, "tcp", port)) == nil)
        return -1;
    if(pipe(pfd) == -1)
        sysfatal("unable to open pipe: %r");
    pid = fork();
    if(pid == -1)
        sysfatal("unable to fork");
    if(pid == 0){
        close(pfd[1]);
        dup(pfd[0], 0);
        dup(pfd[0], 1);
        dprint(1, "exec tlsclient -a %s\n", ds);
        if(auth)
            execl("/bin/tlsclient", "tlsclient", "-a", ds, nil);
        else
            execl("/bin/tlsclient", "tlsclient", ds, nil);
        sysfatal("exec: %r");
    }
    close(pfd[0]);
    c->type = ConnGit9;
    c->rfd = pfd[1];
    c->wfd = dup(pfd[1], -1);
    return githandshake(c, host, path, direction);
}

```

```

⟨function initconn 224b⟩≡ (226c)
void
initconn(Conn *c, int rd, int wr)
{
    c->type = ConnGit;
    c->rfd = rd;
    c->wfd = wr;
}

```

```

⟨function servelocal 224c⟩≡ (226c)
static int
servelocal(Conn *c, char *path, char *direction)
{
    int pid, pfd[2];

    if(pipe(pfd) == -1)
        sysfatal("unable to open pipe: %r");
    pid = fork();
    if(pid == -1)
        sysfatal("unable to fork");
    if(pid == 0){

```

```

        close(pfd[1]);
        dup(pfd[0], 0);
        dup(pfd[0], 1);
        execl("/bin/git/serve", "serve", "-w", nil);
        sysfatal("exec: %r");
    }
    close(pfd[0]);
    c->type = ConnGit;
    c->rfd = pfd[1];
    c->wfd = dup(pfd[1], -1);
    return githandshake(c, nil, path, direction);
}

```

<function localrepo 225a>≡ (226c)

```

static int
localrepo(char *uri, char *path, int npath)
{
    int fd;
    snprintf(path, npath, "%s/.git/../../", uri);
    fd = open(path, OREAD);
    if(fd < 0)
        return -1;
    if(fd2path(fd, path, npath) != 0){
        close(fd);
        return -1;
    }
    close(fd);
    return 0;
}

```

<function writephase 225b>≡ (226c)

```

int
writephase(Conn *c)
{
    char hdr[128];
    int n;

    dprint(1, "start write phase\n");
    if(c->type != ConnHttp)
        return 0;

    if(c->wfd != -1)
        close(c->wfd);
    if(c->cfd != -1)
        close(c->cfd);
    if((c->cfd = webclone(c, c->url)) == -1)
        return -1;
    n = snprintf(hdr, sizeof(hdr), Contenthdr, c->direction);
    if(write(c->cfd, hdr, n) == -1)
        return -1;
    n = snprintf(hdr, sizeof(hdr), Accepthdr, c->direction);
    if(write(c->cfd, hdr, n) == -1)
        return -1;
    if((c->wfd = webopen(c, "postbody", OWRITE)) == -1)
        return -1;
    c->rfd = -1;
    return 0;
}

```

<function readphase 226a>≡ (226c)

```
int
readphase(Conn *c)
{
    dprint(1, "start read phase\n");
    if(c->type != ConnHttp)
        return 0;
    if(close(c->wfd) == -1)
        return -1;
    if((c->rfd = webopen(c, "body", OREAD)) == -1)
        return -1;
    c->wfd = -1;
    return 0;
}
```

<function closeconn 226b>≡ (226c)

```
void
closeconn(Conn *c)
{
    close(c->rfd);
    close(c->wfd);
    switch(c->type){
    case ConnGit:
        break;
    case ConnGit9:
    case ConnSsh:
        free(wait());
        break;
    case ConnHttp:
        close(c->cfd);
        break;
    }
}
```

<git9/proto.c 226c>≡

```
#include <u.h>
#include <libc.h>
#include <ctype.h>

#include "git.h"
```

<constant Useragent 218a>
<constant Contenthdr 218b>
<constant Accepthdr 218c>

```
enum {
    Nproto  = 16,
    Nport   = 16,
    Nhost   = 256,
    Npath   = 128,
    Nbranch = 32,
};
```

<function matchcap 218d>

<function parsecaps 219a>

<function tracepkt 219b>

```

⟨function readpkt 220a⟩
⟨function writepkt 220b⟩

⟨function fmtpkt 221a⟩

⟨function flushpkt 221b⟩

⟨function grab 221c⟩

⟨function parseuri 221d⟩

⟨function webclone 222⟩

⟨function webopen 223a⟩

⟨function issmarthttp 223b⟩

⟨function dialhttp 152a⟩
⟨function dialssh 152b⟩

⟨function githandshake 223c⟩

⟨function dialhgit 224a⟩

⟨function initconn 224b⟩

⟨function dialgit 130b⟩

⟨function servelocal 224c⟩

⟨function localrepo 225a⟩

⟨function gitconnect 130a⟩

⟨function writephase 225b⟩
⟨function readphase 226a⟩

⟨function closeconn 226b⟩

```

D.1.28 git9/query.c

```

⟨function showdir 227⟩≡ (229b)
void
showdir(Hash dh, char *dname, char m)
{
    Dirent *p, *e;
    Object *d;

    path[npath++] = dname;
    if((d = readobject(dh)) == nil)
        sysfatal("bad hash %H", dh);
    assert(d->type == GTree);
    p = d->tree->ent;
    e = p + d->tree->nent;
    for(; p != e; p++){
        if(p->ismod)
            continue;
        if(p->mode & DMDIR)

```

```

        showdir(p->h, p->name, m);
    else
        print("%c %P%s\n", m, p->name);
}
print("%c %P\n", m);
unref(d);
npath--;
}

```

<function show (git9/query.c) 228a>≡ (229b)

```

void
show(Dirent *e, char m)
{
    if(e->mode & DMDIR)
        showdir(e->h, e->name, m);
    else
        print("%c %P%s\n", m, e->name);
}

```

<function difftrees 228b>≡ (229b)

```

void
difftrees(Object *a, Object *b)
{
    Dirent *ap, *bp, *ae, *be;
    int c;

    ap = ae = nil;
    bp = be = nil;
    if(a != nil){
        if(a->type != GTree)
            return;
        ap = a->tree->ent;
        ae = ap + a->tree->nent;
    }
    if(b != nil){
        if(b->type != GTree)
            return;
        bp = b->tree->ent;
        be = bp + b->tree->nent;
    }
    while(ap != ae && bp != be){
        c = entcmp(ap, bp);
        if(c == 0){
            if(ap->mode == bp->mode && hasheq(&ap->h, &bp->h))
                goto next;
            if(ap->mode != bp->mode)
                print("! %P%s\n", ap->name);
            else if(!(ap->mode & DMDIR) || !(bp->mode & DMDIR))
                print("@ %P%s\n", ap->name);
            if((ap->mode & DMDIR) && (bp->mode & DMDIR)){
                if(npath >= nelem(path))
                    sysfatal("path too deep");
                path[npath++] = ap->name;
                if((a = readobject(ap->h)) == nil)
                    sysfatal("bad hash %H", ap->h);
                if((b = readobject(bp->h)) == nil)
                    sysfatal("bad hash %H", bp->h);
                difftrees(a, b);
                unref(a);
                unref(b);
            }
        }
        next:
        ap = ae;
        bp = be;
    }
}

```

```

        npath--;
    }
next:
    ap++;
    bp++;
}else if(c < 0) {
    show(ap, '-');
    ap++;
}else if(c > 0){
    show(bp, '+');
    bp++;
}
}
for(; ap != ae; ap++)
    show(ap, '-');
for(; bp != be; bp++)
    show(bp, '+');
}

```

<function diffcommits 229a>≡ (229b)

```

void
diffcommits(Hash ah, Hash bh)
{
    Object *a, *b, *at, *bt;

    at = nil;
    bt = nil;
    if(!hasheq(&ah, &Zhash) && (a = readobject(ah)) != nil){
        if(a->type != GCommit)
            sysfatal("not commit: %H", ah);
        if((at = readobject(a->commit->tree)) == nil)
            sysfatal("bad hash %H", a->commit->tree);
        unref(a);
    }
    if(!hasheq(&bh, &Zhash) && (b = readobject(bh)) != nil){
        if(b->type != GCommit)
            sysfatal("not commit: %H", ah);
        if((bt = readobject(b->commit->tree)) == nil)
            sysfatal("bad hash %H", b->commit->tree);
        unref(b);
    }
    difftrees(at, bt);
    unref(at);
    unref(bt);
}

```

<git9/query.c 229b>≡

```

#include <u.h>
#include <libc.h>

#include "git.h"

#pragma varargck    type    "P" void

// flags
<global fullpath(query.c) 86d>
<global changes(query.c) 86a>
<global reverse(query.c) 86i>

<global path(query.c) 77f>

```

<global npath(query.c) 77g>
 <function Pfmt 77e>

 <function showdir 227>

 <function show (git9/query.c) 228a>

 <function difftrees 228b>
 <function diffcommits 229a>

 <function usage (git9/query.c) 76a>
 <function main (git9/query.c) 76b>

D.1.29 git9/ref.c

```

<function take 230a>≡ (232)
int
take(Eval *ev, char *m)
{
    int l;

    l = strlen(m);
    if(strncmp(ev->p, m, l) != 0)
        return 0;
    ev->p += l;
    return 1;
}

<function findtwixt 230b>≡ (232)
int
findtwixt(Hash *head, int nhead, Hash *tail, int ntail, Object ***res, int *nres)
{
    return paint(head, nhead, tail, ntail, res, nres, Twixt);
}

<PaintMode other cases 230c>≡ (83e)
    Twixt,

<paint() switch mode cases 230d>+≡ (84b) <83d
    case Twixt:
        dprint(1, "found twixt\n");
        *res = eamalloc(keep.nobj, sizeof(Object*));
        *nres = 0;
        for(i = 0; i < keep.sz; i++){
            o = keep.obj[i];
            if(o != nil && !oshas(&drop, o->hash) && !oshas(&skip, o->hash)){
                (*res)[*nres] = o;
                (*nres)++;
            }
        }
        break;

<function resolveref 230e>≡ (232)
int
resolveref(Hash *r, char *ref)
{
    Eval ev;

    if(evalexp(&ev, ref) == -1){

```

```

        free(ev.stk);
        return -1;
    }
    if(ev.nstk != 1){
        werrstr("ambiguous ref expr");
        free(ev.stk);
        return -1;
    }
    *r = ev.stk[0]->hash;
    free(ev.stk);
    return 0;
}

```

⟨function readrefdir 231a⟩≡ (232)

```

int
readrefdir(Hash **refs, char ***names, int *nrefs, char *dpath, char *dname)
{
    Dir *d, *e, *dir;
    char *path, *name, *sep;
    int ndir;

    if((ndir = slurpdir(dpath, &dir)) == -1)
        return -1;
    sep = (*dname == '\\0') ? "" : "/";
    e = dir + ndir;
    for(d = dir; d != e; d++){
        path = smprint("%s/%s", dpath, d->name);
        name = smprint("%s%s%s", dname, sep, d->name);
        if(d->mode & DMDIR) {
            if(readrefdir(refs, names, nrefs, path, name) == -1)
                goto noref;
        }else{
            *refs = erealloc(*refs, (*nrefs + 1)*sizeof(Hash));
            *names = erealloc(*names, (*nrefs + 1)*sizeof(char*));
            if(resolveref(&(*refs)[*nrefs], name) == -1)
                goto noref;
            (*names)[*nrefs] = name;
            *nrefs += 1;
            goto next;
        }
    }
noref:    free(name);
next:    free(path);
    }
    free(dir);
    return 0;
}

```

⟨function listrefs 231b⟩≡ (232)

```

int
listrefs(Hash **refs, char ***names)
{
    int nrefs;

    *refs = nil;
    *names = nil;
    nrefs = 0;
    if(readrefdir(refs, names, &nrefs, ".git/refs", "") == -1){
        free(*refs);
        return -1;
    }
}

```



```

        return nrefs;
    }

⟨git9/ref.c 232⟩≡
#include <u.h>
#include <libc.h>
#include <ctype.h>

#include "git.h"

typedef struct Eval Eval;

⟨enum PaintQcolor 84a⟩
⟨enum PaintMode 83e⟩

⟨struct Eval 78a⟩

static char *colors[] = {
    [Keep] "keep",
    [Drop] "drop",
    [Blank] "blank",
    [Skip] "skip",
};

⟨global zcommit(ref.c) 34d⟩

⟨function eatspace 80a⟩

⟨function push 81a⟩

⟨function pop 81c⟩

⟨function isword 80g⟩

⟨function word 80f⟩

⟨function take 230a⟩

⟨function paint 84b⟩

⟨function findtwixt 230b⟩

⟨function ancestor 82b⟩

⟨function lca 82a⟩

⟨function parent 81e⟩

⟨function range 83a⟩

⟨function matchpfx 47c⟩

⟨function readref (git9/ref.c) 46a⟩

⟨function evalpostfix 79⟩

⟨function evalexpr 78c⟩

⟨function resolverefs 78b⟩

```

<function resolveref 230e>

<function readrefdir 231a>

<function listrefs 231b>

D.1.30 git9/repack.c

<macro TMPPATH 233a>≡ (233c)
#define TMPPATH(suff) (".git/objects/pack/repack."suff)

<function cleanup 233b>≡ (233c)
int
cleanup(Hash h)
{
 char newpfx[42], dpath[256], fpath[256];
 int i, j, nd;
 Dir *d;

 snprintf(newpfx, sizeof(newpfx), "%H.", h);
 for(i = 0; i < 256; i++){
 snprintf(dpath, sizeof(dpath), ".git/objects/%02x", i);
 if((nd = slurpdir(dpath, &d)) == -1)
 continue;
 for(j = 0; j < nd; j++){
 snprintf(fpath, sizeof(fpath), ".git/objects/%02x/%s", i, d[j].name);
 remove(fpath);
 }
 remove(dpath);
 free(d);
 }
 snprintf(dpath, sizeof(dpath), ".git/objects/pack");
 if((nd = slurpdir(dpath, &d)) == -1)
 return -1;
 for(i = 0; i < nd; i++){
 if(strncmp(d[i].name, newpfx, strlen(newpfx)) == 0)
 continue;
 snprintf(fpath, sizeof(fpath), ".git/objects/pack/%s", d[i].name);
 remove(fpath);
 }
 return 0;
}

<git9/repack.c 233c>≡
#include <u.h>
#include <libc.h>

#include "git.h"

<macro TMPPATH 233a>

<function cleanup 233b>

<function usage (git9/repack.c) 119b>

<function main (git9/repack.c) 119c>

D.1.31 git9/save.c

```
<function gitmode 234a>≡ (239c)
int
gitmode(Dirent *e)
{
    if(e->islink)
        return 0120000;
    else if(e->ismod)
        return 0160000;
    else if(e->mode & DMDIR)
        return 0040000;
    else if(e->mode & 0100)
        return 0100755;
    else
        return 0100644;
}

<function namecmp 234b>≡ (239c)
int
namecmp(void *pa, void *pb)
{
    return strcmp(*(char**)pa, *(char**)pb);
}

<function idxcmp 234c>≡ (239c)
int
idxcmp(void *pa, void *pb)
{
    Idxent *a, *b;
    int c;

    a = (Idxent*)pa;
    b = (Idxent*)pb;
    if((c = strcmp(a->path, b->path)) != 0)
        return c;
    assert(a->order != b->order);
    return a->order < b->order ? -1 : 1;
}

<function bwrite 234d>≡ (239c)
static int
bwrite(void *p, void *buf, int nbuf)
{
    return Bwrite(p, buf, nbuf);
}

<function objbytes 234e>≡ (239c)
static int
objbytes(void *p, void *buf, int nbuf)
{
    Objbuf *b;
    int r, n, o;
    char *s;

    b = p;
    n = 0;
    if(b->off < b->nhdr){
        r = b->nhdr - b->off;
        r = (nbuf < r) ? nbuf : r;
        memcpy(buf, b->hdr, r);
```

```

        b->off += r;
        nbuf -= r;
        n += r;
    }
    if(b->off < b->ndat + b->nhdr){
        s = buf;
        o = b->off - b->nhdr;
        r = b->ndat - o;
        r = (nbuf < r) ? nbuf : r;
        memcpy(s + n, b->dat + o, r);
        b->off += r;
        n += r;
    }
    return n;
}

```

<function writeobj 235a>≡ (239c)

```

void
writeobj(Hash *h, char *hdr, int nhdr, char *dat, int ndat)
{
    Objbuf b = {.off=0, .hdr=hdr, .nhdr=nhdr, .dat=dat, .ndat=ndat};
    char s[64], o[256];
    SHA1state *st;
    Biobuf *f;
    int fd;

    st = sha1((uchar*)hdr, nhdr, nil, nil);
    st = sha1((uchar*)dat, ndat, nil, st);
    sha1(nil, 0, h->h, st);

    snprintf(s, sizeof(s), "%H", *h);
    fd = create(".git/objects", OREAD, DMDIR|0755);
    close(fd);
    snprintf(o, sizeof(o), ".git/objects/%c%c", s[0], s[1]);
    fd = create(o, OREAD, DMDIR | 0755);
    close(fd);
    snprintf(o, sizeof(o), ".git/objects/%c%c/%s", s[0], s[1], s + 2);
    if(readobject(*h) == nil){
        if((f = Bopen(o, OWRITE)) == nil)
            sysfatal("could not open %s: %r", o);
        if(deflatezlib(f, bwrite, &b, objbytes, 9, 0) == -1)
            sysfatal("could not write %s: %r", o);
        Bterm(f);
    }
}

```

<function writetree 235b>≡ (239c)

```

int
writetree(Dirent *ent, int nent, Hash *h)
{
    char *t, *txt, *etxt, hdr[128];
    int nhdr, n;
    Dirent *d, *p;

    t = emalloc((16+256+20) * nent);
    txt = t;
    etxt = t + (16+256+20) * nent;

    /* squeeze out deleted entries */
    n = 0;

```

```

    p = ent;
    for(d = ent; d != ent + nent; d++)
        if(d->name)
            p[n++] = *d;
    nent = n;

    qsort(ent, nent, sizeof(Dirent), entcmp);
    for(d = ent; d != ent + nent; d++){
        if(strlen(d->name) >= 255)
            sysfatal("overly long filename: %s", d->name);
        t = seprint(t, etxt, "%o %s", gitmode(d), d->name) + 1;
        memcpy(t, d->h.h, sizeof(d->h.h));
        t += sizeof(d->h.h);
    }
    nhdr = snprint(hdr, sizeof(hdr), "%T %lld", GTree, (vlong)(t - txt)) + 1;
    writeobj(h, hdr, nhdr, txt, t - txt);
    free(txt);
    return nent;
}

⟨function blobify 236a⟩≡ (239c)
void
blobify(Dir *d, char *path, int *mode, Hash *bh)
{
    char h[64], *buf;
    int f, nh;

    if((d->mode & DMDIR) != 0)
        sysfatal("not file: %s", path);
    *mode = d->mode;
    nh = snprint(h, sizeof(h), "%T %lld", GBlob, d->length) + 1;
    if((f = open(path, OREAD)) == -1)
        sysfatal("could not open %s: %r", path);
    buf = emalloc(d->length);
    if(readn(f, buf, d->length) != d->length)
        sysfatal("could not read blob %s: %r", path);
    writeobj(bh, h, nh, buf, d->length);
    free(buf);
    close(f);
}

⟨function tracked 236b⟩≡ (239c)
int
tracked(char *path)
{
    int r, lo, hi, mid;

    lo = 0;
    hi = nidx-1;
    while(lo <= hi){
        mid = (hi + lo) / 2;
        r = strcmp(path, idx[mid].path);
        if(r < 0)
            hi = mid-1;
        else if(r > 0)
            lo = mid+1;
        else
            return idx[mid].state != 'R';
    }
    return 0;
}

```

<function dirent 237a>≡ (239c)

```

Dirent*
dirent(Dirent **ent, int *nent, char *name)
{
    Dirent *d;

    assert(strchr(name, '/') == nil);

    for(d = *ent; d != *ent + *nent; d++)
        if(d->name && strcmp(d->name, name) == 0)
            return d;
    *nent += 1;
    *ent = erealloc(*ent, *nent * sizeof(Dirent));
    d = *ent + (*nent - 1);
    memset(d, 0, sizeof(*d));
    d->name = estrdup(name);
    return d;
}

```

<function treeify 237b>≡ (239c)

```

int
treeify(Object *t, char **path, char **epath, int off, Hash *h)
{
    int nent, ne, slash, isdir;
    char *s, **p, **ep;
    Dirent *e, *ent;
    Object *o;
    Dir *d;

    nent = t->tree->nent;
    ent = eamalloc(nent, sizeof(*ent));
    memcpy(ent, t->tree->ent, nent*sizeof(*ent));
    for(p = path; p != epath; p = ep){
        s = *p;

        /*
         * paths have been normalized already,
         * no leading or double-slashes allowed.
         */
        assert(off <= strlen(s));
        assert(off == 0 || s[off-1] == '/');
        assert(s[off] != '\0' && s[off] != '/');

        /* get next path element length (from off until '/' or nul) */
        for(ne = 1; s[off+ne] != '\0' && s[off+ne] != '/'; ne++)
            ;

        /* truncate at '/' or nul */
        slash = s[off + ne];
        s[off + ne] = '\0';

        /* skip over children (having s as prefix) */
        for(ep = p + 1; slash && ep != epath; ep++){
            if(strncmp(s, *ep, off + ne) != 0)
                break;
            if((*ep)[off+ne] != '\0' && (*ep)[off+ne] != '/')
                break;
        }

        d = dirstat(s);
    }
}

```

```

e = dirent(&ent, &nent, s + off);
if(e->islink)
    sysfatal("symlinks may not be modified: %s", s);
if(e->ismod)
    sysfatal("submodules may not be modified: %s", s);

s[off + ne] = slash;

isdir = d != nil && (d->mode & DMDIR) != 0;
/*
 * exist? slash? dir?   track?
 * n      -      -      -      -> remove: file gone
 * y      n      n      y      -> blob: tracked non-dir
 * y      n      y      n      -> remove: file untracked
 * y      n      y      n      -> remove: file -> dir
 * y      n      y      y      -> remove: file -> dir
 * y      n      y      n      -> untracked dir, cli junk
 * y      y      y      n      -> recurse
 * y      y      y      y      -> recurse
 */
if(d == nil || !slash && isdir && tracked(s)){
    /*
     * if a tracked file is removed or turned
     * into a dir, we want to delete it. We
     * only want to change files passed in, and
     * not ones along the way, so ignore files
     * that have a '/'.
     */
    e->name = nil;
    s[off + ne] = slash;
    continue;
} else if(slash && isdir){
    /*
     * If we have a list of entries that go into
     * a directory, create a tree node for this
     * entry, and recurse down.
     */
    e->mode = DMDIR | 0755;
    o = readobject(e->h);
    if(o == nil || o->type != GTree)
        o = emptydir();
    /*
     * if after processing deletions, a tree is empty,
     * mark it for removal from the parent.
     *
     * Note, it is still written to the object store,
     * but this is fine -- and ensures that an empty
     * repository will continue to work.
     */
    if(treeify(o, p, ep, off + ne + 1, &e->h) == 0)
        e->name = nil;
} else if(!slash && !isdir){
    /*
     * If the file was explicitly passed in and is
     * not a dir, we want to either remove it or
     * track it, depending on the state of the index.
     */
    if(tracked(s) && !isdir)
        blobify(d, s, &e->mode, &e->h);
    else

```

```

        e->name = nil;
    }
    free(d);
}
if(nent == 0)
    sysfatal("%.s: refusing to update empty directory", off, *path);
nent = writetree(ent, nent, h);
free(ent);
return nent;
}

```

```

⟨function mkcommit 239a⟩≡ (239c)
void
mkcommit(Hash *c, vlong date, Hash tree)
{
    char *s, h[64];
    int ns, nh, i;
    Fmt f;

    fmtstrinit(&f);
    fmtprint(&f, "tree %H\n", tree);
    for(i = 0; i < nparents; i++)
        fmtprint(&f, "parent %H\n", parents[i]);
    fmtprint(&f, "author %s <%s> %lld +0000\n", authername, authoremail, date);
    fmtprint(&f, "committer %s <%s> %lld +0000\n", committername, committeremail, date);
    fmtprint(&f, "\n");
    fmtprint(&f, "%s", commitmsg);
    s = fmtstrflush(&f);

    ns = strlen(s);
    nh = snprintf(h, sizeof(h), "%T %d", GCommit, ns) + 1;
    writeobj(c, h, nh, s, ns);
    free(s);
}

```

```

⟨function findroot 239b⟩≡ (239c)
Object*
findroot(void)
{
    Object *t, *c;
    Hash h;

    if(resolveref(&h, "HEAD") == -1)
        return emptydir();
    if((c = readobject(h)) == nil || c->type != GCommit)
        sysfatal("could not read HEAD %H", h);
    if((t = readobject(c->commit->tree)) == nil)
        sysfatal("could not read tree for commit %H", h);
    return t;
}

```

```

⟨git9/save.c 239c⟩≡
#include <u.h>
#include <libc.h>
#include "git.h"

typedef struct Objbuf Objbuf;
struct Objbuf {
    int off;
    char *hdr;
}

```



```

    int nhdr;
    char *dat;
    int ndat;
};

enum {
    Maxparents = 16,
};

char    *authorname;
char    *authoremail;
char    *committername;
char    *committeremail;
char    *commitmsg;
Hash    parents[Maxparents];
int nparents;
Idxent  *idx;
int idxsz;
int nidx;

⟨function gitmode 234a⟩

⟨function namecmp 234b⟩

⟨function idxcmp 234c⟩

⟨function bwrite 234d⟩

⟨function objbytes 234e⟩

⟨function writeobj 235a⟩

⟨function writetree 235b⟩

⟨function blobify 236a⟩

⟨function tracked 236b⟩

⟨function dirent 237a⟩

⟨function treeify 237b⟩

⟨function mkcommit 239a⟩

⟨function findroot 239b⟩

⟨function usage (git9/save.c) 102a⟩

⟨function main (git9/save.c) 102b⟩

```

D.1.32 git9/send.c

```

⟨function findref 240⟩≡ (244)
int
findref(char **r, int nr, char *ref)
{
    int i;
    for(i = 0; i < nr; i++)

```

```

        if(strcmp(r[i], ref) == 0)
            return i;
    return -1;
}

⟨function findkey 241a⟩≡ (244)
int
findkey(Map *m, int nm, char *ref)
{
    int i;
    for(i = 0; i < nm; i++)
        if(strcmp(m[i].ref, ref) == 0)
            return i;
    return -1;
}

⟨function readours 241b⟩≡ (244)
int
readours(Hash **tailp, char ***refp)
{
    int nu, i, idx;
    char *r, *pfx, **ref;
    Hash *tail;

    if(sendall)
        return listrefs(tailp, refp);
    nu = 0;
    tail = eamalloc((nremoved + nbranch), sizeof(Hash));
    ref = eamalloc((nremoved + nbranch), sizeof(char*));
    for(i = 0; i < nbranch; i++){
        ref[nu] = estrdup(branch[i]);
        if(resolveref(&tail[nu], branch[i]) == -1)
            sysfatal("broken branch %s", branch[i]);
        nu++;
    }
    for(i = 0; i < nremoved; i++){
        pfx = "refs/heads/";
        if(strstr(removed[i], "heads/") == removed[i])
            pfx = "refs/";
        if(strstr(removed[i], "refs/heads/") == removed[i])
            pfx = "";
        if((r = smprint("%s%s", pfx, removed[i])) == nil)
            sysfatal("smprint: %r");
        if((idx = findref(ref, nu, r)) == -1)
            idx = nu++;
        else
            free(ref[idx]);
        assert(idx < nremoved + nbranch);
        memcpy(&tail[idx], &Zhash, sizeof(Hash));
        ref[idx] = r;
    }
    dprint(1, "nu: %d\n", nu);
    for(i = 0; i < nu; i++)
        dprint(1, "update: %H %s\n", tail[i], ref[i]);
    *tailp = tail;
    *refp = ref;
    return nu;
}

```

<function sendpack 242>≡

(244)

```
int
sendpack(Conn *c)
{
    int i, n, idx, nsp, send, first;
    int nours, ntheirs, nmap;
    char buf[Pktmax], *sp[3];
    Hash h, *theirs, *ours;
    Object *a, *b, *p, *o;
    char **refs;
    Map *map, *m;

    first = 1;
    nours = readours(&ours, &refs);
    theirs = nil;
    ntheirs = 0;
    nmap = nours;
    map = eamalloc(nmap, sizeof(Map));
    for(i = 0; i < nmap; i++){
        map[i].theirs = Zhash;
        map[i].ours = ours[i];
        map[i].ref = refs[i];
    }
    while(1){
        n = readpkt(c, buf, sizeof(buf));
        if(n == -1)
            return -1;
        if(n == 0)
            break;
        if(first && n > strlen(buf))
            parsecaps(buf + strlen(buf) + 1, c);
        first = 0;

        if(getfields(buf, sp, nelem(sp), 1, " \t\r\n") != 2)
            sysfatal("invalid ref line %.*s", utfnlen(buf, n), buf);
        theirs = earealloc(theirs, ntheirs+1, sizeof(Hash));
        if(hparse(&theirs[ntheirs], sp[0]) == -1)
            sysfatal("invalid hash %s", sp[0]);
        if((idx = findkey(map, nmap, sp[1])) != -1)
            map[idx].theirs = theirs[ntheirs];
        /*
         * we only keep their ref if we can read the object to add it
         * to our reachability; otherwise, discard it; we only care
         * that we don't have it, so we can tell whether we need to
         * bail out of pushing.
         */
        if((o = readobject(theirs[ntheirs])) != nil){
            ntheirs++;
            unref(o);
        }
    }

    if(writephase(c) == -1)
        return -1;
    send = 0;
    if(force)
        send=1;
    for(i = 0; i < nmap; i++){
        m = &map[i];
        a = readobject(m->theirs);
```

```

    if(hashreq(&m->ours, &Zhash))
        b = nil;
    else
        b = readobject(m->ours);
    p = nil;
    if(a != nil && b != nil)
        p = ancestor(a, b);
    if(!force
    && !hashreq(&m->theirs, &Zhash)
    && !hashreq(&m->ours, &Zhash)
    && (a == nil || p != a)){
        fprintf(2, "remote has diverged\n");
        werrstr("remote diverged");
        flushpkt(c);
        return -1;
    }
    unref(a);
    unref(b);
    unref(p);
    if(hashreq(&m->theirs, &m->ours)){
        print("uptodate %s\n", m->ref);
        continue;
    }
    print("update %s %H %H\n", m->ref, m->theirs, m->ours);
    n = snprintf(buf, sizeof(buf), "%H %H %s", m->theirs, m->ours, m->ref);

    /*
     * Workaround for github.
     *
     * Github will accept the pack but fail to update the references
     * if we don't have capabilities advertised. Report-status seems
     * harmless to add, so we add it.
     *
     * Github doesn't advertise any capabilities, so we can't check
     * for compatibility. We just need to add it blindly.
     */
    if(i == 0 && c->report){
        buf[n++] = '\0';
        n += snprintf(buf + n, sizeof(buf) - n, " report-status");
    }
    if(writepkt(c, buf, n) == -1)
        sysfatal("unable to send update pkt");
    send = 1;
}
flushpkt(c);
if(!send){
    fprintf(2, "nothing to send\n");
    return 0;
}

if(writepack(c->wfd, ours, ours, theirs, ntheirs, &h) == -1)
    return -1;
if(!c->report)
    return 0;

if(readphase(c) == -1)
    return -1;
/* We asked for a status report, may as well use it. */
while((n = readpkt(c, buf, sizeof(buf))) > 0){
    buf[n] = 0;

```

```

    if(chattygit)
        fprintf(2, "done sending pack, status %s\n", buf);
    nsp = getfields(buf, sp, nelem(sp), 1, " \t\n\r");
    if(nsp < 2)
        continue;
    if(nsp < 3)
        sp[2] = "";
    /*
     * Only report errors; successes will be reported by
     * surrounding scripts.
     */
    if(strcmp(sp[0], "unpack") == 0 && strcmp(sp[1], "ok") != 0)
        werrstr("unpack %s", sp[1]);
    else if(strcmp(sp[0], "ng") == 0)
        werrstr("failed update: %s %s", sp[1], sp[2]);
    else
        continue;
    return -1;
}
return 0;
}

```

<git9/send.c 244>≡

```

#include <u.h>
#include <libc.h>

#include "git.h"

typedef struct Map Map;

struct Map {
    char    *ref;
    Hash    ours;
    Hash    theirs;
};

int sendall;
int force;
int nbranch;
char **branch;
char *removed[128];
int nremoved;
int npacked;
int nsent;

```

<function findref 240>

<function findkey 241a>

<function readours 241b>

<function sendpack 242>

<function usage (git9/send.c) 125a>

<function main (git9/send.c) 125b>

D.1.33 git9/serve.c

```
⟨function fail (git9/serve.c) 245a⟩≡ (254)
_Noreturn static void
fail(Conn *c, char *fmt, ...)
{
    char msg[ERRMAX];
    va_list ap;

    va_start(ap, fmt);
    vsnprint(msg, sizeof(msg), fmt, ap);
    va_end(ap);
    fmtpkt(c, "ERR %s\n", msg);
    sysfatal("%s", msg);
}
```

```
⟨function gethead 245b⟩≡ (254)
char*
gethead(Hash *h, char *ref, int nref)
{
    int fd, n;
    char *s;

    if((fd = open(".git/HEAD", OREAD)) == -1)
        return nil;
    if((n = readn(fd, ref, nref-1)) == -1)
        return nil;
    ref[n] = 0;
    strip(ref);
    if(strncmp(ref, "ref: ", 5) != 0)
        return nil;
    s = ref+5;
    if(resolveref(h, s) == -1)
        return nil;
    return s;
}
```

```
⟨function showrefs 245c⟩≡ (254)
int
showrefs(Conn *c)
{
    char **names, *s, buf[256];
    int i, ret, nrefs;
    Hash head, *refs;

    ret = -1;
    nrefs = 0;
    refs = nil;
    names = nil;

    if((s = gethead(&head, buf, sizeof(buf))) == nil)
        memset(&head, 0, sizeof(Hash));
    if(fmtpkt(c, "%H HEAD%csymref=HEAD:%s no-thin\n", head, 0, s) == -1)
        goto error;
    if((nrefs = listrefs(&refs, &names)) == -1)
        fail(c, "listrefs: %r");
    for(i = 0; i < nrefs; i++){
        if(strncmp(names[i], "heads/", strlen("heads/")) != 0)
            continue;
        if(fmtpkt(c, "%H refs/%s\n", refs[i], names[i]) == -1)
```

```

        goto error;
    }
    if(flushpkt(c) == -1)
        goto error;
    ret = 0;
error:
    for(i = 0; i < nrefs; i++)
        free(names[i]);
    free(names);
    free(refs);
    return ret;
}

```

<function servnegotiate 246>≡ (254)

```

int
servnegotiate(Conn *c, Hash **head, int *nhead, Hash **tail, int *ntail)
{
    char pkt[Pktmax];
    int n, acked;
    Object *o;
    Hash h;

    if(showrefs(c) == -1)
        return -1;

    *head = nil;
    *tail = nil;
    *nhead = 0;
    *ntail = 0;
    while(1){
        if((n = readpkt(c, pkt, sizeof(pkt))) == -1)
            goto error;
        if(n == 0)
            break;
        if(strncmp(pkt, "want ", 5) != 0){
            werrstr(" protocol garble %s", pkt);
            goto error;
        }
        if(hparse(&h, &pkt[5]) == -1){
            werrstr(" garbled want");
            goto error;
        }
        if((o = readobject(h)) == nil){
            werrstr("requested nonexistent object");
            goto error;
        }
        unref(o);
        *head = erealloc(*head, (*nhead + 1)*sizeof(Hash));
        (*head)[*nhead] = h;
        *nhead += 1;
    }

    acked = 0;
    while(1){
        if((n = readpkt(c, pkt, sizeof(pkt))) == -1)
            goto error;
        if(strncmp(pkt, "done", 4) == 0)
            break;
        if(n == 0){
            if(!acked && fmtpkt(c, "NAK") == -1)

```

```

        goto error;
    }
    if(strncmp(pkt, "have ", 5) != 0){
        werrstr(" protocol garble %s", pkt);
        goto error;
    }
    if(hparse(&h, &pkt[5]) == -1){
        werrstr(" garbled have");
        goto error;
    }
    if((o = readobject(h)) == nil)
        continue;
    if(!acked){
        if(fmtpkt(c, "ACK %H", h) == -1)
            goto error;
        acked = 1;
    }
    unref(o);
    *tail = erealloc(*tail, (*ntail + 1)*sizeof(Hash));
    (*tail)[*ntail] = h;
    *ntail += 1;
}
if(!acked && fmtpkt(c, "NAK\n") == -1)
    goto error;
return 0;
error:
    fmtpkt(c, "ERR %r\n");
    free(*head);
    free(*tail);
    return -1;
}

```

⟨function servpack 247a⟩≡ (254)

```

int
servpack(Conn *c)
{
    Hash *head, *tail, h;
    int nhead, ntail;

    dprint(1, "negotiating pack\n");
    if(servnegotiate(c, &head, &nhead, &tail, &ntail) == -1)
        fail(c, "negotiate: %r");
    dprint(1, "writing pack\n");
    if(writepack(c->wfd, head, nhead, tail, ntail, &h) == -1)
        fail(c, "send: %r");
    return 0;
}

```

⟨function validref 247b⟩≡ (254)

```

int
validref(char *s)
{
    cleannname(s);
    if(strncmp(s, "refs/", 5) != 0)
        return 0;
    return okref(s);
}

```

⟨function recvnegotiate 247c⟩≡ (254)

```

int

```



```

recvnegotiate(Conn *c, Hash **cur, Hash **upd, char ***ref, int *nupd)
{
    char pkt[Pktmax], refpath[512], *sp[4];
    Hash old, new;
    int n, i;

    if(showrefs(c) == -1)
        return -1;
    *cur = nil;
    *upd = nil;
    *ref = nil;
    *nupd = 0;
    while(1){
        if((n = readpkt(c, pkt, sizeof(pkt))) == -1)
            goto error;
        if(n == 0)
            break;
        if(getfields(pkt, sp, nelem(sp), 1, " \t\n\r") != 3){
            fmtpkt(c, "ERR protocol garble %s\n", pkt);
            goto error;
        }
        if(hparse(&old, sp[0]) == -1){
            fmtpkt(c, "ERR bad old hash %s\n", sp[0]);
            goto error;
        }
        if(hparse(&new, sp[1]) == -1){
            fmtpkt(c, "ERR bad new hash %s\n", sp[1]);
            goto error;
        }
        if(!validref(sp[2])){
            fmtpkt(c, "ERR invalid ref %s\n", sp[2]);
            goto error;
        }
        *cur = erealloc(*cur, (*nupd + 1)*sizeof(Hash));
        *upd = erealloc(*upd, (*nupd + 1)*sizeof(Hash));
        *ref = erealloc(*ref, (*nupd + 1)*sizeof(Hash));
        (*cur)[*nupd] = old;
        (*upd)[*nupd] = new;
        (*ref)[*nupd] = estrdup(sp[2]);
        n = snprintf(refpath, sizeof(refpath), ".git/%s", sp[2]);
        if(n >= sizeof(refpath)-1){
            fmtpkt(c, "ERR invalid ref %s\n", sp[2]);
            goto error;
        }
        if(access(refpath, AWRITE) == -1
            && access(refpath, AEXIST) == 0){
            fmtpkt(c, "ERR read-only ref %s\n", sp[2]);
            goto error;
        }
        *nupd += 1;
    }
    return 0;
error:
    free(*cur);
    free(*upd);
    for(i = 0; i < *nupd; i++)
        free((*ref)[i]);
    free(*ref);
    return -1;
}

```

<function rename (git9/serve.c) 249a>≡ (254)

```
int
rename(char *pack, char *idx, Hash h)
{
    char name[128], path[196];
    Dir st;

    nulldir(&st);
    st.name = name;
    snprintf(name, sizeof(name), "%H.pack", h);
    snprintf(path, sizeof(path), ".git/objects/pack/%s", name);
    if(access(path, AEXIST) == 0)
        fprintf(2, "warning, pack %s already pushed\n", name);
    else if(dirwstat(pack, &st) == -1)
        return -1;
    snprintf(name, sizeof(name), "%H.idx", h);
    snprintf(path, sizeof(path), ".git/objects/pack/%s", name);
    if(access(path, AEXIST) == 0)
        fprintf(2, "warning, pack %s already indexed\n", name);
    else if(dirwstat(idx, &st) == -1)
        return -1;
    return 0;
}
```

<function checkhash (git9/serve.c) 249b>≡ (254)

```
int
checkhash(int fd, vlong sz, Hash *hcomp)
{
    DigestState *st;
    Hash hexpect;
    char buf[Pktmax];
    vlong n, r;
    int nr;

    if(sz < 28){
        werrstr("undersize packfile");
        return -1;
    }

    st = nil;
    n = 0;
    if(seek(fd, 0, 0) == -1)
        sysfatal("packfile seek: %r");
    while(n != sz - 20){
        nr = sizeof(buf);
        if(sz - n - 20 < sizeof(buf))
            nr = sz - n - 20;
        r = readn(fd, buf, nr);
        if(r != nr){
            werrstr("short read");
            return -1;
        }
        st = sha1((uchar*)buf, nr, nil, st);
        n += r;
    }
    sha1(nil, 0, hcomp->h, st);
    if(readn(fd, hexpect.h, sizeof(hexpect.h)) != sizeof(hexpect.h))
        sysfatal("truncated packfile");
    if(!hasheq(hcomp, &hexpect)){
        werrstr("bad hash: %H != %H", *hcomp, hexpect);
    }
}
```

```

        return -1;
    }
    return 0;
}

```

<function mkdir 250a>≡ (254)

```

int
mkdir(char *dir)
{
    char buf[ERRMAX];
    int f;

    if(access(dir, AEXIST) == 0)
        return 0;
    if((f = create(dir, OREAD, DMDIR | 0755)) == -1){
        rerrstr(buf, sizeof(buf));
        if(strstr(buf, "exist") == nil)
            return -1;
    }
    close(f);
    return 0;
}

```

<function mkpath 250b>≡ (254)

```

int
mkpath(char *path)
{
    char *p;

    p = path;
    /*
     * we assume that the path has been
     * checked with validref(), so there
     * are no double '/' or odd characters.
     */
    while(1){
        p = strchr(p, '/');
        if(p == nil)
            return 0;
        *p = 0;
        if(mkdir(path) == -1)
            return -1;
        *p++ = '/';
    }
}

```

<function updatepack 250c>≡ (254)

```

int
updatepack(Conn *c)
{
    char buf[Pktmax], packtmp[128], idxtmp[128], ebuf[ERRMAX];
    int n, pfd, packsiz;
    Hash h;

    /* make sure the needed dirs exist */
    if(mkdir(".git/objects") == -1)
        return -1;
    if(mkdir(".git/objects/pack") == -1)
        return -1;
    if(mkdir(".git/refs") == -1)

```

```

        return -1;
    if(mkdir(".git/refs/heads") == -1)
        return -1;
    snprintf(packtmp, sizeof(packtmp), ".git/objects/pack/recv-%d.pack.tmp", getpid());
    snprintf(idxtmp, sizeof(idxtmp), ".git/objects/pack/recv-%d.idx.tmp", getpid());
    if((pfd = create(packtmp, ORDWR, 0644)) == -1)
        return -1;
    packsz = 0;
    while(1){
        n = read(c->rfd, buf, sizeof(buf));
        if(n == 0)
            break;
        if(n == -1){
            rerrstr(ebuf, sizeof(ebuf));
            if(strstr(ebuf, "hungup") == nil)
                return -1;
            break;
        }
        if(write(pfd, buf, n) != n)
            return -1;
        packsz += n;
    }
    if(checkhash(pfd, packsz, &h) == -1){
        dprint(1, "hash mismatch\n");
        goto error_1;
    }
    if(indexpack(packtmp, idxtmp, h) == -1){
        dprint(1, "indexing failed: %r\n");
        goto error_1;
    }
    if(rename(packtmp, idxtmp, h) == -1){
        dprint(1, "rename failed: %r\n");
        goto error_2;
    }
    return 0;

error_2:    remove(idxtmp);
error_1:    remove(packtmp);
    return -1;
}

```

<function lockrepo 251a>≡ (254)

```

int
lockrepo(void)
{
    int fd, i;

    for(i = 0; i < 10; i++) {
        if((fd = create(".git/_lock", ORCLOSE|ORDWR|OTRUNC|OEXCL, 0644))!= -1)
            return fd;
        sleep(250);
    }
    return -1;
}

```

<function updaterefs 251b>≡ (254)

```

int
updaterefs(Conn *c, Hash *cur, Hash *upd, char **ref, int nupd)
{
    char refpath[512], buf[128];

```

```

int i, newidx, hadref, fd, ret, lockfd;
vlong newtm;
Object *o;
Hash h;

ret = -1;
hadref = 0;
newidx = -1;
/*
 * Date of Magna Carta.
 * Wrong because it was computed using
 * the proleptic gregorian calendar.
 */
newtm = -23811206400;
if((lockfd = lockrepo()) == -1){
    snprintf(buf, sizeof(buf), "repo locked\n");
    return -1;
}
for(i = 0; i < nupd; i++){
    if(resolveref(&h, ref[i]) == 0){
        hadref = 1;
        if(!hasheq(&h, &cur[i])){
            snprintf(buf, sizeof(buf), "old ref changed: %s", ref[i]);
            goto error;
        }
    }
    if(snprintf(refpath, sizeof(refpath), ".git/%s", ref[i]) == sizeof(refpath)){
        snprintf(buf, sizeof(buf), "ref path too long: %s", ref[i]);
        goto error;
    }
    if(hasheq(&upd[i], &Zhash)){
        remove(refpath);
        continue;
    }
    if((o = readobject(upd[i])) == nil){
        snprintf(buf, sizeof(buf), "update to nonexistent hash %H", upd[i]);
        goto error;
    }
    if(o->type != GCommit){
        snprintf(buf, sizeof(buf), "not commit: %H", upd[i]);
        goto error;
    }
    if(o->commit->mtime > newtm){
        newtm = o->commit->mtime;
        newidx = i;
    }
    unref(o);
    if(mkpath(refpath) == -1){
        snprintf(buf, sizeof(buf), "create path: %r");
        goto error;
    }
    if((fd = create(refpath, OWRITE|OTRUNC, 0644)) == -1){
        snprintf(buf, sizeof(buf), "open ref: %r");
        goto error;
    }
    if(fprint(fd, "%H", upd[i]) == -1){
        snprintf(buf, sizeof(buf), "upate ref: %r");
        close(fd);
        goto error;
    }
}

```

```

        close(fd);
    }
    /*
     * Heuristic:
     * If there are no valid refs, and HEAD is invalid, then
     * pick the ref with the newest commits as the default
     * branch.
     *
     * Several people have been caught out by pushing to
     * a repo where HEAD named differently from what got
     * pushed, and this is going to be more of a footgun
     * when 'master', 'main', and 'front' are all in active
     * use. This should make us pick a useful default in
     * those cases, instead of silently failing.
     */
    if(resolveref(&h, "HEAD") == -1 && hadref == 0 && newidx != -1){
        if((fd = create(".git/HEAD", OWRITE|OTRUNC, 0644)) == -1){
            snprintf(buf, sizeof(buf), "open HEAD: %r");
            goto error;
        }
        if(fprint(fd, "ref: %s", ref[newidx]) == -1){
            snprintf(buf, sizeof(buf), "write HEAD ref: %r");
            goto error;
        }
        close(fd);
    }
    ret = 0;
error:
    if(ret != 0)
        fmtpkt(c, "ERR %s", buf);
    close(lockfd);
    werrstr(buf);
    return ret;
}

```

<function recvpack 253a>≡ (254)

```

int
recvpack(Conn *c)
{
    Hash *cur, *upd;
    char **ref;
    int nupd;

    if(!allowwrite)
        fail(c, "read-only repo");
    if(recvnegotiate(c, &cur, &upd, &ref, &nupd) == -1)
        fail(c, "negotiate refs: %r");
    if(nupd != 0 && updatepack(c) == -1)
        sysfatal("update pack: %r");
    if(nupd != 0 && updaterefs(c, cur, upd, ref, nupd) == -1)
        sysfatal("update refs: %r");
    return 0;
}

```

<function parsecmd 253b>≡ (254)

```

char*
parsecmd(char *buf, char *cmd, int ncmd)
{
    int i;
    char *p;

```

```

    for(p = buf, i = 0; *p && i < ncmd - 1; i++, p++){
        if(*p == ' ' || *p == '\t'){
            cmd[i] = 0;
            break;
        }
        cmd[i] = *p;
    }
    while(*p == ' ' || *p == '\t')
        p++;
    return p;
}

```

<git9/serve.c 254>≡

```

#include <u.h>
#include <libc.h>
#include <ctype.h>
#include <auth.h>

```

```

#include "git.h"

```

```

char    *pathpfx = nil;
int allowwrite;

```

<function fail (git9/serve.c) 245a>

<function gethead 245b>

<function showrefs 245c>

<function servnegotiate 246>

<function servpack 247a>

<function validref 247b>

<function recvnegotiate 247c>

<function rename (git9/serve.c) 249a>

<function checkhash (git9/serve.c) 249b>

<function mkdir 250a>

<function mkpath 250b>

<function updatepack 250c>

<function lockrepo 251a>

<function updaterefs 251b>

<function recvpack 253a>

<function parsecmd 253b>

<function usage (git9/serve.c) 131a>

<function main (git9/serve.c) 131b>

D.1.34 git9/util.c

```
<function entcmp 255a>≡ (258)
int
entcmp(void *pa, void *pb)
{
    Dirent *ae, *be;
    uchar *a, *b;
    int ca, cb;

    ae = pa;
    be = pb;
    a = (uchar*)ae->name;
    b = (uchar*)be->name;
    /*
     * If the files have the same name, they're equal.
     * Otherwise, If they're trees, they sort as though
     * there was a trailing slash.
     *
     * Wat.
     */
    while(1){
        ca = *a++;
        cb = *b++;
        /*
         * because these are dir entries in a tree,
         * the only '/' allowable is the virtual '/'
         * at the end of the file name.
         */
        assert(ca != '/' && cb != '/');
        if(ca != cb){
            if(ca == 0 && (ae->mode & DMDIR))
                ca = '/';
            if(cb == 0 && (be->mode & DMDIR))
                cb = '/';
            return (ca > cb) ? 1 : -1;
        }
        if(ca == 0){
            if(ae->mode & DMDIR)
                ca = '/';
            if(be->mode & DMDIR)
                cb = '/';
            if(ca == cb)
                return 0;
            return (ca > cb) ? 1 : -1;
        }
    }
}

<function slurpdir 255b>≡ (258)
int
slurpdir(char *p, Dir **d)
{
    int r, f;

    if((f = open(p, OREAD)) == -1)
        return -1;
    r = dirreadall(f, d);
    close(f);
    return r;
}
```


<function hassuffix 256a>≡ (258)

```
int
hassuffix(char *base, char *suf)
{
    int nb, ns;

    nb = strlen(base);
    ns = strlen(suf);
    if(ns <= nb && strcmp(base + (nb - ns), suf) == 0)
        return 1;
    return 0;
}
```

<function swapsuffix 256b>≡ (258)

```
int
swapsuffix(char *dst, int dstsz, char *base, char *oldsuf, char *suf)
{
    int bl, ol, sl, l;

    bl = strlen(base);
    ol = strlen(oldsuf);
    sl = strlen(suf);
    l = bl + sl - ol;
    if(l + 1 > dstsz || ol > bl)
        return -1;
    memmove(dst, base, bl - ol);
    memmove(dst + bl - ol, suf, sl);
    dst[l] = 0;
    return l;
}
```

<function showprogress 256c>≡ (258)

```
int
showprogress(int x, int pct)
{
    if(!interactive)
        return 0;
    if(x > pct){
        pct = x;
        fprintf(2, "\b\b\b\b\b%3d%%", pct);
    }
    return pct;
}
```

<function murmurhash2 256d>≡ (258)

```
u64int
murmurhash2(void *pp, usize n)
{
    u32int m = 0x5bd1e995;
    u32int r = 24;
    u32int h, k;
    uchar *w, *e;

    h = Seed ^ n;
    e = pp;
    e += n & -4;
    for (w = pp; w != e; w += 4) {
        k = (u32int)w[0] | (u32int)w[1] << 8 | (u32int)w[2] << 16 | (u32int)w[3] << 24;
        k *= m;
        k ^= k >> r;
    }
}
```

```

        k *= m;

        h *= m;
        h ^= k;
    }

    switch (n & 0x3) {
    case 3: h ^= w[2] << 16;
    case 2: h ^= w[1] << 8;
    case 1: h ^= w[0] << 0;
        h *= m;
    }

    h ^= h >> 13;
    h *= m;
    h ^= h >> 15;

    return h;
}

```

⟨function parseqid 257a⟩≡ (258)

```

Qid
parseqid(char *s)
{
    char *e;
    Qid q;

    if(strcmp(s, "NOQID") == 0)
        return (Qid){-1, -1, -1};
    e = s;
    q.path = strtoull(e, &e, 16);
    if(*e != '.')
        sysfatal("corrupt qid: %s (%s)", s, e);
    q.vers = strtoul(e+1, &e, 10);
    if(*e != '.')
        sysfatal("corrupt qid: %s (%s)", s, e);
    q.type = strtoul(e+1, &e, 16);
    if(*e != '\0')
        sysfatal("corrupt qid: %s (%x)", s, *e);
    return q;
}

```

⟨function okref 257b⟩≡ (258)

```

/*
 * Checks the rules for valid ref names, as defined in
 * git/Documentation/protocol-common.txt.
 * It does not check that the ref begins with refs/
 * or is called HEAD, since that's only needed in the
 * clone protocol.
 */
int
okref(char *ref)
{
    int n, slashed;
    char *p;
    Rune r;

    slashed = 0;
    if(*ref == '/' || *ref == '.')
        return 0;
}

```

```

for(p = ref; *p != 0; p += n) {
    n = chartorune(&r, p);
    switch(r){
    case '.':
        if(p[1]== 0 || p[1] == '.')
            return 0;
        if(strcmp(p, ".lock") == 0)
            return 0;
        break;
    case '/':
        if(p[1] == 0 || p[1] == '.' || p[1] == '/')
            return 0;
        slashed = 1;
        break;
    case '@':
        if(p[1] == '{')
            return 0;
        break;
    case ' ':
    case '~':
    case '^':
    case ':':
    case '?':
    case '*':
    case '[':
    case '\\':
    case Runeerror:
    case 0x7f: /* DEL */
        return 0;
    default:
        if(r < 0x20 || isspace(rune(r)))
            return 0;
    }
}
if(ref == p)
    return 0;
return slashed;
}

```

<git9/util.c 258>≡

```

#include <u.h>
#include <libc.h>
#include <ctype.h>

```

```

#include "git.h"

```

```

Reprog *authorpat;

```

<global Zhash 29b>

<global chattygit 158c>

```

int interactive = 1;

```

<global gitdirmode 51g>

```

enum {

```

```

    Seed          = 2928213749ULL

```

```

};

```

<function emptydir 33f>

<function entcmp 255a>

<function hasheq 29c>

<function charval 30b>
 <function emalloc 161a>
 <function eamalloc 161b>
 <function erealloc 161c>
 <function earealloc 162a>
 <function estrdup 162b>
 <function Hfmt 29g>
 <function Tfmt 157c>
 <function Ofmt 157d>
 <function Qfmt 158a>
 <function findrepo 52a>
 <function gitinit 51e>
 <function hparse 30a>
 <function slurpdir 255b>
 <function hassuffix 256a>
 <function swapsuffix 256b>
 <function strip 163c>
 <function _dprint 158e>
 <function showprogress 256c>
 <function qinit 36e>
 <function qclear 36f>
 <function qput 37a>
 <function qpop 37d>
 <function murmurhash2 256d>
 <function parseqid 257a>
 <function okref 257b>

D.1.35 git9/walk.c

```

<constant NCACHE 259a>≡ (265b)
#define NCACHE 4096

<function checkedin 259b>≡ (265b)
int
checkedin(Idxent *e, int change)
{
    char *p;
    int r;

    /* clean index, no need to check tree */
  
```

```

if(cleanidx)
    return e >= &idx[0] && e < &idx[nidx] && e->state != 'R';

if((p = smprint("%s/%s", bdir, e->path)) == nil)
    sysfatal("smprint: %r");

r = access(p, AEXIST);
if(r == 0 && change){
    if(e->state != 'R')
        e->state = 'T';
    staleidx = 1;
}
free(p);

return r == 0;
}

```

<function pathcmp 260a>≡ (265b)

```

/*
 * Tricky; we want to know if a file or dir is indexed,
 * but a dir is only indexed if we have a file with dir/
 * listed in the index.
 *
 * as a result, we need to add a virtual '/' at the end
 * of the path if we're doing it, so if we have:
 * foo.bar/x
 * foo/y
 * and we want to find out if foo is a directory we should
 * descend into, we need to compare as though foo/ ended
 * with a '/', or we'll bsearch down do foo.bar, not foo.
 *
 * this code resembles entcmp() in util.c, but differs
 * because we're comparing whole paths.
 */
int
pathcmp(char *sa, char *sb, int sadir)
{
    unsigned a, b;

    while(1){
        a = *sa++;
        b = *sb++;
        if(a != b){
            if(a == 0 && sadir)
                a = '/';
            if(a == '/' && b == '/')
                return 0;
            return (a > b) ? 1 : -1;
        }
        if(a == 0)
            return 0;
    }
}

```

<function indexed 260b>≡ (265b)

```

int
indexed(char *path, int dir)
{
    int lo, hi, mid, r;

```

```

r = -1;
lo = 0;
hi = nidx-1;
while(lo <= hi){
    mid = (hi + lo) / 2;
    r = pathcmp(path, idx[mid].path, dir);
    if(r < 0)
        hi = mid-1;
    else if(r > 0)
        lo = mid+1;
    else
        break;
}
return r == 0;
}

```

⟨function idxcmp (git9/walk.c) 261a⟩≡ (265b)

```

int
idxcmp(void *pa, void *pb)
{
    Idxent *a, *b;
    int c;

    a = (Idxent*)pa;
    b = (Idxent*)pb;
    if((c = strcmp(a->path, b->path)) != 0)
        return c;
    /* maintain load order if name is identical */
    return a->order < b->order ? -1 : 1;
}

```

⟨function samedata 261b⟩≡ (265b)

```

/*
 * compares whether the indexed entry 'a'
 * has the same contents and mode as
 * the entry on disk 'b'; if the indexed
 * entry is nil, does a deep comparison
 * of the checked out file and the file
 * checked in.
 */
int
samedata(Idxent *a, Idxent *b)
{
    char *gitpath, ba[IOUNIT], bb[IOUNIT];
    int fa, fb, na, nb, same;
    Dir *da, *db;

    if(a != nil){
        if(a->qid.path == b->qid.path
           && a->qid.vers == b->qid.vers
           && a->qid.type == b->qid.type
           && a->mode == b->mode
           && a->mode != 0)
            return 1;
    }

    same = 0;
    da = nil;
    db = nil;
    if((gitpath = smprint("%s/%s", bdir, b->path)) == nil)

```

```

        sysfatal("smprint: %r");
fa = open(gitpath, OREAD);
fb = open(b->path, OREAD);
if(fa == -1 || fb == -1)
    goto mismatch;
da = dirfststat(fa);
db = dirfststat(fb);
if(da == nil || db == nil)
    goto mismatch;
if((da->mode&0100) != (db->mode&0100))
    goto mismatch;
if(da->length != db->length)
    goto mismatch;
while(1){
    if((na = readn(fa, ba, sizeof(ba))) == -1)
        goto mismatch;
    if((nb = readn(fb, bb, sizeof(bb))) == -1)
        goto mismatch;
    if(na != nb)
        goto mismatch;
    if(na == 0)
        break;
    if(memcmp(ba, bb, na) != 0)
        goto mismatch;
}
if(a != nil){
    a->qid = db->qid;
    a->mode = db->mode;
    staleidx = 1;
}
same = 1;

mismatch:
    free(da);
    free(db);
    if(fa != -1)
        close(fa);
    if(fb != -1)
        close(fb);
    return same;
}

```

<function loadent 262>≡ (265b)

```

void
loadent(char *dir, Dir *d, int fullpath)
{
    char *path;
    Idxent *e;

    if(fullpath)
        path = estrdup(dir);
    else if((path = smprint("%s/%s", dir, d->name)) == nil)
        sysfatal("smprint: %r");

    cleanname(path);
    if(!intree && (printflg & Uflg) == 0 && !indexed(path, d->qid.type & QTDIR)){
        free(path);
        return;
    }
    if(d->qid.type & QTDIR){

```

```

        loadwdir(path);
        free(path);
    }else{
        if(nwdir == wdirs){
            wdirs += wdirs/2;
            wdir = erealloc(wdir, wdirs*sizeof(Idxtent));
        }
        e = wdir + nwdir;
        e->path = path;
        e->qid = !intree? d->qid : (Qid){-1,-1,-1};
        e->mode = d->mode;
        e->order = nwdir;
        e->state = 'T';
        nwdir++;
    }
}

```

<function loadwdir 263a>≡ (265b)

```

void
loadwdir(char *path)
{
    int fd, i, n;
    Dir *d, *e;

    d = nil;
    e = nil;
    cleannname(path);
    if(!intree
    && strncmp(path, ".git", 4) == 0
    && (path[4] == '/' || path[4] == 0))
        return;

    if((fd = open(path, OREAD)) < 0)
        goto error;
    if((e = dirfstat(fd)) == nil)
        fprintf(2, "fstat: %r");
    if(e->qid.type & QTDIR)
        while((n = dirread(fd, &d)) > 0){
            for(i = 0; i < n; i++)
                loadent(path, &d[i], 0);
            free(d);
        }
    else
        loadent(path, e, 1);
error:
    free(e);
    if(fd != -1)
        close(fd);
}

```

<function pfxmatch 263b>≡ (265b)

```

int
pfxmatch(char *p, char **pfx, int *pfxlen, int npfx)
{
    int i;

    if(p == nil)
        return 0;
    if(npfx == 0)
        return 1;

```



```

for(i = 0; i < npfx; i++){
    if(strncmp(p, pfx[i], pfxlen[i]) != 0)
        continue;
    if(p[pfxlen[i]] == '/' || p[pfxlen[i]] == 0)
        return 1;
    if(strcmp(pfx[i], ".") == 0 || *pfx[i] == 0)
        return 1;
}
return 0;
}

```

⟨function reporel 264a⟩≡ (265b)

```

char*
reporel(char *s)
{
    char *p;
    int n;

    if(*s == '/')
        s = estrdup(s);
    else if((s = smprint("%s/%s", wdirpath, s)) == nil)
        sysfatal("smprint: %x");

    p = cleannname(s);
    n = strlen(repopath);
    if(strncmp(s, repopath, n) != 0)
        sysfatal("path outside repo: %s", s);
    p += n;
    if(*p == '/')
        p++;
    memmove(s, p, strlen(p)+1);
    return s;
}

```

⟨function show (git9/walk.c) 264b⟩≡ (265b)

```

void
show(Biobuf *o, int flg, char *str, char *path)
{
    char *pa, *pb;
    int n;

    dirty |= flg;
    if(!quiet && (printflg & flg)){
        Bprint(o, str);
        n = nslash;
        if(n){
            for(pa = relapath, pb = path; *pa && *pb; pa++, pb++){
                if(*pa != *pb)
                    break;
                if(*pa == '/'){
                    n--;
                    path = pb+1;
                }
            }
            while(n-- > 0)
                Bprint(o, "../");
        }
        Bprint(o, "%s\n", path);
    }
}

```

<function findslashes 265a>≡ (265b)

```
void
findslashes(char *path)
{
    char *p;

    p = cleanname(path);
    if(p[0] == '.') {
        if(p[1] == '\\0')
            return;
        else if(p[1] == '.' && (p[2] == '/' || p[2] == '\\0'))
            sysfatal("relative path escapes git root");
    }

    snprintf(relapath, sizeof relapath, "%s/", p);
    p = relapath;
    if(*p == '/')
        p++;

    for(; *p; p++)
        if(*p == '/')
            nslash++;
}
```

<git9/walk.c 265b>≡

```
#include <u.h>
#include <libc.h>
#include "git.h"

typedef struct Idxed      Idxed;
typedef struct Idxent     Idxent;
```

<constant NCACHE 259a>

```
enum {
    Rflg      = 1 << 0,
    Mflg      = 1 << 1,
    Aflg      = 1 << 2,
    Uflg      = 1 << 3,
    /* everything after this is not an error */
    Tflg      = 1 << 4,
};
```

```
struct Idxed {
    char**  cache;
    int n;
    int max;
};
```

```
Idxed      idxtab[NCACHE];
```

```
char      repopath[1024];
<global wdirpath(walk.c) 88b>
char      relapath[1024];
```

```
char      *rstr   = "R ";
char      *mstr   = "M ";
char      *astr   = "A ";
char      *ustr   = "U ";
char      *tstr   = "T ";
```

```

char    *bdir = ".git/fs/HEAD/tree";

int nslash;
int nrel;
int quiet;
int dirty;
int isindexed = 1;
int intree;
int printflg;

Idxent *idx;
int idxsz;
int nidx;

int cleanidx = 0;    /* skip tree check for checkedin() */
int staleidx = 0;

Idxent *wdir;
int wdirs;
int nwdir;

void    loadwdir(char*);

<function checkedin 259b>

<function pathcmp 260a>

<function indexed 260b>

<function idxcmp (git9/walk.c) 261a>

<function samedata 261b>

<function loadent 262>
<function loadwdir 263a>

<function pfxmatch 263b>

<function reporel 264a>

<function show (git9/walk.c) 264b>

<function findslashes 265a>

<function usage (git9/walk.c) 88a>
<function main (git9/walk.c) 88c>

```

D.2 diff/

D.2.1 diff/diff.c

```

<diff/diff.c 266>≡
#include <u.h>
#include <libc.h>
#include <bio.h>

#include "diff.h"

```

<function done(diff) 137a>

<function usage(diff) 136a>

<function main(diff) 136b>

D.2.2 diff/diff.h

<constant MAXPATHLEN(diff) 267a>≡ (267e)
#define MAXPATHLEN 1024

<constant MAXLINELEN(diff) 267b>≡ (267e)
#define MAXLINELEN 4096

<macro DIRECTORY(diff) 267c>≡ (267e)
#define DIRECTORY(s) ((s)->qid.type&QTDIR)

<macro REGULAR_FILE(diff) 267d>≡ (267e)
#define REGULAR_FILE(s) ((s)->type == 'M' && !DIRECTORY(s))

<diff/diff.h 267e>≡
typedef struct Line Line;
typedef struct Cand Cand;
typedef struct Diff Diff;
typedef struct Change Change;

struct Line {
 int serial;
 int value;
};

struct Cand {
 int x;
 int y;
 int pred;
};

struct Change
{
 int oldx;
 int oldy;
 int newx;
 int newy;
};

struct Diff {
 Cand cand;
 Line *file[2], line;
 int len[2];
 int binary;
 int bindiff;
 Line *sfile[2]; /*shortened by pruning common prefix and suffix*/
 int slen[2];
 int pref, suff; /*length of prefix and suffix*/
 int *class; /*will be overlaid on file[0]*/
 int *member; /*will be overlaid on file[1]*/
 int *klist; /*will be overlaid on file[0] after class*/
 Cand *clist; /* merely a free storage pot for candidates */

```

int clen;
int *J;    /*will be overlaid on class*/
long *ixold; /*will be overlaid on klist*/
long *ixnew; /*will be overlaid on file[1]*/
char *file1;
char *file2;
Biobuf *input[2];
Biobuf *b0;
Biobuf *b1;
int firstchange;
Change *changes;
int nchanges;
};

```

```

extern char mode;
extern char bflag;
extern char rflag;
extern char mflag;
extern int anychange;
extern Biobuf stdout;

```

<constant MAXPATHLEN(diff) 267a>
 <constant MAXLINELEN(diff) 267b>

<macro DIRECTORY(diff) 267c>
 <macro REGULAR_FILE(diff) 267d>

```

int mkpathname(char *, char *, char *);
char *mktmpfile(int, Dir **);
char *statfile(char *, Dir **);
void *emalloc(unsigned);
void *erealloc(void *, unsigned);
void diff(char *, char *, int);
void diffreg(char*, char*, char*, char*);
void diffdir(char *, char *, int);
void calcdiff(Diff *, char *, char *, char *, char *);
Biobuf *prepare(Diff*, int, char *, char *);
void check(Diff *, Biobuf *, Biobuf *);
void change(Diff *, int, int, int, int);
void freediff(Diff *);
void flushchanges(Diff *);
void fetch(Diff *d, long *f, int a, int b, Biobuf *bp, char *s);
int readline(Biobuf*, char*, int);

```

D.2.3 diff/diffdir.c

<function itemcmp(diff) 268a>≡ (271a)

```

static int
itemcmp(void *v1, void *v2)
{
    char **d1 = v1, **d2 = v2;

    return strcmp(*d1, *d2);
}

```

<function scandir(diff) 268b>≡ (271a)

```

static char **
scandir(char *name)
{

```

```

char **cp;
Dir *db;
int nitems;
int fd, n;

if ((fd = open(name, OREAD)) < 0) {
    fprintf(2, "%s: can't open %s: %r\n", argv0, name);
    /* fake an empty directory */
    cp = emalloc(sizeof(char*));
    cp[0] = 0;
    return cp;
}
cp = 0;
nitems = 0;
if((n = dirreadall(fd, &db)) > 0){
    while (n--) {
        cp = erealloc(cp, (nitems+1)*sizeof(char*));
        cp[nitems] = emalloc(strlen((db+n)->name)+1);
        strcpy(cp[nitems], (db+n)->name);
        nitems++;
    }
    free(db);
}
cp = erealloc(cp, (nitems+1)*sizeof(char*));
cp[nitems] = 0;
close(fd);
qsort((char *)cp, nitems, sizeof(char*), itemcmp);
return cp;
}

```

<function isdotordotdot(diff) 269a>≡ (271a)

```

static int
isdotordotdot(char *p)
{
    if (*p == '.') {
        if (!p[1])
            return 1;
        if (p[1] == '.' && !p[2])
            return 1;
    }
    return 0;
}

```

<function diffdir(diff) 269b>≡ (271a)

```

void
diffdir(char *f, char *t, int level)
{
    char **df, **dt, **dirf, **dirt;
    char *from, *to;
    int res;
    char fb[MAXPATHLEN+1], tb[MAXPATHLEN+1];

    df = scandir(f);
    dt = scandir(t);
    dirf = df;
    dirt = dt;
    while (*df || *dt) {
        from = *df;
        to = *dt;
        if (from && isdotordotdot(from)) {

```

```

        df++;
        continue;
    }
    if (to && isdotordotdot(to)) {
        dt++;
        continue;
    }
    if (!from)
        res = 1;
    else if (!to)
        res = -1;
    else
        res = strcmp(from, to);
    if (res < 0) {
        if (mode == 0 || mode == 'n')
            Bprint(&stdout, "Only in %s: %s\n", f, from);
        df++;
        continue;
    }
    if (res > 0) {
        if (mode == 0 || mode == 'n')
            Bprint(&stdout, "Only in %s: %s\n", t, to);
        dt++;
        continue;
    }
    if (mkpathname(fb, f, from))
        continue;
    if (mkpathname(tb, t, to))
        continue;
    diff(fb, tb, level+1);
    df++; dt++;
}
for (df = dirf; *df; df++)
    free(*df);
for (dt = dirt; *dt; dt++)
    free(*dt);
free(dirf);
free(dirt);
}

```

<function diff(diff) 270>≡ (271a)

```

void
diff(char *f, char *t, int level)
{
    char *fp, *tp, *p, fb[MAXPATHLEN+1], tb[MAXPATHLEN+1];
    Dir *fsb, *tsb;

    fsb = nil;
    tsb = nil;
    if ((fp = statfile(f, &fsb)) == 0)
        goto Return;
    if ((tp = statfile(t, &tsb)) == 0)
        goto Return;
    if (DIRECTORY(fsb) && DIRECTORY(tsb)) {
        if (rflag || level == 0)
            diffdir(fp, tp, level);
        else
            Bprint(&stdout, "Common subdirectories: %s and %s\n", fp, tp);
    }
    else if (REGULAR_FILE(fsb) && REGULAR_FILE(tsb))

```

```

        diffreg(fp, f, tp, t);
    else {
        if (REGULAR_FILE(fsb)) {
            if ((p = utfrrune(f, '/')) == 0)
                p = f;
            else
                p++;
            if (mkpathname(tb, tp, p) == 0)
                diffreg(fp, f, tb, t);
        } else {
            if ((p = utfrrune(t, '/')) == 0)
                p = t;
            else
                p++;
            if (mkpathname(fb, fp, p) == 0)
                diffreg(fb, f, tp, t);
        }
    }
}
Return:
    free(fsb);
    free(tsb);
}

```

`<diff/diffdir.c 271a>`≡

```

#include <u.h>
#include <libc.h>
#include <bio.h>
#include "diff.h"

```

`<function itemcmp(diff) 268a>`

`<function scandir(diff) 268b>`

`<function isdotordotdot(diff) 269a>`

`<function diffdir(diff) 269b>`

`<function diff(diff) 270>`

D.2.4 diff/diffio.c

`<macro MIN(diff) 271b>`≡ (278a)

```

#define MIN(x, y)    ((x) < (y) ? (x) : (y))

```

`<function readline(diff) 271c>`≡ (278a)

```

int
readline(Biobuf *bp, char *buf, int nbuf)
{
    int c;
    char *p, *e;

    p = buf;
    e = p + nbuf-1;
    do {
        c = Bgetc(bp);
        if (c < 0) {
            if (p == buf)
                return -1;
            break;
        }
    } while (p < e);
}

```



```

    }
    *p++ = c;
    if (c == '\n')
        break;
} while (p < e);
*p = 0;
if (c != '\n' && c >= 0) {
    do c = Bgetc(bp);
    while (c >= 0 && c != '\n');
}
return p - buf;
}

```

<constant HALFLONG(diff) 272a>≡ (278a)
 #define HALFLONG 16

<macro low(diff) 272b>≡ (278a)
 #define low(x) (x&((1L<<HALFLONG)-1))

<macro high(diff) 272c>≡ (278a)
 #define high(x) (x>>HALFLONG)

<function readhash(diff) 272d>≡ (278a)
 /*
 * hashing has the effect of
 * arranging line in 7-bit bytes and then
 * summing 1-s complement in 16-bit hunks
 */
 static int
 readhash(Biobuf *bp, char *buf, int nbuf)
 {
 long sum;
 unsigned shift;
 char *p;
 int len, space;

 sum = 1;
 shift = 0;
 if ((len = readline(bp, buf, nbuf)) == -1)
 return 0;
 p = buf;
 switch(bflag) /* various types of white space handling */
 {
 case 0:
 while (len--) {
 sum += (long)*p++ << (shift &= (HALFLONG-1));
 shift += 7;
 }
 break;
 case 1:
 /*
 * coalesce multiple white-space
 */
 for (space = 0; len--; p++) {
 if (isspace(*p)) {
 space++;
 continue;
 }
 if (space) {
 shift += 7;

```

        space = 0;
    }
    sum += (long)*p << (shift &= (HALFLONG-1));
    shift += 7;
}
break;
default:
    /*
     * strip all white-space
     */
    while (len--) {
        if (isspace(*p)) {
            p++;
            continue;
        }
        sum += (long)*p++ << (shift &= (HALFLONG-1));
        shift += 7;
    }
    break;
}
sum = low(sum) + high(sum);
return ((short)low(sum) + (short)high(sum));
}

```

<function prepare(diff) 273)≡ (278a)

```

Biobuf *
prepare(Diff *d, int i, char *arg, char *orig)
{
    Line *p;
    int j, h;
    Biobuf *bp;
    char *cp, buf[MAXLINELEN];
    int nbytes;
    Rune r;

    if (i == 0) {
        d->file1 = orig;
        d->firstchange = 0;
    } else
        d->file2 = orig;
    bp = Bopen(arg, OREAD);
    if (!bp)
        sysfatal("cannot open %s: %r", arg);
    if (d->binary)
        return bp;
    nbytes = Bread(bp, buf, MIN(1024, MAXLINELEN));
    if (nbytes > 0) {
        cp = buf;
        while (cp < buf+nbytes-UTFmax) {
            /*
             * heuristic for a binary file in the
             * brave new UNICODE world
             */
            cp += chartorune(&r, cp);
            if (r == 0 || (r > 0x7f && r <= 0xa0)) {
                d->binary++;
                return bp;
            }
        }
    }
    Bseek(bp, 0, 0);
}

```

```

}
p = emalloc(3*sizeof(Line));
for (j = 0; h = readhash(bp, buf, sizeof(buf)); p[j].value = h)
    p = erealloc(p, (++j+3)*sizeof(Line));
d->len[i] = j;
d->file[i] = p;
d->input[i] = bp;
return bp;
}

```

⟨function squishspace(*diff*) 274a⟩≡ (278a)

```

static int
squishspace(char *buf)
{
    char *p, *q;
    int space;

    for (space = 0, q = p = buf; *q; q++) {
        if (isspace(*q)) {
            space++;
            continue;
        }
        if (space && bflag == 1) {
            *p++ = ' ';
            space = 0;
        }
        *p++ = *q;
    }
    *p = 0;
    return p - buf;
}

```

⟨function check(*diff*) 274b⟩≡ (278a)

```

/*
 * need to fix up for unexpected EOF's
 */
void
check(Diff *d, Biobuf *bf, Biobuf *bt)
{
    int f, t, flen, tlen;
    char fbuf[MAXLINELEN], tbuf[MAXLINELEN];

    d->ixold[0] = 0;
    d->ixnew[0] = 0;
    for (f = t = 1; f < d->len[0]; f++) {
        flen = readline(bf, fbuf, sizeof(fbuf));
        d->ixold[f] = d->ixold[f-1] + flen; /* ftell(bf) */
        if (d->J[f] == 0)
            continue;
        do {
            tlen = readline(bt, tbuf, sizeof(tbuf));
            d->ixnew[t] = d->ixnew[t-1] + tlen; /* ftell(bt) */
        } while (t++ < d->J[f]);
        if (bflag) {
            flen = squishspace(fbuf);
            tlen = squishspace(tbuf);
        }
        if (flen != tlen || strcmp(fbuf, tbuf))
            d->J[f] = 0;
    }
}

```

```

while (t < d->len[1]) {
    tlen = readline(bt, tbuf, sizeof(tbuf));
    d->ixnew[t] = d->ixnew[t-1] + tlen; /* fseek(bt) */
    t++;
}
}

```

⟨function range(diff) 275a)≡ (278a)

```

static void
range(int a, int b, char *separator)
{
    Bprint(&stdout, "%d", a > b ? b: a);
    if (a < b)
        Bprint(&stdout, "%s%d", separator, b);
}

```

⟨function fetch(diff) 275b)≡ (278a)

```

void
fetch(Diff *d, long *f, int a, int b, Biobuf *bp, char *s)
{
    char buf[MAXLINELEN];
    int maxb, len;

    if(a <= 1)
        a = 1;
    if(bp == d->input[0])
        maxb = d->len[0];
    else
        maxb = d->len[1];
    if(b > maxb)
        b = maxb;
    if(a > maxb)
        return;
    Bseek(bp, f[a-1], 0);
    while (a++ <= b) {
        len = readline(bp, buf, sizeof(buf));
        if(len == 0 || buf[len-1] != '\n'){
            Bprint(&stdout, "%s%s\n", s, buf);
            Bprint(&stdout, "\\ No newline at end of file\n");
        }else
            Bprint(&stdout, "%s%s", s, buf);
    }
}

```

⟨function change(diff) 275c)≡ (278a)

```

void
change(Diff *df, int a, int b, int c, int d)
{
    char verb;
    char buf[4];
    Change *ch;

    if (a > b && c > d)
        return;
    anychange = 1;
    if (mflag && df->firstchange == 0) {
        if(mode) {
            buf[0] = '-';
            buf[1] = mode;
            buf[2] = ' ';

```

```

        buf[3] = '\0';
    } else {
        buf[0] = '\0';
    }
    Bprint(&stdout, "diff %s%s %s\n", buf, df->file1, df->file2);
    df->firstchange = 1;
}
verb = a > b ? 'a': c > d ? 'd': 'c';
switch(mode) {
case 'e':
    range(a, b, ",");
    Bputc(&stdout, verb);
    break;
case 0:
    range(a, b, ",");
    Bputc(&stdout, verb);
    range(c, d, ",");
    break;
case 'n':
    Bprint(&stdout, "%s:", df->file1);
    range(a, b, ",");
    Bprint(&stdout, " %c ", verb);
    Bprint(&stdout, "%s:", df->file2);
    range(c, d, ",");
    break;
case 'f':
    Bputc(&stdout, verb);
    range(a, b, " ");
    break;
case 'c':
case 'a':
case 'u':
    if(df->nchanges%1024 == 0)
        df->changes = erealloc(df->changes, (df->nchanges+1024)*sizeof(df->changes[0]));
    ch = &df->changes[df->nchanges++];
    ch->oldx = a;
    ch->oldy = b;
    ch->newx = c;
    ch->newy = d;
    return;
}
Bputc(&stdout, '\n');
if (mode == 0 || mode == 'n') {
    fetch(df, df->ixold, a, b, df->input[0], "< ");
    if (a <= b && c <= d)
        Bprint(&stdout, "---\n");
}
fetch(df, df->ixnew, c, d, df->input[1], mode == 0 || mode == 'n' ? "> ": "");
if (mode != 0 && mode != 'n' && c <= d)
    Bprint(&stdout, ".\n");
}

```

<function changeset(diff) 276)≡ (278a)

```

int
changeset(Diff *d, int i)
{
    while(i < d->nchanges && d->changes[i].oldy + 1 + 2*Lines > d->changes[i+1].oldx)
        i++;
    if(i < d->nchanges)
        return i+1;
}

```

```

    return d->nchanges;
}

⟨function flushchanges(diff) 277⟩≡ (278a)
void
flushchanges(Diff *df)
{
    vlong a, b, c, d, at, hdr;
    vlong i, j;

    if(df->nchanges == 0)
        return;

    hdr = 0;
    for(i=0; i < df->nchanges; ){
        j = changeset(df, i);
        a = df->changes[i].oldx - Lines;
        b = df->changes[j-1].oldy + Lines;
        c = df->changes[i].newx - Lines;
        d = df->changes[j-1].newy + Lines;
        if(a < 1)
            a = 1;
        if(c < 1)
            c = 1;
        if(b > df->len[0])
            b = df->len[0];
        if(d > df->len[1])
            d = df->len[1];
        if(mode == 'a'){
            a = 1;
            b = df->len[0];
            c = 1;
            d = df->len[1];
            j = df->nchanges;
        }
        if(mode == 'u'){
            if(!hdr){
                Bprint(&stdout, "--- %s\n", df->file1);
                Bprint(&stdout, "+++ %s\n", df->file2);
                hdr = 1;
            }
            Bprint(&stdout, "@@ -%lld,%lld +%lld,%lld @@ \n", a, b-a+1, c, d-c+1);
        }else{
            Bprint(&stdout, "%s:", df->file1);
            range(a, b, ",");
            Bprint(&stdout, " - ");
            Bprint(&stdout, "%s:", df->file2);
            range(c, d, ",");
            Bputc(&stdout, '\n');
        }
        at = a;
        for(; i<j; i++){
            fetch(df, df->ixold, at, df->changes[i].oldx-1, df->input[0], mode == 'u' ? " " : " ");
            fetch(df, df->ixold, df->changes[i].oldx, df->changes[i].oldy, df->input[0], mode == 'u' ? "-" : "-");
            fetch(df, df->ixnew, df->changes[i].newx, df->changes[i].newy, df->input[1], mode == 'u' ? "+" : "+");
            at = df->changes[i].oldy+1;
        }
        fetch(df, df->ixold, at, b, df->input[0], mode == 'u' ? " " : " ");
    }
    df->nchanges = 0;
}

```

```

}

<diff/diffio.c 278a>≡
#include <u.h>
#include <libc.h>
#include <bio.h>
#include <ctype.h>
#include "diff.h"

<macro MIN(diff) 271b>

<function readline(diff) 271c>

<constant HALFLONG(diff) 272a>
<macro low(diff) 272b>
<macro high(diff) 272c>

<function readhash(diff) 272d>

<function prepare(diff) 273>

<function squishspace(diff) 274a>

<function check(diff) 274b>

<function range(diff) 275a>

<function fetch(diff) 275b>

<function change(diff) 275c>

enum
{
    Lines = 3, /* number of lines of context shown */
};

<function changeset(diff) 276>

<function flushchanges(diff) 277>

```

D.2.5 diff/diffreg.c

```

<function sort(diff) 278b>≡ (284c)
static void
sort(Line *a, int n) /*shellsort CACM #201*/
{
    int m;
    Line *ai, *aim, *j, *k;
    Line w;
    int i;

    m = 0;
    for (i = 1; i <= n; i *= 2)
        m = 2*i - 1;
    for (m /= 2; m != 0; m /= 2) {
        k = a+(n-m);
        for (j = a+1; j <= k; j++) {
            ai = j;
            aim = ai+m;

```

```

do {
    if (aim->value > ai->value ||
        aim->value == ai->value &&
        aim->serial > ai->serial)
        break;
    w = *ai;
    *ai = *aim;
    *aim = w;

    aim = ai;
    ai -= m;
} while (ai > a && aim >= ai);
}
}
}

```

⟨function unsort(*diff*) 279a)≡ (284c)

```

static void
unsort(Line *f, int l, int *b)
{
    int *a;
    int i;

    a = malloc((l+1)*sizeof(int));
    for(i=1;i<=l;i++)
        a[f[i].serial] = f[i].value;
    for(i=1;i<=l;i++)
        b[i] = a[i];
    free(a);
}

```

⟨function prune(*diff*) 279b)≡ (284c)

```

static void
prune(Diff *d)
{
    int i,j;

    for(d->pref = 0; d->pref < d->len[0] && d->pref < d->len[1] &&
        d->file[0][d->pref+1].value == d->file[1][d->pref+1].value;
        d->pref++);
    for(d->suff=0; d->suff < d->len[0] - d->pref && d->suff < d->len[1] - d->pref &&
        d->file[0][d->len[0] - d->suff].value == d->file[1][d->len[1] - d->suff].value;
        d->suff++);
    for(j=0;j<2;j++) {
        d->sfile[j] = d->file[j]+d->pref;
        d->slen[j] = d->len[j]-d->pref-d->suff;
        for(i=0;i<=d->slen[j];i++)
            d->sfile[j][i].serial = i;
    }
}

```

⟨function equiv(*diff*) 279c)≡ (284c)

```

static void
equiv(Line *a, int n, Line *b, int m, int *c)
{
    int i, j;

    i = j = 1;
    while(i<=n && j<=m) {
        if(a[i].value < b[j].value)

```



```

        a[i++].value = 0;
    else if(a[i].value == b[j].value)
        a[i++].value = j;
    else
        j++;
}
while(i <= n)
    a[i++].value = 0;
b[m+1].value = 0;
j = 0;
while(++j <= m) {
    c[j] = -b[j].serial;
    while(b[j+1].value == b[j].value) {
        j++;
        c[j] = b[j].serial;
    }
}
c[j] = -1;
}

```

<function newcand(diff) 280a)≡ (284c)

```

static int
newcand(Diff *d, int x, int y, int pred)
{
    Cand *q;

    d->clist = erealloc(d->clist, (d->clen+1)*sizeof(Cand));
    q = d->clist + d->clen;
    q->x = x;
    q->y = y;
    q->pred = pred;
    return d->clen++;
}

```

<function search(diff) 280b)≡ (284c)

```

static int
search(Diff *d, int *c, int k, int y)
{
    int i, j, l;
    int t;

    if(d->clist[c[k]].y < y) /*quick look for typical case*/
        return k+1;
    i = 0;
    j = k+1;
    while((l=(i+j)/2) > i) {
        t = d->clist[c[l]].y;
        if(t > y)
            j = l;
        else if(t < y)
            i = l;
        else
            return l;
    }
    return l+1;
}

```

<function stone(diff) 280c)≡ (284c)

```

static int
stone(Diff *d, int *a, int n, int *b, int *c)

```

```

{
    int i, k,y;
    int j, l;
    int oldc, tc;
    int oldl;

    k = 0;
    c[0] = newcand(d, 0, 0, 0);
    for(i=1; i<=n; i++) {
        j = a[i];
        if(j==0)
            continue;
        y = -b[j];
        oldl = 0;
        oldc = c[0];
        do {
            if(y <= d->clist[oldc].y)
                continue;
            l = search(d, c, k, y);
            if(l!=oldl+1)
                oldc = c[l-1];
            if(l<=k) {
                if(d->clist[c[l]].y <= y)
                    continue;
                tc = c[l];
                c[l] = newcand(d, i, y, oldc);
                oldc = tc;
                oldl = l;
            } else {
                c[l] = newcand(d, i,y,oldc);
                k++;
                break;
            }
        } while((y=b[++j]) > 0);
    }
    return k;
}

```

<function unravel(diff) 281a ≡ (284c)

```

static void
unravel(Diff *d, int p)
{
    int i;
    Cand *q;

    for(i=0; i<=d->len[0]; i++) {
        if (i <= d->pref)
            d->J[i] = i;
        else if (i > d->len[0]-d->suff)
            d->J[i] = i+d->len[1] - d->len[0];
        else
            d->J[i] = 0;
    }
    for(q=d->clist+p; q->y != 0; q= d->clist + q->pred)
        d->J[q->x+d->pref] = q->y+d->pref;
}

```

<constant BUF(diff) 281b ≡ (284c)

```

#define BUF 4096

```

⟨function cmp(diff) 282a⟩≡ (284c)

```
static int
cmp(Biobuf* b1, Biobuf* b2)
{
    int n;
    uchar buf1[BUF], buf2[BUF];
    int f1, f2;
    vlong nc = 1;
    uchar *b1s, *b1e, *b2s, *b2e;

    f1 = Bfildes(b1);
    f2 = Bfildes(b2);
    seek(f1, 0, 0);
    seek(f2, 0, 0);
    b1s = b1e = buf1;
    b2s = b2e = buf2;
    for(;;){
        if(b1s >= b1e){
            if(b1s >= &buf1[BUF])
                b1s = buf1;
            n = read(f1, b1s, &buf1[BUF] - b1s);
            b1e = b1s + n;
        }
        if(b2s >= b2e){
            if(b2s >= &buf2[BUF])
                b2s = buf2;
            n = read(f2, b2s, &buf2[BUF] - b2s);
            b2e = b2s + n;
        }
        n = b2e - b2s;
        if(n > b1e - b1s)
            n = b1e - b1s;
        if(n <= 0)
            break;
        if(memcmp((void *)b1s, (void *)b2s, n) != 0){
            return 1;
        }
        nc += n;
        b1s += n;
        b2s += n;
    }
    if(b1e - b1s == b2e - b2s)
        return 0;
    return 1;
}
```

⟨function calcdiff(diff) 282b⟩≡ (284c)

```
void
calcdiff(Diff *d, char *f, char *fo, char *t, char *to)
{
    Biobuf *b0, *b1;
    int k;

    b0 = prepare(d, 0, f, fo);
    if (!b0)
        return;
    b1 = prepare(d, 1, t, to);
    if (!b1) {
        Bterm(b0);
        return;
    }
}
```

```

}
if (d->binary){
    // could use b0 and b1 but this is simpler.
    if(cmp(b0, b1))
        d->bindiff = 1;
    Bterm(b0);
    Bterm(b1);
    return;
}
d->cflen = 0;
prune(d);
sort(d->sfile[0], d->slen[0]);
sort(d->sfile[1], d->slen[1]);

d->member = (int *)d->file[1];
equiv(d->sfile[0], d->slen[0], d->sfile[1], d->slen[1], d->member);
d->member = erealloc(d->member, (d->slen[1]+2)*sizeof(int));

d->class = (int *)d->file[0];
unsort(d->sfile[0], d->slen[0], d->class);
d->class = erealloc(d->class, (d->slen[0]+2)*sizeof(int));

d->klist = emalloc((d->slen[0]+2)*sizeof(int));
d->clist = emalloc(sizeof(Cand));
k = stone(d, d->class, d->slen[0], d->member, d->klist);
free(d->member);
free(d->class);

d->J = emalloc((d->len[0]+2)*sizeof(int));
unravel(d, d->klist[k]);
free(d->clist);
free(d->klist);

d->ixold = emalloc((d->len[0]+2)*sizeof(long));
d->ixnew = emalloc((d->len[1]+2)*sizeof(long));
Bseek(b0, 0, 0); Bseek(b1, 0, 0);
check(d, b0, b1);
}

```

$\langle$ function output(*diff*) 283 $\rangle \equiv$ (284c)

```

static void
output(Diff *d)
{
    int m, i0, i1, j0, j1;

    if(d->bindiff)
        print("binary files %s %s differ\n", d->file1, d->file2);
    if(d->binary)
        return;
    m = d->len[0];
    d->J[0] = 0;
    d->J[m+1] = d->len[1]+1;
    if (mode != 'e') {
        for (i0 = 1; i0 <= m; i0 = i1+1) {
            while (i0 <= m && d->J[i0] == d->J[i0-1]+1)
                i0++;
            j0 = d->J[i0-1]+1;
            i1 = i0-1;
            while (i1 < m && d->J[i1+1] == 0)
                i1++;
        }
    }
}

```

```

        j1 = d->J[i1+1]-1;
        d->J[i1] = j1;
        change(d, i0, i1, j0, j1);
    }
} else {
    for (i0 = m; i0 >= 1; i0 = i1-1) {
        while (i0 >= 1 && d->J[i0] == d->J[i0+1]-1 && d->J[i0])
            i0--;
        j0 = d->J[i0+1]-1;
        i1 = i0+1;
        while (i1 > 1 && d->J[i1-1] == 0)
            i1--;
        j1 = d->J[i1-1]+1;
        d->J[i1] = j1;
        change(d, i1, i0, j1, j0);
    }
}
if (m == 0)
    change(d, 1, 0, 1, d->len[1]);
flushchanges(d);
}

```

<function diffreg(diff) 284a)≡ (284c)

```

void
diffreg(char *f, char *fo, char *t, char *to)
{
    Diff d;

    memset(&d, 0, sizeof(d));
    calcdiff(&d, f, fo, t, to);
    output(&d);
    freediff(&d);
}

```

<function freediff(diff) 284b)≡ (284c)

```

void
freediff(Diff *d)
{
    if(d->input[0] != nil)
        Bterm(d->input[0]);
    if(d->input[1] != nil)
        Bterm(d->input[1]);
    free(d->J);
    free(d->ixold);
    free(d->ixnew);
}

```

<diff/diffreg.c 284c)≡

```

#include <u.h>
#include <libc.h>
#include <bio.h>

#include "diff.h"

/* diff - differential file comparison
 *
 * Uses an algorithm due to Harold Stone, which finds
 * a pair of longest identical subsequences in the two
 * files.
 *

```

```

* The major goal is to generate the match vector J.
* J[i] is the index of the line in file1 corresponding
* to line i file0. J[i] = 0 if there is no
* such line in file1.
*
* Lines are hashed so as to work in core. All potential
* matches are located by sorting the lines of each file
* on the hash (called value). In particular, this
* collects the equivalence classes in file1 together.
* Subroutine equiv replaces the value of each line in
* file0 by the index of the first element of its
* matching equivalence in (the reordered) file1.
* To save space equiv squeezes file1 into a single
* array member in which the equivalence classes
* are simply concatenated, except that their first
* members are flagged by changing sign.
*
* Next the indices that point into member are unsorted into
* array class according to the original order of file0.
*
* The cleverness lies in routine stone. This marches
* through the lines of file0, developing a vector klist
* of "k-candidates". At step i a k-candidate is a matched
* pair of lines x,y (x in file0 y in file1) such that
* there is a common subsequence of length k
* between the first i lines of file0 and the first y
* lines of file1, but there is no such subsequence for
* any smaller y. x is the earliest possible mate to y
* that occurs in such a subsequence.
*
* Whenever any of the members of the equivalence class of
* lines in file1 matable to a line in file0 has serial number
* less than the y of some k-candidate, that k-candidate
* with the smallest such y is replaced. The new
* k-candidate is chained (via pred) to the current
* k-1 candidate so that the actual subsequence can
* be recovered. When a member has serial number greater
* than the y of all k-candidates, the klist is extended.
* At the end, the longest subsequence is pulled out
* and placed in the array J by unravel.
*
* With J in hand, the matches there recorded are
* check'ed against reality to assure that no spurious
* matches have crept in due to hashing. If they have,
* they are broken, and "jackpot " is recorded--a harmless
* matter except that a true match for a spuriously
* mated line may now be unnecessarily reported as a change.
*
* Much of the complexity of the program comes simply
* from trying to minimize core utilization and
* maximize the range of doable problems by dynamically
* allocating what is needed and reusing what is not.
* The core requirements for problems larger than somewhat
* are (in words) 2*length(file0) + length(file1) +
* 3*(number of k-candidates installed), typically about
* 6n words for files of length n.
*/

```

(function sort(diff) 278b)

<function unsort(diff) 279a>

<function prune(diff) 279b>

<function equiv(diff) 279c>

<function newcand(diff) 280a>

<function search(diff) 280b>

<function stone(diff) 280c>

<function unravel(diff) 281a>

<constant BUF(diff) 281b>

<function cmp(diff) 282a>

<function calcdiff(diff) 282b>

<function output(diff) 283>

<function diffreg(diff) 284a>

<function freediff(diff) 284b>

D.2.6 diff/merge3.c

<macro min(diff) 286a>≡ (289)
#define min(a, b) ((a)<(b)?(a):(b))

<macro max(diff) 286b>≡ (289)
#define max(a, b) ((a)>(b)?(a):(b))

<function changecmp(diff) 286c>≡ (289)
static int
changecmp(void *a, void *b)
{
 return ((Change*)a)->oldx - ((Change*)b)->oldx;
}

<function addchange(diff) 286d>≡ (289)
static void
addchange(Diff *df, int a, int b, int c, int d)
{
 Change *ch;

 if (a > b && c > d)
 return;
 if(df->nchanges%1024 == 0)
 df->changes = erealloc(df->changes, (df->nchanges+1024)*sizeof(df->changes[0]));
 ch = &df->changes[df->nchanges++];
 ch->oldx = a;
 ch->oldy = b;
 ch->newx = c;
 ch->newy = d;
}

<function collect(diff) 287a>≡ (289)

```
static void
collect(Diff *d)
{
    int m, i0, i1, j0, j1;

    m = d->len[0];
    d->J[0] = 0;
    d->J[m+1] = d->len[1]+1;
    for (i0 = m; i0 >= 1; i0 = i1-1) {
        while (i0 >= 1 && d->J[i0] == d->J[i0+1]-1 && d->J[i0])
            i0--;
        j0 = d->J[i0+1]-1;
        i1 = i0+1;
        while (i1 > 1 && d->J[i1-1] == 0)
            i1--;
        j1 = d->J[i1-1]+1;
        d->J[i1] = j1;
        addchange(d, i1, i0, j1, j0);
    }
    if (m == 0)
        addchange(d, 1, 0, 1, d->len[1]);
    qsort(d->changes, d->nchanges, sizeof(Change), changecmp);
}
```

<function overlaps(diff) 287b>≡ (289)

```
static int
overlaps(int lx, int ly, int rx, int ry)
{
    if(lx <= rx)
        return ly >= rx;
    else
        return ry >= lx;
}
```

<function same(diff) 287c>≡ (289)

```
static int
same(Diff *l, Change *lc, Diff *r, Change *rc)
{
    char lbuf[MAXLINELEN], rbuf[MAXLINELEN];
    int i, ll, rl, lx, ly, rx, ry;

    lx = lc->newx;
    ly = lc->newy;
    rx = rc->newx;
    ry = rc->newy;
    if(ly - lx != ry - rx)
        return 0;
    assert(ly <= l->len[1] && ry <= r->len[1]);
    Bseek(l->input[1], l->ixnew[lx-1], 0);
    Bseek(r->input[1], r->ixnew[rx-1], 0);
    for(i = 0; i <= (ly - lx); i++){
        ll = readline(l->input[1], lbuf, sizeof(lbuf));
        rl = readline(r->input[1], rbuf, sizeof(rbuf));
        if(ll != rl)
            return 0;
        if(memcmp(lbuf, rbuf, ll) != 0)
            return 0;
    }
    return 1;
}
```


<function merge(diff) 288>≡

(289)

```

char*
merge(Diff *l, Diff *r)
{
    int lx, ly, rx, ry;
    int il, ir, delta;
    Change *lc, *rc;
    char *status;
    vlong ln;

    il = 0;
    ir = 0;
    ln = 0;
    status = nil;
    collect(l);
    collect(r);
    while(il < l->nchanges || ir < r->nchanges){
        lc = nil;
        rc = nil;
        lx = -1;
        ly = -1;
        rx = -1;
        ry = -1;
        if(il < l->nchanges){
            lc = &l->changes[il];
            lx = (lc->oldx < lc->oldy) ? lc->oldx : lc->oldy;
            ly = (lc->oldx < lc->oldy) ? lc->oldy : lc->oldx;
        }
        if(ir < r->nchanges){
            rc = &r->changes[ir];
            rx = (rc->oldx < rc->oldy) ? rc->oldx : rc->oldy;
            ry = (rc->oldx < rc->oldy) ? rc->oldy : rc->oldx;
        }
        if(lc != nil && rc != nil && overlaps(lx, ly, rx, ry)){
            /*
             * align the edges of the chunks, expanding them
             * so that when we compare for sameness, we are
             * comparing same-sized chunks.
             */
            if(lc->oldx < rc->oldx){
                delta = rc->oldx - lc->oldx;
                rc->oldx = max(rc->oldx-delta, 1);
                rc->newx = max(rc->newx-delta, 1);
            }else{
                delta = lc->oldx - rc->oldx;
                lc->oldx = max(lc->oldx-delta, 1);
                lc->newx = max(lc->newx-delta, 1);
            }
            if(lc->oldy > rc->oldy){
                delta = lc->oldy - rc->oldy;
                rc->oldy = min(rc->oldy+delta, r->len[0]);
                rc->newy = min(rc->newy+delta, r->len[1]);
            }else{
                delta = rc->oldy - lc->oldy;
                lc->oldy = min(lc->oldy+delta, l->len[0]);
                lc->newy = min(lc->newy+delta, l->len[1]);
            }
            if(same(l, lc, r, rc)){
                fetch(l, l->ixold, ln, lc->oldx-1, l->input[0], "");
                fetch(l, l->ixnew, lc->newx, lc->newy, l->input[1], "");
            }
        }
    }
}

```

```

    }else{
        fetch(l, l->ixold, ln, lc->oldx-1, l->input[0], "");
        Bprint(&stdout, "<<<<<<<<< %s\n", l->file2);
        fetch(l, l->ixnew, lc->newx, lc->newy, l->input[1], "");
        Bprint(&stdout, "===== original\n");
        fetch(l, l->ixold, lc->oldx, lc->oldy, l->input[0], "");
        Bprint(&stdout, "===== %s\n", r->file2);
        fetch(r, r->ixnew, rc->newx, rc->newy, r->input[1], "");
        Bprint(&stdout, ">>>>>>>>\n");
        status = "conflict";
    }
    ln = lc->oldy+1;
    il++;
    ir++;
}else if(lc != nil && (rc == nil || lx < rx)){
    fetch(l, l->ixold, ln, lc->oldx-1, l->input[0], "");
    fetch(l, l->ixnew, lc->newx, lc->newy, l->input[1], "");
    ln = lc->oldy+1;
    il++;
}else if(rc != nil && (lc == nil || rx < lx)){
    fetch(l, r->ixold, ln, rc->oldx-1, r->input[0], "");
    fetch(r, r->ixnew, rc->newx, rc->newy, r->input[1], "");
    ln = rc->oldy+1;
    ir++;
}else
    abort();
}
if(ln <= l->len[0])
    fetch(l, l->ixold, ln, l->len[0], l->input[0], "");
return status;
}

```

<diff/merge3.c 289>≡

```

#include <u.h>
#include <libc.h>
#include <bio.h>

```

```

#include "diff.h"

```

<macro min(diff) 286a>

<macro max(diff) 286b>

<function changecmp(diff) 286c>

<function addchange(diff) 286d>

<function collect(diff) 287a>

<function overlaps(diff) 287b>

<function same(diff) 287c>

<function merge(diff) 288>

<function usage (diff/merge3.c)(diff) 139a>

<function main (diff/merge3.c)(diff) 139b>

D.2.7 diff/util.c

```
<function mkpathname(diff) 290a>≡ (291a)
int
mkpathname(char *pathname, char *path, char *name)
{
    if (strlen(path) + strlen(name) > MAXPATHLEN)
        sysfatal("pathname %s/%s too long", path, name);
    sprintf(pathname, "%s/%s", path, name);
    return 0;
}

<function mktmpfile(diff) 290b>≡ (291a)
char *
mktmpfile(int input, Dir **sb)
{
    int fd, i;
    char *p;
    char buf[8192];

    p = mktemp(tmp[whichtmp++]);
    /*
     * Because we want this file to stick around
     * for the entire run of the program, we leak
     * the fd intentionally here; when we exit,
     * the system will remove the file for us.
     */
    fd = create(p, OWRITE|ORCLOSE, 0600);
    if (fd < 0)
        sysfatal("cannot create %s: %r", p);
    while ((i = read(input, buf, sizeof(buf))) > 0) {
        if ((i = write(fd, buf, i)) < 0)
            break;
    }
    *sb = dirfstat(fd);
    if (i < 0)
        sysfatal("cannot read/write %s: %r", p);
    return p;
}

<function statfile(diff) 290c>≡ (291a)
char *
statfile(char *file, Dir **sb)
{
    Dir *dir;
    int input;

    dir = dirstat(file);
    if (dir == nil) {
        if (strcmp(file, "-") || (dir = dirfstat(0)) == nil)
            sysfatal("cannot stat %s: %r", file);
        free(dir);
        return mktmpfile(0, sb);
    } else if (!REGULAR_FILE(dir) && !DIRECTORY(dir)) {
        free(dir);
        if ((input = open(file, OREAD)) == -1)
            sysfatal("cannot open %s: %r", file);
        file = mktmpfile(input, sb);
        close(input);
    } else
```

```

        *sb = dir;
    return file;
}

⟨diff/util.c 291a⟩≡
#include <u.h>
#include <libc.h>
#include <bio.h>
#include "diff.h"

Biobuf  stdout;
char    mode;           /* '\0', 'e', 'f', 'h' */
char    bflag;          /* ignore multiple and trailing blanks */
char    rflag;          /* recurse down directory trees */
char    mflag;          /* pseudo flag: doing multiple files, one dir */
int anychange;

static char *tmp[] = {"/tmp/diff1XXXXXXXXXX", "/tmp/diff2XXXXXXXXXX"};
static int whichtmp;

⟨function emalloc(diff) 162c⟩

⟨function erealloc(diff) 162d⟩

⟨function mkpathname(diff) 290a⟩

⟨function mktmpfile(diff) 290b⟩

⟨function statfile(diff) 290c⟩

```

D.3 misc/

D.3.1 misc/patch.c

```

⟨function readline(patch.c) 291b⟩≡ (303)
char*
readline(Biobuf *f, int *lnum)
{
    char *ln;

    if((ln = Brdstr(f, '\n', 0)) == nil)
        return nil;
    *lnum += 1;
    return ln;
}

⟨function fail(patch.c) 291c⟩≡ (303)
void
fail(char *fmt, ...)
{
    char msg[ERRMAX];
    va_list ap;

    finish(0);
    va_start(ap, fmt);
    vsnprint(msg, sizeof(msg), fmt, ap);
    va_end(ap);
    fprintf(2, "%s\n", msg);
}

```

```

    exits(msg);
}

```

<function fileheader(patch.c) 292a>≡ (303)

```

int
fileheader(char *s, char *pfx, char **name)
{
    int len, n, nnull;
    char *e;

    if((strcmp(s, pfx, strlen(pfx))) != 0)
        return -1;
    for(s += strlen(pfx); *s; s++)
        if(!isspace(*s))
            break;
    for(e = s; *e; e++)
        if(isspace(*e))
            break;
    if(s == e)
        return -1;
    nnull = strlen("/dev/null");
    if((e - s) != nnull || strcmp(s, "/dev/null", nnull) != 0){
        n = strip;
        while(s != e && n > 0){
            while(s != e && *s == '/')
                s++;
            while(s != e && *s != '/')
                s++;
            n--;
        }
        while(*s == '/')
            s++;
        if(*s == '\0')
            fail("too many components stripped");
    }
    len = (e - s) + 1;
    *name = emalloc(len);
    strcpy(*name, *name + len, s);
    return 0;
}

```

<function hunkheader(patch.c) 292b>≡ (303)

```

int
hunkheader(Hunk *h, char *s, char *oldpath, char *newpath, int lnum)
{
    char *e;

    memset(h, 0, sizeof(*h));
    h->lnum = lnum;
    h->oldpath = strdup(oldpath);
    h->newpath = strdup(newpath);
    h->origlen = 0;
    h->origsz = 32;
    h->orig = emalloc(h->origsz);
    h->oldlen = 0;
    h->oldsz = 32;
    h->old = emalloc(h->oldsz);
    h->newlen = 0;
    h->newsz = 32;
    h->new = emalloc(h->newsz);
}

```

```

cleanname(h->oldpath);
cleanname(h->newpath);
if(strncmp(s, "@@ -", 4) != 0)
    return -1;
e = s + 4;
h->oldln = strtol(e, &e, 10);
h->oldcnt = 1;
if(*e == ','){
    e++;
    h->oldcnt = strtol(e, &e, 10);
}
while(*e == ' ' || *e == '\t')
    e++;
if(*e != '+')
    return -1;
e++;
h->newln = strtol(e, &e, 10);
h->newcnt = 1;
if(e == s)
    return -1;
if(*e == ','){
    e++;
    h->newcnt = strtol(e, &e, 10);
}
if(e == s || *e != ' ')
    return -1;
if(strncmp(e, " @@", 3) != 0)
    return -1;
/*
 * empty files have line number 0: keep that,
 * otherwise adjust down.
 */
if(h->oldln > 0)
    h->oldln--;
if(h->newln > 0)
    h->newln--;
if(h->oldln < 0 || h->newln < 0 || h->oldcnt < 0 || h->newcnt < 0)
    fail("malformed hunk %s", s);
return 0;
}

```

<function addorig(patch.c) 293a> ≡ (303)

```

void
addorig(Hunk *h, char *ln)
{
    int n;

    n = strlen(ln);
    while(h->origlen + n >= h->origsz){
        h->origsz *= 2;
        h->orig = erealloc(h->orig, h->origsz);
    }
    memcpy(h->orig + h->origlen, ln, n);
    h->origlen += n;
}

```

<function addnew(patch.c) 293b> ≡ (303)

```

void
addnew(Hunk *h, char *ln)
{

```

```

int n;

ln++;
n = strlen(ln);
while(h->newlen + n >= h->newsz){
    h->newsz *= 2;
    h->new = erealloc(h->new, h->newsz);
}
memcpy(h->new + h->newlen, ln, n);
h->newlen += n;
}

```

⟨function addold(patch.c) 294a⟩≡ (303)

```

void
addold(Hunk *h, char *ln)
{
    int n;

    ln++;
    n = strlen(ln);
    while(h->oldlen + n >= h->oldsz){
        h->oldsz *= 2;
        h->old = erealloc(h->old, h->oldsz);
    }
    memcpy(h->old + h->oldlen, ln, n);
    h->oldlen += n;
}

```

⟨function addmiss(patch.c) 294b⟩≡ (303)

```

int
addmiss(Hunk *h, char *ln, int *nold, int *nnew)
{
    if(ln == nil)
        return 1;
    else if(ln[0] != '-' && ln[0] != '+')
        return 0;
    if(ln[0] == '-'){
        addold(h, ln);
        *nold += 1;
    }else{
        addnew(h, ln);
        *nnew += 1;
    }
    return 1;
}

```

⟨function addhunk(patch.c) 294c⟩≡ (303)

```

void
addhunk(Patch *p, Hunk *h)
{
    p->hunk = erealloc(p->hunk, ++p->nhunk*sizeof(Hunk));
    p->hunk[p->nhunk-1] = *h;
}

```

⟨function hunkcmp(patch.c) 294d⟩≡ (303)

```

int
hunkcmp(void *a, void *b)
{
    int c;

```

```

    c = strcmp(((Hunk*)a)->oldpath, ((Hunk*)b)->oldpath);
    if(c != 0)
        return c;
    return ((Hunk*)a)->oldln - ((Hunk*)b)->oldln;
}

```

⟨function swapint(patch.c) 295a⟩≡ (303)

```

void
swapint(int *a, int *b)
{
    int t;

    t = *a;
    *a = *b;
    *b = t;
}

```

⟨function swapstr(patch.c) 295b⟩≡ (303)

```

void
swapstr(char **a, char **b)
{
    char *t;

    t = *a;
    *a = *b;
    *b = t;
}

```

⟨function trimhunk(patch.c) 295c⟩≡ (303)

```

void
trimhunk(char c, Hunk *h)
{
    if((c == ' ' || c == '-') && h->oldlen > 0 && h->old[h->oldlen-1] == '\n'){
        h->oldcnt--;
        h->oldlen--;
    }
    if((c == ' ' || c == '+') && h->newlen > 0 && h->new[h->newlen-1] == '\n'){
        h->newcnt--;
        h->newlen--;
    }
}

```

⟨function parse(patch.c) 295d⟩≡ (303)

```

Patch*
parse(Biobuf *f, char *name)
{
    char *ln, *old, *new, c;
    int i, oldcnt, newcnt, lnum;
    Patch *p;
    Hunk h, *ph;

    ln = nil;
    lnum = 0;
    p = emalloc(sizeof(Patch));
comment:
    free(ln);
    while((ln = readline(f, &lnum)) != nil){
        if(strncmp(ln, "--- ", 4) == 0)
            goto patch;
        free(ln);
    }
}

```



```

}
if(p->nhunk == 0)
    fail("%s: could not find start of patch", name);
goto out;

patch:
    if(fileheader(ln, "--- ", &old) == -1)
        goto comment;
    free(ln);

    if((ln = readline(f, &lnum)) == nil)
        goto out;
    if(fileheader(ln, "+++ ", &new) == -1)
        goto comment;
    free(ln);

    if((ln = readline(f, &lnum)) == nil)
        goto out;
hunk:
    oldcnt = 0;
    newcnt = 0;
    if(hunkheader(&h, ln, old, new, lnum) == -1)
        goto comment;
    free(ln);

    while(1){
        if((ln = readline(f, &lnum)) == nil){
            if(oldcnt != h.oldcnt)
                fail("%s:%d: malformed hunk: mismatched -hunk size %d != %d", name, lnum, oldcnt, h.oldcnt);
            if(newcnt != h.newcnt)
                fail("%s:%d: malformed hunk: mismatched +hunk size %d != %d", name, lnum, newcnt, h.newcnt);
            addhunk(p, &h);
            break;
        }
        c = ln[0];
        addorig(&h, ln);
        switch(ln[0]){
            default:
                fail("%s:%d: malformed hunk: leading junk", name, lnum);
            case '\\':
                if(strncmp(ln, "\\ No newline", nelem("\\ No newline")-1) == 0)
                    trimhunk(c, &h);
                /* ignore unknown directives */
                break;
            case '-':
                addold(&h, ln);
                oldcnt++;
                break;
            case '+':
                addnew(&h, ln);
                newcnt++;
                break;
            case '\\n':
                addold(&h, " \n");
                addnew(&h, " \n");
                oldcnt++;
                newcnt++;
                break;
            case ' ':
                addold(&h, ln);

```

```

        addnew(&h, ln);
        oldcnt++;
        newcnt++;
        break;
    }
    free(ln);
    if(oldcnt > h.oldcnt || newcnt > h.newcnt)
        fail("%s:%d: malformed hunk: oversized hunk", name, lnum);
    if(oldcnt < h.oldcnt || newcnt < h.newcnt)
        continue;

    addhunk(p, &h);
    if((ln = readline(f, &lnum)) == nil)
        goto out;
    if(strncmp(ln, "\\ No newline", nelem("\\ No newline")-1) == 0)
        trimhunk(c, &p->hunk[p->nhunk-1]);
    if(strncmp(ln, "--- ", 4) == 0)
        goto patch;
    if(strncmp(ln, "@@ ", 3) == 0)
        goto hunk;
    goto comment;
}

out:
if(reverse){
    for(i = 0; i < p->nhunk; i++){
        ph = &p->hunk[i];
        swapint(&ph->oldln, &ph->newln);
        swapint(&ph->oldcnt, &ph->newcnt);
        swapint(&ph->oldlen, &ph->newlen);
        swapint(&ph->oldsz, &ph->newsz);
        swapstr(&ph->oldpath, &ph->newpath);
        swapstr(&ph->old, &ph->new);
    }
}
qsort(p->hunk, p->nhunk, sizeof(Hunk), hunkcmp);
free(old);
free(new);
free(ln);
return p;
}

```

⟨function rename(patch.c) 297a⟩≡ (303)

```

int
rename(int fd, char *name)
{
    Dir st;
    char *p;

    nulldir(&st);
    if((p = strrchr(name, '/')) == nil)
        st.name = name;
    else
        st.name = p + 1;
    return dirfwstat(fd, &st);
}

```

⟨function mkpath(patch.c) 297b⟩≡ (303)

```

int
mkpath(char *path)

```

```

{
    char *p, buf[ERRMAX];
    int f;

    if(*path == '\0')
        return 0;
    for(p = strchr(path+1, '/'); p != nil; p = strchr(p+1, '/')){
        *p = '\0';
        if(access(path, AEXIST) != 0){
            if((f = create(path, OREAD, DMDIR | 0777)) == -1){
                rerrstr(buf, sizeof(buf));
                if(strstr(buf, "exist") == nil)
                    return -1;
            }
            close(f);
        }
        *p = '/';
    }
    return 0;
}

```

⟨function blat(patch.c) 298a⟩≡ (303)

```

void
blat(char *old, char *new, char *o, usize len, int mode)
{
    char *tmp;
    int fd;

    tmp = nil;
    if(strcmp(old, "/dev/null") == 0 && !dryrun)
        if(access(new, AEXIST) != -1)
            fail("would clobber: %s", new);
    if(strcmp(new, "/dev/null") != 0 && !dryrun){
        if(mkpath(new) == -1)
            fail("mkpath %s: %r", new);
        if((tmp = smprint("%s.tmp%d", new, getpid())) == nil)
            fail("smprint: %r");
        if((fd = create(tmp, OWRITE, mode|0200)) == -1)
            fail("open %s: %r", tmp);
        if(write(fd, o, len) != len)
            fail("write %s: %r", tmp);
        close(fd);
    }
    if((changed = realloc(changed, (nchanged+1)*sizeof(Fchg))) == nil)
        fail("realloc: %r");
    if((changed[nchanged].new = strdup(new)) == nil)
        fail("strdup: %r");
    if((changed[nchanged].old = strdup(old)) == nil)
        fail("strdup: %r");
    changed[nchanged].tmp = tmp;
    nchanged++;
}

```

⟨function finish(patch.c) 298b⟩≡ (303)

```

int
finish(int ok)
{
    Fchg *c;
    int i, fd;

```

```

for(i = 0; i < nchanged; i++){
    c = &changed[i];
    if(!ok){
        if(c->tmp != nil && remove(c->tmp) == -1)
            fprintf(2, "remove %s: %r\n", c->tmp);
        goto Free;
    }
    if(!dryrun){
        if(strcmp(c->new, "/dev/null") == 0){
            if(remove(c->old) == -1){
                ok = 0;
                fprintf(2, "remove %s: %r\n", c->old);
                goto Free;
            }
            goto Print;
        }
        if((fd = open(c->tmp, ORDWR)) == -1){
            ok = 0;
            fprintf(2, "open %s: %r\n", c->tmp);
            goto Free;
        }
        if(strcmp(c->old, c->new) == 0 && remove(c->old) == -1
            || rename(fd, c->new) == -1
            || close(fd) == -1){
            ok = 0;
            fprintf(2, "cleanup: %r");
        }
    }
}

Print:
if(strcmp(c->new, "/dev/null") == 0)
    print("%s\n", c->old);
else
    print("%s\n", c->new);

Free:
    free(c->tmp);
    free(c->old);
    free(c->new);
}
free(changed);
return ok;
}

```

```

⟨function slurp(patch.c) 299⟩≡ (303)
int
slurp(Fbuf *f, char *path)
{
    int n, i, fd, sz, len, nlines, linesz;
    char *buf;
    int *lines;
    Dir *d;

    if((fd = open(path, OREAD)) == -1){
        fprintf(2, "open %s: %r", path);
        return -1;
    }
    if((d = dirfstat(fd)) == nil)
        fail("stat %s: %r", path);
    sz = 8192;
    len = 0;
    buf = emalloc(sz);

```

```

while(1){
    if(len == sz){
        sz *= 2;
        buf = erealloc(buf, sz);
    }
    n = read(fd, buf + len, sz - len);
    if(n == 0)
        break;
    if(n == -1)
        fail("read %s: %r", path);
    len += n;
}

nlines = 0;
linesz = 32;
lines = emalloc(linesz*sizeof(int));
lines[nlines++] = 0;
for(i = 0; i < len; i++){
    if(buf[i] != '\n')
        continue;
    if(nlines+2 == linesz){
        linesz *= 2;
        lines = erealloc(lines, linesz*sizeof(int));
    }
    lines[nlines++] = i+1;
}
lines[nlines] = len;
f->len = len;
f->buf = buf;
f->lines = lines;
f->nlines = nlines;
f->lastln = 0;
f->lastfuzz = 0;
f->mode = d->mode;
free(d);
close(fd);
return 0;
}

```

<function searchln(patch.c) 300a>≡ (303)

```

char*
searchln(Fbuf *f, Hunk *h, int ln)
{
    int off;

    off = f->lines[ln];
    if(off + h->oldlen > f->len)
        return nil;
    if(memcmp(f->buf + off, h->old, h->oldlen) != 0)
        return nil;
    f->lastln = ln + h->oldcnt;
    f->lastfuzz = ln - h->oldln;
    return f->buf + off;
}

```

<function search(patch.c) 300b>≡ (303)

```

char*
search(Fbuf *f, Hunk *h)
{
    int ln, oldln, fuzz, scanning;

```

```

char *p;

oldln = h->oldln + f->lastfuzz;
if(oldln + h->oldcnt > f->nlines)
    oldln = f->nlines - h->oldcnt;
scanning = oldln >= f->lastln;
for(fuzz = 0; scanning && fuzz < 250; fuzz++){
    scanning = 0;
    ln = oldln - fuzz;
    if(ln >= f->lastln){
        scanning = 1;
        if((p = searchln(f, h, ln)) != nil)
            return p;
    }
    ln = oldln + fuzz + 1;
    if(ln + h->oldcnt <= f->nlines){
        scanning = 1;
        if((p = searchln(f, h, ln)) != nil)
            return p;
    }
}
return nil;
}

```

```

⟨function rejected(patch.c) 301a⟩≡ (303)
void
rejected(Hunk *h, char *fname)
{
    fprintf(2, "%s:%d: skipping failed hunk %s:%d\n", fname, h->lnum, h->oldpath, h->oldln);
    fprintf(rejfd, "--- %s:%d \n", h->oldpath, h->oldln);
    fprintf(rejfd, "+++ %s:%d \n", h->newpath, h->newln);
    fprintf(rejfd, "@@ -%d,%d +%d,%d @@\n", h->oldln, h->oldcnt, h->newln, h->newcnt);
    write(rejfd, h->orig, h->origlen);
}

```

```

⟨function append(patch.c) 301b⟩≡ (303)
char*
append(char *o, int *sz, char *s, char *e)
{
    int n;

    n = (e - s);
    o = erealloc(o, *sz + n);
    memcpy(o + *sz, s, n);
    *sz += n;
    return o;
}

```

```

⟨function apply(patch.c) 301c⟩≡ (303)
int
apply(Patch *p, char *fname)
{
    char *o, *s, *n, *curfile, *nextfile;
    int i, osz;
    Hunk *h, *prevh;
    Fbuf f;

    n = nil;
    o = nil;
    osz = 0;

```

```

curfile = nil;
h = nil;
prevh = nil;
nchanged = 0;
changed = nil;
for(i = 0; i < p->nhunk; i++){
    h = &p->hunk[i];
    if(strcmp(h->newpath, "/dev/null") == 0)
        nextfile = h->oldpath;
    else
        nextfile = h->newpath;
    if(curfile == nil || strcmp(curfile, nextfile) != 0){
        if(curfile != nil){
            if(!dryrun)
                o = append(o, &osz, n, f.buf + f.len);
            blat(prevh->oldpath, prevh->newpath, o, osz, f.mode);
            osz = 0;
        }
        if(!dryrun){
            if(slurp(&f, h->oldpath) == -1){
                rejected(h, fname);
                goto Next;
            }
            n = f.buf;
        }
        curfile = nextfile;
    }
    if(!dryrun){
        char *e;
        s = n;
        e = search(&f, h);
        if(e == nil){
            if(rejfd == -1)
                fail("%s:%d: unable to find hunk offset near %s:%d", fname, h->lnum, h->oldpath, h->oldln);
            else{
                rejected(h, fname);
                goto Next;
            }
        }
        o = append(o, &osz, s, e);
        o = append(o, &osz, h->new, h->new + h->newlen);
        n = e + h->oldlen;
    }
}
Next:
    prevh = h;
}
if(curfile != nil){
    if(!dryrun)
        o = append(o, &osz, n, f.buf + f.len);
    blat(h->oldpath, h->newpath, o, osz, f.mode);
}
free(o);
return 0;
}

```

```

⟨function freepatch(patch.c) 302⟩≡ (303)
void
freepatch(Patch *p)
{
    Hunk *h;

```

```

int i;

for(i = 0; i < p->nhunk; i++){
    h = &p->hunk[i];
    free(h->oldpath);
    free(h->newpath);
    free(h->old);
    free(h->new);
}
free(p->hunk);
free(p->name);
free(p);
}

```

```

<misc/patch.c 303>≡
#include <u.h>
#include <libc.h>
#include <ctype.h>
#include <bio.h>

typedef struct Patch Patch;
typedef struct Hunk Hunk;
typedef struct Fbuf Fbuf;
typedef struct Fchg Fchg;

struct Patch {
    char    *name;
    Hunk    *hunk;
    usize   nhunk;
};

struct Hunk {
    int lnum;

    char    *orig;
    int origlen;
    int origsz;

    char    *oldpath;
    int oldln;
    int oldcnt;
    int oldlen;
    int oldsz;
    char    *old;

    char    *newpath;
    int newln;
    int newcnt;
    int newlen;
    int newsz;
    char    *new;
};

struct Fbuf {
    int *lines;
    int nlines;
    int lastln;
    int lastfuzz;
    char *buf;
    int len;
}

```



```

    int mode;
};

struct Fchg {
    char    *tmp;
    char    *old;
    char    *new;
};

int finish(int);
void    fail(char*, ...);

int strip;
int reverse;
int rejfd = -1;
char    *rejfile;
char    *workdir;
Fchg    *changed;
int nchanged;
int dryrun;

⟨function readline(patch.c) 291b⟩

⟨function emalloc(patch.c) 162e⟩

⟨function erealloc(patch.c) 163a⟩

⟨function fail(patch.c) 291c⟩

⟨function fileheader(patch.c) 292a⟩

⟨function hunkheader(patch.c) 292b⟩

⟨function addorig(patch.c) 293a⟩

⟨function addnew(patch.c) 293b⟩

⟨function addold(patch.c) 294a⟩

⟨function addmiss(patch.c) 294b⟩

⟨function addhunk(patch.c) 294c⟩

⟨function hunkcmp(patch.c) 294d⟩

⟨function swapint(patch.c) 295a⟩

⟨function swapstr(patch.c) 295b⟩

⟨function trimhunk(patch.c) 295c⟩

⟨function parse(patch.c) 295d⟩

⟨function rename(patch.c) 297a⟩

⟨function mkpath(patch.c) 297b⟩

⟨function blat(patch.c) 298a⟩

⟨function finish(patch.c) 298b⟩

```

⟨function slurp(patch.c) 299⟩

⟨function searchln(patch.c) 300a⟩

⟨function search(patch.c) 300b⟩

⟨function rejected(patch.c) 301a⟩

⟨function append(patch.c) 301b⟩

⟨function apply(patch.c) 301c⟩

⟨function freepatch(patch.c) 302⟩

⟨function usage(patch.c) 137b⟩

⟨function main(patch.c) 138⟩

Glossary

VCS = Version Control System

DVCS = Distributed Version Control System

SHA1 = Secure Hash Algorithm 1

URL = Uniform Resource Locator

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Bibliography

- [CS14] Scott Chacon and Ben Straub. *Pro Git*. 2014. Available at <https://git-scm.com/book/en/v2>. cited page(s) 13, 14
- [EJ01] D. Eastlake and P. Jones. US Secure Hash Algorithm 1 (SHA1), September 2001. Available at <http://www.ietf.org/rfc/rfc3174.txt>. cited page(s) 18
- [Gru86] Dick Grune. Concurrent versions system, a method for independent cooperation. Technical report, Vrije Universiteit, Amsterdam, 1986. Unpublished but available at https://dickgrune.com/Books/Publications/Concurrent_Versions_System,_a_method_for_independent_cooperation.ps. cited page(s) 12
- [Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 14
- [KR88] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988. cited page(s) 14
- [O’S09] Brian O’Sullivan. *Mercurial: The Definitive Guide*. O’Reilly, 2009. Available at <http://hgbook.red-bean.com/>. cited page(s) 13
- [Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. <https://github.com/aryx/syncweb>. cited page(s) 14
- [Pad14] Yoann Padioleau. *Principia Softwarica: The Plan 9 Kernel 9*. 2014. cited page(s) 18, 23
- [Pad18] Yoann Padioleau. *Principia Softwarica: The Plan 9 Shell rc*. 2018. cited page(s) 12, 14
- [Pad25] Yoann Padioleau. *Principia Softwarica: The Plan 9 Utilities*. 2025. cited page(s) 14
- [Roc75] Marc J. Rochkind. The source code control system. *IEEE Transactions on Software Engineering*, pages 364–370, December 1975. Available at <http://www.basepath.com/aup/talks/SCCS-Slideshow.pdf>. cited page(s) 12
- [sha93] Secure hash standard, April 1993. Latest version available at <https://csrc.nist.gov/publications/detail/fips/180/4/final>. cited page(s) 18
- [Tic85] Walter F. Tichy. Rcs - a system for version control. *Software Practice and Experience*, pages 637–654, July 1985. Available at <http://www.gnu.org/s/rcs/tichy-paper.pdf>. cited page(s) 12